

머릿말

RAG 관련 자료를 처음 찾아보셨을 때, 혹시 이런 느낌 아니었나요? "LangChain 공식 문서는 있는데... 이걸 어디서 어떻게 시작하지?"

"예제는 따라 했는데, 막상 내 문서를 넣으면 검색이 왜 이렇게 안 되지?" "기본 RAG는 만들었는데, 정확도를 올리려면 뭘 건드려야 하지?"

이 책은 그 질문들에서 시작했습니다.

이 책에서 만드는 것

처음부터 끝까지 하나의 프로젝트를 만듭니다.

사내 AI 비서 **ConnectHR**입니다. 직원 정보를 조회하고, 인사 규정 문서를 검색하고, 두 가지를 한 번에 답해주는 시스템입니다. CH01에서 LLM 환각을 체험하고, CH11에서 튜닝을 모두 적용한 완성 에이전트로 마무리합니다.

조각난 예제가 아닙니다. 처음부터 끝까지, 하나의 프로젝트입니다.

다른 RAG 자료와 다른 점

첫째, 실패부터 시작합니다.

각 챕터는 잘 동작하는 코드가 아니라, 에러나 한계 상황에서 출발합니다. "왜 이게 안 되지?"를 먼저 경험하고, 그 이유를 찾고, 해결하는 순서입니다. 그 과정이 이해를 만듭니다.

둘째, 튜닝까지 다룹니다.

대부분의 RAG 자료는 검색 결과가 나오는 순간 멈춥니다. 이 책은 "왜 엉뚱한 문서를 가져오나", "왜 같은 뜻인데 못 찾나"까지 파고듭니다. 검색 품질 튜닝(CH08 하이브리드 검색), 질문 해석 개선(CH09 Query Rewrite), PDF·이미지 처리(CH10 Vision LLM), 그리고 이 모든 튜닝을 한 서비스로 엮어 내는 완성 에이전트(CH11)까지 포함되어 있습니다.

셋째, 이야기로 읽힙니다.

API 명세가 아니라 스토리입니다. 팀장의 지시에서 시작해서, 동료의 불만을 들으면서, ConnectHR이 한 단계씩 성장하는 이야기입니다. 비유와 상황으로 먼저 개념을 잡고, 그 다음에 코드로 들어갑니다.

이 책의 구조

챕터 하나는 상황 → 코드 → 개념 → 결과의 흐름으로 이어집니다. 현실에서 마주치는 문제가 먼저 등장하고, 그 문제를 푸는 코드가 뒤따르고, 코드가 왜 그렇게 생겼는지 개념으로 정리한 뒤, 실제 실행 결과로 닫습니다.

코드가 낯설다면 상황 설명과 결과 해석 부분을 먼저 따라가도 흐름이 이해됩니다. 코드는 그 뒤에 돌아와 다시 읽어도 늦지 않습니다.

이 책이 맞는 분

- Python 기초 문법은 알지만, LLM이나 RAG는 처음인 분
- 예제는 따라 해봤지만, 처음부터 끝까지 하나의 프로젝트를 만들어본 적 없는 분
- "이론은 알겠는데, 실제로 작동하는 걸 보고 싶다"는 분

마지막으로

이 책은 정답을 알려주지 않습니다.

에러를 만나고, 왜 그런지 고민하고, 고치는 과정을 함께 걸어갑니다. ConnectHR이 완성될 때쯤이면, 단순히 코드를 복붙한 게 아니라 "왜 이렇게 만들었는지"를 이해하게 됩니다.

그게 이 책이 하고 싶은 것입니다.

자, 이제 시작해보겠습니다.

들어가며: 사서를 키우다

ConnectHR 대시보드에 질문이 흘러가고 있었습니다. "연차 신청 절차 알려줘"가 들어오고 2초 뒤 출처와 함께 답변이 올라갑니다. 옆자리 동료가 "A 사원 연차 며칠 남았어? 사용 규정도 알려줘"라고 치자 DB에서 숫자를 꺼내고 문서에서 규정을 찾아 한 번에 답합니다. 캐시에 있던 질문은 0.1초 만에 돌아왔습니다.

오픈이는 턱을 괴고 모니터를 바라봤습니다. 커서가 깜빡이는 입력창 위로 질문이 하나 더 올라갑니다. 또 답합니다. 아무도 놀라지 않습니다. 당연한 것처럼 질문하고 당연한 것처럼 답변을 받습니다.

4개월 전에는 환각이 뭔지도 몰랐는데.

예전에는 저 질문에 AI가 자신 있게 거짓말을 했습니다. 지금은 아무도 그 시절을 기억하지 못합니다. 다만 시작은 또렷하게 남아 있습니다.

모니터를 바라보던 시선이 4개월 전으로 되감깁니다.

팀장 : "AI로 사내 문서 검색 시스템 만들어봐. 직원들이 규정이나 정책 찾는 게 번거롭다고 해서."

AI로? 사내 문서를? 나 혼자서?

사수도 없었습니다. 옆자리는 비어 있고 물어볼 사람도 없었습니다. 노트북을 열고 ChatGPT에 그대로 쳐봤습니다.

"우리 회사 연차 규정이 어떻게 되나요?"

AI가 답합니다. 연차는 15일이고 3일 전까지 신청하면 된다고요.

오, 이거면 되는 거 아니야?

서랍에서 규정집을 꺼냈습니다. 모니터 왼쪽에 세워놓고 오른쪽 화면의 AI 답변과 한 줄씩 대조하기 시작했습니다. 첫 줄부터 달랐습니다. 연차 일수가 틀렸습니다. 신청 기한도 틀렸습니다. 존재하지 않는 조항까지 지어냈습니다. 열 줄을 비교하는 동안 한 줄도 맞는 게 없었는데 화면 속 AI는 여전히 확신에 찬 어조였습니다. 규정집을 쥔 손가락 끝이 하얗졌습니다.

세상의 모든 공개 자료는 섭렵했지만 우리 회사 내부 문서는 본 적 없는 외부인. 모르면 모른다고 하면 될 텐데 그럴듯한 답을 만들어냅니다.

이게 **환각(Hallucination)**입니다.

그러면 문서를 직접 넣어주면 되지 않을까. 규정 내용을 통째로 프롬프트에 붙여봤습니다. 연차 규정 한 페이지를 넣었더니 제대로 답합니다. 됐다 싶어서 규정집 200페이지를 한꺼번에 넣었습니다. 토큰 한도를 넘겨서 잘려나갔습니다. 절반도 읽지 못한 AI가 앞부분만 가지고 다시 자신 있게 답하기 시작합니다.

환각이 돌아왔습니다.

팀장 : "전부 외우게 하지 말고 필요한 것만 찾아서 읽게 해."

그날 밤 샤워하다가 문득 떠오른 게 있었습니다. 대학교 오픈북 시험. 교과서 전체를 외울 필요 없이 문제가 나오면 해당 페이지를 펼쳐서 읽으면 됐습니다. AI한테도 똑같이 하면 됩니다. 질문이 들어올 때마다 200페이지 전체가 아니라 관련된 두세 페이지만 골라서 건네주면 되는 겁니다.

이게 **RAG**입니다. 모든 걸 외우게 하는 대신 필요한 문서만 찾아서 읽게 하는 구조. 문서를 찾으려면 문서가 정리된 곳이 있어야 합니다. 그래서 도서관을 짓기로 했습니다.

도서관을 짓는 일은 생각보다 지저분했습니다.

건물부터 세워야 했습니다. "팀원 연차 며칠 남았어?" 같은 질문은 문서 어디에도 답이 없었습니다. 개인별 잔여 연차는 인사 DB에 들어 있었습니다. AI가 DB를 직접 뒤지게 할 수는 없으니 데이터를 꺼내주는 API를 따로 만들었습니다. 주방에 손님이 직접 들어가는 식당은 없으니까요. 주문을 받으면 웨이터가 주방에서 가져다줍니다.

그다음은 장서입니다. 공유 드라이브를 열었습니다. PDF, DOCX, XLSX, 심지어 HWP까지 300개가 넘는 파일이 한 폴더에 쌓여 있었습니다. "인사규정_최종.docx" 옆에 "인사규정_최종_진짜최종.docx"가 있었고 어느 게 현행 문서인지 파일명만 봐서는 알 수 없었습니다. 스크롤을 내릴수록 눈앞이 아득해졌습니다.

팀장 : "쓰레기를 넣으면 쓰레기가 나와."

폐기 문서를 걸러내야 했습니다. 하나씩 열어보고 날짜를 확인하고 현행 여부를 체크합니다. 살릴 문서만 추려냈습니다. 형식별로 파싱하고 메타데이터를 붙이고 폴더 구조를 잡는 데만 며칠. 사서가 새 책을 받으면 바로 서가에 꽂지 않는 것처럼. 분류표를 확인하고 라벨을 붙이고 청구기호를 매긴 다음에야 서가에 올립니다.

문서를 골랐으면 서가에 꽂을 차례입니다. 마트에서 사온 재료를 봉지째 냉장고에 던지면 나중에 찾을 수 없습니다. 양파가 어디 있는지 고기는 아직 쓸 수 있는지 뒤져봐야 합니다. 제대로 하려면 손질하고 먹기 좋게 다듬고 용기에 나눠 담아야 합니다. 문서도 똑같았습니다. 텍스트를 꺼내고 적당한 크기로 자르고 숫자 배열로 변환해서 벡터DB에 저장합니다.

사람한테는 "연차"와 "유급 휴가"가 같은 말입니다. 기계한테는 아닙니다. 글자가 다르면 다른 단어입니다. 서가에 꽂힌 문서끼리 의미가 가까운지 먼지를 기계가 판단할 수 있도록 만드는 데까지 한 달이 걸렸습니다.

한 달 뒤. 서가에 문서가 꽂혀 있고 검색하면 관련 문서가 유사도 점수와 함께 나왔습니다. 동료 자리로 걸어가서 화면을 돌렸습니다.

동료 : "검색 결과 다섯 개를 던져주지 말고 그냥 답을 알려줘."

걸음을 멈추고 자리로 돌아왔습니다. 서가에서 책을 찾아오는 건 됐는데 그 책을 읽고 정리해서 답해주는 사람이 없었습니다.

도서관은 완성됐는데 사서가 없었습니다.

사서를 앉혔습니다. 질문을 듣고 서가에서 문서를 찾고 읽어서 정리하고 출처까지 알려주는 사서. "어떤 문서에서 이 답을 찾았는지" 반드시 보여주게 만들었습니다. 출처 없는 답변은 근거 없는 주장이니깐요. 한번 물어보고 끝이 아니라 대화를 이어갈 수 있는 기억력도 붙여줬습니다.

그제야 사서가 일을 시작합니다.

기빠할 틈도 없이 새 문제가 터졌습니다.

동료 : "A 사원 연차 며칠? 그리고 연차 신청 절차 알려줘."

한 문장에 두 종류가 섞여 있었습니다. 잔여 연차는 DB에 있고 신청 절차는 문서에 있습니다. 사서는 서가의 문서밖에 모릅니다. DB 쪽 담당자를 부를 줄도 모릅니다. 이들 동안 모니터 앞에서 화이트보드에 화살표를 그리고 지우고 다시 그렸습니다.

팀장 : "1층 안내데스크 가봤어? 거기 직원이 뭘 해?"

가만히 생각해봤습니다. 안내데스크 직원은 모든 업무를 직접 처리하지 않습니다. 이걸 인사팀, 이걸 총무팀, 이걸 시설관리팀. 누구에게 물어봐야 하는지를 아는 게 그 사람의 역할입니다. 질문이 들어오면 유형을 분류하고 맞는 담당자를 호출합니다. DB를 조회하는 담당자, 문서를 검색하는 담당자, 둘 다 호출해서 합치는 담당자.

사서를 안내데스크 직원으로 승진시켰습니다.

이렇게 **ConnectHR**이 태어났습니다.

ConnectHR을 동료들에게 풀어줬습니다.

동료 : "아까도 물어봤는데 또 20초나 기다려?"

같은 질문이 들어올 때마다 매번 처음부터 답을 만들고 있었습니다. 사서가 같은 책을 꺼내서 같은 페이지를 다시 읽고 같은 답을 다시 쓰고 있었던 겁니다. 10번 물어보면 10번 서가를 뒤집니다. 사서에게 메모장을 줬습니다. 한 번 답한 건 적어두고 같은 질문이 오면 메모를 보여줍니다. 20초가 0.1초가 됐습니다. 다만

메모장에는 유통기한을 뒀습니다. 규정이 바뀌었을 수도 있으니까요.

업무 일지도 쥐어줬습니다. 하루에 토큰을 얼마나 쓰는지, 비용은 얼마인지 기록하게 했습니다. 느린 질문이 어떤 건지도 따로 남겼습니다.

운영은 안정됐습니다. 빨라지니까 쓰는 사람이 늘었고 늘어나니까 그동안 보이지 않던 문제가 하나씩 터지기 시작합니다.

동료 : "병가 규정을 물어봤는데 출장 규정이 나왔어요."

뭐라고?

퇴근길 지하철에서 노트북을 꺼냈습니다. 흔들리는 객차 안에서 로그를 열어봤습니다. LLM은 멀쩡했습니다. 받은 문서를 기반으로 성실하게 답했을 뿐입니다. 문제는 그 문서였습니다. 검색 결과를 하나씩 열어보니 사서가 서가에서 엉뚱한 책을 꺼내온 겁니다. 500자마다 기계적으로 잘라놓은 문서 조각에서 병가 규정 뒷부분과 출장 규정 앞부분이 한 덩어리로 섞여 있었습니다.

한 달 전에 꽃았던 서가를 다시 정리해야 했습니다.

가위로 일정하게 자르던 걸 의미 단위로 바꿨습니다. 문서에서 주제가 바뀌는 지점을 감지하고 거기서 끊게 했습니다. 검색 결과도 한 번 더 훑어서 관련 없는 문서를 걸러내게 합니다. 키워드로만 찾던 검색에 의미 검색을 합쳤습니다. 같은 질문을 넣었는데 답이 달라졌습니다. 검색을 바꿨을 뿐인데 사서가 다른 사람이 된 것 같았습니다.

검색이 좋아지니까 이번엔 질문이 문제였습니다.

동료 : "WFH 정책 알려줘."

ConnectHR이 멈췄습니다. 문서에는 "재택근무"라고 적혀 있었으니까요. "휴가 규정 알려줘"도 마찬가지로였습니다. 문서에는 "연차유급휴가"라고 돼 있습니다. 검색 엔진이 아무리 좋아도 질문 자체를 이해 못하면 소용없습니다.

초보 사서는 서가에서 "WFH"라는 글자만 찾습니다. 당연히 없습니다. 경험 많은 사서는 다릅니다.

WFH라면... 재택근무? Work From Home? 혹시 원격근무?

떠올릴 수 있는 표현을 전부 떠올려서 각각 찾아봅니다. 사서에게 그 감각을 심어줬습니다. 약어를 풀어쓰고 동의어를 확장하고 한 질문을 여러 각도로 바꿔서 검색하게 했습니다. 그리고 답변에 근거를 붙였습니다. "이 문서의 3페이지에서 찾았습니다"라고 원본 이미지와 함께.

마지막 관문은 스캔 PDF였습니다.

총무팀이 보내준 오래된 사규집을 열었습니다. 조직도 PDF. 텍스트를 추출하려고 했는데 아무것도 나오지 않았습니다. 종이 문서를 스캐너에 올려서 찍은 거라 페이지 전체가 이미지입니다. 사람 눈에는 조직도의 박스와 화살표가 또렷하게 보이는데 기계한테는 그냥 점들의 배열일 뿐입니다. 텍스트 추출기를 아무리 돌려도 "대표이사경영지원본부기술개발본부"가 한 줄로 붙어 나왔습니다.

글자가 보이는데 읽을 수가 없어.

사서에게 눈을 달아줬습니다. 글만 읽던 사서가 이미지를 보고 거기 적힌 내용을 파악하게 됐습니다.

그리고 성적표를 만들었습니다. "잘 되는 것 같다"는 느낌만으로는 어디를 고쳐야 할지 모릅니다. 찾아온 문서 중 몇 개가 정답이었는지. 정답 문서를 몇 개나 찾았는지. AI가 지어낸 답변은 없었는지. 이 지표들로 ConnectHR의 실력을 숫자로 측정했습니다. 느낌이 아니라 숫자입니다. 튜닝 전과 후를 비교했습니다. 숫자가 올라가는 걸 확인하고 나서야 의자 등받이에 등을 기댔습니다.

튜닝을 한 가지씩 더할 때마다 코드가 여기저기 흩어졌습니다. 청킹 재설계, 하이브리드 검색, 리랭킹, 쿼리 풀어쓰기, 비전 OCR이 각자 다른 챕터에 한 줄씩 있었습니다. 한 번 더 정리가 필요했습니다. 질문이 들어오는 입구는 하나로, 답이 나가는 출구도 하나로. 뒷단에서 사서가 무엇을 부르든 바깥에서는 같은 모양이 보이게. 부품별로 책임을 쪼개고 한 파이프라인에 엮었습니다.

그렇게 **ConnectHR**이 완성된 얼굴로 대시보드에 섰습니다.

4개월이 지났습니다.

돌이켜보면 티켓 열 장이었습니다. 매번 모르는 단어 앞에서 멈췄고 검색하고 틀리고 다시 읽었습니다. 대단한 깨달음 같은 건 없었습니다. 하나를 풀면 다음 문제가 터졌고 그걸 또 풀었을 뿐입니다.

다만 한 가지 달라진 게 있습니다.

예전에는 "AI가 엉뚱한 답을 해"라는 말을 들으면 LLM을 의심했습니다. 지금은 검색을 의심합니다. 청킹을 확인하고 리랭킹을 떠올리고 쿼리를 바꿔봅니다. 이름을 외운 게 아니라 문제와 해결을 한 쌍으로 기억하게 된 겁니다.

이 책은 그 열 쌍의 기록입니다. **오픈이**가 부딪혔던 문제가 있고 **팀장**이 던져준 비유로 감을 잡은 뒤 직접 만들어보며 넘어가는 과정이 있습니다.

문제를 보고 어디를 건드려야 하는지 보이는 감각. 이 책이 드리고 싶은 건 그겁니다.

첫 번째 과제부터 시작하겠습니다.

시작하기 전에

1. Git 레포지토리

이 책에서는 두 개의 Git 레포지토리를 사용합니다.

레포	용도	주소
rag-start	실습용. import와 데이터가 준비된 파일에 TODO를 채워넣습니다	https://github.com/metacoding-18-ai-applied-v4/rag-start
rag-end	완성 코드. 막히면 여기서 정답을 확인합니다	https://github.com/metacoding-18-ai-applied-v4/rag-end

1.1 실습 흐름

1. **rag-start** 레포를 클론합니다.
2. 챗터를 보면서 **[실습]** 파일의 TODO 부분에 코드를 작성합니다.
3. 막히면 **rag-end** 완성 코드를 참고합니다.

```
git clone https://github.com/metacoding-18-ai-applied-v4/rag-start.git
cd rag-start
```

1.2 폴더 구조

각 챗터는 하나의 예제 폴더에 대응합니다. 챗터를 진행할수록 폴더 번호가 올라가며 시스템이 한 단계씩 성장합니다.

rag-start/		
├── ex01/	←	CH01: 환각과 RAG의 첫 만남
├── ex02/	←	CH02: 일단 사내 시스템부터
├── ex04/	←	CH04: 문서를 지식으로 바꾸다
├── ex05/	←	CH05: 드디어 답해준다
├── ex06/	←	CH06: 연차도 규정도 한번에
├── ex07/	←	CH07: 실제로 써보니
├── ex08/	←	CH08: 엉뚱한 문서를 가져온다
├── ex09/	←	CH09: 질문을 제대로 이해 못한다

ex10/ ← CH10: PDF 이미지까지 잡아라
ex11/ ← CH11: 커넥트HR 에이전트의 완성

CH03은 이야기 챕터로 코드가 없습니다.

1.3 코드 분류

각 챕터의 코드 파일에는 세 가지 분류가 붙어 있습니다.

분류	의미	rag-start 상태	독자 액션
[실습]	챕터 핵심 코드	import + 데이터 + TODO 주석	TODO를 채워 코드 작성
[설명]	중요하지만 핵심은 아닌 코드	완성 코드 그대로	코드를 읽고 이해
[참고]	이 챕터 주제가 아닌 코드	완성 코드 그대로	파일명과 한 줄 설명만 확인

[실습] 파일에는 import, 상수, 더미 데이터, 테스트 입력값이 미리 준비되어 있습니다. 챕터를 따라 하며 #
TODO: 주석 부분만 채워넣으면 됩니다. [설명] 과 [참고] 파일은 실행에 필요한 완성 코드가 이미 들어
있으므로 수정하지 않습니다.

2. 환경 설정

2.1 Python 설치

이 책의 모든 예제는 Python 3.12 를 기준으로 작성되었습니다. 3.10~3.12에서 동작하며 3.13 이상에서는 일부
패키지 호환성 문제가 있을 수 있습니다.

macOS

```
# Homebrew로 설치
brew install python@3.12
```

Windows

공식 사이트(<https://www.python.org/downloads/>)에서 Python 3.12를 다운로드합니다. 설치 시 "Add
Python to PATH" 체크박스를 반드시 선택하세요.

2.2 가상환경 설정

예제마다 패키지 버전이 다를 수 있으므로 반드시 가상환경을 만들어서 진행하세요.

```
# 가상환경 생성
python3.12 -m venv .venv

# 활성화 (macOS/Linux)
source .venv/bin/activate

# 활성화 (Windows)
.venv\Scripts\activate

# 패키지 설치
pip install -r requirements.txt
```

가상환경이 활성화되면 터미널 프롬프트 앞에 `(.venv)` 가 표시됩니다.

2.3 Ollama 설치

Ollama는 로컬 LLM을 실행하는 도구입니다. 공식 사이트(<https://ollama.com>)에서 설치하세요.

```
# 설치 확인
ollama --version
```

2.4 LLM 모델 다운로드

챕터별로 사용하는 모델이 다릅니다.

모델	사용 챕터	용도	RAM
<code>deepseek-r1:8b</code>	CH01~05	추론(Chain-of-Thought) 특화	16GB 이상
<code>nomic-embed-text</code>	CH01	임베딩 (CH04부터 ko-sroberta로 교체)	—
<code>llama3.1:8b</code>	CH06~CH11	Tool Calling 지원	16GB 이상
<code>qwen2.5vl:7b</code>	CH10·CH11	비전 LLM (스캔 PDF 처리)	16GB 이상

```
# CH01~05 모델
ollama pull deepseek-r1:8b
ollama pull nomic-embed-text

# CH06~CH11 모델
ollama pull llama3.1:8b

# CH10·CH11 비전 모델
ollama pull qwen2.5vl:7b
```

RAM이 부족하거나 응답이 너무 느리면: `.env` 에서 `LLM_PROVIDER=openai` 로 바꿔서 GPT-4o-mini를 쓸 수 있습니다. 단, API 비용이 발생합니다.

2.5 .env 파일 설정

각 예제 폴더의 `.env.example` 을 `.env` 로 복사한 뒤 값을 채워넣으세요.

```
cp .env.example .env
```

```
# LLM 설정
LLM_PROVIDER=ollama
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=deepseek-r1:8b          # CH01~05
# OLLAMA_MODEL=llama3.1:8b          # CH06~CH11 (Tool Calling 필요)

# OpenAI 사용 시
# LLM_PROVIDER=openai
# OPENAI_API_KEY=sk- ...
# OPENAI_MODEL=gpt-4o-mini

# PostgreSQL (ex02 이후)
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=rag_db
POSTGRES_USER=rag_user
POSTGRES_PASSWORD=rag_password

# 벡터DB + 임베딩
CHROMA_PERSIST_DIR=./data/chroma_db
EMBEDDING_MODEL=jhgan/ko-sroberta-multitask
```

```
# 비전 LLM (ex10·ex11)
VISION_MODEL=qwen2.5vl:7b
VISION_PROVIDER=ollama
# VISION_PROVIDER=openai # 로컬 사양 부족 시
```

2.6 Docker (PostgreSQL)

ex02 이후 예제는 PostgreSQL이 필요합니다. Docker Compose로 실행합니다.

```
docker compose up -d
```

PostgreSQL 16 Alpine 이미지를 사용하며 `data/schema.sql` 이 자동으로 초기화됩니다.

2.7 핵심 패키지 요약

패키지	버전	용도
<code>langchain</code>	0.3.x	RAG 파이프라인, 에이전트
<code>chromadb</code>	1.5.x	벡터 데이터베이스
<code>fastapi</code>	0.115.x	API 서버
<code>sentence-transformers</code>	3.3.x	한국어 임베딩 + Cross-Encoder Reranker (CH11)
<code>psycopg2-binary</code>	2.9.x	PostgreSQL 연결
<code>pypdf</code>	4.3.x	PDF 파싱
<code>python-docx</code>	1.1.x	DOCX 파싱
<code>openpyxl</code>	3.1.x	XLSX 파싱
<code>rank-bm25</code>	0.2.x	하이브리드 검색 (CH08·CH11)
<code>easyocr</code>	1.7.x	OCR (CH10)

각 예제 폴더의 `requirements.txt` 로 한 번에 설치할 수 있습니다.

```
pip install -r requirements.txt
```


3. 자주 만나는 오류

3.1 `python` 명령어가 안 될 때

macOS/Linux에서는 `python` 대신 `python3.12` 를 사용해야 할 수 있습니다.

```
# python이 안 되면
python3.12 --version
python3.12 -m venv .venv
```

3.2 `pip install` 에서 권한 오류

가상환경 없이 시스템 Python에 설치하려고 하면 권한 오류가 발생합니다. 가상환경을 먼저 활성화하세요.

```
# 이렇게 하면 안 됩니다
pip install langchain # PermissionError 또는 externally-managed-environment

# 이렇게 하세요
source .venv/bin/activate # 먼저 가상환경 활성화
pip install -r requirements.txt
```

3.3 `pip` 대신 `pip3`

`pip` 명령이 안 되면 `pip3` 를 사용하세요. 가상환경 안에서는 둘 다 동일합니다.

3.4 `psycopg2-binary` 설치 실패 (macOS Apple Silicon)

M1/M2/M3 Mac에서 `psycopg2-binary` 설치가 실패할 수 있습니다.

```
# libpq 먼저 설치
brew install libpq
pip install psycopg2-binary
```

3.5 Ollama 연결 오류

`ConnectionRefusedError: Connection refused`

Ollama 서버가 실행 중인지 확인하세요.

```
# Ollama 서버 실행  
ollama serve
```

```
# 또는 Ollama 앱을 실행하면 자동으로 서버가 시작됩니다
```

3.6 모델 다운로드 오류

```
model not found
```

해당 모델을 아직 다운로드하지 않은 경우입니다. `ollama pull 모델명` 으로 다운로드하세요.

챕터 1. AI한테 물어봤더니 거짓말을 합니다. 환각과 RAG

이번 챕터가 끝나면

- LLM이 모르는 것을 자신 있게 지어낸다는 걸 직접 확인합니다 (환각)
- 문서를 직접 건네주면 거짓말을 멈춘다는 걸 배웁니다 (Context Injection)
- 이걸 자동화한 RAG 파이프라인을 직접 만들어봅니다 (RAG)

챕터 1은 맛보기 챕터입니다

이 챕터는 **본격 여정에 들어가기 전** RAG의 원리만 빠르게 체험하는 시간입니다. 더미 데이터 3건으로 "환각 → 문서 주입 → RAG"가 어떤 흐름인지 몸으로 느끼는 것이 목적입니다. 실제 사내 AI 비서 **커넥트HR 에이전트**를 만드는 본격 여정은 **챕터 2부터** 시작됩니다. 지금은 가볍게 훑는다는 기분으로 따라오세요.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 준비

이 책의 실습은 GitHub 레포를 클론해서 진행합니다.

레포	용도	주소
rag-start	실습용 (빈 파일)	github.com/metacoding-18-ai-applied-v4/rag-start
rag-end	완성 코드 (참고)	github.com/metacoding-18-ai-applied-v4/rag-end

아직 클론하지 않았다면 터미널에서 실행합니다.

터미널 레포 클론

```
git clone https://github.com/metacoding-18-ai-applied-v4/rag-start.git
cd rag-start/ex01
```

파일 구조는 이렇습니다.

ex01 디렉토리

```
ex01/
├── step1_fail.py           # [실습] LLM 단독 호출 → 환각 체험
├── step2_context.py       # [실습] 컨텍스트 직접 주입 → 임시 해결
├── step3_rag.py           # [실습] RAG 기본 파이프라인 구성
├── step4_no_chunking.py   # [실습] 청킹 없이 비교 → 차이 체감
└── step5_rag.py           # [실습] 추론 심화 (Chain-of-Thought)
```

컨텍스트, 청킹, Chain-of-Thought 같은 단어가 낯설어도 괜찮습니다. 파일 이름에 미리 등장한 것이고, 각 실습에서 하나씩 풀어 설명합니다.

[실습] 파일에는 import와 데이터가 미리 준비되어 있습니다. 챕터를 따라 하며 TODO 부분을 채워넣으세요. 막히면 rag-end의 완성 코드를 참고하세요.

2. 실습 환경 구축

터미널 Python 환경 구성. macOS / Linux

```
cd ex01
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

터미널 Python 환경 구성. Windows

```
cd ex01
py -3.12 -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Windows는 PowerShell이 아니라 **명령 프롬프트(cmd)** 를 권장합니다. PowerShell 사용 시 `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser` 로 스크립트 실행을 허용한 뒤 `.venv\Scripts\Activate.ps1` 을 실행하세요.

Ollama 모델이 아직 없다면 다운로드합니다.

터미널 Ollama 모델 다운로드

```
ollama pull deepseek-r1:8b
ollama pull nomic-embed-text
```

LLM 선택

기본값은 Ollama + `deepseek-r1:8b` 입니다 (16GB RAM 이상 권장). RAM이 부족하거나 응답이 너무 느리면 `.env` 에서 `LLM_PROVIDER=openai` 로 바꿔서 GPT-4o-mini를 쓸 수도 있습니다. 단, API 비용이 발생합니다. `.env` 파일에 `OPENAI_API_KEY=sk-xxxxxx` 형태로 키를 등록하세요.

3. 사용할 라이브러리

이번 챕터에서는 **LangChain**이라는 프레임워크를 사용합니다. LLM 호출, 벡터 검색, 체인 조립처럼 RAG에 필요한 부품을 제공하는 도구입니다. 여기서는 맛보기로만 쓰고 챕터 5에서 본격적으로 다룹니다.

패키지	역할
<code>langchain-ollama</code>	Ollama LLM/임베딩 연동
<code>langchain-chroma</code>	ChromaDB 벡터 저장소
<code>langchain-classic</code>	RetrievalQA 체인
<code>chromadb</code>	벡터 DB

지금은 개념만 잡으세요

LangChain, 임베딩, 벡터 DB 같은 용어가 한꺼번에 나와서 부담스러울 수 있습니다. 지금은 "문서를 넣어주면 LLM이 정확하게 답한다"는 **RAG의 개념**만 잡으면 충분합니다. 각 기술의 동작 원리는 챕터 4~챕터 5에서 차근차근 다룹니다.

4. 실습 순서

이번 챕터의 실습 5개는 이 순서로 진행합니다.

1. `ex01/step1_fail.py` . LLM 단독 질문 (환각 체험)
2. `ex01/step2_context.py` . 문서 직접 전달
3. `ex01/step3_rag.py` . RAG 파이프라인
4. `ex01/step4_no_chunking.py` . 청킹 없이 비교
5. `ex01/step5_rag.py` . 추론 심화

환각을 직접 체험하고(step1), 문서를 넣으면 달라지는 걸 확인한 뒤(step2), RAG로 조립합니다(step3). 그다음 청킹 없이 돌려서 차이를 체감하고(step4), 추론이 필요한 질문까지 던져봅니다(step5). **step1부터 순서대로 실행하세요.**

1.1 그럴듯한 거짓말: LLM 환각(Hallucination)



그림 1-1. 입사 3일 차, 첫 번째 미션

입사 3일 차.

키보드 두드리는 소리만 또룩또룩 울리는 사무실. 모니터에는 사내 Wi-Fi 비밀번호가 적힌 포스트잇이 하나 붙어 있습니다. 오전 10시, 팀장이 커피잔을 들고 자리로 다가왔습니다. 잔에서 김이 올라오고 있었습니다.

팀장 : "AI로 사내 문서 검색 시스템 만들어봐요. 직원들이 규정이나 정책 찾는 게 번거롭다고 해서."

손이 멈췄습니다.

AI 비서. 사내 문서. 대화식 검색. 나 혼자서?

오픈이 : "언제까지요?"

팀장 : "급하진 않아요. 2주 내로 간단한 프로토타입만."

팀장이 자리로 돌아간 뒤에도 한참 모니터를 쳐다봤습니다. 옆자리에서 동료가 의자를 돌려 저를 보더니 한마디 던졌습니다.

동료 : "그거 왜 만들어요? ChatGPT한테 그냥 물어보면 되지 않나요."

...그러게. 직접 물어보면 되는 거 아니야?

마침 로컬에 `deepseek-r1` 모델을 올려두었습니다. 연차 규정부터 물어보기로 했습니다.

`ex01/step1_fail.py`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 1 ex01/step1_fail.py. LLM에게 직접 물어보기

```
# TODO: ChatOllama로 deepseek-r1:8b 모델 연결 (temperature=0)
# 1. Ollama에서 deepseek-r1:8b 모델을 로드 (temperature=0: 가장 확률 높은 답변)
llm = ChatOllama(model="deepseek-r1:8b", temperature=0)

question = "우리 회사(커넥트)의 신입사원 연차 발생 규정이 어떻게 돼?"

# TODO: 질문 출력 → llm.invoke로 답변 받기 → 답변 출력
# 2. 질문을 터미널에 출력
console.print(f"[bold]질문:[/bold] {question}\n")
# 3. LLM에 질문을 보내고 답변을 받음
response = llm.invoke(question)
# 4. 답변을 출력
console.print(f"[bold]답변:[/bold]\n{response.content}")
```

`ChatOllama`는 LangChain이 Ollama LLM을 호출할 때 쓰는 래퍼입니다. `temperature=0`은 LLM이 창의적 변형 없이 가장 확률 높은 답변을 내놓게 하는 설정입니다. 이제 실행합니다.

터미널 실행

```
python step1_fail.py
```

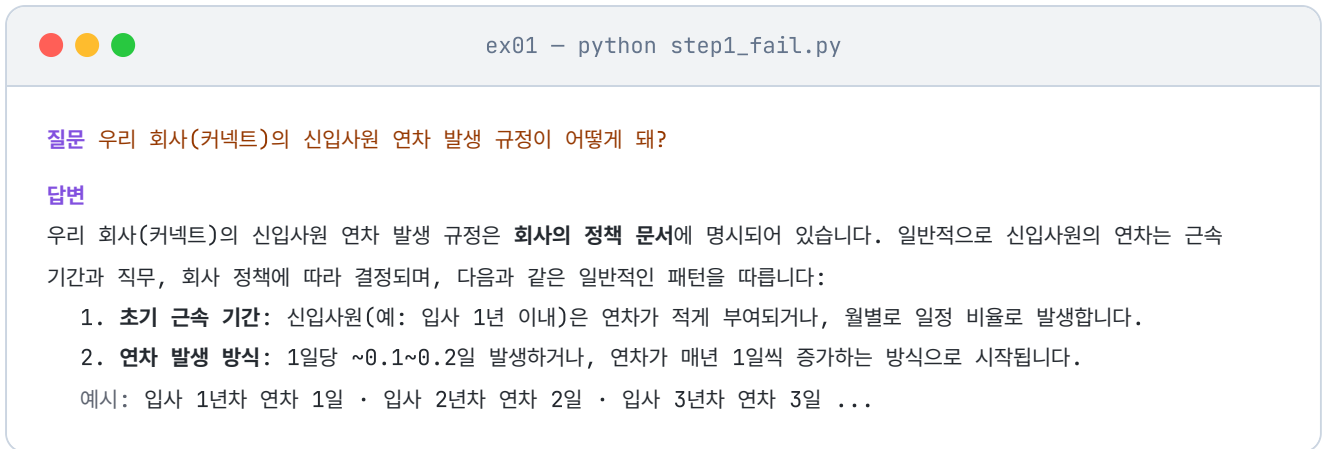


그림 1-2. `step1_fail.py` 실행 결과. 자신감 있게 답하지만 실제 커넥트 규정과 다릅니다

답변을 보면 "근로기준법에 따라 1년 미만은 매월 1일..." 같은 내용이 나옵니다. 공식적인 느낌도 나고 그럴듯합니다. 입사할 때 받은 규정집을 꺼내 비교해봤습니다. 커넥트의 실제 규정은 이랬습니다.

신입사원은 입사 후 3년 동안은 연차가 없다. 대신 매월 1회 '리프레시 데이'를 유급으로 제공한다. 3년 근무 시 30일의 연차가 일시에 발생한다.

잠깐, 뭐라고?

다시 읽었습니다. 완전히 다른 내용이었습니다. LLM이 방금 그럴듯한 거짓말을 한 것입니다.

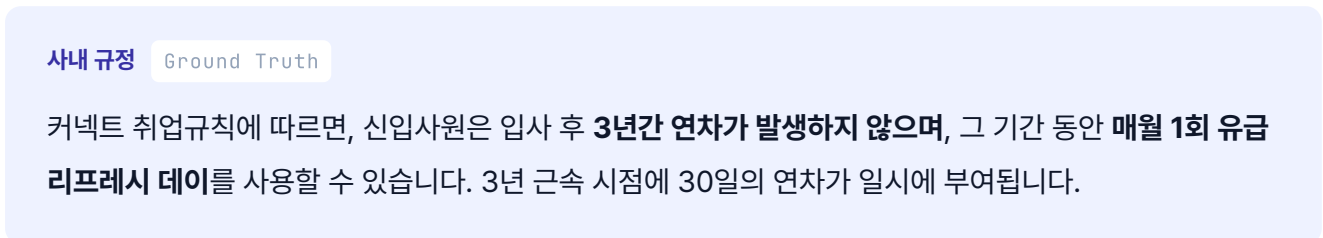
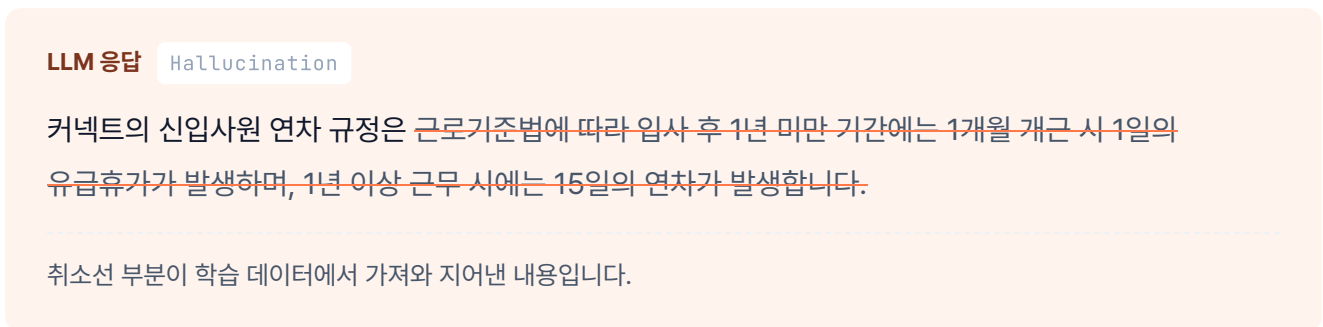


그림 1-3. LLM 응답과 사내 규정을 나란히 비교한 모습입니다

여기서 의문이 생깁니다. LLM은 왜 자신 있게 틀린 대답을 했을까요?

LLM을 이렇게 가정해보겠습니다. 입사 면접을 보러 온 외부인이라고요. 이 외부인은 세상에 공개된 거의 모든 자료를 읽었습니다. 인터넷, 뉴스, 책, 논문까지. 공개된 텍스트라면 뭐든 섭렵했습니다. 그래서 근로기준법은 완벽하게 알고 일반적인 회사 연차 제도도 줄줄 외웁니다. 그런데 커넥트의 내부 규정집은 공개된 적이 없습니다. 이 외부인이 읽을 방법이 없었습니다.

문제는 이 외부인이 "모른다"고 솔직히 말하지 못한다는 점입니다. 질문을 받으면 자기가 아는 것 중에서 가장 비슷해 보이는 걸 자신감 있게 말합니다. 마치 확실히 아는 것처럼 들립니다. 이게 **LLM 환각 (Hallucination)** 입니다.

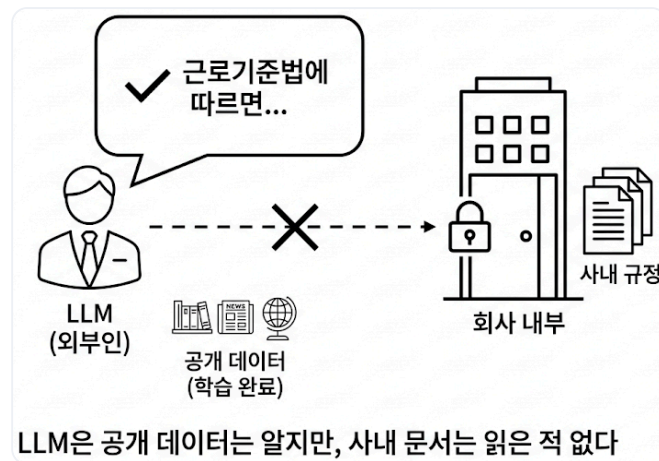


그림 1-4. LLM은 세상의 공개 데이터는 학습했지만, 우리 회사 내부 문서는 읽은 적이 없습니다

1.2 교재를 펼쳐놓으면: 컨텍스트 주입(Context Injection)

규정집을 덮고 잠시 생각했습니다. 모니터 옆 자리에서 팀장이 다시 지나가며 한 마디 던졌습니다.

팀장 : "전부 외우게 하지 말고, 필요한 것만 찾아서 읽게 해요."

그 말이 귀에 남았습니다.

...외우게 하지 말고, 읽게 한다.

학창 시절 오픈북 시험이 떠올랐습니다. 시험 범위를 전부 외우는 대신, 책을 책상 위에 펼쳐놓고 필요한 페이지만 찾아서 답을 쓰는 방식. AI한테도 규정집을 그대로 건네주면 어떻게 될까요?

지금은 DB도 검색 시스템도 없으니, 우선은 đơn순하게 **프롬프트에 규정 내용을 그대로 붙여 넣는** 방법부터 시도합니다. `ex01/step2_context.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 2 ex01/step2_context.py. 문서를 직접 넣어보기

```
# 1. 규정 문서를 변수에 담습니다 (아직 DB 안 씀)
context_data = """
[커넥트 취업규칙]
1. 신입사원은 입사 후 3년 동안은 연차가 없다. (파격적인 규정)
2. 대신 매월 1회 '리프레시 데이'를 유급으로 제공한다.
3. 3년 근속 시 30일의 연차가 일시에 발생한다.
"""

question = "우리 회사(커넥트)의 신입사원 연차 발생 규정이 어떻게 돼?"

# TODO: f-string으로 context_data와 question을 포함한 프롬프트 작성

# 2. context_data를 프롬프트에 직접 넣어서 LLM에 전달
prompt = f"""
아래 [참고 정보]를 보고 질문에 답해줘.
[참고 정보]
{context_data}

질문: {question}
"""

# 3. LLM 호출 → 답변 출력
response = llm.invoke(prompt)
console.print(f"답변: {response.content}")
```

step1과 달라진 부분은 `context_data`를 프롬프트에 직접 넣었다는 것뿐입니다.

터미널 실행

```
python step2_context.py
```



ex01 - python step2_context.py

질문 우리 회사(커넥트)의 신입사원 연차 발생 규정이 어떻게 돼?

답변

[커넥트 취업규칙]에 따르면, 신입사원의 연차 발생 규정은 다음과 같습니다:

1. 신입사원은 입사 후 3년 동안 연차가 없다.
2. 대신 매월 1회 '리프레시 데이'를 유급으로 제공한다.
3. 3년 근속 시 30일의 연차가 일시에 발생한다.

이러한 규정은 신입사원에게 유연한 근무 환경을 제공하고, 장기적으로는 경력 초기부터 회사에 대한 충성도를 높이는 방식으로 설계된 것으로 보입니다.

그림 1-5. `step2_context.py` 실행 결과. 문서를 직접 넣으니 정확하게 답합니다

답변이 규정집과 한 글자도 다르지 않았습니다. 방금 전 거짓말하던 그 LLM이 맞나 싶을 정도였습니다. 코드가 바뀐 것도 아니고, 달라진 건 `context_data` 를 프롬프트에 붙인 것 하나뿐. LLM에게 "정답지"를 건네준 셈입니다.

이걸 **Context Injection (컨텍스트 주입)** 이라고 합니다.

됐다. 이거면 끝난 거 아니야?

기쁨은 오래가지 않았습니다.

손으로 규정집 파일철을 들추다 보니 문서가 한두 개가 아니었습니다. 인사규정 옆에 복지 정책, 보안 지침, 업무 가이드, 출장 규정, 회의록, 프로젝트 문서.... 탭마다 제목이 달려 있고 어떤 건 수십 페이지짜리였습니다.

이걸 매번 전부 복사해서 프롬프트에 붙인다고?

LLM에는 한 번에 처리할 수 있는 텍스트 길이 한도(**컨텍스트 윈도우**)가 있습니다. 문서가 쌓일수록 한도를 넘깁니다. 무엇보다 연차 하나 물어보는데 보안 지침과 복지 정책까지 같이 보내면, LLM이 엉뚱한 조항을 들고 와서 답할 가능성이 커집니다.

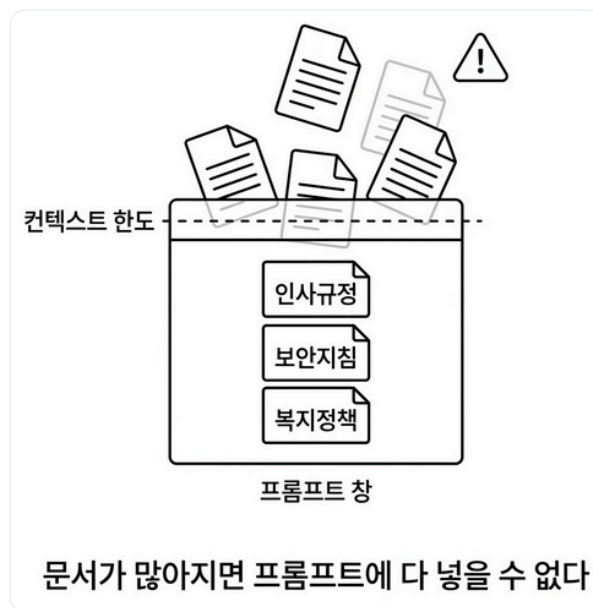


그림 1-6. 문서를 통째로 넣는 방식의 한계. 문서가 늘어나면 프롬프트 창이 넘칩니다

1.3 사서가 필요합니다: RAG(Retrieval-Augmented Generation)

파일철을 다시 내려놓고 한숨을 쉬었습니다. 옆자리 동료가 노트북을 덮으며 말했습니다.

동료 : "문서 찾아주는 사람이 있으면 좋겠네요. 도서관 사서처럼."

사서...

맞는 말이었습니다. 도서관에선 누군가 와서 질문하면 사서가 그 질문에 맞는 책을 서가에서 골라 건네줍니다. 방문자는 그 책만 읽으면 됩니다. 전부 외울 필요도, 서가를 통째로 훑길 필요도 없습니다.

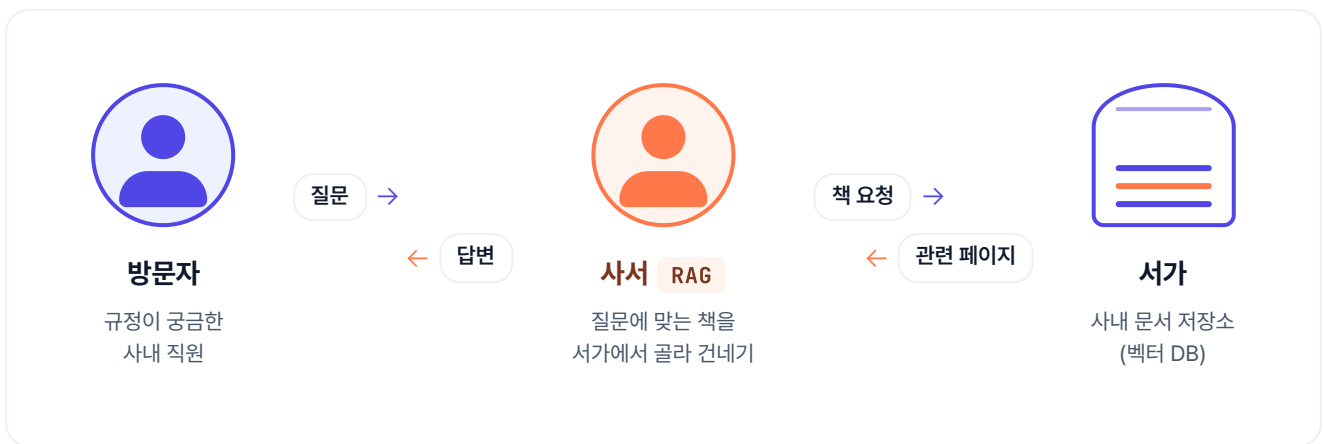


그림 1-7. 사서가 방문자의 질문을 듣고 서가에서 관련 책을 골라 건네줍니다. RAG는 이 '사서'의 역할을 LLM 옆에 앉히는 것입니다

LLM도 같은 방식이면 됩니다. 사내 문서 전체를 외우게 할 필요가 없습니다. 질문이 들어왔을 때 **그 질문에** 해당하는 문서 조각만 찾아서 LLM에게 건네주면 됩니다.

이것이 **RAG (Retrieval-Augmented Generation, 검색 증강 생성)** 입니다. 이름은 거창하지만 하는 일은 단순합니다. 사서를 하나 앉히는 겁니다.



그림 1-8. 위는 RAG 내부 3단계(저장·검색·생성)의 사서 비유,
아래는 같은 질문에서 LLM 단독(점선·근거 없음)과 RAG(실선·출처 포함)의 경로 차이입니다

1.3.1 서가에 책 꽂기: 임베딩 + ChromaDB 인덱싱

`ex01/step3_rag.py`에는 사내 규정 3개가 더미 데이터로 미리 준비되어 있습니다. 이 파일을 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 3 ex01/step3_rag.py. RAG 파이프라인 (1/2: 문서 저장)

```
# 1. 더미 데이터 준비
docs = [
    Document(page_content="[인사규정] 신입사원 휴가 및 연차...",
              metadata={"source": "인사규정"}),
    Document(page_content="[보안규정] 업무 보안: 모든 임직원은...",
              metadata={"source": "보안규정"}),
    Document(page_content="[복지규정] 식대 지원: 점심 식사는...",
              metadata={"source": "복지규정"}),
]

# TODO: OllamaEmbeddings(nomic-embed-text)로 임베딩 생성 → Chroma.from_documents로 벡터D
# B 저장

# 2. 문서를 숫자 벡터로 변환하는 임베딩 모델 로드
embeddings = OllamaEmbeddings(model="nomic-embed-text")

# 3. 문서 3개를 벡터로 변환해서 ChromaDB에 저장
vectorstore = Chroma.from_documents(documents=docs, embedding=embeddings)

# TODO: vectorstore.as_retriever로 검색기 생성 (search_kwargs={"k": 3})

# 4. 질문과 가장 비슷한 문서 3개를 가져오는 검색기 생성
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```

OllamaEmbeddings, ChromaDB란? `OllamaEmbeddings`는 Ollama에 올려둔 임베딩 모델을 불러와 텍스트 → 벡터 변환을 해주는 래퍼입니다. `ChromaDB`는 이 벡터를 저장하고 유사도 검색을 해주는 오픈소스 벡터 데이터베이스입니다.

`OllamaEmbeddings`는 각 문서를 수백 차원의 숫자 배열(벡터)로 변환합니다. 의미가 비슷한 문서일수록 벡터 공간에서 가까이 위치하게 됩니다. `Chroma.from_documents()`가 이 벡터를 ChromaDB에 저장합니다. `k=3`은 "질문과 가장 비슷한 문서 3개를 가져오라"는 설정입니다. `as_retriever()`는 벡터스토어에서 검색 기능만 떼어낸 **Retriever(검색기)**를 만듭니다. 다음 코드에서 RetrievalQA가 이 검색기를 받아 문서 검색부터 답변 생성까지 한 번에 처리합니다.

이 챕터의 임베딩 모델

`nommic-embed-text`를 사용합니다. 챕터 4에서 한국어에 최적화된 `ko-sroberta-multitask`로 교체합니다.

1.3.2 사서에게 질문하기: RetrievalQA 체인 조립

이어서 같은 `ex01/step3_rag.py` 파일의 다음 TODO를 찾아 `pass`를 지우고 아래 코드를 작성합니다.

실습 3 ex01/step3_rag.py. RAG 파이프라인 (2/2: 체인 연결)

```
# 5. 프롬프트 템플릿 - LLM에게 "참고 정보에서만 답해"라고 제약을 걸
template = """당신은 회사의 규정에 대해 설명해주는 AI 비서입니다.
아래의 참고 정보를 바탕으로 질문에 답하세요. 반드시 한국어로 답변해야 합니다.

참고 정보: {context}

질문: {question}
답변: """

PROMPT = PromptTemplate(template=template, input_variables=["context", "question"])

# TODO: ChatOllama(deepseek-r1:8b) → RetrievalQA.from_chain_type으로 체인 조립

# 6. LLM 로드
llm = ChatOllama(model="deepseek-r1:8b", temperature=0)
# 7. 검색기 + LLM을 체인으로 연결 (검색된 문서를 LLM에 자동 전달)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True,
    chain_type_kwargs={"prompt": PROMPT},
)

# TODO: qa_chain.invoke로 질문 실행 → 검색된 문서(근거) 출력 → AI 답변 출력

# 8. 질문 실행 - 검색 + LLM 답변이 한 번에 동작
result = qa_chain.invoke({"query": "신입사원 휴가 규정에 대해 알려줘."})
```

"참고 정보를 바탕으로 질문에 답하세요"라는 한 줄이 핵심입니다. LLM에게 제공된 문서 안에서만 답하도록 제약을 거는 것입니다.

그라운드링(Grounding). LLM이 답변을 **제공된 문서에만** 근거하도록 강제하는 기법. 프롬프트에 "참고 정보에서만 답하라"를 심는 가장 단순한 형태부터, "찾을 수 없으면 모른다고 답하라", "답변 끝에 출처 파일명 명시" 같은 규칙을 추가하는 강한 형태까지 스펙트럼이 있습니다. 챕터 5에서 **출처 강제(Source Grounding)** 라는 강한 버전을 규칙 4개짜리 프롬프트로 구현합니다.

`chain_type_kwargs={"prompt": PROMPT}` 는 위에서 만든 프롬프트 템플릿을 체인에 주입하는 옵션입니다. 이것 넣지 않으면 LangChain 기본 프롬프트가 쓰이는데, 우리가 원하는 한국어 답변 규칙을 적용하려면 직접 넘겨줘야 합니다. `return_source_documents=True` 는 LLM의 답변뿐 아니라 **검색에 사용된 원본 문서도 결과에 포함**시키는 옵션입니다. 이 옵션이 꺼져 있으면 `result["result"]` (답변)만 돌아오고, 켜면 `result["source_documents"]` 에 어떤 문서를 참고했는지까지 함께 돌아옵니다.

RetrievalQA란? LangChain이 제공하는 RAG 전용 체인 클래스. "질문 받기 → Retriever로 문서 검색 → LLM에 문서와 질문 함께 전달 → 답변 반환" 이 전 과정을 한 줄 호출로 돌려줍니다. 수동으로 이어붙여야 할 코드를 내장 추상화로 대체한 셈입니다.

터미널 실행

```
python step3_rag.py
```

ex01 - python step3_rag.py

문서를 학습(임베딩) 중입니다...

질문 신입사원 휴가 규정에 대해 알려줘.

검색된 문서(근거)

[복지규정] 점심 식사는 무제한 법인카드로 지원하며, 저녁 식사는 오후 9시 이후 야근 시에만 사용이 가능합니다.

[보안규정] 모든 임직원은 회사에서 지급한 승인된 보안 USB만 사용해야 하며, 개인 USB나 외부 저장 매체 사용은 엄격히 금지됩니다.

[인사규정] 신입사원은 입사 후 처음 3년 동안은 법정 연차가 발생하지 않습니다. 대신 매월 1회의 유급 '리프레시 데이'를 휴가로 사용할 수 있습니다.

AI 답변

신입사원은 입사 후 처음 3년 동안 법정 연차가 발생하지 않습니다. 대신 매월 1회의 유급 '리프레시 데이'를 휴가로 사용할 수 있습니다. 이는 회사에서 제공하는 특별 휴가로, 사용 시에는 일반 연차와 마찬가지로 업무 일정 조정이 필요합니다.

그림 1-9. `step3_rag.py` 실행 결과. [인사규정] 문서를 찾아서 답변하고 출처까지 보여줍니다

이제 답변과 함께 어느 문서를 참고했는지가 나옵니다. step2에서는 문서를 수동으로 넣어줬지만 이번에는 질문에 맞는 문서를 자동으로 찾아왔습니다. 환각이 사라지고 출처가 생겼습니다.

1.4 청킹, 있을 때와 없을 때: 청킹(Chunking)

옆자리 동료가 화면을 힐끗 보고 물었습니다.

동료 : "근데 왜 문서 3개를 따로따로 넣어요? 하나로 합쳐서 넣으면 더 간단하지 않아요?"

사실 그게 더 편해 보입니다. 인사규정, 보안규정, 복지규정을 한 개의 긴 문자열로 이어붙여 Document 하나로 저장하면 됩니다. `ex01/step4_no_chunking.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다. step3과 달라진 점은 두 가지뿐.

실습 4 ex01/step4_no_chunking.py. 청킹 없이

```
# 1. docs_bad를 벡터DB에 저장 (세 규정을 하나로 이어붙인 거대한 Document)
vectorstore = Chroma.from_documents(documents=docs_bad, embedding=embeddings)

# 2. 검색기 생성 - 통째로 하나뿐이므로 k=1로 검색해도 전체가 다 나옴
retriever = vectorstore.as_retriever(search_kwargs={"k": 1})
```

나머지 흐름(임베딩 생성 → 체인 조립 → 질문 실행)은 step3과 동일합니다.

터미널 실행

```
python step4_no_chunking.py
```

ex01 - python step4_no_chunking.py

문서를 학습(임베딩) 중입니다... (청킹 미적용)

질문 신입사원 휴가 규정에 대해 알려줘.

검색된 문서(근거)

[전체규정] [인사규정] 신입사원 휴가 및 연차 ... [보안규정] 업무 보안 ... [복지규정] 식대 지원 ...

→ 세 규정이 한 덩어리로 섞여서 반환됨

AI 답변

1. 신입사원은 입사 후 3년 동안 법정 연차가 발생하지 않습니다.
2. 대신 매월 1회 유급 '리프레시 데이' 휴가를 사용할 수 있습니다.
3. 리프레시 데이는 유급 휴가로 제공됩니다.

그림 1-10. 청킹 없이 한 덩어리로 넣으면 관련 없는 규정까지 뭉쳐 떨어웁니다

보시는 것처럼, 합쳐서 넣으니 "신입사원 휴가 규정"을 물어봐도 인사규정, 보안규정, 복지규정이 한 덩어리로 달려옵니다. 관련 없는 내용이 섞이면 LLM이 정작 필요한 부분을 놓치기 쉽습니다.

동료 : "아, 그래서 따로따로 잘라서 넣는 거였네요."

문서를 의미 단위로 쪼개는 것을 **청킹(Chunking)** 이라고 합니다. 사서가 책 한 권을 통째로 건네지 않고, 필요한 페이지만 뜯어서 건네는 것과 같습니다. 지금은 "규정별로 하나씩" 정도로만 나눠 봤지만, 실제 사내 문서에서는 더 정교한 전략이 필요합니다. 청킹 전략 상세 비교는 **챕터 8 (검색 품질 튜닝)** 에서 다룹니다.

1.5 복잡한 질문 던져보기: 사슬 추론(Chain-of-Thought)

동료가 커피를 가지러 가며 물었습니다.

동료 : "이제 진짜 질문 한번 던져볼까요. 사람들이 실제로 할 만한 거."

좋은 도전입니다. step3까지는 "규정이 뭐야?" 수준의 단순 검색이었습니다. 이번엔 **규정을 찾아서 읽고 계산까지 해야 하는 질문**을 던져봅니다. `ex01/step5_rag.py` 의 코드 구조는 step3과 동일하고, 달라진 건 파일에 준비된 **질문**뿐입니다. 이 파일을 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 5 ex01/step5_rag.py. 추론 심화

```
question = "입사 6개월차 신입인데 리프레시 데이 2번 썼어. 몇 번 남았는지 규정 기반으로 계산해줘."
```

"매월 1회 제공" → "6개월이면 6번" → "2번 썼으면 4번 남음"까지, 규정을 읽고 계산해야 하는 질문입니다.

터미널 실행

```
python step5_rag.py
```

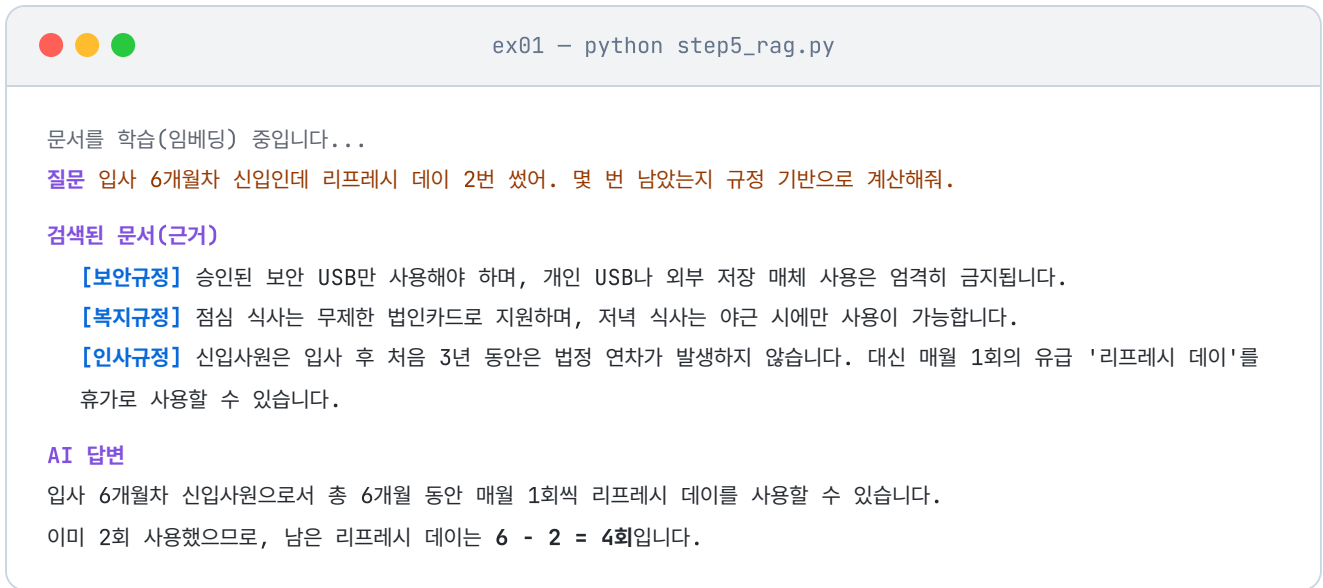


그림 1-11. `step5_rag.py` 실행 결과. 규정을 바탕으로 연차를 스스로 계산하고 추론한 모습

사슬 추론(Chain-of-Thought)이란? LLM이 최종 답을 곧바로 내놓지 않고, 먼저 문제를 작게 쪼개어 하나씩 따져본 뒤 답을 내놓게 하는 방식입니다. 생각의 고리가 사슬처럼 이어진다고 해서 "사슬 추론"이라 부릅니다. 연차 계산처럼 여러 단계가 필요한 질문에서 답의 정확도가 크게 올라갑니다.

실행하면 검색된 문서(근거)와 함께 계산 과정이 담긴 답변이 돌아옵니다.

동료가 커피잔을 들고 돌아와 화면을 들여다봤습니다.

동료 : "오, 이제 진짜 AI 비서 같은데요. 그럼 이거 우리 회사 실제 문서 다 넣으면 바로 되는 거예요?"

웃음이 나왔습니다.

...아니, 여기부터가 시작이지.

더미 데이터 3개로 원리만 보여준 상태입니다. 사내 문서는 PDF, DOCX, 엑셀로 존재하고, 파일 수도 수십 개, 분량도 천 페이지가 넘어갑니다. 이걸 수집하고, 파싱하고, 적절한 크기로 쪼개서 벡터 DB에 저장하는 과정이 필요합니다. 그리고 운영에 올리려면 캐시, 모니터링, 튜닝까지.

용어 정리

이야기에서 사용한 비유가 실제로 어떤 기술 용어에 해당하는지 정리합니다.

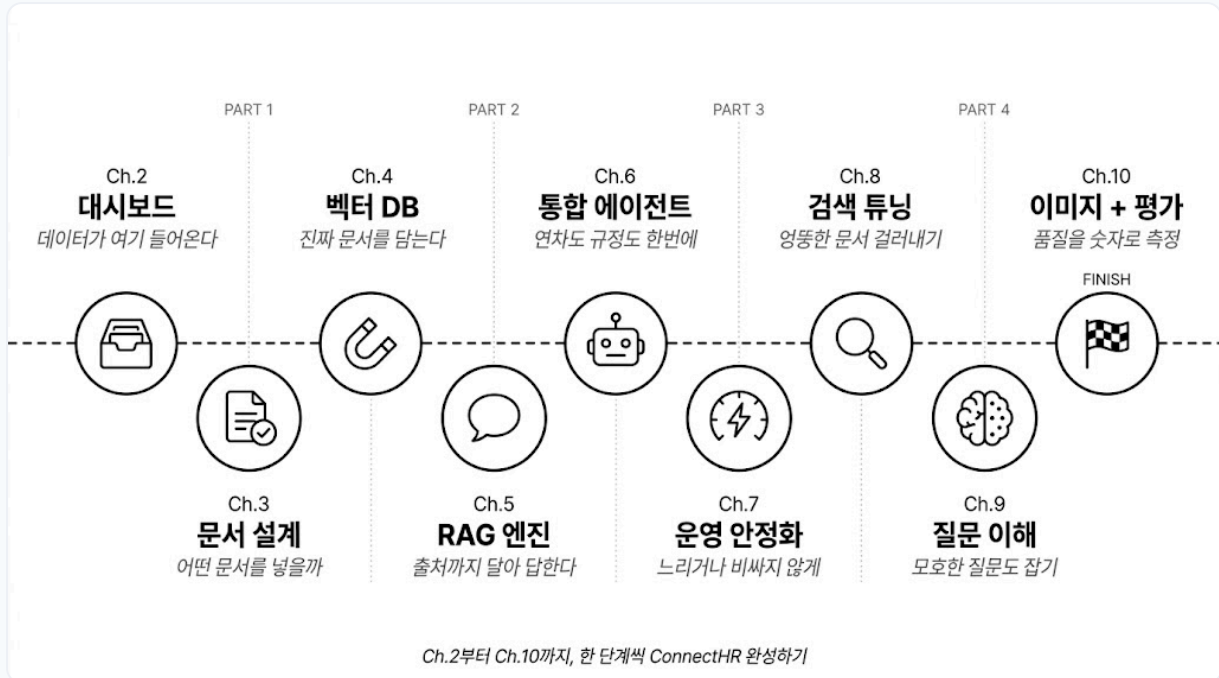
이야기 속 표현	진짜 용어	정식 정의
"자신감 넘치는 거짓말"	LLM 환각 (Hallucination)	LLM이 학습 데이터에 없는 정보를 그럴듯하게 만들어내는 현상
"문서를 직접 붙여 넣기"	Context Injection	관련 정보를 프롬프트에 직접 넣어서 LLM에 제공하는 방법
"오픈북 시험", "사서"	RAG (Retrieval-Augmented Generation)	외부 지식 저장소에서 관련 문서를 검색해 LLM 생성에 활용하는 방식
"서가에 책 꽂기"	임베딩 + 벡터 DB 저장	문서를 수치 벡터로 변환해 ChromaDB에 인덱싱하는 과정
"사서가 책 찾기"	벡터 유사도 검색	질문 벡터와 문서 벡터 간 코사인 유사도를 계산해 가장 관련 있는 문서를 반환
"관련 페이지만 건네기"	청킹 (Chunking)	긴 문서를 의미 단위로 조각내어 검색 정확도를 높이는 기법

이것만은 기억하자

- **LLM은 우리 회사 문서를 읽은 적이 없습니다.** 아무리 자신감 있게 답해도 사내 정보는 우리가 직접 넣어줘야 합니다.
- **RAG는 오픈북 시험입니다.** LLM이 모든 걸 외울 필요 없이 질문마다 관련 문서를 찾아보면서 답합니다.
- **청킹과 추론까지 있어야 답이 단단해집니다.** 문서를 의미 단위로 쪼개고, 규정을 읽고 계산하는 추론을 시키면 그제야 "AI 비서 같다"는 말이 나옵니다.

본격 여정은 여기서부터

챕터 1에서 맞본 RAG 원리를 기반으로, 챕터 2부터 **커넥트HR 에이전트**를 한 단계씩 쌓아갑니다. 4개 파트가 본문의 전체 구조입니다.



앞으로의 여정. 10개 챕터를 4파트로 묶은 전체 로드맵

PART 1 · 사내 시스템 만들기

AI 비서가 조회할 데이터를 먼저 마련합니다. FastAPI로 직원, 연차, 매출 CRUD API를 만들고, 어떤 문서를 어떻게 정리해서 넣을지 설계합니다.

챕터 2 **사내 시스템 API** FastAPI · PostgreSQL · 직원·연차·매출 CRUD

챕터 3 **문서 설계와 메타데이터** 어떤 문서를 어떤 형식으로 넣을지, 재인덱싱 전략까지

PART 2 · RAG 엔진 만들기

챕터 1의 맛보기를 실전 수준으로 끌어올립니다. 진짜 사내 문서를 파싱, 청킹하고 영구 벡터 DB에 저장한 뒤, 출처가 함께 오는 RAG 엔진을 조립합니다.

챕터 4 **벡터 DB 구축** PDF·DOCX 파싱, 한국어 임베딩, ChromaDB 영구 저장

챕터 5 **LCEL 파이프라인** 출처 강제, 멀티턴 대화, 정식 RAG 엔진

PART 3 · 진짜 비서 만들기

문서 검색만으로는 부족합니다. "김대리 연차 며칠 남았어?" 같은 DB 조회 질문까지 자연스럽게 처리하는 에이전트를 만들고, 운영에 올립니다.

챕터 6 통합 에이전트 Tool Calling · ReAct Agent · 사후 분류

챕터 7 운영 안정화 응답 캐시 · 토큰 추적 · 모니터링

PART 4 · 튜닝과 평가

기본 RAG는 출발점일 뿐. 엉뚱한 문서를 가져오거나 질문 의도를 놓치는 문제를 하나씩 고쳐 품질을 끌어올리고, 마지막엔 정량 평가로 마무리합니다.

챕터 8 검색 품질 튜닝 Hybrid Search (BM25+Vector) · Cross-Encoder 리랭킹

챕터 9 질문 이해 강화 HyDE · Multi-Query · 답변 근거 시스템

챕터 10 이미지 처리 + 평가 Vision LLM · OCR · Precision@k · Hallucination Rate

PART 1의 첫 챕터인 챕터 2에서는 AI 비서가 조회할 실제 사내 시스템(직원, 연차, 매출 DB)을 FastAPI로 만들어봅니다.

챕터 2. 일단 사내 시스템부터. 직원, 연차, 매출

이번 챕터가 끝나면

- AI 비서가 조회할 **사내 시스템**(FastAPI + PostgreSQL)이 실제로 돌아가는 걸 눈으로 확인합니다
- **REST API**와 **CRUD** 네 동작(등록, 조회, 수정, 삭제)의 감을 잡습니다
- 앞으로 만들 AI 비서의 정식 이름 **커넥트HR 에이전트**를 갖게 됩니다

준비하기. 실습 시작 전 한 번만 설정

1. 실습 폴더 이동

챕터 1에서 클론한 레포의 `ex02` 폴더로 이동합니다.

터미널 폴더 이동

```
cd rag-start/ex02
```

파일 구조는 다음과 같습니다.

ex02 디렉토리

```

ex02/
├── run.py                # [참고] 서버 실행 진입점
├── docker-compose.yml    # [참고] PostgreSQL 컨테이너
├── requirements.txt       # [참고] 의존성 목록
├── .env.example          # [참고] 환경 변수 템플릿
├── app/
│   ├── main.py          # [참고] FastAPI 진입점
│   ├── api.py            # [참고] REST API 엔드포인트
│   ├── crud.py           # [참고] DB CRUD 로직
│   ├── database.py       # [참고] PostgreSQL 연결
│   ├── models.py         # [참고] SQLAlchemy 모델
│   ├── schemas.py        # [참고] Pydantic 데이터 검증
│   └── views.py          # [참고] 관리자 웹 라우터
├── data/schema.sql       # [참고] 기본 테이블 + 샘플 데이터
├── templates/           # [참고] 관리자 웹 UI HTML
└── static/              # [참고] 관리자 웹 CSS/JS

```

이번 챕터의 모든 파일은 [참고]입니다. 직접 작성하지 않고 실행해서 동작만 확인합니다. AI 비서가 조회할 "사내 시스템"이 어떻게 생겼는지 감을 잡는 것이 목적입니다. FastAPI, PostgreSQL 내부 구현에 집착할 필요가 없습니다.

2. 실습 환경 구축

Docker로 PostgreSQL을 띄우고, Python 가상환경에 의존성을 설치합니다.

터미널 환경 구성. macOS / Linux

```

cd ex02
cp .env.example .env
python3.12 -m venv .venv
source .venv/bin/activate
docker compose up -d
pip install -r requirements.txt

```

터미널 환경 구성. Windows

```
cd ex02
copy .env.example .env
py -3.12 -m venv .venv
.venv\Scripts\activate
docker compose up -d
pip install -r requirements.txt
```

`docker compose up -d` 를 실행하면 PostgreSQL 컨테이너가 시작되면서 `data/schema.sql` 이 자동으로 실행됩니다. 직원 5명, 연차 기록, 매출 데이터가 미리 입력되어 있어 서버를 띄우자마자 바로 조회할 수 있습니다.

Apple Silicon(M1/M2/M3)에서 psycopg2 설치 실패 시

`brew install libpq` 를 먼저 실행한 뒤 `pip install -r requirements.txt` 를 다시 시도하세요. libpq는 PostgreSQL 클라이언트 라이브러리, psycopg2가 내부에서 사용합니다.

3. 사용할 라이브러리

패키지	역할
<code>fastapi</code>	웹 API 서버 프레임워크
<code>uvicorn</code>	ASGI 서버 (FastAPI 구동)
<code>psycopg2-binary</code>	PostgreSQL 드라이버
<code>pydantic</code>	요청/응답 데이터 검증
<code>jinja2</code>	관리자 웹 UI HTML 템플릿
<code>python-dotenv</code>	환경 변수 관리

4. 실습 순서

이번 챕터는 **실행만** 해봅니다. 코드는 읽지 않아도 됩니다.

1. `python run.py` . 서버 시작
2. 브라우저 `http://localhost:8000/docs` . Swagger UI로 API 목록 훑어보기

3. 브라우저 <http://localhost:8000/admin/>. 관리자 웹 UI 확인

2.1 AI가 대답 못 하는 질문: 정형 데이터의 필요

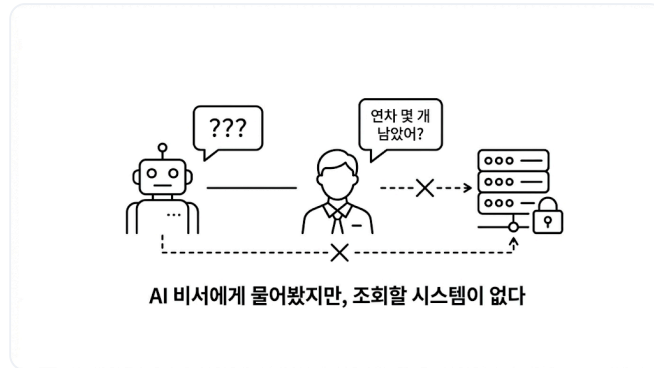


그림 2-1. 문서 검색만으로는 답할 수 없는 질문이 있습니다

월요일 아침 9시 반. 창밖에 비가 줄줄 내렸습니니다.

어제 RAG 맛을 봤다는 흥분이 아직 가시지 않았는데, 팀장이 커피잔을 들고 자리로 다가왔습니다. 잔에서 김이 올라왔고, 키보드 위에 얹힌 손끝이 미지근했습니다.

팀장 : "문서 검색만 되면 안 되죠."

팀장 : "'김대리 연차 며칠 남았어?', '이번 달 개발팀 매출 얼마야?' 이런 것도 답해야죠."

손이 멈췄습니다.

잠깐. 규정집 어딘가에 있을 법한데.

책장에서 **인사규정 v2.3**을 꺼냈습니다. 페이지를 넘겼습니다. "연차는 입사 1년차에 15일..." 규정은 나옵니다. 그런데 **"김대리가 지금 며칠 남았는지"**는 어디에도 없었습니다. 당연한 이야기였습니다. 규정집은 "어떻게 발생하는가"를 적는 곳이지, "누가 얼마 남았는가"를 적는 곳이 아닙니다.

아, 이건 문서 문제가 아니구나.

매출도 마찬가지입니다. 작년 매출 보고서 PDF는 있지만 "이번 달"은 DB에만 있죠. 실시간 숫자는 문서에 없었습니다. 규정집을 덮고 의자에 기댔습니다. 천장 형광등이 한 번 깜빡였습니다.

AI 비서가 진짜 업무를 도우려면 **사내 데이터를 꺼낼 수 있는 시스템**이 먼저 있어야 했습니다. AI에게 "연차 잔여일 알려줘"라고 시킬 대상. 지금까지 그게 없었던 것입니다.

그래서 이번 챕터는 AI 비서보다 **사내 시스템**을 먼저 실행해 봅니다. 코드를 한 줄씩 뜯지는 않습니다. 완성된 시스템을 띄워 "이런 데이터가 이런 모양으로 들어온다"를 눈으로 확인하는 것이 목적입니다.

2.2 직원, 연차, 매출 세 테이블

화이트보드 앞에 나란히 썼습니다. 팀장이 마커를 집어 들더니 왼쪽부터 박스 세 개를 그려 나갑니다. 마커 냄새가 한 번 훅 올라왔습니다.

팀장 : "세 개면 충분해요. 직원, 연차, 매출."

직원(Employee). 사번, 이름, 부서, 직급, 입사일. 예: EMP001 김민준 개발팀 과장 2019-03-02 .

연차(LeaveBalance). 누가, 몇 년도에, 총 며칠이고, 사용한 게 며칠인지. 예: 이서연(EMP002) 2025년 총 15일, 사용 3일, 잔여 12일 .

매출(Sale). 어느 부서가, 언제, 얼마를, 뭘 팔았는지. 예: 개발팀 2025-02-10 12,000,000원 솔루션 개발 C사 .

팀장이 직원과 연차 박스 사이에 화살표를 그었습니다.

팀장 : "직원 한 명에 연차 기록은 해마다 하나씩. 1대N."

매출은 혼자 떨어뜨려 그렸습니다. 부서 단위라 직원과 직접 연결되지 않는다는 뜻이었습니다.

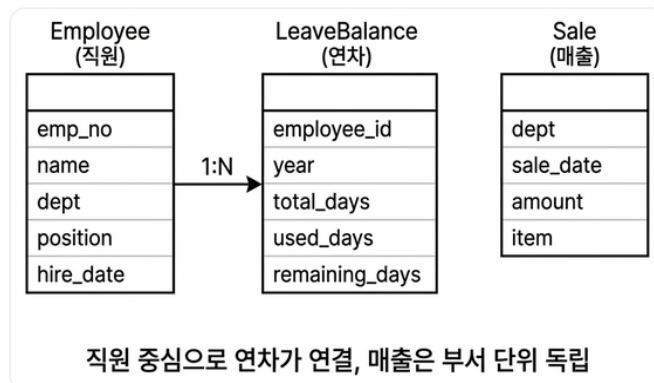


그림 2-2. 세 테이블의 관계. 직원을 중심으로 연차가 연결되고, 매출은 부서 단위로 독립 관리됩니다

세 테이블 모두에 **CRUD**(등록, 조회, 수정, 삭제) API를 제공하는 것이 이번 챕터에서 띄울 시스템입니다. 챕터 6에서 커넥트HR 에이전트가 이 API를 직접 호출하게 되니, 지금은 **"AI가 나중에 쓸 시스템을 먼저 열어두는"** 단계입니다.

2.3 이름이 없으면 부를 수 없다: 커넥트HR 에이전트의 탄생

테이블 구조를 정리한 화이트보드를 들여다보던 팀장이 한마디 던졌습니다.

팀장 : "이 AI 비서, 이름이 뭐예요?"

이름이요? 그냥 'AI 비서'라고 부르고 있었는데....

팀장 : "프로젝트에 이름이 없으면 회의할 때 불편하잖아요. 우리 회사가 **커넥트**니까요. HR 데이터를 다루는 AI 에이전트니까... **커넥트HR 에이전트** 어때요?"

커넥트의 HR 에이전트. 짧고, 무엇을 하는지 바로 전달됩니다.

오픈이 : "좋네요. 커넥트HR 에이전트."

이름이 붙으니 프로젝트가 진짜 시작된 느낌입니다. 지금은 사내 시스템 껍데기뿐이지만, 앞으로 챕터를 거듭하며 이 에이전트가 한 단계씩 자랍니다. 문서를 읽고, 질문에 답하고, DB를 조회하고, 결국 "진짜 비서"에 가까워지는 여정입니다.

이제 실제로 실행해 볼 차례입니다.

2.4 서버를 띄우고 Swagger UI에서 두드려보기

`python run.py` 로 서버를 띄웁니다.

터미널 서버 실행

```
python run.py
```

터미널에 `Uvicorn running on http://0.0.0.0:8000` 이 찍히면 준비 완료입니다. 브라우저에서 `http://localhost:8000/docs` 를 엽니다.



그림 2-3. FastAPI가 자동 생성한 Swagger UI. 직원, 연차, 매출 세 영역의 API가 한눈에 보입니다

Swagger UI란? FastAPI가 Pydantic 스키마에서 API 문서를 자동으로 뽑아 주는 웹 UI입니다. 각 엔드포인트를 브라우저에서 바로 호출할 수 있어, 별도의 테스트 도구 없이 개발 중 API 동작을 확인할 수 있습니다.

이 책에서는 Swagger UI로 하나하나 호출해 보는 실습은 따로 하지 않습니다. "이 시스템엔 이런 API가 있구나" 정도만 눈으로 확인하고 넘어가세요. 챕터 6에서는 사람 대신 커넥트HR 에이전트가 이 API들을 두드리게 됩니다.

2.5 관리자 웹 UI로도 확인하기

Swagger UI는 개발자용입니다. 같은 시스템에 **일반 사용자용 관리자 화면**도 함께 제공됩니다.

브라우저에서 `http://localhost:8000/admin/` 을 엽니다.

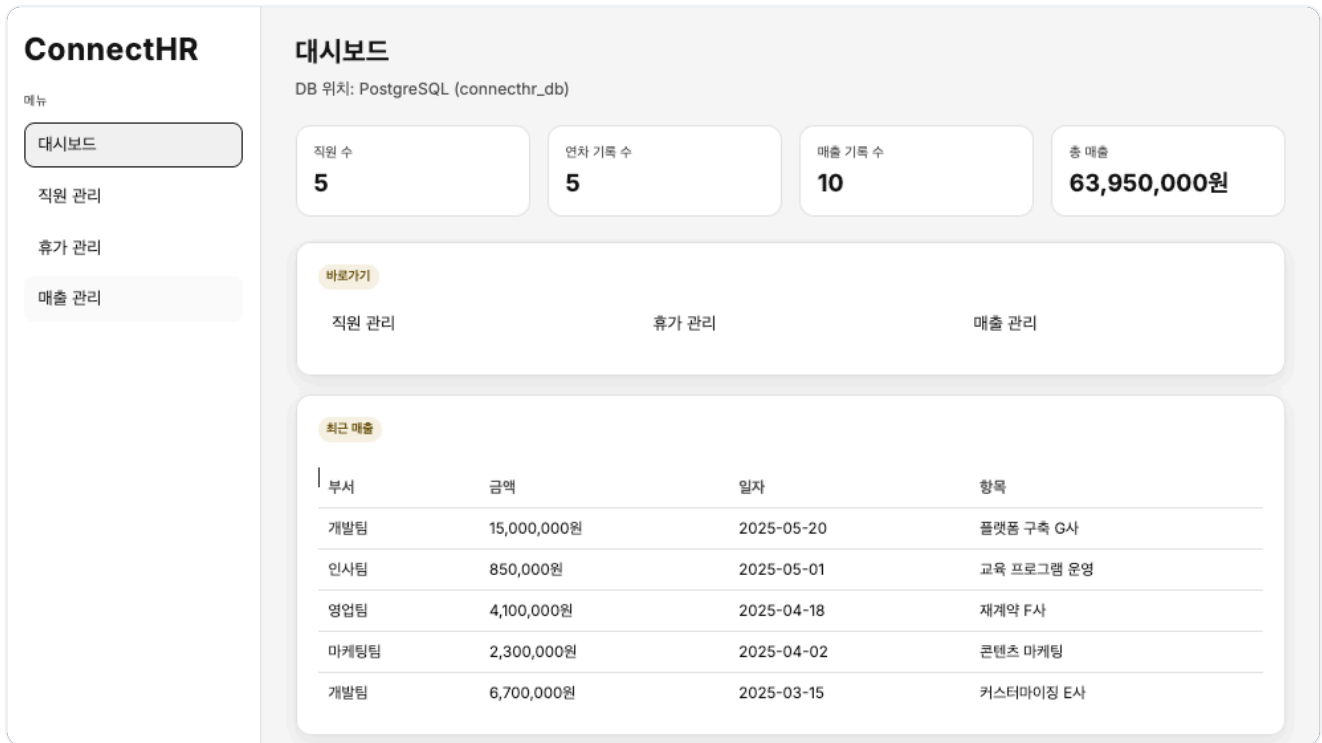


그림 2-4. Jinja2 템플릿으로 만든 관리자 대시보드. 직원, 연차, 매출 현황을 한눈에 보여 줍니다

좌측 **직원 관리** 메뉴에서 이름 "홍길동"과 부서, 직급, 입사일을 입력하고 저장을 누르면 아래 목록에 한 줄이 바로 추가됩니다. 사번은 시스템이 자동으로 부여합니다.

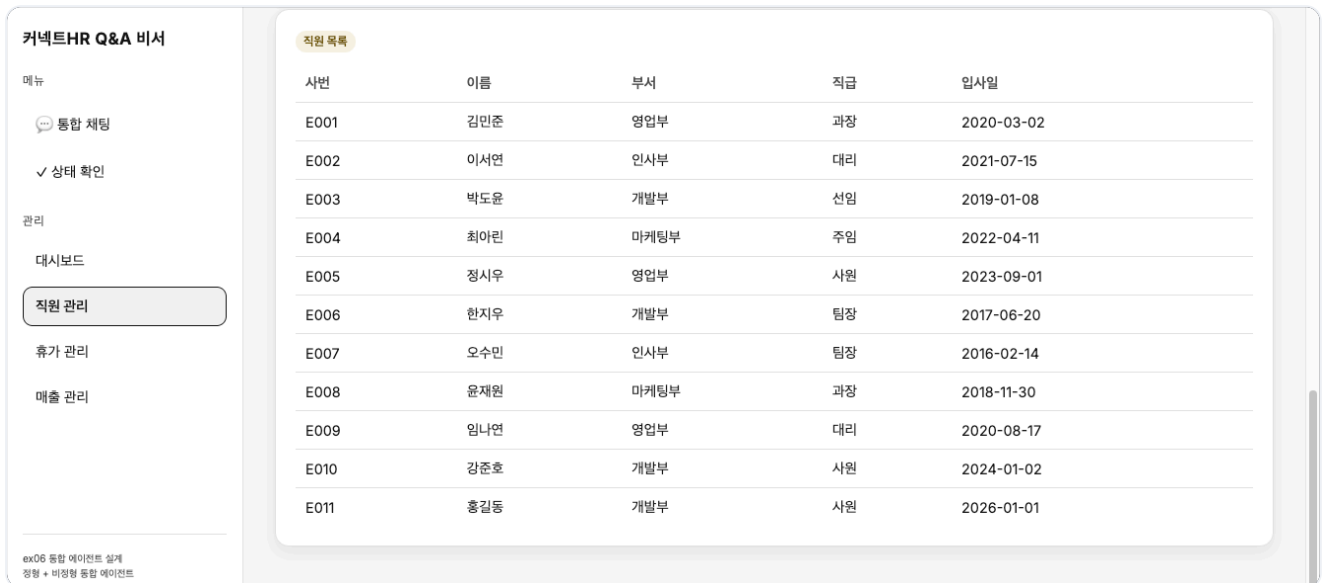


그림 2-5. 직원 관리 목록. 맨 아래에 홍길동이 새 사번으로 추가된 모습입니다. API 호출법을 몰라도 CRUD가 됩니다

지금까지 만든 부분이 전체 시스템에서 어디에 놓이는지 한번 보겠습니다. 아래는 이 책 전체에서 만들 **커넥트HR 에이전트**의 구성도입니다. 진하게 표시된 두 박스가 이번 챕터에서 실행한 부분입니다. 나머지는 다음 챕터들에서 하나씩 채워 갑니다.

CHAPTER 02

시스템 아키텍처

대시보드 + SQL



그림 2-6. 챗터 2가 만든 부분. 순서대로 학습합니다.

종료합니다

`Ctrl + C` 로 서버를 종료하고 `docker compose down` 으로 컨테이너를 내립니다.

2.6 API 엔드포인트 목록

이 시스템이 제공하는 API 전체 목록입니다. 챗터 6에서 커넥트HR 에이전트가 MCP 도구로 이 중 몇 개를 호출하게 됩니다.

메서드	경로	설명
GET	<code>/api/employees</code>	직원 목록 조회 (이름·부서 필터)
POST	<code>/api/employees</code>	직원 등록
GET	<code>/api/employees/{emp_no}</code>	직원 상세 조회
PATCH	<code>/api/employees/{emp_no}</code>	직원 정보 수정
DELETE	<code>/api/employees/{emp_no}</code>	직원 삭제
GET	<code>/api/leaves</code>	연차 목록 조회 (직원·연도 필터)
POST	<code>/api/leaves</code>	연차 등록
POST	<code>/api/leaves/usage</code>	연차 사용 등록
PATCH	<code>/api/leaves/{emp_no}/{year}</code>	연차 수정
GET	<code>/api/sales</code>	매출 목록 조회 (부서·기간 필터)
POST	<code>/api/sales</code>	매출 등록
GET	<code>/api/sales/dept-summary</code>	부서별 매출 합계

`/api/sales/dept-summary`는 부서별 매출 합계를 반환하는 특수 엔드포인트입니다. 챗터 6에서 에이전트의 `sales_sum` 도구가 이 엔드포인트를 호출해 "개발팀 매출 얼마야?" 같은 질문에 답하게 됩니다.

Swagger UI를 마저 훑어보고 있는데 팀장이 모니터를 흘끗 보더니 한마디 던졌습니다.

팀장 : "이제 AI 비서가 조회해볼 곳이 생겼네요."

오프닝에 "문서 검색만 되면 안 되지"라며 판을 뒤집었던 그 사람입니다. 이제 그 판에 올려둘 시스템이 준비됐습니다.

용어 정리

본문 속 표현	진짜 용어	정식 정의
"사내 데이터를 꺼내는 시스템"	REST API	HTTP 메서드 (GET/POST/PATCH/DELETE)로 자원을 조회·생성·수정·삭제하는 인터페이스
"등록·조회·수정·삭제 네 동작"	CRUD	Create·Read·Update·Delete. 데이터 조작의 기본 네 가지
"직원·연차·매출이 들어 있는 저장소"	PostgreSQL	오픈소스 관계형 데이터베이스. 테이블 구조로 저장하고 SQL로 조회
"요청·응답 JSON의 모양을 정한 양식"	Pydantic	Python에서 요청·응답 데이터의 구조와 검증 규칙을 선언하는 라이브러리
"버튼 한 번으로 API를 테스트하는 웹 화면"	Swagger UI	FastAPI가 Pydantic 스키마에서 자동 생성하는 API 문서·테스트 UI
"API는 그대로, 호출자만 바뀐다"	MCP Tools	챗터 6에서 붙일 "AI가 API를 직접 호출"할 수 있게 해 주는 도구 계층

이것만은 기억하자

- 문서로는 답할 수 없는 질문이 있습니다. "김대리 연차 며칠?" 같은 **실시간 정형 데이터**는 규정집이 아니라 DB에서 꺼내야 합니다. RAG만으로는 이 영역을 덮을 수 없습니다.
- **API는 그대로, 호출자만 바뀝니다.** 오늘 만든 FastAPI는 지금은 사람이 웹 UI로 호출하지만, 챗터 6부터는 커넥트HR 에이전트가 **같은 엔드포인트**를 대신 호출합니다.
- **커넥트HR 에이전트의 이름이 정해졌습니다.** 다음 챗터부터는 이 에이전트에게 먹일 사내 문서를 어떤 형식으로 수집하고 정리할지 설계합니다.

챕터 3. 어떤 문서를 넣을까. 문서 표준과 메타데이터

이번 챕터가 끝나면

- 왜 문서 전처리가 RAG 품질을 결정하는지 체감합니다 (문서 품질 = 답변 품질)
- PDF, DOCX, XLSX, HWP 네 형식의 특징과 Markdown 통일 전략을 이해합니다
- 메타데이터, 청킹, 재인덱싱 기본 규칙을 머릿속에 잡아 챕터 4 구현으로 넘어갑니다

이번 챕터는 코드 실습이 없습니다. 챕터 4에서 문서를 실제로 파싱하고 벡터 DB에 저장하기 전에, "어떤 문서를 어떤 규칙으로 넣을 것인가"를 먼저 정리합니다. 규칙 없이 바로 구현하면 검색 품질이 엉망이 됩니다.

3.1 공유 드라이브를 전부 넣었더니

 그림 3-1. 공유 드라이브의 문서 더미. 정리 없이 이걸 전부 넣으면 어떻게 될까

수요일 오후 3시. 사무실에 커피 머신 돌아가는 소리만 낮게 깔려 있었습니다.

챕터 1 더미 문서 3개로 RAG는 이미 잘 돌아왔습니다. 챕터 2 사내 시스템도 켜졌으니 이제 남은 절반. 문서 검색을 본격으로 채워 넣을 차례였습니다.

이건 금방이겠지.

공유 드라이브를 열고 폴더 전체를 드래그. 300여 개 파일을 파이프라인에 그대로 밀어 넣었습니다. "긱어서 넣고, 청킹하고, 임베딩"까지 30분. 커피를 한 잔 새로 내리고 자리로 돌아왔습니다.

다음 날 아침이었습니다. 옆자리 동료가 의자를 돌려 저를 봤습니다.

동료 : "어제 비서한테 연차 물어봤다고 했잖아요."

동료 : "15일이라고 했다면서요. 인사팀 가니까 20일이래요. 혼났어요."

손이 키보드에서 멈췄습니다.

...어?

검색 로그를 열어 봤습니다. 마우스 휠을 굽는 손끝에 식은 커피잔이 닿았습니다. 답변 상단 출처. [인사규정_2022_폐기본.pdf](#). 2년 전 폐기된 문서였습니다. 그 밑에 [복지정책_v1.xlsx](#), [업무가이드_초안.docx](#) 같은 이름이 줄지어 뒀습니다.

이게... 환각보다 나쁘네.

환각은 "모르는데 지어낸" 거라 의심이라도 하지만, 이건 **출처까지 달려 있으니 사용자가 믿어 버립니다.** 동료는 폐기본을 근거로 인사팀에 가서 혼났고, 커넥트HR 에이전트가 그 과정에 공범이 된 셈이었습니다.

업계에서 이런 상황을 한 문장으로 부릅니다. **Garbage In, Garbage Out.**

Garbage In, Garbage Out (GIGO). "쓰레기를 넣으면 쓰레기가 나온다." 입력 데이터의 품질이 그대로 출력 품질을 결정한다는 컴퓨터 과학의 오래된 원칙입니다. 1950년대 메인프레임 시대부터 쓰인 말이고, 그대로 RAG에도 적용됩니다. 폐기본, 초안, 중복본 같은 "쓰레기"가 벡터 DB에 들어가면 아무리 뛰어난 LLM이라도 그 쓰레기를 근거로 답합니다. 튜닝, 프롬프트, 리랭커 같은 고급 기법보다 **앞에 놓이는 가장 기본적인 품질 조건입니다.**

그제야 공유 드라이브를 다시 들여다봤습니다. 마우스 휠이 끝없이 내려갔습니다. [취업규칙.pdf](#), [보안지침_v3_최종_진짜최종.docx](#), [2024년_복지정책.xlsx](#), [회의록_0301.hwp](#), [프로젝트_보고서.pptx](#)...

300개. 이걸 하나하나 다 열어봐야 하나?

형식이 제각각이었습니다. PDF, DOCX, XLSX, HWP에 PPT까지. 최신본 옆에 2년 전 초안이 버젓이 섞여 있었습니다. **정리하지 않고 통째로 넣은 것이 원인**이었습니다.

3.2 사서처럼 정리부터

자리를 지나가던 팀장이 모니터를 흘끗 보더니 한마디 던졌습니다.

팀장: "도서관 가면 책 아무 데나 꽂아요?"

한마디에 머리가 맑아졌습니다. 새 책이 기증되면 사서는 바로 서가에 꽂지 않습니다.

1. **먼저 분류합니다.** 이 책이 어느 분야인지, 대여 가능한지, 최신판인지 확인합니다.

2. **라벨을 붙입니다.** 청구기호(위치), 저자, 출판년도, 키워드를 기록합니다.

3. **서가에 꽂습니다.** 분류에 맞는 위치에 넣습니다.

라벨 없이 마구잡이로 꽂으면 나중에 찾을 수 없습니다. 사내 문서도 마찬가지입니다. 벡터 DB에 넣기 전에 **분류하고 라벨을 붙이고 정리하는 단계**가 필요합니다. 이 단계를 건너뛰면 검색 품질이 엉망이 됩니다.

 그림 3-2. 사내 문서를 벡터 DB에 넣기까지. 정리 없이 넣으면 검색 품질이 떨어집니다

3.3 넣을 문서와 뺄 문서

모든 문서를 넣을 필요는 없습니다. 오히려 불필요한 문서가 섞이면 검색 품질이 떨어집니다. 동료가 당한 것처럼 폐기된 규정이 검색되면 곤란합니다.

넣어야 할 것. 현재 유효한 규정, 정책, 가이드. 자주 질문받는 내용이 담긴 문서.

빼야 할 것. 폐기된 문서, 개인 메모, 초안, 중복 문서(같은 내용의 v1/v2/v3).

간단한 기준 하나면 충분합니다. **"이 문서를 신입사원에게 줘도 되나?"** 된다면 넣고, 아니라면 뺍니다.

3.4 형식 통일: Markdown

PDF, DOCX, XLSX, HWP는 그대로 벡터 DB에 넣을 수 없습니다. 벡터 DB는 **텍스트**만 이해하기 때문입니다.

해외 지사 보고서에 비유해 보겠습니다. 미국 지사는 영어로, 일본 지사는 일본어로, 프랑스 지사는 프랑스어로 문서를 보냈습니다. 우리 팀 누구나 검색하고 읽으려면 먼저 **한국어로 번역**해 하나의 언어로 통일해야 합니다. PDF, DOCX, XLSX도 똑같습니다. 형식이 제각각이면 벡터 DB가 읽지 못합니다. 먼저 **하나의 텍스트 형태로** 바뀌어야 합니다.

이 책에서는 **Markdown**으로 통일합니다. 이유는 네 가지입니다.

- LLM이 가장 잘 이해합니다. 훈련 데이터에 Markdown이 대량 포함되어 있어 **# 제목**, **- 목록** 같은 구조를 자연스럽게 인식합니다.
- 제목이 보존됩니다. PDF의 큰 글씨, DOCX의 "제목 1" 스타일이 **# 제목**으로 바뀝니다.

- 표도 보존됩니다. 엑셀 시트가 | 열1 | 열2 | 형태로 변환됩니다.
- 사람도 읽을 수 있습니다. 변환 결과가 제대로인지 눈으로 바로 검증할 수 있습니다.

 그림 3-3. 파서가 다양한 형식을 Markdown 텍스트로 통일합니다. 그래야 청킹하고 검색할 수 있습니다

반드시 Markdown일 필요는 없습니다

일반 텍스트(plain text)나 JSON으로 변환해도 벡터 DB에 넣을 수 있습니다. 다만 Markdown은 **제목, 표, 목록 같은 문서 구조를 보존하면서도 가볍다**는 점에서 RAG 파이프라인에 가장 널리 쓰입니다.

3.5 메타데이터 태깅

도서관에서 책에 붙이는 정보(청구기호, 저자, 분야, 출판년도)를 문서 세계에서는 **메타데이터**라 부릅니다.

왜 중요할까요? 에이전트에게 "보안 관련 규정 알려줘"라고 물었을 때 메타데이터에 `file_name: SEC_보안규정` 이 있으면 보안 문서를 즉시 식별합니다. 없으면 모든 문서를 처음부터 뒤져야 합니다.

어떤 메타데이터를 붙일지는 프로젝트마다 다릅니다. 우리는 최소한으로 가져갑니다. **파서가 파일명과 경로에서 자동 추출할 수 있는 것만** 사용합니다.

메타데이터	추출 방식	예시
<code>file_name</code>	파일에서 자동	<code>HR_취업규칙_v1.0.pdf</code>
<code>file_type</code>	확장자에서 자동	<code>pdf</code>
<code>source_path</code>	폴더 구조에서 자동	<code>docs/hr/HR_취업규칙_v1.0.pdf</code>
<code>doc_id</code>	파일명에서 자동 생성	<code>hr_취업규칙_v1_0</code>
<code>page</code>	파싱 시 자동	<code>1</code>

사람이 직접 입력하는 항목이 하나도 없습니다. 대신 **파일명에 정보를 담는 것이** 핵심입니다. `HR_취업규칙_v1.0.pdf` 처럼 분류(HR)와 버전(v1.0)을 파일명에 넣으면 파서가 알아서 메타데이터로 만들어 줍니다.

3.6 청킹 전략 사전 설계

챕터 1에서 이미 경험했습니다. 문서를 통째로 넣으면 검색 정확도가 떨어집니다. 적절한 크기로 **조각(chunk)** 을 해야 합니다.

조각 크기를 어떻게 정할까요? 너무 작으면 문맥이 잘립니다. "신입사원은 3년 동안 연차가 없다" 다음 줄 "대신 리프레시 데이를 제공한다"가 있는데 잘못 자르면 앞 문장만 나옵니다. 너무 크면 챕터 1의 청킹 없는 실험처럼 관련 없는 내용이 딸려옵니다.

지금은 설계만 해 두겠습니다. 실제 구현은 챕터 4에서 진행합니다.

전략	크기	오버랩	적합한 경우
고정 크기	500자	100자	규정·매뉴얼 (구조화된 문서)
문단 기준	문단 단위	없음	보고서·회의록 (자연스러운 구분)
의미 기준	가변	없음	긴 문서·주제 전환이 잦은 문서

오버랩이 왜 필요한가

500자마다 딱 잘라 버리면 문장이 청크 경계에서 끊깁니다. 그래서 앞뒤 청크가 일정 구간(여기선 100자) 겹치게 자릅니다. 실제 사내 규정 문장으로 어떻게 잘리는지 보겠습니다.

원본

신입사원은 첫 3년간 연차가 없습니다. 대신 매월 1회 리프레시 데이를 유급으로 제공합니다. 3년 근속 시점에 30일의 연차가 일시에 발생합니다.

↓ 청크 사이즈에 맞춰 자르되, 100자씩 겹치게

청크 1

신입사원은 첫 3년간 연차가 없습니다. **대신 매월 1회 리프레시 데이를 유급으로 제공합니다.**

청크 2

대신 매월 1회 리프레시 데이를 유급으로 제공합니다. 3년 근속 시점에 **30일의 연차가 일시에 발생합니다.**

청크 3

30일의 연차가 일시에 발생합니다. 단, 입사 후 3년이 경과하기 전 ...

노란색이 **오버랩 영역**입니다. 같은 문장이 양쪽 청크에 모두 들어가 있어, 어느 청크가 검색돼도 경계 문장이 살아 있습니다. 오버랩이 0이라면 "리프레시 데이..."가 어느 청크에서도 온전히 나오지 않는 상황이 생깁니다.

이 책에서는 챕터 4에서 **고정 크기(500자 + 100자 오버랩)**으로 시작하고, 챕터 8에서 의미 기준 청킹과 다른 오버랩 값을 비교 실험합니다.

3.7 재인덱싱 전략

사내 문서는 살아 있습니다. 취업규칙이 개정되고, 새 보안지침이 나오고, 복지정책이 바뀝니다. 벡터 DB에 한 번 넣으면 끝이 아닙니다.

재인덱싱을 안 하면 어떻게 될까요? 3.1에서 동료가 당한 일이 반복됩니다. 규정이 바뀌었는데 벡터 DB에는 옛 문서가 그대로 남아 있기 때문입니다.

재인덱싱 방식은 두 가지입니다. 그림으로 한 번 보면 차이가 선명해집니다.



그림 3-4. 별표(*)는 개정된 문서. 전체 재인덱싱은 멀쩡한 네 개까지 지웠다가 다시 넣고, 증분 재인덱싱은 바뀐 하나만 갈아 끼웁니다

이 책에서는 문서 수가 적으므로 **전체 재인덱싱**으로 충분합니다. 문서가 수천 개 이상이면 증분 방식을 고려합니다.

3.8 docs 폴더 구조

정리해 보겠습니다. 커넥트HR 에이전트가 먹을 문서는 다음 구조로 관리합니다.

data/docs/ 구조

```
data/docs/
├── hr/
│   ├── HR_취업규칙_v1.0.pdf
│   └── HR_정보보안서약서.pdf
├── security/
│   └── SEC_보안규정_v1.0.docx
├── finance/
│   ├── FIN_2025_상반기_매출현황.xlsx
│   └── FIN_부서별_예산기안서.xlsx
└── ops/
    └── OPS_신규서비스_런칭전략.pdf
```

별도의 메타데이터 파일은 만들지 않습니다. **폴더 구조와 파일명이 곧 메타데이터**이기 때문입니다.

3.9 더 알아보기

문서 품질 체크리스트. 벡터 DB에 넣기 전에 한 번 더 점검합니다.

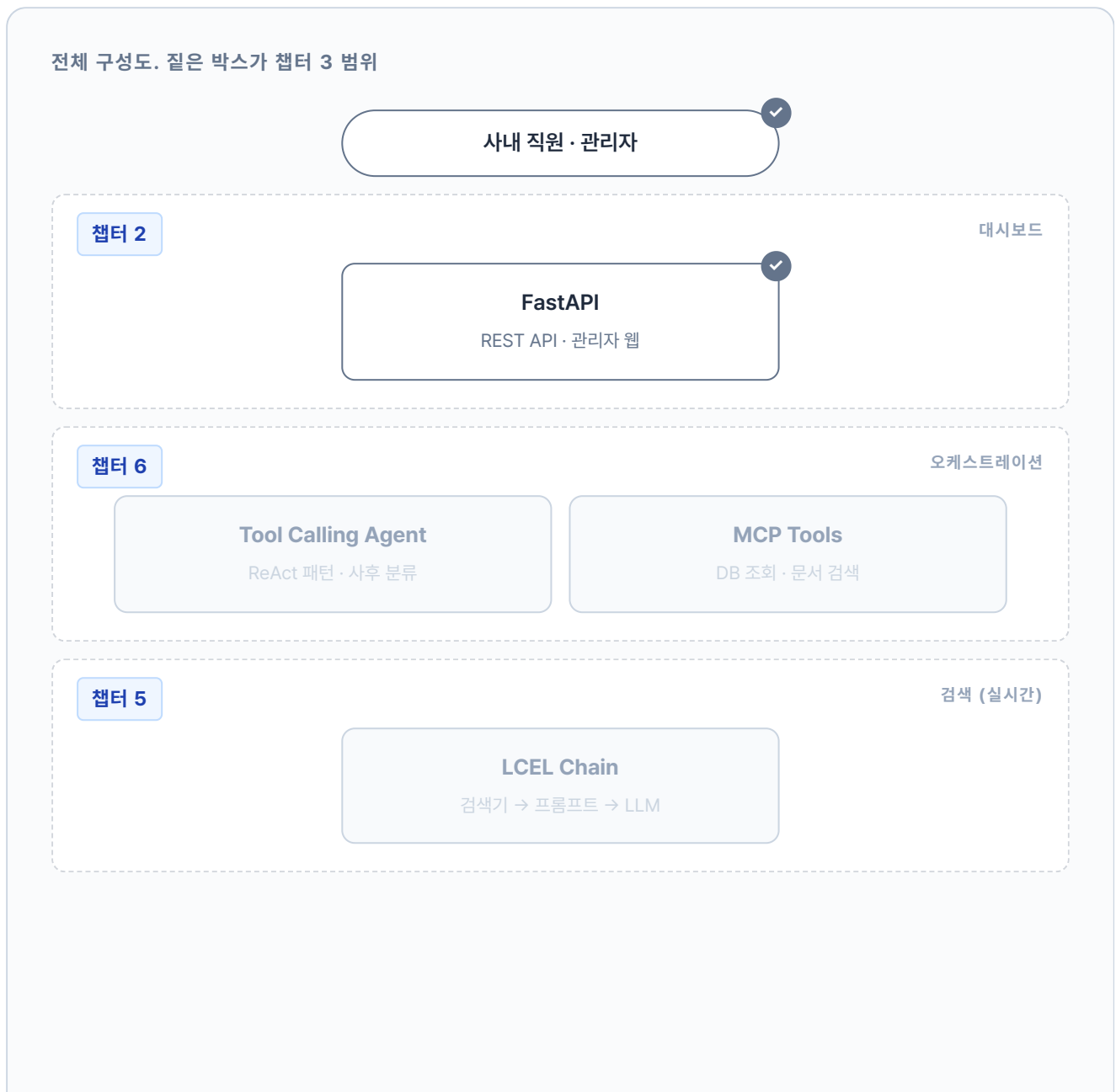
1. 현재 유효한 문서인가? (폐기본, 초안이 아닌가)
2. 중복 문서가 없는가? (같은 내용의 v1/v2/v3)
3. 텍스트 추출이 가능한가? (이미지만 있는 PDF는 챕터 10에서 OCR로 처리)
4. 파일명 규칙을 지켰는가? (**분류_제목_버전.확장자**)

한국어 청킹의 특수성. 영어는 단어 사이에 공백이 있어 토큰 수 기반 청킹이 자연스럽습니다. 한국어는 띄어쓰기 단위가 영어보다 크고 조사가 붙어 같은 500자라도 정보 밀도가 다를 수 있습니다. 챕터 8에서 의미 기반 청킹(Semantic Chunking)과 비교 실험합니다.

메타데이터 필터링. 벡터DB는 저장할 때 메타데이터를 함께 넣어 두고, 검색할 때 `{"file_type": "pdf"}` 처럼 필터를 걸 수 있습니다. 지금은 메타데이터를 출처 표시용으로만 쓰지만, 문서가 많아지면 필터링으로 검색 범위를 좁힐 수 있습니다. 구체적인 필터 문법은 다음 챕터에서 실제로 써 보면서 익힙니다.

3.10 전체 구성도에서 챕터 3의 자리

이번 챕터가 설계한 "문서 규칙"은 챕터 4 파싱, 벡터화 파이프라인의 **입구**에 놓입니다. 다음 챕터에서 이 규칙에 맞춰 정리된 문서를 실제로 파싱하고 청킹해서 벡터DB에 저장합니다.



챕터 4

파싱·벡터화 (오프라인)

오늘 설계한 규칙

챕터 3 문서 규칙

PDF·Word·Excel·HWP

Doc Pipeline

파싱·청킹·임베딩

벡터 DB

벡터 저장소

챕터 2

데이터

PostgreSQL

직원·연차·매출

Ollama LLM (외부)

오늘은 코드 대신 **규칙**을 설계했습니다. 챕터 4에서 이 규칙을 따르는 문서를 실제로 파싱, 청킹해서 **벡터 DB**에 저장합니다.

용어 정리

본문 속 표현	진짜 용어	정식 정의
"같은 말로 번역"	파싱 (Parsing)	다양한 형식(PDF·DOCX·XLSX)에서 텍스트를 추출해 통일된 형태로 변환하는 과정
"도서관 분류 라벨"	메타데이터 (Metadata)	문서 내용이 아닌, 문서에 대한 정보 (파일명·형식·경로 등)
"문서 조각내기"	청킹 (Chunking)	긴 문서를 벡터 DB에 저장할 수 있는 크기로 분할하는 과정
"조각이 겹치는 부분"	오버랩 (Overlap)	청크 경계에서 문맥이 잘리지 않도록 앞뒤를 겹치게 자르는 기법
"문서 다시 넣기"	재인덱싱 (Re-indexing)	변경되거나 추가된 문서를 벡터 DB에 반영하는 과정
"쓰레기 넣으면 쓰레기"	GIGO (Garbage In, Garbage Out)	입력 데이터 품질이 출력 품질을 결정한다는 원칙

이것만은 기억하자

- 좋은 답을 원하면 좋은 문서를 넣어야 합니다. 쓰레기를 넣으면 쓰레기가 나옵니다. 현행 문서만 선별하고 폐기본, 초안, 중복본은 제외합니다.
- PDF, DOCX, XLSX는 먼저 Markdown으로 통일합니다. 형식이 다르면 청킹도 검색도 불가능합니다. 제목, 표 구조가 보존되는 Markdown이 RAG에 가장 잘 맞습니다.
- 폴더 구조와 파일명이 곧 메타데이터입니다. **분류_제목_버전** 규칙만 지키면 파서가 자동으로 메타데이터를 뽑아 냅니다. 다음 챕터에서 이 규칙을 실제 파서로 구현합니다.

챕터 4. 문서를 지식으로 바꾸다. 파싱, 청킹, 임베딩, ChromaDB

이번 챕터가 끝나면

- PDF, DOCX, XLSX를 **Markdown**으로 변환하는 통합 파서를 이해합니다 (Parsing)
- 고정 크기 **500자 + 100자 오버랩** 청킹으로 문맥을 보존한 조각을 만듭니다 (Chunking)
- **ko-sroberta-multitask** 임베딩으로 텍스트를 768차원 벡터로 바꾸고 **ChromaDB**에 저장합니다
- CLI 검색으로 "휴가 규정" 같은 쿼리가 실제로 원하는 문서를 찾아오는 걸 눈으로 확인합니다

준비하기. 실습 시작 전 한 번만 설정

1. 실습 폴더 이동

터미널 폴더 이동

```
cd rag-start/ex04
```

파일 구조는 다음과 같습니다.

ex04 디렉토리

```

ex04/
├── requirements.txt
├── data/
│   ├── docs/                # 사내 문서 원본 (챕터 3 분류 구조)
│   ├── markdown/           # 파싱 결과 (자동 생성)
│   └── chroma_db/           # 벡터 DB 영속 저장소 (자동 생성)
└── src/
    ├── main.py              # [실습] 파이프라인 진입점 (step1, step2 분기)
    ├── chunker.py           # [실습] Fixed-size 청킹 (500자, 100자 오버랩)
    ├── store.py             # [실습] ko-sroberta 임베딩, ChromaDB 저장과 검색
    ├── cli_search.py        # [실습] 벡터 검색 CLI
    ├── extractor.py         # [설명] PDF, DOCX, XLSX 통합 파서
    ├── _pipeline_utils.py   # [참고] argparse, 마크다운 저장 헬퍼
    ├── _chunk_utils.py      # [참고] chunker 내부 헬퍼
    ├── _store_utils.py      # [참고] store 내부 헬퍼
    └── _search_utils.py     # [참고] cli_search 내부 헬퍼
    
```

2. 실습 환경 구축

터미널 환경 구성. macOS / Linux

```

cd ex04
cp .env.example .env
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
    
```

터미널 환경 구성. Windows

```

cd ex04
copy .env.example .env
py -3.12 -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
    
```

임베딩 모델은 첫 실행 시 약 400MB 다운로드

`ko-sroberta-multitask` 는 최초 실행 시 HuggingFace에서 자동 다운로드됩니다.
이후부터는 로컬 캐시를 사용하므로 한 번만 다운받으면 됩니다.

3. 사용할 라이브러리

패키지	역할
<code>pypdf</code>	PDF 텍스트 추출 (페이지별)
<code>python-docx</code>	DOCX 단락·표 추출
<code>openpyxl</code>	XLSX 셀 데이터 추출
<code>sentence-transformers</code>	ko-sroberta 임베딩 모델 로딩·실행
<code>chromadb</code>	벡터 DB (로컬 영속 저장소)

4. 실습 순서

1. `python src/main.py --step 1` . 파싱 결과만 먼저 확인 (Markdown)
2. `data/markdown/` 폴더 열어 변환 결과 눈으로 검증
3. `python src/main.py` . 전체 파이프라인 (파싱 → 청킹 → 임베딩 → 저장)
4. `python src/cli_search.py --query "휴가 규정"` . 검색 동작 확인

4.1 "정리는 했는데, 어떻게 넣지?"

 그림 4-1. 문서를 지식으로 바꾸는 주방. 손질, 다지기, 양념, 냉장고

화요일 오전 10시. 사무실 창으로 햇살이 길게 들어오고, 모니터에는 어제 정리한 폴더 트리가 그대로 떠 있었습니다. 키보드에 얹은 손끝에 가벼운 온기가 돌았습니다.

`data/docs/hr/HR_취업규칙_v1.0.pdf` ... 6개 파일명이 단정하게 줄 맞춰져 있습니다. 옆자리 동료의 의자를 드르륵 돌렸습니다. 어제 인사팀에 한 번 혼나고 온 그 동료입니다.

동료 : "이제 진짜 잘 나오는 거죠?"

잘 나와야지.

챗터 3에서 머릿속에 그려둔 규칙(Markdown 통일, 폴더 메타데이터, 500자에 100자 오버랩)을 이제 **코드로 옮길 차례**입니다. 손질하고, 다지고, 양념하고, 냉장고에 정리하면 6개 문서가 검색 가능한 지식이 됩니다.

4.2 네 단계 파이프라인

팀장 : "재료 손질 안 하고 요리해 본 적 있어요?"

마트에서 사 온 재료를 봉지째 냉장고에 던지면 어떻게 될까요? 나중에 찾을 수 없습니다. 양파가 어디 있는지, 고기가 아직 썰 만한지 다 뒤져야 합니다. 제대로 하려면 네 단계를 거칩니다.

1. **손질**. 흙 씻고, 껍질 벗기고, 뼈 발라냄. 먹을 수 없는 부분 제거.
2. **다지기**. 요리에 맞게 적당한 크기로 자름.
3. **양념**. 소금에 절이거나 밀간. 나중에 바로 쓸 수 있게.
4. **냉장고 정리**. 라벨 붙이고 구분해서 넣음.

사내 문서도 같습니다.

주방	문서 처리	무슨 일
손질	파싱 (Parsing)	PDF·DOCX·XLSX에서 텍스트를 꺼내 Markdown으로 통일
다지기	청킹 (Chunking)	Markdown을 500자 조각으로 자름 (100자 오버랩)
양념	임베딩 (Embedding)	조각을 768차원 숫자 벡터로 변환
냉장고	벡터 DB 저장	벡터를 ChromaDB에 넣어 유사도 검색 가능하게

 그림 4-2. 손질 → 다지기 → 양념 → 냉장고 정리. 이 네 단계가 오프라인 파이프라인의 본체입니다

4.3 손질: 파싱부터 돌려본다

호기심에 `HR_취업규칙_v1.0.pdf` 를 파이썬으로 그냥 `open()` 해봤습니다. 화면에 쏟아진 건 `%PDF-1.6\n%âãĬÓ...` 로 시작하는 뜻 모를 문자열이었습니다.

아, 그냥 열면 안 되는구나.

![(../assets/챕터 4/gemini/04_example_hr_rules.png) 그림 4-3. 취업규칙 PDF 원본. 사람 눈에는 글자지만 컴퓨터에겐 바이너리입니다]

첫 단계는 **손질**입니다. 사람 눈에는 "제1조 제1항"이 보이지만 컴퓨터에겐 PDF 파일이 그냥 바이너리입니다. 텍스트를 꺼내려면 **파서(Parser)** 가 필요합니다. 형식마다 쓰는 도구가 다릅니다.

형식	파서	특징
PDF	<code>pypdf</code>	텍스트 기반 PDF에서 페이지별 추출. 이미지 PDF는 챕터 10에서 OCR·Vision으로 처리
DOCX	<code>python-docx</code>	단락(Paragraph)과 표(Table) 추출, 제목을 Markdown으로 변환
XLSX	<code>openpyxl</code>	시트별 셀 데이터를 행 단위로 읽음

![(../assets/챕터 4/diagram/04_parser-select.png) 그림 4-4. 파일 확장자에 따라 적절한 파서를 자동 분기합니다. 통합 함수 하나로 처리합니다]

`ex04/src/extractor.py` 의 통합 함수가 확장자를 보고 파서를 자동 선택합니다. 이 함수는 이미 구현된 상태이고 우리는 `ex04/src/main.py` 에서 호출만 합니다.

extractor의 전략 패턴. `ex04/src/extractor.py` 의 `extract_text()` 함수는 `{" .pdf": extract_from_pdf, ".docx": extract_from_docx, ".xlsx": extract_from_xlsx}` 같은 딕셔너리 매핑으로 파서를 고릅니다. 새 형식을 추가할 때 딕셔너리에 한 줄만 추가하면 되는 구조입니다.

욕심 같아선 `python src/main.py` 한 번에 전체 파이프라인을 돌리고 싶지만, **파싱 결과부터 따로 확인**하는 게 안전합니다. 챕터 3에서 본 **Garbage In, Garbage Out** 원칙이 여기서 적용됩니다. 입력이 깨지면 뒤에서 아무리 청킹과 임베딩을 잘해도 복구되지 않습니다.

그래서 `ex04/src/main.py` 는 4단계를 두 **Step**으로 묶어 쪼개 뒀습니다.

단계	함수	하는 일	실행 옵션
Step 1	<code>step1_python_parsing()</code>	문서(파싱) → 결과를 <code>data/markdown/</code> 에 저장	<code>--step 1</code>
Step 2	<code>step2_embed_and_store()</code>	저장(칭킹) → 양념 (임베딩) → 냉장고 (ChromaDB 저장)	<code>--step 2</code> (없으면 1, 2 모두)

Step 1을 먼저 돌려 `data/markdown/` 결과물을 눈으로 검증하고, 괜찮으면 Step 2로 넘어갑니다. `--step` 인자를 빼면 전체가 순서대로 실행됩니다. 이 섹션(4.3)에서는 Step 1을, 다음 섹션(4.4)에서 Step 2를 채웁니다. 먼저 `step1_python_parsing` 의 TODO부터.

실습 1 ex04/src/main.py. `step1_python_parsing`

```
def step1_python_parsing(docs_dir: str) → List[dict]:
    # TODO: extract_all_from_directory()로 문서 추출 → 결과 출력 → 마크다운 저장
    # 1. docs_dir(data/docs/) 안의 PDF/DOCX/XLSX를 한꺼번에 파싱
    results = extract_all_from_directory(docs_dir)
    # 2. 파싱 결과를 data/markdown/에 .md 파일로 저장 (눈으로 확인용)
    save_results_as_markdown(results)
    return results
```

`extract_all_from_directory()` 는 `ex04/src/extractor.py` 에 이미 구현돼 있고 `save_results_as_markdown()` 은 `_pipeline_utils.py` 의 헬퍼입니다. 우리가 하는 일은 둘을 순서대로 호출하는 것뿐입니다. `main()` 끝부분에는 `--step` 인자 분기 TODO가 있는데, 여기에 `step1_python_parsing` 을 연결합니다.

실습 1 ex04/src/main.py. `main()` 분기

```
def main() → None:
    args = parse_arguments()
    steps_to_run = sorted(set(args.step))
    python_results: List[dict] = []

    # TODO: 1 in steps_to_run이면 step1_python_parsing 실행
    if 1 in steps_to_run:
        python_results = step1_python_parsing(docs_dir=args.docs_dir)
```


이제 파싱만 돌려봅니다.

터미널 파싱만 실행

```
python src/main.py --step 1
```

`--step 1` 은 파싱과 Markdown 변환만 수행합니다. 끝나면 `data/markdown/` 에 변환 결과가 생깁니다. `HR_취업규칙_v1.0.md` 를 열어 확인합니다.

```
data/markdown/HR_취업규칙_v1.0.md
```

```
# HR_취업규칙_v1.0.pdf
```

- 파일 형식: pdf
- 추출 글자 수: 1916자

```
---
```

```
## 페이지 1
```

```
취업규칙 (다단 편집형) 문서번호: HR-2026-001
```

```
버전: v2.0 (Draft)
```

```
...
```

파일 정보가 상단에 요약돼 있고 `## 페이지 1` 단위로 텍스트가 잘 추출됐습니다.

이번엔 XLSX 결과를 봅니다. `data/markdown/FIN_부서별_예산기안서.md` 를 열면 시트가 **Markdown 표 형식**(`| 열1 | 열2 |`)으로 변환돼 있어요.

data/markdown/FIN_부서별_예산기안서.md

FIN_부서별_예산기안서.xlsx

- 파일 형식: xlsx
- 추출 글자 수: 833자

[시트: 예산기안서_최종]

2025년도	주요사업부	예산	집행	기안서					
---	---	---	---	---	---				
문서번호:	FIN-BDG-2025-001	결재	기안	검토	승인				
기안부서:	재무전략실								
#	본부명	부서명	예산	계정	Q1	배정액(원)	Q2	배정액(원)	
1	경영지원본부	인사총무팀	복리후생비		25000000		30000000		

...

표 모양이 살짝 삐뚤해 보여도 벡터 DB 입장에서는 **원본 엑셀보다 훨씬 이해하기 쉬운** 형태입니다. 헤더 행과 구분선(---)이 있고 파이프(|) 기호로 행, 열 관계가 텍스트 흐름에 담겨 있어, "경영지원본부 예산"을 검색하면 표의 해당 데이터가 정확히 잡힙니다.

4.4 다지기, 양념, 냉장고, 전체 파이프라인

파싱이 깨끗한 걸 확인했으면 나머지 세 단계를 한 번에 돌립니다.

다지기: 청킹

취업규칙 전문을 한 덩어리로 넣으면 챕터 1에서 겪은 바로 그 문제가 반복됩니다. 관련 없는 내용까지 딸려옵니다. 챕터 3에서 설계한 대로 **고정 크기 500자 + 100자 오버랩**으로 자릅니다. 청크가 어떻게 잘리고 어떻게 겹치는지는 챕터 3.6 오버랩 도식에서 이미 눈으로 봤으니, 여기서는 구현만 짚습니다.

ex04/src/chunker.py 의 split_text_into_chunks() 함수 안에 TODO가 하나 있어요.

실습 2 ex04/src/chunker.py. split_text_into_chunks

```
def split_text_into_chunks(
    text: str,
    chunk_size: int = DEFAULT_CHUNK_SIZE,
    overlap: int = DEFAULT_OVERLAP,
) → List[str]:
    if chunk_size ≤ overlap:
        raise ValueError(
            f"chunk_size({chunk_size})는 overlap({overlap})보다 커야 합니다."
        )

    text = text.strip()
    if not text:
        return []

    # TODO: chunk_size 단위로 텍스트를 자르되, overlap만큼 겹치게 합니다
    chunks = []                                # 결과를 담은 리스트
    step = chunk_size - overlap                 # 다음 청크 시작 위치 (500-100=400자씩 이동)
    start = 0                                  # 현재 위치

    while start < len(text):                    # 텍스트 끝까지 반복
        end = start + chunk_size                 # 현재 위치에서 500자 잘라내기
        chunk = text[start:end].strip()         # 앞뒤 공백 제거
        if chunk:                                # 빈 문자열이 아니면
            chunks.append(chunk)                 # 결과에 추가
            start += step                        # 400자 뒤로 이동 (100자 겹침)

    return chunks
```

DEFAULT_CHUNK_SIZE (500), DEFAULT_OVERLAP (100)은 chunker.py 상단에 선언된 모듈 상수입니다. main.py 에서 --chunk-size 로 넘겨 덮어쓸 수 있습니다.

step = chunk_size - overlap 이 핵심입니다. 500자를 자르지만 다음 청크는 400자 뒤에서 시작하므로 결과적으로 **100자가 겹칩니다**. while 루프가 텍스트 끝까지 반복하며 청크 리스트를 쌓아 올립니다.

각 조각에는 **메타데이터**도 함께 붙습니다. 출처 파일명, 페이지, 분류. 챕터 3에서 설계한 라벨이 여기서 쓰이고, 나중에 에이전트가 "이 답변의 출처는 취업규칙 3페이지입니다"라고 당당하게 말할 수 있게 됩니다.

청크 한 조각 = 본문 + 메타데이터 라벨

제5조(연차유급휴가) 신입사원은 입사 후 3년 동안은 연차가 없다. 대신 매월 1회 '리프레시 데이'를 유급으로 제공한다. 3년 근속 시점에 30일의 연차가 일시에 발생한다. ...

출처 HR_취업규칙_v1.0.pdf

페이지 3

분류 HR

chunk_id hr_취업규칙_v1_0_text_p003_c0002

이 라벨들이 ChromaDB에 `metadata` 필드로 같이 저장됩니다. 검색할 때 "HR 폴더의 3페이지 근처 문서만"처럼 필터링도 가능하고, 답변에 출처를 붙일 때도 이 값을 그대로 씁니다.

양념: 임베딩

컴퓨터는 글자를 모릅니다. 숫자만 연산합니다. **임베딩(Embedding)**은 텍스트의 의미를 숫자 벡터(768개 숫자 리스트)로 바꾸는 과정입니다. 핵심은 **의미가 비슷한 텍스트는 비슷한 숫자**가 된다는 점입니다.



그림 4-5. 왼쪽(문장 임베딩 벡터 예시. 앞 몇 자리만 봐도 비슷/다름이 드러남), 오른쪽(임베딩 공간에 찍힌 좌표. 의미가 가까운 문서가 한 곳에 모임)

"숫자가 비슷하다"는 걸 어떻게 판단할까요? 벡터를 좌표 위의 **화살표**라고 생각하시면 됩니다. 두 화살표가 같은 방향이면 의미가 비슷하고, 반대면 다릅니다. **코사인 유사도(Cosine Similarity)**가 이 각도를 측정합니다.

- 같은 방향(각도 0°) → 유사도 **1** (완전히 같은 의미)
- 직각(90°) → 유사도 **0** (관련 없음)
- 반대 방향(180°) → 유사도 **-1** (반대 의미)

 그림 4-6. 코사인 유사도. 각도로 의미 유사도를 측정합니다

임베딩 모델은 여럿 있지만 이 책은 **ko-sroberta-multitask**를 씁니다. 한국어 문장 유사도에 파인튜닝돼 있고("연차"와 "휴가"가 가깝다는 걸 잘 잡습니다), 로컬 실행이라 API 비용이 없고, 사내 민감 정보를 외부로 보내지 않아도 됩니다.

챕터 8에서 다른 모델과 비교

챕터 8에서 ko-sroberta와 다른 임베딩 모델의 검색 품질을 비교 실험합니다. 지금은 기본값으로 시작하고, 더 나은 선택지가 있는지는 챕터 8에서 확인합니다.

냉장고: ChromaDB 저장

양념까지 끝난 재료를 **ChromaDB**에 정리합니다. 챕터 1에서 `Chroma.from_documents()` 한 줄로 썼지만, 이번에는 `ex04/src/store.py` 가 직접 넣습니다.

ChromaDB란? 텍스트 의미를 담은 **벡터**(768차원 숫자 배열)를 저장하고, 주어진 질문 벡터와 가장 가까운 벡터를 빠르게 찾아 주는 **벡터 데이터베이스**입니다. 일반 관계형 DB(PostgreSQL, MySQL)가 "이름이 홍길동인 행"처럼 정확한 값을 조회한다면, 벡터 DB는 "이 질문과 의미가 비슷한 문서 Top-K"를 조회합니다. 로컬에 파일로 영속 저장이 가능해 서버 재시작 후에도 데이터가 남아 있고, Python 한 줄로 띄울 수 있을 정도로 가볍습니다. 실무에서는 Pinecone, Weaviate, Milvus 같은 유료/대규모용 옵션도 쓰지만, 이 책처럼 로컬 실습, 소규모 운영에는 ChromaDB가 표준에 가깝습니다.

저장 항목	내용	예시
<code>id</code>	청크 고유 ID	<code>hr_취업규칙_v1_0_text_p001_c0003</code>
<code>document</code>	청크 텍스트 원문	"제5조 연차유급휴가는..."
<code>embedding</code>	768차원 벡터	<code>[0.12, -0.34, ...]</code>
<code>metadata</code>	출처 정보	<code>{file_name: "HR_취업규칙_v1.0.pdf", page: 3}</code>

저장은 **upsert**. 같은 ID가 있으면 덮어쓰고 없으면 새로 추가합니다. 파이프라인을 여러 번 실행해도 중복이 쌓이지 않습니다. 챕터 3에서 설계한 **전체 재인덱싱** 전략과 정확히 맞닿습니다.

`ex04/src/store.py` 에는 `_store_utils.py` 에서 import해 쓰는 **모듈 상수 3개**가 함수 시그니처 기본값으로 등장합니다. CLI 옵션으로 덮어쓸 수 있습니다.

상수	기본값	역할
<code>DEFAULT_CHROMA_DIR</code>	<code>./data/chroma_db</code>	ChromaDB가 벡터를 저장할 로컬 디렉토리. 서버가 재시작돼도 데이터 유지
<code>DEFAULT_COLLECTION_NAME</code>	<code>metacoding_documents</code>	컬렉션 이름(일반 DB의 "테이블" 같은 개념). 프로젝트별로 다르게 쓰면 여러 벡터DB를 한 디렉토리에 공존 가능
<code>DEFAULT_EMBEDDING_MODEL</code>	<code>jhgan/ko-sroberta-multitask</code>	텍스트를 768차원 벡터로 변환할 임베딩 모델(HuggingFace ID). 검색 시에도 같은 모델 을 써야 의미가 맞음

`store_chunks_to_chroma()` 함수 안에 TODO가 있습니다. 4단계를 순서대로 구현합니다.

실습 3 ex04/src/store.py. store_chunks_to_chroma

```
def store_chunks_to_chroma(
    chunks: list[dict],
    chroma_dir: str = DEFAULT_CHROMA_DIR,
    collection_name: str = DEFAULT_COLLECTION_NAME,
    embedding_model_name: str = DEFAULT_EMBEDDING_MODEL,
) → dict:
    if not chunks:
        raise ValueError("저장할 청크가 없습니다.")
    chroma_dir = str(Path(chroma_dir).resolve())

    # TODO: 위 4단계를 순서대로 구현합니다

    # 1. ko-sroberta 임베딩 모델 로드 (최초 실행 시 ~400MB 다운로드, 이후 캐시)
    model = load_embedding_model(embedding_model_name)

    # 2. ChromaDB 클라이언트 생성 - 디스크에 영속 저장 (프로그램 종료해도 유지)
    client = chromadb.PersistentClient(
        path=chroma_dir,
        settings=Settings(anonymized_telemetry=False),
    )
    # 컬렉션이 없으면 생성, 있으면 기존 컬렉션 반환. cosine 유사도 사용
    collection = get_or_create_collection(client, collection_name)

    # 3. 청크 텍스트를 768차원 벡터로 변환 + ChromaDB용 데이터 정리
    ids, documents, embeddings, metadatas = embed_chunks(chunks, model)

    # 4. 배치 단위로 upsert - 같은 ID가 있으면 덮어쓰기 (중복 방지)
    for batch_start in range(0, len(ids), BATCH_SIZE):
        batch_end = batch_start + BATCH_SIZE
        collection.upsert(
            ids=ids[batch_start:batch_end],
            documents=documents[batch_start:batch_end],
            embeddings=embeddings[batch_start:batch_end],
            metadatas=metadatas[batch_start:batch_end],
        )

    return {
        "collection_name": collection_name,
        "chroma_dir": chroma_dir,
        "total_chunks": len(chunks),
        "collection_count": collection.count(),
    }
```

`_store_utils.py`의 헬퍼 함수 세 개가 코드에 쓰였습니다. 직접 구현하진 않지만 역할은 알아 두세요.

함수	역할
<code>load_embedding_model(name)</code>	HuggingFace에서 <code>ko-sroberta-multitask</code> 모델을 로드해 <code>SentenceTransformer</code> 객체를 반환. 최초 1회만 다운로드, 이후 캐시
<code>get_or_create_collection(client, name)</code>	ChromaDB 컬렉션을 가져오거나 없으면 생성. 유사도 계산 방식을 cosine 으로 지정
<code>embed_chunks(chunks, model)</code>	청크 리스트를 <code>(ids, documents, embeddings, metadatas)</code> 네 리스트로 분해. 문서 텍스트를 배치 단위로 768차원 벡터로 변환

우리가 할 일은 이 세 함수를 **순서대로 호출**하는 것뿐입니다.

2단계에서 `chromadb.PersistentClient(path=chroma_dir)` 가 **벡터DB를 여는** 부분입니다. 챕터 1에서는 `Chroma.from_documents()` 한 줄이 메모리에 임시 저장소를 만들고 프로그램이 끝나면 사라졌어요. 사내 실서비스에선 매번 문서를 다시 임베딩하는 건 낭비라서, 이번엔 **디스크에 파일로 영속 저장**합니다. `path` 에 적은 폴더(`data/chroma_db/`)에 벡터가 파일로 쌓이고, 프로그램을 껐다 켜도 그대로 남아 있습니다. `Settings(anonymized_telemetry=False)` 는 ChromaDB가 기본으로 켜 둔 익명 사용 통계 수집을 끄는 옵션. 사내 문서를 다루는 실습에선 꺼 두는 편이 안전합니다.

`chromadb.Client` vs `chromadb.PersistentClient` . 같은 ChromaDB지만 여는 방식이 다릅니다. `Client()` 는 프로세스 메모리에만 존재하는 임시 저장소(프로그램 종료 시 휘발)고, `PersistentClient(path=...)` 는 지정 디렉토리에 SQLite + Parquet 파일로 저장해 **재시작 후에도 유지**됩니다. 대규모 운영에선 Pinecone, Weaviate처럼 서버를 별도로 띄우지만, 로컬 실습이나 소규모 사내 시스템에는 `PersistentClient` 만으로 충분합니다.

3단계에서 `embed_chunks()` 가 청크를 768차원 벡터로 일괄 변환하고, 4단계 `collection.upsert()` 가 이를 `BATCH_SIZE` (기본값 64)씩 끊어 저장합니다. `upsert` 는 같은 ID가 있으면 덮어쓰는 연산이라, 파이프라인을 여러 번 돌려도 중복이 쌓이지 않습니다. 메모리가 부족하면 `BATCH_SIZE` 를 32로 줄이고, 여유가 있고 임베딩이 느리면 128로 늘릴 수 있습니다.

실습: main.py로 한 번에 돌리기

`ex04/src/main.py` 의 `step2_embed_and_store` 함수에 청킹, 임베딩, 저장을 조합하는 TODO가 있습니다. 앞에서 파싱이 돌려둔 결과(`python_results`)를 받아 두 함수를 순서대로 호출합니다.

실습 4 ex04/src/main.py. step2_embed_and_store

```
def step2_embed_and_store(
    python_results: list[dict],
    chroma_dir: str,
    collection_name: str,
    embedding_model_name: str,
    chunk_size: int = DEFAULT_CHUNK_SIZE,
    overlap: int = DEFAULT_OVERLAP,
) → dict:
    # TODO: chunk_all_documents()로 청킹 → store_chunks_to_chroma()로 저장
    # 1. 파싱 결과를 500자 + 100자 오버랩으로 청킹
    all_chunks = chunk_all_documents(python_results, chunk_size, overlap)
    # 2. 청크를 임베딩하여 ChromaDB에 저장
    store_result = store_chunks_to_chroma(
        chunks=all_chunks,
        chroma_dir=chroma_dir,
        collection_name=collection_name,
        embedding_model_name=embedding_model_name,
    )
    return store_result
```

마지막으로 `main()` 의 `--step 2` 분기 TODO를 채워 step2와 연결합니다. (`--step 2` 만 골라 돌리면 파싱이 없으므로 내부에서 step1을 다시 부릅니다.)

실습 4 ex04/src/main.py. main() 의 step2 분기

```
# TODO: 2 in steps_to_run이면 step2_embed_and_store 실행
if 2 in steps_to_run:
    if not python_results:
        python_results = step1_python_parsing(docs_dir=args.docs_dir)
    step2_embed_and_store(
        python_results=python_results,
        chroma_dir=args.chroma_dir,
        collection_name=args.collection,
        embedding_model_name=args.embedding_model,
        chunk_size=args.chunk_size,
        overlap=args.overlap,
    )
```

이제 전체 파이프라인을 한 번에 돌립니다.

터미널 전체 파이프라인 실행

```
python src/main.py
# 청크 크기를 바꾸고 싶다면
python src/main.py --chunk-size 300 --overlap 50
```

명령 한 줄로 파싱부터 저장까지 어떤 순서로 흘러가는지 한눈에 보면 다음과 같습니다.

PYTHON SRC/MAIN.PY 실행 흐름

\$ python src/main.py



main()

`parse_arguments()` → `steps_to_run = [1, 2]`



1 step1_python_parsing **파싱**

`extract_all_from_directory(docs_dir)`
 data/docs/ 안 6개 파일(PDF-DOCX-XLSX) 파싱 → list[dict]
`save_results_as_markdown(results)`
 data/markdown/*.md 저장 (눈으로 확인 가능)

↓ python_results 전달

2 step2_embed_and_store **청킹·임베딩·저장**

`chunk_all_documents(python_results, 500, 100)`
 500자 + 100자 오버랩 → 약 18개 청크
`store_chunks_to_chroma(chunks, ...)`
 ① `load_embedding_model()` → ko-sroberta 로드
 ② `PersistentClient(path=data/chroma_db/)` → DB 열기
 ③ `embed_chunks(chunks, model)` → 768차원 벡터 변환
 ④ `collection.upsert(...)` → 배치 단위로 저장



완료

data/markdown/ 6개 md 파일 + data/chroma_db/ 청크 18개가 벡터로 저장

그림 4-7 (왼쪽). Step 1 — 6개 사내 문서가 Markdown으로 변환

그림 4-7 (오른쪽). Step 2 — 18개 청크가 ChromaDB에 저장

4.5 검색 확인: "휴가 규정"을 찾아오는가

파이프라인이 돌았으면 검색이 되는지 확인합니다. 검색 로직은 `ex04/src/store.py` 의 `search_chroma()` 함수에 있고, TODO가 한 군데 있어요.

실습 5 ex04/src/store.py. search_chroma

```
def search_chroma(
    query: str,
    chroma_dir: str = DEFAULT_CHROMA_DIR,
    collection_name: str = DEFAULT_COLLECTION_NAME,
    embedding_model_name: str = DEFAULT_EMBEDDING_MODEL,
    top_k: int = 5,
) → List[dict]:
    chroma_dir_path = Path(chroma_dir)
    if not chroma_dir_path.exists():
        print(f"ChromaDB 디렉토리가 없습니다: {chroma_dir}")
        print("main.py를 먼저 실행하여 문서를 색인하십시오.")
        sys.exit(1)

    # TODO: 쿼리 임베딩 → collection.query() → 결과 정리

    # 1. 질문 텍스트를 벡터로 변환 (저장할 때와 같은 모델 사용)
    model = load_embedding_model(embedding_model_name)
    query_embedding = model.encode([query], normalize_embeddings=True).tolist()

    # 2. 저장된 ChromaDB를 열고 컬렉션 가져오기
    client = chromadb.PersistentClient(
        path=str(chroma_dir_path.resolve()),
        settings=Settings(anonymized_telemetry=False),
    )
    collection = client.get_collection(name=collection_name)

    # 3. 쿼리 벡터와 가장 가까운 청크 top_k개 검색
    results = collection.query(
        query_embeddings=query_embedding,
        n_results=top_k,
        include=["documents", "distances", "metadatas"],
    )

    # 4. 검색 결과를 순위별 딕셔너리 리스트로 정리
    search_results = []
    docs = results.get("documents", [[]])[0]
    dists = results.get("distances", [[]])[0]
    metas = results.get("metadatas", [[]])[0]

    for rank, (doc, dist, meta) in enumerate(zip(docs, dists, metas), start=1):
        search_results.append({
            "rank": rank, "text": doc,
            "distance": round(dist, 4), "metadata": meta,
        })

    return search_results
```

저장과 똑같은 임베딩 모델을 써야 합니다. 질문 벡터와 저장된 청크 벡터가 같은 공간에서 비교되어야 거리가 의미를 갖기 때문입니다. 검색 결과는 `{rank, text, distance, metadata}` 형태의 리스트로 정리돼 돌아옵니다.

`cli_search.py`는 `_search_utils.py`의 `format_distance_as_similarity()` 헬퍼를 써서 ChromaDB 거리(0~2)를 백분율 유사도(0~100%)로 바꿔 출력합니다. 거리가 0에 가까울수록 유사도는 100%에 가까워집니다.

터미널 CLI 검색

```
python src/cli_search.py --query "휴가 규정" --top-k 3
```

그림 4-8 (왼쪽). 쿼리와 상위 1, 2위 검색 결과

그림 4-8 (오른쪽). 3위 결과. 취업규칙의 휴가, 연차 관련 조항이 잡혔습니다

1~3위가 모두 취업규칙 휴가 관련 내용입니다. 키워드가 정확히 일치하지 않아도 의미가 가까우면 찾아냅니다.

옆자리 동료가 화면을 슬쩍 들여다봤습니다.

동료: "이제 인사팀 안 가도 되겠네요."

웃음이 한 번 새어 나왔습니다. 그러나 화면을 다시 보니 1위 유사도가 **71%**였습니다. `--top-k 5`로 늘려 보니 4, 5위에 결이 다른 청크가 섞여 있었어요.

아직 완벽하지는 않다.

벡터 DB는 "그나마 가까운 순서"로 k개를 **무조건** 채웁니다. 그래서 검색 개수를 늘리면 관련 없는 문서가 억지로 달려와요. 검색 품질을 본격적으로 끌어올리는 작업은 챕터 8에서 다룹니다. 지금은 "휴가 규정"에 취업규칙이 1위로 잡히는 것까지가 이번 챕터의 도착선입니다.

검색 결과는 PC, 모델 버전에 따라 조금 다를 수 있습니다

임베딩 모델 버전이나 실행 환경에 따라 순서, 유사도 수치가 책과 달라질 수 있습니다. 최상위(Top 1~3)에 취업규칙의 휴가, 연차 관련 내용이 잡히면 정상입니다.

4.6 더 알아보기

왜 Markdown으로 변환할까요? 임베딩 모델은 훈련 데이터에 Markdown이 많이 포함되어 있어 `# 제목`, `| 표 |`, `- 목록` 같은 구조를 잘 이해합니다. PDF 바이너리를 그대로 넣는 것보다 Markdown 텍스트가 검색 품질이 좋습니다. 그리고 `data/markdown/`의 결과물을 **사람이 눈으로 검증**할 수 있다는 이점도 있습니다.

이미지 기반 PDF의 한계. 예제 파일 중 `HR_정보보안서약서.pdf`는 스캔한 이미지로만 구성되어 있어 `pypdf`가 텍스트를 뽑지 못합니다. `extractor.py`의 PDF 파서는 이런 파일에서 빈 문자열을 돌려주고, 결국 청크가 0개로 나옵니다. 이미지 기반 PDF는 OCR이나 Vision LLM이 필요한데, 이 문제는 **챕터 10**에서 해결합니다.

extractor.py의 전략 패턴. 파일 확장자에 따라 파서를 자동으로 고르는 구조는 딕셔너리 매핑 한 줄로 끝납니다.

설명 ex04/src/extractor.py. 전략 패턴 (발췌)

```
extractor_map = {
    ".pdf": extract_from_pdf,
    ".docx": extract_from_docx,
    ".xlsx": extract_from_xlsx,
}
return extractor_map[suffix](file_path)
```

새 형식(예: HWP)을 추가하려면 이 딕셔너리에 한 줄 추가하고 해당 파서 함수만 구현하면 됩니다.

`if/elif`로 분기하는 것보다 확장이 쉽고, 파서끼리 결합도도 낮습니다.

청킹 · 임베딩 옵션 한눈에. 자주 손보는 값들입니다.

상수	기본값	언제 바꾸나
<code>chunk_size</code>	500자	규정처럼 짧은 조항은 300자, 긴 보고서는 800자까지 실험
<code>overlap</code>	100자	<code>chunk_size</code> 의 20% 전후가 무난. 경계에서 잘려도 되는 문서면 0으로
<code>BATCH_SIZE</code>	64	메모리 부족하면 32, 여유 있으면 128
<code>top_k</code> (검색)	5	정확도만 본다면 3, 리랭커 입력 후보군 만들려면 10+

4.7 전체 구성도에서 챗터 4의 자리

오프라인 파이프라인의 본체를 완성했습니다. 챗터 3 규칙 → Doc Pipeline → ChromaDB. 이제 이 저장소를 챗터 5에서 실시간 검색 엔진으로 연결합니다.



챕터 5

검색 (실시간)

LCEL Chain

검색기 → 프롬프트 → LLM

챕터 4

파싱·벡터화 (오프라인)

챕터 3 문서 규칙

PDF·Word·Excel·HWP

오늘 만든 부분

Doc Pipeline

파싱·청킹·임베딩

오늘 채운 벡터 DB

ChromaDB

벡터 저장소

챕터 2

데이터

PostgreSQL

직원·연차·매출

Ollama LLM (외부)

오늘 띄운 건 **Doc Pipeline + ChromaDB** 두 박스입니다. 벡터 DB에 18개 청크가 들어갔고, CLI로 검색이 됩니다.
다음 챕터 5에서는 이 저장소 위에 **LCEL Chain**을 얹어 "검색 → 프롬프트 → LLM" 실시간 RAG 엔진을 조립합니다.

용어 정리

본문 속 표현	진짜 용어	정식 정의
"손질"	파싱 (Parsing)	PDF·DOCX·XLSX에서 순수 텍스트를 추출하는 과정
"다지기"	청킹 (Chunking)	긴 텍스트를 벡터 DB에 적합한 크기로 분할. 고정 500자 + 오버랩 100자
"양념"	임베딩 (Embedding)	텍스트 의미를 768차원 숫자 벡터로 변환
"냉장고"	벡터 DB (Vector Database)	벡터를 저장하고 유사한 벡터를 빠르게 검색하는 특수 DB
"의미 유사도"	코사인 유사도 (Cosine Similarity)	두 벡터 사이 각도로 유사도를 측정. 1에 가까울수록 유사
"덮어쓰기"	업서트 (Upsert)	같은 ID가 있으면 업데이트, 없으면 삽입하는 연산

이것만은 기억하자

- 문서 → 지식은 네 단계를 거칩니다. 파싱(손질), 청킹(다지기), 임베딩(양념), ChromaDB 저장(냉장고). 어느 한 단계라도 소홀하면 검색이 엉망이 됩니다.
- 파싱 결과를 먼저 눈으로 확인하세요. 챗터 3의 **Garbage In, Garbage Out** 원칙이 여기서 적용됩니다. 텍스트가 깨져 있으면 뒤에서 아무리 고쳐도 소용없습니다. `data/markdown/`의 결과를 반드시 검증합니다.
- 벡터 검색은 키워드 매칭이 아닙니다. "휴가 규정"이라고 물어도 "연차 유급휴가"가 잡힙니다. 의미가 가까우면 찾아냅니다. 다음 챗터 5에서는 이 검색 결과를 **LCEL 체인**에 붙여 실제 답변을 만들어 냅니다.

챕터 5. 사서가 답해준다. LCEL 파이프라인

이번 챕터가 끝나면

- **LCEL**(LangChain Expression Language) 파이프 연산자로 Retriever → Prompt → LLM → Parser를 조립합니다
- **출처 강제(Source Grounding)** 프롬프트로 환각을 줄이고 답변에 출처를 항상 붙입니다
- **WindowMemory(k=5)** 로 최근 5턴 멀티턴 대화를 유지합니다
- 채팅 UI에서 커넥트HR 에이전트의 전신인 RAG Q&A 엔진과 실제로 대화해 봅니다

준비하기. 실습 시작 전 한 번만 설정

1. 실습 폴더 이동

터미널 폴더 이동

```
cd rag-start/ex05
```

파일 구조는 다음과 같습니다.

ex05 디렉토리

```

ex05/
├── run.py                # [참고] 서버 실행 진입점
├── .env                  # [참고] 환경 변수
├── data/
│   ├── docs/            # [참고] 원본 문서 (ex04와 동일 샘플)
│   ├── markdown/        # [참고] 파싱 결과
│   └── chroma_db/        # [참고] 벡터 DB (첫 실행 시 자동 생성)
├── src/
│   ├── rag_chain.py      # [실습] LCEL 파이프라인 + 출처 강제
│   ├── conversation.py   # [실습] WindowMemory(k=5)
│   ├── llm_factory.py    # [참고] Ollama/OpenAI 분기
│   ├── vectorstore.py    # [참고] Retriever 생성
│   ├── session_manager.py # [참고] 세션별 Memory + TTL
│   └── response_parser.py # [설명] <think> 제거 + 출처 추출
├── templates/           # [참고] 채팅 UI HTML
├── static/               # [참고] 채팅 UI CSS/JS
└── app/
    ├── main.py           # [참고] FastAPI 진입점
    ├── chat_api.py       # [설명] POST /api/chat
    └── session.py         # [참고] 세션 쿠키
  
```

2. 실습 환경 구축

터미널 환경 구성. macOS / Linux

```

cd ex05
cp .env.example .env
python3.12 -m venv .venv
source .venv/bin/activate
ollama pull deepseek-r1:8b
pip install -r requirements.txt
  
```

터미널 환경 구성. Windows

```

cd ex05
copy .env.example .env
py -3.12 -m venv .venv
.venv\Scripts\activate
ollama pull deepseek-r1:8b
pip install -r requirements.txt
  
```

.env 핵심 상수입니다. 코드를 보기 전에 역할부터 파악해 두면 흐름이 잘 읽힙니다.

상수	기본값	역할
LLM_PROVIDER	ollama	사용할 LLM 제공자 (ollama 또는 openai)
OLLAMA_MODEL	deepseek-r1:8b	Ollama에서 쓸 모델 이름
EMBEDDING_MODEL	jhgan/ko-sroberta-multitask	문서 임베딩 모델. 챕터 4와 같은 값이어야 검색이 맞음
CHROMA_PERSIST_DIR	./data/chroma_db	벡터 DB 저장 경로
RETRIEVER_TOP_K	5	질문당 검색해서 가져올 관련 문서 조각 개수
CONVERSATION_WINDOW_SIZE	5	프롬프트에 포함할 최근 대화 턴 수

3. 사용할 라이브러리

패키지	역할
langchain	체인 조립 프레임워크
langchain-community	HuggingFaceEmbeddings 등 커뮤니티 통합
langchain-ollama	Ollama LLM 연결
langchain-chroma	ChromaDB Retriever 래퍼
fastapi · uvicorn	웹 API + ASGI 서버

4. 실행 순서

1. `src/rag_chain.py` . LCEL 파이프라인 + 출처 강제 프롬프트
2. `src/conversation.py` . WindowMemory(k=5)
3. `python run.py` . 서버 실행 후 브라우저 `http://localhost:8000/chat` 에서 대화

5.1 "그냥 답을 알려줘"

 그림 5-1. 사서가 답해준다. 책을 찾아서 읽고, 출처와 함께 답해 줍니다

수요일 점심 직후, 사무실 라디에이터가 한 번 울컥 소리를 내고 다시 잠잠해졌습니다.

챕터 4에서 벡터 검색까지 돌렸고, 동료에게 CLI 사용법을 알려 주었습니다. 점심 먹고 돌아오니 그 동료가 또 의자를 돌려 제 쪽을 봤어요.

동료 : "병가 쓸 때 증빙 서류 필요해요?"

동료 : "연차 며칠 남았는지 어떻게 확인해요?"

CLI가 켜진 터미널을 그대로 보여 주었습니다.

벡터 검색 결과

1 HR_취업규칙_v1.0 (p.3)

유사도 87.2%

"제15조(병가) 질병이나 부상으로 인하여 직무를 수행할 수 없을 때에는..."

2 HR_취업규칙_v1.0 (p.4)

유사도 81.5%

"병가 기간이 3일 이상인 경우에는 의사의 진단서를..."

동료 : "이걸 제가 읽어요? 그냥 답을 알려주세요."

맞다. 사람들은 문서 조각을 원하는 게 아니라 **답변**을 원한다.

벡터 검색은 **재료를 찾아 주는 일**이지 **요리를 해 주는 일**이 아닙니다. 이번 챕터에서는 그 "요리"를 해 주는 **RAG Q&A 엔진**을 만듭니다. 질문하면 문서를 검색하고, 읽어서, 자연어로 답해 주는 시스템입니다.

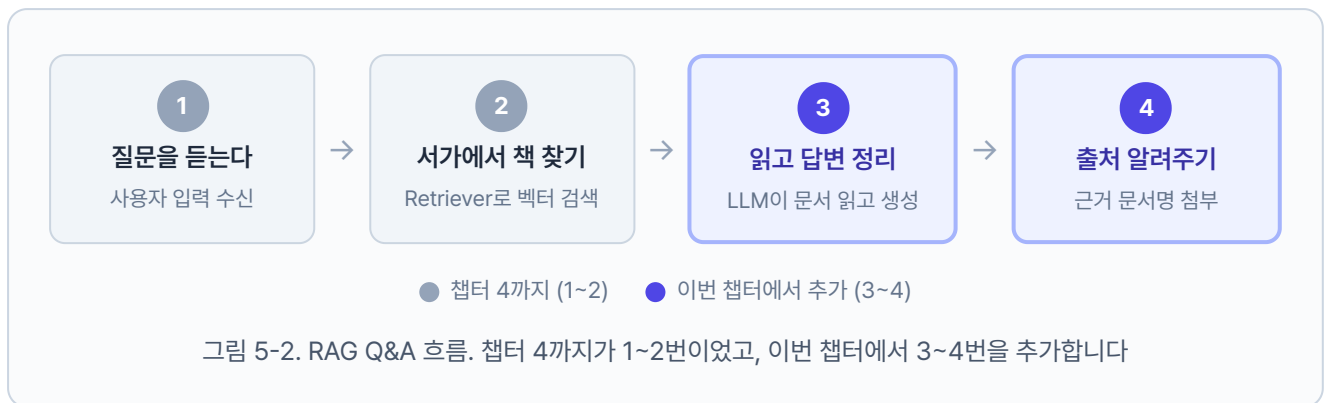
5.2 사서가 일하는 네 단계

지금까지 만든 벡터 검색은 **도서관의 검색 시스템**과 비슷합니다. "한국 역사"를 검색하면 관련 책이 어디 있는지 알려주지만, 책을 꺼내서 읽고 정리해 주지는 않습니다.

우리에게 필요한 건 **사서**입니다. 사서에게 "조선시대 과거 제도가 뭐야?"라고 물으면 네 단계가 일어납니다.

1. 질문을 듣는다. "조선 과거 제도"가 핵심이군
2. 서가에서 책을 찾는다. 한국사 개론 3장 근처를 꺼냄
3. 읽고 답변을 정리한다. "문과, 무과, 잡과 세 종류가 있었습니다"
4. 출처를 알려준다. "한국사 개론 3장에 나와 있어요"

이 네 단계가 바로 RAG Q&A 파이프라인입니다. 챗터 4가 2번까지였다면, 이번 챗터가 **3, 4번을** 추가합니다.



5.3 LCEL 조립과 출처 강제(rag_chain.py)

검색 → 프롬프트 → LLM → 파싱. 각 단계를 직접 이어 붙여도 되지만, **LangChain**이 이걸 부품화해 줍니다. 조립 방식은 **파이프 연산자(|)**. 이를 **LCEL(LangChain Expression Language)** 이라 부릅니다.



파이프 연산자(`|`) 는 앞 블록의 출력을 뒤 블록의 입력으로 흘려보내는 역할을 합니다. 유닉스 셸의 `ls | grep` 파이프와 같은 발상입니다. 덕분에 Retriever, Prompt, LLM, Parser 어느 블록이든 **같은 인터페이스만 지키면 자리 바꿔 끼울 수 있습니다**. LLM을 DeepSeek에서 GPT-4o로 갈아 끼울 때도 체인 구조는 건드리지 않고 LLM 블록만 교체하면 됩니다. 챗터 8~10에서 튜닝할 때 이 구조의 힘이 드러납니다.

출처 강제(Source Grounding)

챗터 1에서 맞본 **그라운드링**의 강한 버전입니다. 그때는 "참고 정보에서만 답하세요" 한 줄이었지만, 실서비스에서는 그걸로 부족합니다. LLM이 여전히 "문서에서 찾을 수 없으면" 외부 지식을 슬쩍 섞어 지어냅니다.

이번 챗터는 규칙을 **네 개로 늘립니다**. (1) 제공된 문서에서만, (2) 없으면 "확인되지 않습니다", (3) 답변 끝에 `[출처: 파일명]`, (4) 추측 금지. `ex05/src/rag_chain.py` 상단에 상수 `RAG_SYSTEM_PROMPT` 로 박혀 있습니다.

실습(rag_chain.py)

설명 ex05/src/rag_chain.py. 출처 강제 프롬프트 (발췌)

```
RAG_SYSTEM_PROMPT = """당신은 커넥트HR 사내 문서 Q&A 비서입니다.
아래에 제공된 문서(Context)만 사용하여 질문에 답변하십시오.
```

규칙:

1. 반드시 제공된 문서에서만 근거를 찾아 답변하십시오.
2. 문서에서 답을 찾을 수 없으면 "해당 내용은 제공된 문서에서 확인되지 않습니다."라고 답하십시오.
3. 답변 마지막에 근거 문서명을 반드시 명시하십시오. 형식: `[출처: 문서명]`
4. 추측이나 외부 지식을 사용하지 마시오.

Context (제공된 문서):

```
{context}
```

이전 대화:

```
{history}
```

```
"""
```

`{context}` 에는 검색된 문서가, `{history}` 에는 이전 대화가 자동으로 채워집니다. 규칙 2번이 **환각 방지**(모르면 모른다고 답하라), 규칙 3번이 **출처 명시**를 담당합니다. 같은 파일에 `RAG_HUMAN_PROMPT = "질문: {question}"` 상수도 함께 선언돼 있어, 뒤에서 `("human", RAG_HUMAN_PROMPT)` 자리에 그대로 끼워 넣습니다. `_format_docs()` 와 `build_rag_chain()` 두 함수에 TODO가 있습니다. TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 1 ex05/src/rag_chain.py. _format_docs

```
def _format_docs(docs):
    # TODO: 검색된 Document를 "[문서 N] 출처: ..." 텍스트로 변환
    # 1. 검색된 Document 리스트를 프롬프트에 넣을 텍스트로 변환
    parts = []
    for i, doc in enumerate(docs, start=1):
        source = doc.metadata.get("source", "알 수 없음")
        page = doc.metadata.get("page", "-")
        parts.append(f"[문서 {i}] 출처: {source} (p.{page})\n{doc.page_content}")
    return "\n\n".join(parts)
```

각 문서에 [출처: 파일명 (p.N)] 라벨을 붙여 한 문자열로 합칩니다. 이 결과가 프롬프트 변수 {context} 로 들어갑니다. 이어서 본체 조립. 챕터 1에서 RetrievalQA가 내부에서 알아서 해주던 검색·프롬프트·LLM 호출을, 이번에는 Retriever부터 하나씩 꺼내서 파이프로 직접 조립합니다.

실습 1 ex05/src/rag_chain.py. build_rag_chain

```
def build_rag_chain():
    # TODO: build_llm()으로 LLM 생성 ~ (chain, retriever) 튜플 반환
    # 1. LLM 인스턴스 생성 (llm_factory.py에서 Ollama/OpenAI 선택)
    llm = build_llm()
    # 2. ChromaDB에서 문서를 검색하는 Retriever 생성
    retriever = build_retriever()

    # 3. 시스템 프롬프트 + 사용자 프롬프트 조립
    prompt = ChatPromptTemplate.from_messages([
        ("system", RAG_SYSTEM_PROMPT),
        ("human", RAG_HUMAN_PROMPT),
    ])

    # 4. LCEL 파이프로 체인 조립. 질문→검색→포맷→프롬프트→LLM→텍스트
    chain = (
        {
            "context": itemgetter("question") | retriever | _format_docs,
            "history": itemgetter("history"),
            "question": itemgetter("question"),
        }
        | prompt
        | llm
        | StrOutputParser()
    )

    # 5. 체인과 검색기를 함께 반환
    return chain, retriever
```


파이프 연산자(`|`)가 각 단계의 출력을 다음 단계의 입력으로 넘깁니다. `itemgetter("question")`은 입력 dict에서 `question` 키만 꺼내는 호출가능 객체입니다. 등장한 LangChain 부품 세 개를 한 줄씩 정리하면 아래와 같습니다.

부품	역할
<code>ChatPromptTemplate.from_messages([...])</code>	<code>system</code> / <code>human</code> 메시지로 프롬프트 조립. <code>{context}</code> , <code>{question}</code> 같은 변수는 체인 실행 시 자동으로 채워짐
<code>build_llm()</code>	<code>.env</code> 의 <code>LLM_PROVIDER</code> 에 따라 <code>ChatOllama</code> 또는 <code>ChatOpenAI</code> 인스턴스를 만들어 줌 (<code>llm_factory.py</code>)
<code>StrOutputParser()</code>	LLM이 돌려주는 <code>AIMessage</code> 객체에서 <code>.content</code> 문자열만 꺼내는 최종 파서

왜 chain과 retriever를 같이 반환할까요? 체인은 최종 답변 텍스트만 돌려주지만, 출처 표시용으로 검색된 원본 Document도 따로 필요합니다. `ex05/app/chat_api.py`가 `retriever.invoke(question)`을 한 번 더 호출해 출처 목록을 만들어 응답 JSON에 붙입니다.

5.4 WindowMemory. conversation.py

도서관 방문자가 사서에게 이어서 묻습니다.

방문자 : "한국 역사 관련 책 어디 있어요?"

사서 : "2층 인문학 서가에 있습니다."

방문자 : "거기 세계사도 있어요?"

여기서 "거기"는 **2층 인문학 서가**입니다. 사람 사서는 당연히 기억하지만 LLM은 기본적으로 **기억력이 없습니다**. 매 요청이 독립적이라 이전 질문을 모릅니다. 그래서 **대화 히스토리**를 직접 관리해야 합니다.

모든 대화를 다 기억할 수는 없으니 **최근 N턴만 유지**하는 슬라이딩 윈도우 방식을 씁니다. 사서가 메모장을 들고 있다고 생각하시면 됩니다. 새 메모가 들어오면 가장 오래된 메모를 지우고 최근 5장만 남깁니다.

 그림 5-4. 6번째 메모가 들어오면 가장 오래된 1번째가 빠집니다. 항상 `maxlen = 5`

Python에서는 `collections.deque(maxlen=5)` 를 쓰면 이 구조가 자동으로 구현됩니다.

실습(conversation.py)

`ex05/src/conversation.py` 는 `deque(maxlen=k)` 위에 얹은 래퍼를 씌워 "이전 대화"를 프롬프트에 넣을 형태로 바꿉니다. `save_turn`, `get_history`, `clear` 세 메서드에 TODO가 있습니다. TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 2 ex05/src/conversation.py. WindowMemory

```
class WindowMemory:
    def __init__(self, k=5, human_prefix="사용자", ai_prefix="AI 비서"):
        self.k = k
        self.human_prefix = human_prefix
        self.ai_prefix = ai_prefix
        self._turns = deque(maxlen=k)

    def save_turn(self, question, answer):
        # TODO: (question, answer) 튜플을 self._turns에 추가
        # 1. 질문-답변 1턴을 메모장에 저장
        self._turns.append((question, answer))

    def get_history(self):
        # TODO: self._turns의 (question, answer) 쌍을 순회하며
        # 1. 저장된 대화를 "사용자: 질문 / AI: 답변" 텍스트로 변환
        lines = []
        for question, answer in self._turns:
            lines.append(f"{self.human_prefix}: {question}")
            lines.append(f"{self.ai_prefix}: {answer}")
        return "\n".join(lines)

    def clear(self):
        # TODO: self._turns 초기화
        # 1. 메모장 비우기
        self._turns.clear()
```

`deque(maxlen=5)` 가 자동으로 오래된 턴을 제거해 주므로 우리가 할 일은 "append → 텍스트 변환 → 비우기" 셋뿐입니다. `ex05/src/session_manager.py` 가 세션별로 이 `WindowMemory` 를 하나씩 보관하고, 일정 시간 동안 호출이 없으면 TTL로 만료시킵니다.

5.5 서버 실행 + 채팅 UI

두 파일의 TODO를 모두 채웠으면 서버를 띄워 브라우저에서 직접 대화해 봅니다.

터미널 서버 실행

```
python run.py
```

터미널에 `Uvicorn running on http://0.0.0.0:8000` 이 찍히면 준비 완료입니다. 브라우저에서 `http://localhost:8000/chat` 을 열면 채팅 UI가 나타납니다.

 그림 5-5. 커넥트HR 에이전트의 전신. RAG Q&A 채팅 UI

"병가 쓸 때 증빙 서류 필요해요?"라고 물어보면 사내 취업규칙을 근거로 답이 돌아오고, 답변 끝에 출처 파일명이 붙습니다.

 그림 5-6. 답변이 자연어로 돌아오고, 출처까지 명시됩니다

이어서 "그럼 며칠 이상일 때 진단서가 필요해?"라고 물어보면 **이전 대화의 "병가"를 기억**해서 맥락을 이어 답합니다. WindowMemory가 동작한 결과입니다.

 그림 5-7. 멀티턴 대화. 이전 질문을 기억해 "거기", "그럼"을 해석합니다

옆자리에서 동료가 화면을 넘겨다봤습니다.

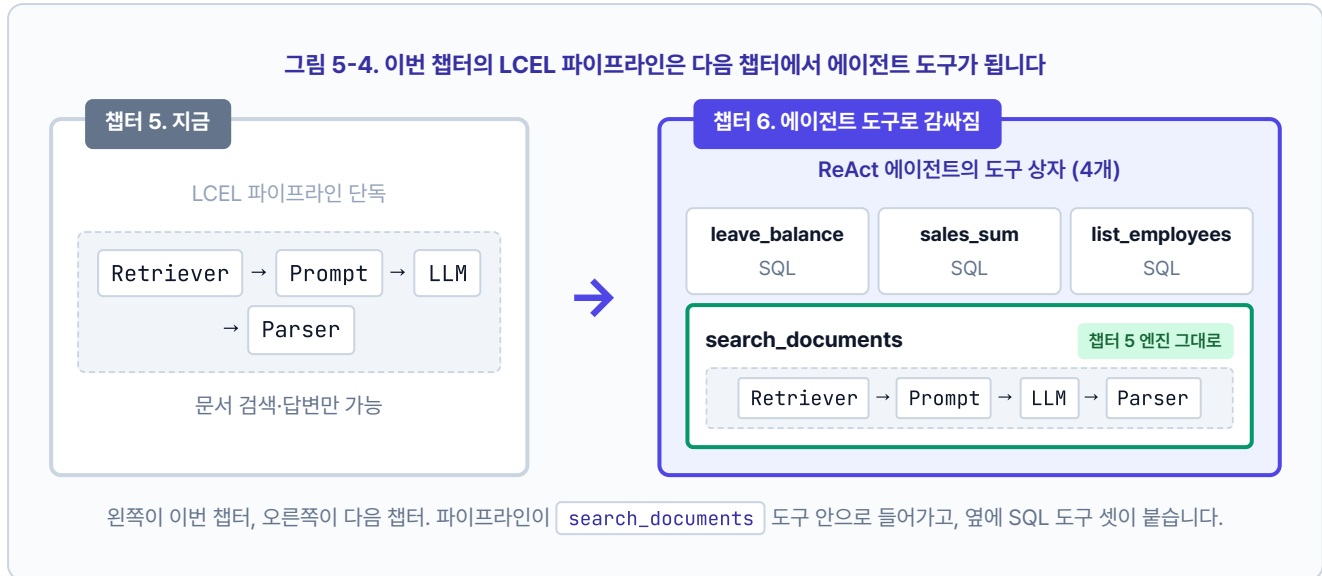
동료 : "아, 이제 답을 주네요. 아까는 문서 조각만 던지더니."

오프닝에 "이걸 내가 읽어?" 하던 바로 그 사람입니다. 이번엔 읽을 것도, 해석할 것도 없이 답이 나왔습니다.

처음으로 **사용자가 쓸 수 있는 형태**의 비서가 생겼습니다. 벡터 검색 결과가 자연어 답변으로 바뀌고, 출처가 붙고, 이어지는 질문의 맥락까지 이해합니다. 다만 "김대리 연차 며칠 남았어?" 같은 **실시간 DB 조회 질문**은 아직 못 풅니다. 챕터 6에서 문서 질문과 DB 질문을 한 에이전트가 동시에 처리하도록 **Tool Calling + ReAct 패턴**을 엮습니다.

이 파이프라인, 다음 챕터에서 어떻게 되는가

지금 만든 LCEL 파이프라인은 여기서 끝이 아닙니다. 다음 챕터에서 이 구조가 그대로 에이전트 도구 하나 (`search_documents`)로 감싸지고, 옆에 DB 조회 도구(`leave_balance`, `sales_sum`, `list_employees`) 셋이 나란히 놓입니다. 에이전트가 질문을 보고 어떤 도구를 열지 스스로 판단하게 되죠. 파이프라인 자체는 버려지지 않습니다.



5.6 전체 구성도에서 챕터 5의 자리



챕터 6

오케스트레이션

Tool Calling Agent

ReAct 패턴 · 사후 분류

@tool 도구

DB 조회 · 문서 검색

챕터 5

검색 (실시간)

오늘 만든 부분

LCEL Chain

검색기 → 프롬프트 → LLM · 출처 강제 ·

WindowMemory

챕터 4

파싱·벡터화 (오프라인)

챕터 3 문서 규칙

PDF·Word·Excel·HWP

Doc Pipeline

파싱·청킹·임베딩

ChromaDB

벡터 저장소

챕터 2

데이터

PostgreSQL

직원·연차·매출

Ollama LLM (외부)

오늘 엮은 건 **LCEL Chain** 한 박스입니다. 이제 문서 기반 질문에는 출처를 붙여 답합니다. 하지만 "김대리 연차 며칠?" 같은 DB 조회 질문은 아직 못 풅니다. 챕터 6에서 **Tool Calling Agent + ReAct 패턴**을 엮어 두 종류 질문을 한 에이전트가 처리하게 만듭니다.

용어 정리

본문 속 표현	진짜 용어	정식 정의
"사서의 업무 순서"	LCEL 파이프라인	LangChain Expression Language. 파이프(<code> </code>)로 Retriever → Prompt → LLM → Parser를 연결
"근거를 대"	출처 강제(Source Grounding)	프롬프트로 "제공된 문서에서만 답하고 출처 명시"를 강제하는 기법
"5장짜리 메모장"	WindowMemory	최근 N턴 대화만 유지하는 슬라이딩 윈도우. <code>deque(maxlen=k)</code> 기반
"메모장 관리인"	ConversationManager	세션별 WindowMemory를 보관하고 TTL로 만료 세션을 정리하는 클래스
"입력 → 출력 연결"	파이프 연산자(<code> </code>)	<code>A B</code> 는 A의 출력을 B의 입력으로 전달. Unix 파이프와 같은 개념

이것만은 기억하자

- **벡터 검색은 재료, LCEL은 요리입니다.** 검색 결과를 던져 주는 게 아니라 읽고 답해 주는 체인을 엮어야 사용자가 쓸 수 있는 비서가 됩니다.
- **출처 강제는 환각의 안전망입니다.** "제공된 문서에서만 답하라 + 출처를 명시하라" 한 줄이 거짓말을 크게 줄여 줍니다. 답변에 출처 파일명을 항상 붙이세요.
- **기억 없는 LLM에게 메모장을 쥐여 줍니다.** WindowMemory(k=5)면 일상 대화에 충분합니다. 챗터 6에서는 에이전트가 도구를 직접 골라 "문서 질문"과 "DB 조회 질문"을 한 번에 처리합니다.

챕터 6. 연차도 규정도 한번에. ReAct Agent와 Tool Calling

이번 챕터가 끝나면

- **@tool** 데코레이터로 DB 조회, 문서 검색을 에이전트가 선택할 수 있는 도구로 감쌉니다
- **ReAct 패턴**(Reason, Act, Observe 반복)으로 복잡한 질문을 단계적으로 해결합니다
- 에이전트가 **어떤 도구를 호출했는지**로 질문 유형(정형·비정형·복합)을 사후 판정합니다
- "A 사원 연차 며칠? 그리고 신청 절차도 알려줘" 같은 복합 질문에 한 번의 답으로 응답합니다

준비하기. 실습 시작 전 한 번만 설정

1. 실습 폴더 이동

터미널 폴더 이동

```
cd rag-start/ex06
```

파일 구조는 다음과 같습니다.

ex06 디렉토리

```

ex06/
├── run.py
├── src/
│   ├── mcp_tools.py      # [실습] @tool 4개
│   ├── agent.py          # [실습] ReAct 에이전트 조립
│   ├── llm_factory.py    # [참고] Ollama/OpenAI 분기
│   ├── db_helper.py      # [참고] PostgreSQL + ChromaDB 헬퍼
│   └── agent_helpers.py  # [참고] 결과 파싱 유틸
├── app/
│   ├── main.py           # [참고] FastAPI 진입점
│   ├── chat_api.py       # [설명] 에이전트/RAG 모드 선택 API
│   ├── admin_views.py    # [참고] 관리자 대시보드 라우터
│   └── database.py        # [참고] PostgreSQL 연결
├── templates/            # [참고] 채팅 UI + 관리자 대시보드
└── static/               # [참고] UI 스타일
    
```

2. 실습 환경 구축

터미널 환경 구성. macOS / Linux

```

cd ex06
cp .env.example .env
python3.12 -m venv .venv
source .venv/bin/activate
docker compose up -d
ollama pull llama3.1:8b
pip install -r requirements.txt
    
```

터미널 환경 구성. Windows

```

cd ex06
copy .env.example .env
py -3.12 -m venv .venv
.venv\Scripts\activate
docker compose up -d
ollama pull llama3.1:8b
pip install -r requirements.txt
    
```


이전 챗터 Docker가 떠 있으면 포트 충돌

챗터 2의 Docker가 실행 중이라면 `cd ex02 && docker compose down`으로 먼저 종료해 주세요. 같은 포트(5432)를 씁니다.

3. 사용할 라이브러리

패키지	역할
<code>langchain</code>	체인·에이전트 프레임워크
<code>langchain-ollama</code>	Ollama LLM 연결 (Tool Calling 지원 모델 필수)
<code>langchain-chroma</code> · <code>chromadb</code>	벡터 DB 래퍼·저장소
<code>psycopg2-binary</code>	PostgreSQL 드라이버
<code>fastapi</code> · <code>uvicorn</code>	웹 API 서버

Tool Calling 지원 모델을 써야 합니다

모든 Ollama 모델이 Tool Calling을 지원하지는 않습니다. `llama3.1:8b`는 추론과 Tool Calling을 둘 다 안정적으로 해내서 이 챗터에 적합합니다 (16GB RAM에서 무리 없이 돌고 한국어 품질도 무난). 반면 `deepseek-r1`은 **추론에 특화된 대신** Tool Calling이 지원되지 않습니다. 에이전트가 도구를 스스로 호출해야 하는 이 챗터에서는 쓸 수 없으니 `llama3.1:8b`로 교체해 주세요.

`.env` 핵심 상수입니다. 챗터 5에서 쓰던 `.env`와 비교해 LLM 모델이 바뀌고, DB 접속 정보가 추가됩니다.

상수	기본값	역할
<code>LLM_PROVIDER</code>	<code>ollama</code>	LLM 제공자 (<code>ollama</code> 또는 <code>openai</code>). 이 챗터는 Tool Calling 지원 모델만 가능
<code>OLLAMA_MODEL</code>	<code>llama3.1:8b</code>	챗터 5의 <code>deepseek-r1:8b</code> 에서 변경. Tool Calling 지원
<code>OPENAI_MODEL</code>	<code>gpt-4o-mini</code>	OpenAI 쓸 때 권장. Ollama보다 Tool Calling이 안정적
<code>POSTGRES_*</code>	<code>rag_db</code> / <code>rag_user</code> / ...	챗터 2에서 띄운 DB 접속 정보
<code>CHROMA_PERSIST_DIR</code>	<code>./data/chroma_db</code>	ex06 자체 벡터 DB. 첫 실행 시 <code>data/docs/</code> 로부터 자동 구축
<code>RETRIEVER_TOP_K</code>	<code>3</code>	<code>search_documents</code> 도구가 가져올 청크 수

4. 실습 순서

1. `src/mcp_tools.py` . @tool 4개 (연차, 매출, 직원, 문서)
2. `src/agent.py` . ReAct 에이전트 조립 + 사후 분류
3. `python run.py` . 서버 실행 후 `http://localhost:8000/chat` 에서 복합 질문 테스트

6.1 RAG가 답 못 하는 질문

 그림 6-1. 서가뿐 아니라 장부실 열쇠까지 쥔 사서. 질문 하나에 두 자료함을 모두 살피는 통합 에이전트

목요일 오후 4시. 창밖이 노랗게 물들고 있었습니다. 키보드 소리가 드문드문 울리는 사무실이었습니다.

챗터 5에서 채팅 UI를 띄워 두고 동료들이 한 번씩 써 보게 했습니다. 첫 반응은 나쁘지 않았습니다. 그런데 얼마 후 다른 동료가 노트북을 들고 책상 앞까지 왔습니다.

동료 : "이게 그 AI 비서예요? 한번 써봐도 되죠?"

오픈이 : "당연하죠. 뭐든 물어보세요."

동료가 채팅창에 입력을 시작했습니다.

동료 : "A 사원 남은 연차는? 그리고 연차 신청 절차 알려줘요."

잠시 후 돌아온 답변은 "해당 직원에 대한 정보를 찾지 못했습니다"였습니다.

오픈이는 당황해서 질문을 바꿔 넣어 봤습니다. "이번 달 매출 합계." 같은 대답. "직원 목록 보여줘." 역시 같은 대답.

동료 : "직원 이름도 모르는 AI 비서예요?"

동료가 돌아간 자리에는 노트북 화면 위의 에러 문구만 남았습니다.

사서는 여전히 서가만 뒤지고 있었다. 챗터 2에서 만들어 둔 장부실이 옆방에 있는데, 그쪽 문을 열 줄 모른다.

옆자리 팀장이 모니터를 흘끔 보고 한 마디 던졌습니다.

팀장 : "사서한테 책만 찾게 하지 말고, 장부도 꺼내게 해봐요."

지금의 사서는 서가에서 규정집만 꺼내 줍니다. 하지만 'A 사원'은 서가 어디에도 없습니다. 직원 연차 정보는 책이 아니라 **PostgreSQL 장부**에 적혀 있습니다. 장부실을 여는 법을 배우지 못한 사서는 "없어요"만 반복했습니다.

6.2 정형 데이터와 비정형 데이터

지금 만들어 둔 것을 정리하면:

- **챗터 2**: 직원, 연차, 매출을 PostgreSQL에 저장하는 API
- **챗터 4~5**: 사내 문서를 벡터 DB에 넣고 검색, 답변하는 RAG 엔진

둘이 따로 놓고 있습니다. AI 비서는 RAG만 쓸 줄 알지 DB에 접근하는 법을 모릅니다.

질문	경로
"A 사원 연차 몇 개?"	PostgreSQL 조회 (정형)
"연차 신청 절차?"	취업규칙 문서 검색 (비정형)
"A 사원 연차 + 신청 절차?"	둘 다 (복합)

진짜 비서가 되려면 이 둘을 상황에 맞게 고르거나, 필요하면 동시에 써야 합니다.

6.3 사서가 두 자료함을 연다

팀장이 말한 "장부까지 꺼내는 사서"를 그려 보겠습니다. 지금까지 서가 하나만 맡던 사서에게 옆방 장부실 열쇠를 새로 쥐여 주는 장면입니다.

이용자 : "A 사원 연차 며칠 남았어요?"

사서 : (장부실로 가서 연차 장부를 펼친다) "3일 남았습니다."

며칠 후 복잡한 질문.

이용자 : "A 사원 연차 며칠 남았고, 신청 절차도 알려주세요."

사서 : (장부실에서 연차를 확인한 뒤 서가로 가서 규정집까지 찾아온다) "3일 남았고, 규정은 이렇습니다..."

한 명의 사서가 자료의 성격을 보고 **장부실과 서가를 스스로 오갑니다**. 바깥에 안내 직원을 따로 두고 "너는 장부실, 너는 서가" 하고 미리 나누지 않습니다. 사서가 질문을 보자마자 필요한 자료함을 직접 엽니다.

 그림 6-2. 어느 자료함을 열지는 사서가 알아서 판단합니다.

AI 비서도 같은 구조가 됩니다. 한 명의 **에이전트**가 질문을 보고 장부 도구(`leave_balance` 등)를 열지, 문서 도구(`search_documents`)를 열지, 둘 다 열지 판단합니다. 판단 주체는 에이전트 속 LLM 자신이지, 바깥의 분류기가 아닙니다. 이번 챕터의 **통합 에이전트**가 이 "만능 사서"입니다.

6.3.1 파이프라인에서 에이전트로 (챗터 5 엔진은 유지됩니다)

챗터 5에서 만든 건 **고정된 파이프라인**이었습니다. 질문이 들어오면 언제나 같은 순서로 흐릅니다. Retriever로 문서 조각을 가져오고, Prompt로 감싸고, LLM에 보내고, 답을 돌려받죠. 입력 질문이 "휴가 규정"이든 "A 사원 연차"든 똑같은 경로를 탑니다. 문제는 그 경로 중간에 **PostgreSQL 장부를 들여다보는 단계가 없다는** 점입니다. 체인을 새로 짤다 해도 "이 질문에는 DB를 볼까 문서만 볼까?"를 **실시간으로 판단할 자리가 없습니다**. 고정 파이프는 분기가 약합니다.

에이전트는 다릅니다. 질문을 받으면 LLM이 먼저 "지금 어느 자료함을 열어야 하지?"를 스스로 판단합니다. 결과를 보고 "충분한가, 아니면 한 곳 더?"를 다시 판단합니다. 이 **생각(Reason) → 행동(Act) → 관찰(Observe)** 루프가 **ReAct 패턴**입니다. 경로가 질문마다 달라지죠.

그림 6-3. LCEL 체인 vs ReAct 에이전트

항목	LCEL 체인 (챗터 5)	ReAct 에이전트 (이 챗터)
흐름	고정 순서. Retriever → Prompt → LLM	동적. 질문을 보고 매 턴 경로 결정
도구 선택	없음 (문서 검색만)	LLM이 스스로 고름
분기	파이프를 짤 때 하드코딩	런타임에 판단
적합한 질문	순수 문서 Q&A	정형·비정형·복합을 한 창구로

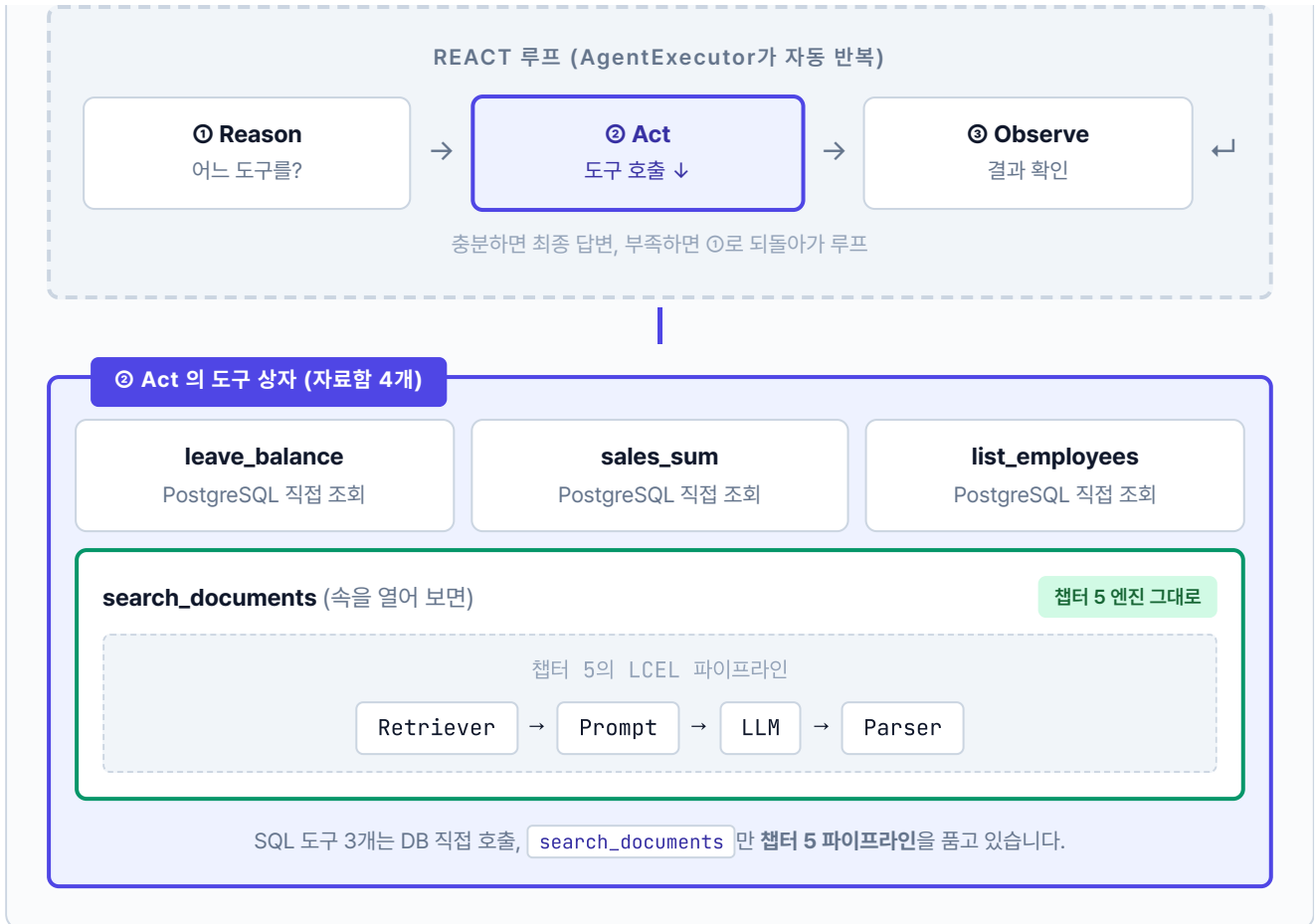
"그럼 파이프라인에서 **Tool Calling**만 붙이면 되지 않나?" 사실 LCEL에서도

`llm.bind_tools([...])` 로 도구를 묶을 수 있고, LLM이 어떤 도구를 호출할지 고르는 것까진 됩니다. 여기까지는 체인 한 번으로 충분하죠. 진짜 문제는 **루프**입니다. 도구가 답을 내놓으면 LLM은 그걸 보고 "충분한가? 한 번 더?"를 다시 판단해야 합니다. LCEL 체인은 입력 → 출력 한 번 흐르고 끝나는 구조라 이 **재판단**을 자동으로 돌리지 않습니다. 필요할 때마다 수동으로 while 루프를 짜야 하죠.

ReAct 에이전트(AgentExecutor)는 그 루프를 **한 줄 호출로 자동화**합니다. 도구 선택 → 실행 → 결과 관찰 → 다음 판단 → (끝까지) 를 AgentExecutor가 알아서 돌립니다. 복합 질문("A 사원 연차 + 신청 절차")처럼 도구를 두세 번 연달아 써야 할 때 이 자동 루프가 빛을 발합니다.

그림으로 보면 파이프라인이 어디에 박혀 있는지 한눈에 보입니다. 바깥을 **ReAct 루프**가 감싸고, 그 안에서 에이전트가 도구 4개를 골라 씁니다. 그중 `search_documents` 도구의 속을 열어 보면 챗터 5에서 만든 **LCEL 파이프라인이 그대로** 들어 있습니다.

그림 6-4. ReAct 에이전트 안의 파이프라인. 챗터 5 엔진은 버려지지 않고 도구 하나로 감싸져 들어옵니다



정리하면 세 층입니다. **바깥층**은 ReAct 루프가 질문마다 경로를 정합니다. **중간층**은 도구 4개 중 어떤 걸 쓸지 선택하는 자리입니다. **안쪽층**이 `search_documents` 안에서 돌아가는 챕터 5의 LCEL 파이프라인이고요. 복합 질문이 들어오면 바깥 루프가 두 번 돕니다. 먼저 `leave_balance` 로 SQL 한 번, 다음 턴에 `search_documents` 안의 파이프라인이 문서 한 번. 두 결과를 합쳐 최종 답변을 만듭니다.

여기서 중요한 건 **챕터 5 엔진을 버리지 않는다**는 점입니다. `rag_chain.py` 의 LCEL 파이프라인은 그대로 보존됩니다. 다만 이번 챕터에서는 그 엔진을 **에이전트의 도구 하나**(`search_documents`)로 감싸 넣고, 옆에 `leave_balance` · `sales_sum` · `list_employees` 세 도구를 나란히 둡니다. 엔진을 버리는 게 아니라 **더 큰 판단 구조(에이전트) 아래에 배치**하는 것입니다. 다음 챕터 7, 10도 이 에이전트 구조를 그대로 이어받습니다.

다음 절부터 이 사서의 손에 들릴 **도구**를 하나씩 만들어 보겠습니다.

6.4 MCP Tools: @tool 데코레이터

사서가 손에 들 **자료함(도구)** 을 네 개 준비합니다. 앞의 세 개는 장부실, 마지막 하나는 서가에 있습니다.

도구	역할	데이터 원천
<code>leave_balance</code>	특정 직원 연차 잔여일	PostgreSQL <code>employees</code> + <code>leave_balance</code>
<code>sales_sum</code>	부서·기간별 매출 합계	PostgreSQL <code>sales</code>
<code>list_employees</code>	이름·부서로 직원 목록	PostgreSQL <code>employees</code>
<code>search_documents</code>	문서 기반 질문 답변	ChromaDB (챗터 4~5 파이프라인)

6.4.1 @tool vs MCP — 도구 호출의 두 가지 방식

AI 에이전트가 외부 기능을 호출하는 방법은 크게 두 가지가 있습니다. 하나는 Anthropic이 만든 **MCP(Model Context Protocol)** 입니다. 도구를 별도 서버로 분리하고 표준 메시지 규격으로 통신하는 방식이라 어떤 프레임워크에서든 같은 도구를 재사용할 수 있습니다. 다른 하나는 LangChain의 `@tool` 데코레이터입니다. 같은 프로세스 안에서 함수를 직접 호출하니 설정이 간단합니다.



그림 6-3. LangChain `@tool` 데코레이터와 MCP 방식의 도구 호출 비교. 전자는 에이전트와 함수가 한 프로세스, 후자는 MCP 서버로 분리된 별도 프로세스입니다

MCP 쪽 화살표에 적힌 `stdio · HTTP` 는 에이전트와 MCP 서버가 대화할 때 쓰는 통신 규격입니다. 두 프로세스가 따로 떼 있으니 직접 함수 호출은 못 하고, 중간에 약속된 메시지 형식이 필요합니다. 같은 컴퓨터 안에서는 **stdio**(표준 입출력 스트림)로, 원격 서버는 **HTTP**로 메시지를 주고받습니다. Claude Desktop·Cursor·IDE 에이전트 같은 여러 앱이 같은 MCP 서버에 붙을 수 있는 이유도 이 표준 규격 덕분입니다. 반대로 `@tool` 은 파이썬 함수를 그대로 호출하므로 이런 프로토콜이 필요 없습니다.

핵심 개념은 같습니다. "AI가 함수의 이름과 설명을 보고 스스로 호출한다." 이 책에서는 `@tool` 로 원리를 익혀 보겠습니다. 원리를 이해하면 MCP로 확장하는 건 어렵지 않습니다.

각 도구 함수에 `@tool` 데코레이터와 독스트링(설명)을 붙여두면 LangChain이 도구의 이름과 설명, 파라미터를 자동으로 읽어옵니다. 에이전트는 이 정보를 보고 "이 질문에는 어떤 도구가 맞지?"를 판단합니다. 이처럼 LLM이 상황에 맞는 도구를 스스로 골라서 호출하는 기능을 **도구 호출(Tool Calling)** 이라고 합니다. 모든 LLM이 지원하는 건 아니라서 OpenAI의 `gpt-4o-mini` 나 Ollama의 `llama3.1:8b` 처럼 Tool Calling을 지원하는 모델을 골라야 합니다.

`@tool` 데코레이터. 파이썬 `@decorator` 문법은 함수 정의 위에 붙여 **함수를 다른 함수로 감싸 주는** 표시입니다. `@tool` 은 LangChain이 제공하는 데코레이터로, 일반 함수를 LLM이 호출 가능한 `Tool` 객체로 자동 변환해 줍니다. 함수 이름·인자 타입·반환 타입·docstring을 읽어 스키마를 만들고, 에이전트에게 "이 도구는 `(emp_no: str) -> dict` 를 받습니다"처럼 구조화된 정보로 전달합니다. 직접 `Tool(name=..., func=..., description=...)` 를 수동 조립하지 않아도 됩니다.

docstring. 함수 본문 첫 줄에 놓이는 삼중 따옴표 문자열(`"""..."""`)입니다. 원래는 사람 개발자용 설명이지만, `@tool` 에서는 **LLM이 도구를 고를 때 읽는 사용 설명서**로 쓰입니다. "언제 이 도구를 쓰는가"를 명확히 적어야 에이전트가 올바르게 선택합니다. 예: `"""주어진 사번의 연차 잔여일수를 반환합니다."""`

6.4.2 도구 구현하기

`src/mcp_tools.py` 에 네 개의 함수 TODO가 있습니다. 각 TODO의 `pass` 를 지우고 `@tool` 과 docstring, 인자를 채웁니다.

실습 2 ex06/src/mcp_tools.py. @tool 데코레이터

```
from src.db_helper import run_query, get_vectorstore, DB_ERROR_MSG

@tool
def leave_balance(emp_no: str) → dict:
    """직원의 연차 잔여 일수를 조회한다."""
    # TODO: leave_balance - 사번으로 연차 조회

    # 1. DB 조회 (employees + leave_balance 조인)
    rows = run_query(
        """SELECT e.name, l.total_days, l.used_days,
               (l.total_days - l.used_days) AS remaining_days
        FROM employees e
        LEFT JOIN leave_balance l ON e.emp_no = l.emp_no
        WHERE e.emp_no = %s""",
        (emp_no,)
    )

    # 2. 결과 있으면 반환, 없으면 에러
    return rows[0] if rows else {"error": DB_ERROR_MSG}

@tool
def search_documents(query: str, k: int = 3) → dict:
    """사내 문서에서 관련 내용을 벡터 검색한다."""
    # TODO: search_documents - ChromaDB로 관련 청크 검색

    # 1. ChromaDB 컬렉션 가져오기
    collection = get_vectorstore()

    # 2. 벡터 검색 수행
    results = collection.query(query_texts=[query], n_results=k)

    # 3. content / source 형태로 가공
    docs = [
        {"content": doc, "source": results["metadatas"][0][i].get("source", "unknown")}
        for i, doc in enumerate(results["documents"][0])
    ]
    return {"results": docs, "total_found": len(docs)}
```

`run_query` · `get_vectorstore` 는 `src/db_helper.py` 의 보조 함수로, PostgreSQL 커넥션과 ChromaDB 컬렉션을 감싼 헬퍼입니다. 직접 작성하지 않고 import만 합니다.

나머지 두 개는 DB 조회의 다른 변형입니다. `sales_sum` 은 집계 쿼리(SUM + GROUP BY), `list_employees` 는 동적 WHERE 조립(부서·이름 필터).

실습 2 ex06/src/mcp_tools.py. 집계·동적 필터

@tool

```
def sales_sum(dept: str = "", start_date: str = "", end_date: str = "") → dict:
    """부서별 또는 전체 매출 합계를 조회한다."""
```

TODO: sales_sum - 기간·부서 필터로 매출 집계

1. 파라미터 기본값 처리

start = start_date or "2024-11-01"

end = end_date or "2024-12-31"

2. DB 조회 (부서 필터 적용)

dept_filter = f"AND e.department LIKE '%{dept}%' " if dept else ""

```
rows = run_query(
    f"""SELECT SUM(s.amount) AS total_amount
        FROM sales s JOIN employees e ON s.emp_no = e.emp_no
        WHERE s.sale_date BETWEEN %s AND %s {dept_filter}""",
    (start, end),
)
```

3. 결과 가공해서 반환

```
return {"total_amount": rows[0]["total_amount"] if rows else 0,
        "period": f"{start} ~ {end}"}
```

@tool

```
def list_employees(dept: str = "", name: str = "") → dict:
```

"""직원 목록을 조회한다. 부서·이름으로 필터 가능."""

TODO: list_employees - WHERE 조건을 동적으로 조립

1. 조건 조립 (부서/이름 필터)

conditions, params = [], []

if dept:

conditions.append("department LIKE %s"); params.append(f"%{dept}%")

if name:

conditions.append("name LIKE %s"); params.append(f"%{name}%")

2. SQL 문자열 완성 + 실행

sql = "SELECT emp_no, name, department, position, hire_date FROM employees"

if conditions:

sql += " WHERE " + " AND ".join(conditions)

rows = run_query(sql + " ORDER BY name", tuple(params))

3. 결과 반환

```
return {"employees": rows, "count": len(rows)}
```

도구의 docstring이 **LLM의 판단 근거**가 되므로 "이 도구는 언제 써야 하는가"를 명확히 써 주세요. 설명이 불명확하면 에이전트가 엉뚱한 도구를 고릅니다. AI 비서에게 각 도구가 무슨 일을 하는지 제대로 알려줘야 제대로 골라 쓰는 것과 마찬가지입니다.

MCP Tools? Tool? 이 책에서는 LangChain `@tool` 로 만든 함수를 포괄해 **MCP Tools**라고 부릅니다. Model Context Protocol의 의미를 확장해 "AI가 호출 가능한 도구 계층"이라는 개념적 명칭으로 씁니다. 구현 수단은 LangChain의 Tool Calling이지만, 같은 역할(도구 호출 계층)을 MCP 서버로도 구현할 수 있습니다.

6.5 ReAct 패턴 & 에이전트 조립

챗터 1에서 "리프레시 데이 몇 번 남았어?"라는 질문을 던졌습니다. LLM이 규정을 읽고 "매월 1회 → 6개월이면 6번 → 2번 썼으면 4번"까지 스스로 계산해 냈죠. 생각의 고리가 사슬처럼 이어지는 **사슬 추론 (Chain-of-Thought)** 입니다. ReAct는 여기에 **도구 호출**을 끼워 넣은 확장판입니다.

챗터 1 사슬 추론	챗터 6 ReAct
하는 일: 문서를 읽고 단계별 계산 반복: 머릿속에서 1회 예시: "6개월이면 6번, 2번 쓰면 4번" 핵심: 읽고 생각	하는 일: 도구를 고르고 호출하고 결과 확인 반복: Reason→Act→Observe 여러 바퀴 예시: "DB 조회 → 결과 보고 → 문서 검색" 핵심: 생각하고 행동

"생각만 하는" 사슬 추론에서 "생각하고 행동하고 결과를 보는" ReAct로 진화합니다.

사서는 복합 질문을 한 번에 풀지 않습니다. 장부실과 서가 사이를 왔다 갔다 하면서 자료를 조각조각 모읍니다. 그 왕복의 머릿속 패턴은 이렇습니다.

1. **Reason (추론)**. "연차 잔여일이 먼저 필요하겠네. 그다음 신청 절차."
2. **Act (행동)**. `leave_balance(emp_no="E003")` 호출
3. **Observe (관찰)**. "총 15일, 잔여 8일이 돌아왔군"
4. **Reason**. "이제 신청 절차를 문서에서 찾자"
5. **Act**. `search_documents(query="연차 신청 절차")`

6. **Observe.** "그룹웨어 → 근태관리 → 연차신청"

7. 최종 답변 생성

이 Reason, Act, Observe 반복이 **ReAct 패턴**입니다. LangChain의

`create_tool_calling_agent` + `AgentExecutor` 가 루프를 자동으로 돌려 줍니다.



6.5.1 에이전트 조립하기

`src/agent.py` 에 에이전트 조립 TODO가 있습니다. TODO의 `pass` 를 지우고 아래 코드를 작성합니다. `create_tool_calling_agent` + `AgentExecutor` 로 ReAct 루프를 만듭니다.

실습 3 ex06/src/agent.py. ReAct 에이전트

```
from src.mcp_tools import ALL_TOOLS

SYSTEM_PROMPT = """당신은 사내 HR 및 업무 질문에 답변하는 AI 어시스턴트입니다.
- 정형 데이터(숫자·통계·목록)는 DB 조회 도구를 사용하세요.
- 비정형 질문(절차·정책·설명)은 search_documents를 사용하세요.
- 복합 질문은 두 종류의 도구를 모두 사용하세요."""

# TODO: agent 조립 - 도구 + LLM + 프롬프트

# 1. 프롬프트 구성 (system + history + input + scratchpad)
prompt = ChatPromptTemplate.from_messages([
    ("system", SYSTEM_PROMPT),
    MessagesPlaceholder(variable_name="chat_history", optional=True),
    ("human", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad"),
])

# 2. Tool Calling Agent 생성
agent = create_tool_calling_agent(llm=llm, tools=ALL_TOOLS, prompt=prompt)

# 3. AgentExecutor 래핑 (중간 단계 반환 활성화)
executor = AgentExecutor(
    agent=agent, tools=ALL_TOOLS,
    verbose=False, return_intermediate_steps=True,
    max_iterations=10, handle_parsing_errors=True,
)
```

등장한 LangChain 부품 세 개를 한 줄씩 정리합니다.

부품	역할
<code>create_tool_calling_agent(llm, tools, prompt)</code>	LLM·도구·프롬프트를 묶어 "판단만 하는" 에이전트 코어 생성. 도구 호출 결정까지만 책임짐
<code>AgentExecutor(agent, tools, ...)</code>	에이전트 코어를 받아 ReAct 루프를 실제로 돌리는 실행기. 도구 호출, 결과 주입, 다음 판단 요청을 반복. <code>max_iterations</code> 로 무한 루프 방지
<code>MessagesPlaceholder("agent_scratchpad")</code>	ReAct 루프의 메모장 . 각 Reason·Act·Observe 단계가 여기에 쌓이고, 에이전트가 매 턴 이전 기록을 참고해 다음 결정을 내림

"LLM이 도구를 한 번 고르는 것"과 "복잡한 질문을 풀기 위해 도구를 여러 번 반복 호출하는 것"은 다릅니다. 전자는 `create_tool_calling_agent` 만으로 되지만, 후자는 `AgentExecutor` 가 루프를 돌려야 가능합니다. `agent_scratchpad` 는 그 루프의 기억 창구입니다.

6.6 서버 실행 + 복합 질문 테스트

터미널 서버 실행

```
python run.py
```

브라우저에서 `http://localhost:8000/chat` 을 엽니다. 이번엔 모드 토글이 생겼습니다. **Agent 모드**로 바꾸면 화면 상단에 **정형·비정형·복합** 세 그룹으로 묶인 샘플 질문 버튼이 나타납니다. 하나씩 눌러 에이전트가 어떤 경로로 답변하는지 확인합니다.

질문 유형	예시 버튼	에이전트가 부르는 도구
정형	"김민준 연차 잔여일수 알려줘" · "영업부 11월 매출 합계가 얼마야?" · "개발부 직원 목록 보여줘"	<code>leave_balance</code> · <code>sales_sum</code> · <code>list_employees</code>
비정형	"온보딩 절차가 어떻게 되나요?" · "보안 정책에 대해 설명해줘" · "출장 비용 규정 알려줘"	<code>search_documents</code>
복합	"매출 상위 부서의 위케이션 규정은?" · "정시우 사원의 연차 잔여와 휴가 규정을 설명해주세요"	여러 도구 연쇄 호출

 그림 6-5. 복합 질문 "정시우 사원의 연차 잔여와 휴가 규정을 설명해주세요"에 대한 답변입니다. DB에서 숫자를 꺼내고 문서에서 절차를 찾아 한 답으로 정리합니다

"정시우 사원의 연차 잔여와 휴가 규정을 설명해주세요"라는 복합 질문이 들어왔을 때, 에이전트 머릿속의 사슬 추론은 이렇게 흘러갑니다.

Reason 1: "연차 잔여일은 DB에 있겠지. leave_balance를 호출하자." **Act 1:**

`leave_balance(emp_no="E003")` → **Observe 1:** "총 15일, 사용 7일, 잔여 8일" **Reason 2:**

"잔여일은 알았고, 휴가 규정은 문서에 있겠지." **Act 2:** `search_documents("휴가 규정")` →

Observe 2: "취업규칙 제15조 연차유급휴가..." **Reason 3:** "두 정보 다 모였다. 답변을 만들자."

챕터 1의 사슬 추론은 머릿속에서 한 번에 끝났지만, ReAct는 **도구를 호출하고 결과를 확인하는 행동**이 추론 사이사이에 끼어듭니다. 생각의 고리가 행동으로 이어지고, 행동의 결과가 다음 생각의 입력이 됩니다.

![(../assets/챕터 6/terminal/06_agent-run-real.png) 그림 6-6. `python -m src.agent` 실행 결과입니다. 도구 호출 → 사후 분류 → 최종 답변까지 실제 단계별 로그가 찍힙니다

동료: "이제 진짜 비서 같은데요."

오픈이가 수첩을 꺼냈습니다. 입사 3일 차에 처음 AI 비서를 켜올 때와 지금의 차이를 한 장 안에 적어 두고 싶었습니다.

— 생각 정리 —

1. 정형 질문

- "A 사원 연차 잔여?" 같은 숫자·필드 조회
- DB 도구 하나면 빠르게 답함

2. 비정형 질문

- "연차 신청 절차?" 같은 규정·매뉴얼 조회
- 문서 검색으로 정확한 근거 제공

3. 복합 질문

- 정형 + 비정형이 한 문장에 섞인 경우
- ReAct로 도구를 연쇄 호출해 한 번에 답함

→ 에이전트가 "어느 자료함을 열지" 스스로 판단

6.7 전체 구성도에서 챕터 6의 자리

전체 구성도. 짙은 박스가 챕터 6 범위

사내 직원·관리자

챕터 2

대시보드

FastAPI

REST API · 관리자 웹

챕터 6

오케스트레이션

오늘 만든 부분

Tool Calling Agent

ReAct 패턴 · 사후 분류

오늘 만든 부분

MCP Tools

DB 조회 · 문서 검색

챕터 5

검색 (실시간)

LCEL Chain

검색기 → 프롬프트 → LLM

챕터 4

파싱·벡터화 (오프라인)

챕터 3 문서 규칙

PDF·Word·Excel·HWP

Doc Pipeline

파싱·청킹·임베딩

ChromaDB

벡터 저장소

챕터 2

데이터

PostgreSQL

직원·연차·매출

Ollama LLM (외부)

오늘 채운 건 **Tool Calling Agent**와 **MCP Tools** 두 박스입니다. 위쪽 사용자 입력부터 아래쪽 데이터, 문서 저장소까지 전 구간이 하나로 연결됐습니다. 챕터 7에서는 이 에이전트를 실제 운영에 올렸을 때 생기는 속도, 비용 문제를 ResponseCache와 TokenTracker로 잡습니다.

종료합니다

`Ctrl + C` 로 서버를 종료합니다. Docker 컨테이너는 챕터 7에서도 같은 PostgreSQL-ChromaDB를 쓰므로 그대로 두세요.

용어 정리

본문 속 표현	진짜 용어	정식 정의
"만능 사서"	에이전트 (Agent)	질문을 받아 스스로 판단하고 도구를 선택·실행해 답변을 만드는 자율 프로그램
"자료함" (장부실 · 서가)	도구 (Tool)	<code>@tool</code> 데코레이터로 감싼 함수. 에이전트가 선택 실행하는 원자 작업 단위
"호출한 도구로 라벨 붙이기"	사후 분류 (Post-hoc)	에이전트가 실행 후 실제 호출한 도구 종류로 질문 유형 (structured-unstructured-hybrid) 을 확정하는 방식
"생각 → 행동 → 관찰"	ReAct 패턴	Reason·Act·Observe 반복으로 복잡 질문을 단계적 해결하는 에이전트 전략
"반복 실행기"	AgentExecutor	ReAct 루프를 돌리고 도구 호출을 관리하는 LangChain 컴포넌트

이것만은 기억하자

- **문서 검색만으론 진짜 비서가 될 수 없습니다.** 정형 데이터(DB)와 비정형 데이터(문서)를 한 에이전트에서 통합해야 "연차 며칠?", "신청 절차?", "둘 다"를 모두 답할 수 있습니다.
- **분류는 사전 판정이 아니라 사후 분류입니다.** 에이전트가 자율적으로 도구를 고르고, 실제 호출된 도구 종류로 질문 유형이 확정됩니다. 두 도구가 모두 호출되면 그게 바로 복합(hybrid)입니다.
- **ReAct 루프가 복합 질문을 푼다.** Reason, Act, Observe 반복으로 에이전트가 도구를 여러 번 호출하고 결과를 묶어 한 답으로 돌려 줍니다. 챕터 7에서는 이 에이전트의 속도, 비용을 잡는 운영 기술을 다룹니다.

챕터 7. 실제로 써보니. 캐시와 모니터링

이번 챕터가 끝나면

- **ResponseCache(TTL)** 로 같은 질문에 같은 답을 0.1초에 돌려줍니다
- **EmbeddingCache** 로 같은 텍스트의 임베딩 재계산을 줄입니다
- **TokenTracker** 로 LLM 호출별 토큰, 비용, 응답시간을 집계합니다
- **Retry 로직**(최대 3회, 간격 2초)으로 네트워크, 파싱 오류에서 자동 회복합니다
- 이 모든 것을 **ConnectHRAgent**라는 운영용 래퍼로 한 번에 감쌉니다

준비하기. 실습 시작 전 한 번만 설정

1. 실습 폴더 이동

터미널 폴더 이동

```
cd rag-start/ex07
```

파일 구조는 다음과 같습니다.

ex07 디렉토리

```

ex07/
├── run.py
├── src/
│   ├── agent_config.py      # [실습] ConnectHRAgent 운영 래퍼
│   ├── cache.py             # [실습] ResponseCache + EmbeddingCache
│   ├── monitoring.py         # [실습] TokenTracker + JSON 로깅
│   ├── llm_factory.py        # [참고] LLM 인스턴스 생성
│   ├── agent_helpers.py      # [참고] RAG 체인 + 라우팅 매핑
│   ├── router.py             # [참고] 사후 분류 로직 (챕터 6)
│   └── tools/                # [참고] 도구 4개 (파일 분리)
│       ├── leave_balance.py
│       ├── sales_sum.py
│       ├── list_employees.py
│       └── search_documents.py
├── app/
│   ├── main.py               # [참고] FastAPI 진입점
│   ├── chat_api.py           # [참고] Agent API 엔드포인트
│   └── database.py            # [참고] PostgreSQL 연결
├── templates/chat.html       # [참고] 채팅 UI
└── static/                   # [참고] UI 스타일

```

2. 실습 환경 구축

터미널 환경 구성. macOS / Linux

```

cd ex07
cp .env.example .env
python3.12 -m venv .venv
source .venv/bin/activate
docker compose up -d
ollama pull llama3.1:8b
pip install -r requirements.txt

```

터미널 환경 구성. Windows

```

cd ex07
copy .env.example .env
py -3.12 -m venv .venv
.venv\Scripts\activate
docker compose up -d
ollama pull llama3.1:8b
pip install -r requirements.txt

```

3. 사용할 라이브러리

패키지	역할
<code>langchain</code> · <code>langchain-ollama</code>	에이전트 프레임워크·LLM 연결
<code>langchain-chroma</code> · <code>chromadb</code>	벡터 DB
<code>sentence-transformers</code>	ko-sroberta 임베딩
<code>psycopg2-binary</code>	PostgreSQL 드라이버
<code>fastapi</code> · <code>uvicorn</code>	웹 API 서버
<code>rich</code>	운영 로그 가독성 (컬러·테이블 출력)

`.env` 핵심 상수입니다. 이번 챕터에서 캐시·로깅 관련 항목이 새로 등장합니다.

상수	기본값	역할
<code>CACHE_TTL</code>	<code>3600</code>	응답 캐시 TTL (초)
<code>CACHE_MAX_SIZE</code>	<code>1000</code>	응답 캐시 최대 항목 수
<code>EMBEDDING_CACHE_DIR</code>	<code>./outputs/embedding_cache</code>	임베딩 벡터 저장 경로
<code>LOG_LEVEL</code>	<code>INFO</code>	로그 레벨
<code>USE_JSON_LOG</code>	<code>false</code>	JSON 구조화 로그 여부

4. 실습 순서

각 주제별로 이론을 읽고 바로 관련 파일 TODO를 채우는 구조입니다.

- 7.2 캐시 — ResponseCache·EmbeddingCache 이론 → `src/cache.py` 작성
- 7.3 모니터링 — TokenTracker 이론 → `src/monitoring.py` 작성
- 7.4 Retry + ConnectHRAgent — 재시도·운영 래퍼 이론 → `src/agent_config.py` 작성
- 7.5 서버 실행 — 같은 질문 두 번 던져 캐시 적중 확인

7.1 "같은 질문에 또 20초?"

 그림 7-1. 운영 안정화. 캐시와 업무 일지, 재시도까지 붙은 사서

챕터 6에서 통합 에이전트를 완성한 뒤 일주일이 흘렀습니다. 동료들 자리에 한 번씩 깔리던 웃음이 이제는 슬랙 채널에 모이고 있었습니다. 월요일 오전 9시 반. 오픈이는 커피잔을 책상에 내려놓고 슬랙을 열었습니다.

동료 : "병가 증빙 필요한지 아까도 물어봤는데, 또 20초 기다려야 해요?"

20초. 커서만 깜빡이는 화면에서 20초는 깃니다. 오픈이가 터미널을 열어 같은 질문을 던져 봤습니다. 로딩 스피너가 또 20초 동안 돌아갔습니다.

같은 질문인데 왜 매번 처음부터 찾는 거지.

건너편 자리의 다른 동료가 의자를 돌리며 한마디 더 없었습니다.

동료 : "우리 팀에서 하루에 30번은 쓰는 것 같은데, 얼마나 쓰고 있는 건지 파악이 안 돼요."

얼마나 쓰는지. 호출 횟수, 응답 시간, 토큰 수. 어디에도 기록이 없었습니다. 슬랙 채널을 위로 끌어올려 보니 간간이 "에러가 났어요" 메시지도 섞여 있었습니다. 확인해 보면 네트워크 타임아웃이거나 LLM 파싱 실패. 잠시 뒤 다시 물어보면 정상인데 사용자 입장에서 "고장났다"였습니다.

속도, 사용량, 실패. 세 가지가 한꺼번에 문제가 됐다.

옆 자리 팀장이 지나가다 모니터를 훑듯 봤습니다.

팀장 : "같은 질문에 매번 서가를 뒤지는 사서가 있으면 어떻겠어요?"

오픈이 : "비효율적이죠."

팀장 : "그럼 뭐가 필요해요?"

잠시 생각했습니다. 팀장은 답을 끌고 가는 스타일이 아니라 늘 질문을 던지는 쪽이었습니다.

메모장. 한 번 찾은 답을 적어 두는 메모장이 있으면 되잖아.

이번 챕터에서 **챕터 6의 IntegratedAgent** 는 그대로 둡니다. 엔진을 재설계하는 게 아니라 바깥을 감싸는 작업이죠. 캐시·업무 일지·재시도 세 층을 차례로 엮고, 그 결과물을 **ConnectHRAgent** 라고 부르겠습니다. 이름만 새 옷일 뿐 속은 챕터 6 그대로입니다.

그림 7-2. 챗터 6 에이전트는 그대로, 바깥에 운영 래퍼 3층을 씌웁니다



같은 엔진(IntegratedAgent)이지만 바깥에 3층이 더 감싸져 있습니다. 호출이 들어오면 캐시 → 재시도 → 코어 → 토큰 기록 순으로 흐릅니다. 각 층의 상세 파이프라인은 7.4절 그림 7-6에서 다룹니다.

7.2 ResponseCache + EmbeddingCache

메모장 이야기는 팀장 자리로도 이어졌습니다. 오픈이가 화이트보드 앞에 서서 마커 뚜껑을 딸깍 열었습니다.

팀장 : "메모장에 유통기한은 어떻게 할 건데요?"

오픈이 : "1시간 정도요. 규정이 자주 바뀌는 건 아니지만, 하루를 넘기면 옛날 답을 줄 수도 있으니깐요."

팀장 : "그 감각이 맞아요. 메모장은 오래 들고 있을수록 독이 돼요."

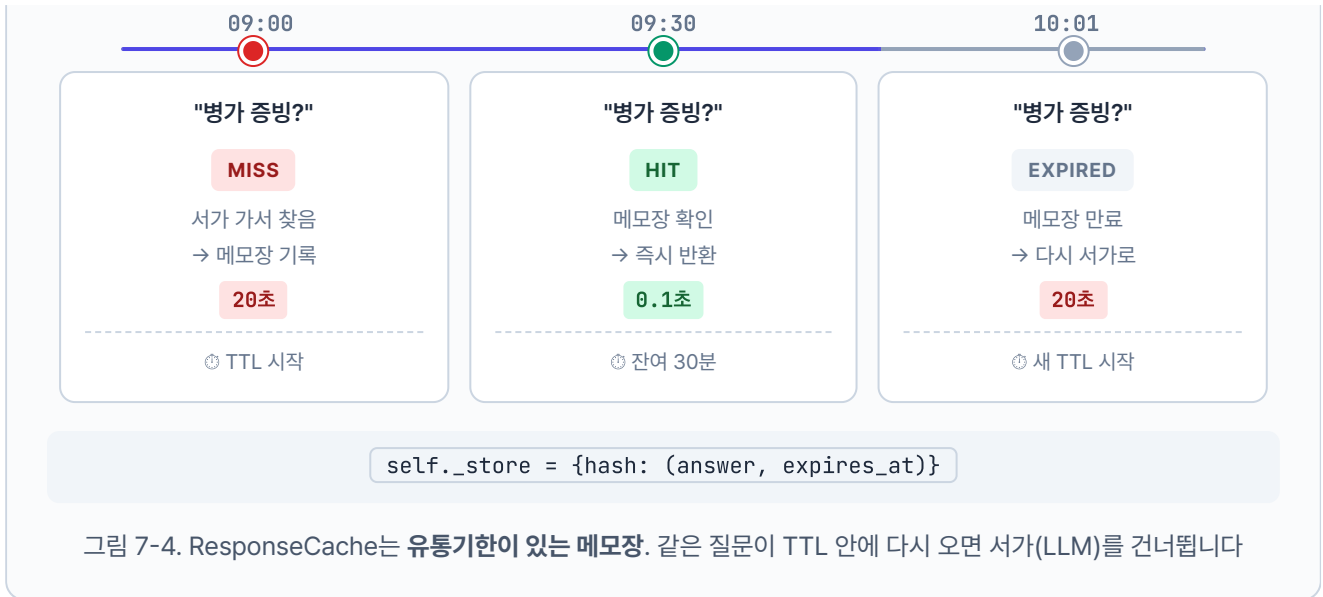
챗터 5에서 사서에게 대화용 메모장(**WindowMemory**)을 줬던 기억이 떠올랐습니다. 이번에는 **메모장을 두 권 더** 얹었습니다. 하나는 답변을 적는 메모장, 또 하나는 임베딩을 적는 메모장입니다.

답변 메모장 (ResponseCache)

같은 질문이 또 들어오면 서가까지 다시 갈 필요가 없습니다. 이전에 적어 둔 답을 바로 읽어 주면 됩니다. 단, 메모장에는 **유통기한(TTL)** 이 있습니다. 1시간 지나면 지웁니다. 사내 규정이 바뀌었을 수도 있기 때문입니다.

ResponseCache

같은 질문을 시간 순서로 추적



임베딩 메모장 (EmbeddingCache)

질문을 벡터로 바꾸는 데에도 시간이 듭니다. 같은 문장을 여러 번 변환할 필요는 없습니다. 한 번 계산한 벡터를 파일로 저장해 두면 다음엔 바로 꺼내 씁니다. EmbeddingCache는 유통기한이 없습니다. 벡터 값은 텍스트만 같으면 언제나 같기 때문입니다.



실습 1. cache.py

`ex07/src/cache.py` 상단에는 캐시 동작을 조절하는 상수 세 개가 이미 선언돼 있습니다. `.env` 값이 없으면 이 기본값을 씁니다.

설명 ex07/src/cache.py. 모듈 상수

```
DEFAULT_RESPONSE_TTL = 3600          # 응답 캐시 TTL (초). 1시간
DEFAULT_RESPONSE_CACHE_MAX_SIZE = 1000 # 최대 저장 항목 수
DEFAULT_EMBEDDING_CACHE_DIR = "./outputs/embedding_cache"
```

이제 두 클래스의 `get` · `set` TODO의 `pass` 를 지우고 아래 코드를 채워 보겠습니다. 키 생성(SHA-256 해시)과 파일 I/O는 `_cache_utils.py` 에 헬퍼로 이미 준비돼 있어서, 우리는 **TTL 검사**와 **최대 크기 관리**에만 집중하면 됩니다.

먼저 클래스 골격과 `__init__` 입니다. 메모장의 **용량 한도**(`max_size`), **유통기한**(`ttl`), 그리고 히트·미스 카운터를 준비합니다.

실습 1 ex07/src/cache.py. ResponseCache — 초기화

```
class ResponseCache:
    """TTL 기반 인메모리 응답 캐시."""

    def __init__(self, ttl=DEFAULT_RESPONSE_TTL, max_size=DEFAULT_RESPONSE_CACHE_MAX_SIZE):
        # 1. TTL·max_size는 .env 또는 기본값에서 주입 (3600초, 1000개)
        self.ttl = ttl
        self.max_size = max_size
        # 2. 메모장 본체: key → (value, expires_at) 튜플
        self._store = {}
        # 3. 통계 카운터 - 운영 중 적중률을 보기 위한 값
        self._hits = 0
        self._misses = 0
        logger.info("[ResponseCache] 초기화 완료 (TTL: %d초, 최대 크기: %d)", ttl, max_size)
```

`self._store` 가 **메모장 본체**입니다. 키 하나당 (실제 답변, 만료 시각) 튜플을 담습니다. 만료 시각을 같이 저장해 두는 것이 TTL 구현의 핵심입니다.

이제 조회 메서드 `get()` 입니다. **세 가지 경로**로 갈라집니다. 키 없음 → 만료됨 → 적중. 어느 쪽이든 카운터 갱신과 로깅을 잊지 않아야 합니다.

실습 1

ex07/src/cache.py. ResponseCache.get

```
def get(self, query, context=""):
    """캐시에서 응답을 조회합니다 (TTL 만료 체크)."""
    # TODO: get - TTL 검사 후 HIT/MISS 반환
    # 1. 질문(+문맥)을 SHA-256 해시로 변환해 고정 길이 키로
    key = make_response_key(query, context)

    # 2. 저장소에 키가 아예 없으면 MISS
    entry = self._store.get(key)
    if entry is None:
        self._misses += 1
        logger.debug("[ResponseCache] 미스: key=%s...", key[:12])
        return None

    # 3. 있더라도 만료 시각이 지났으면 삭제 후 MISS
    value, expires_at = entry
    if time.time() > expires_at:
        del self._store[key]
        self._misses += 1
        logger.debug("[ResponseCache] 만료: key=%s...", key[:12])
        return None

    # 4. 살아 있으면 HIT - 남은 TTL 로그로 남겨 디버깅 편하게
    self._hits += 1
    logger.info("[ResponseCache] 적중: key=%s... (잔여 TTL: %.0f초)", key[:12], expires_at - time.time())
    return value
```

핵심은 **2·3·4 분기가 모두 카운터를 갱신**한다는 점입니다. 만료된 항목은 그 자리에서 삭제하고 MISS로 처리합니다. 같은 질문이어도 유통기한이 지났으면 새 답을 받아 와야 하기 때문입니다.

저장할 때는 **용량 관리**가 함께 들어갑니다. 메모장 페이지가 `max_size` (기본 1000)를 넘으면 **만료 시각이 가장 이른 항목부터** 한 건 지우고 새 항목을 넣습니다.

실습 1 ex07/src/cache.py. ResponseCache.set

```
def set(self, query, value, context=""):
    """캐시에 응답을 저장합니다 (max_size 초과 시 오래된 항목 제거)."""
    # TODO: set - max_size 초과 시 가장 오래된 항목 제거
    # 1. 조회와 같은 방식으로 키 생성
    key = make_response_key(query, context)

    # 2. 용량 초과 시 만료 임박 항목 한 건 제거
    if len(self._store) > self.max_size:
        oldest_key = min(self._store, key=lambda k: self._store[k][1])
        del self._store[oldest_key]
        logger.debug("[ResponseCache] 최대 크기 초과로 항목 제거: key=%s...", oldest_key[:12])

    # 3. 만료 시각 = 지금 + TTL. 튜플로 저장
    expires_at = time.time() + self.ttl
    self._store[key] = (value, expires_at)
    logger.info("[ResponseCache] 저장: key=%s... (만료: %.0f초 후)", key[:12], self.ttl)
```

`min(self._store, key=lambda k: self._store[k][1])` 이 가장 먼저 만료될 항목을 찾는 부분입니다. 튜플의 두 번째 요소(`expires_at`)가 작을수록 만료가 가깝습니다. 용량을 딱 채우기 전에 한 건을 빼 주므로, 메모장은 항상 `max_size` 아래로 유지됩니다.

마지막으로 클래스 아래에는 **운영용 편의 메서드 두 개**가 이미 완성된 채로 들어 있습니다. 우리가 구현할 부분은 없고, 작동만 기억하면 됩니다.

설명 ex07/src/cache.py. ResponseCache.clear / stats

```
def clear(self):
    """만료된 캐시 항목을 제거합니다."""
    return response_cache_clear(self)

def stats(self):
    """캐시 통계를 반환합니다 (_hits / _misses / 현재 크기 등)."""
    return response_cache_stats(self)
```

`clear()` 는 만료된 항목을 일괄 정리하고, `stats()` 는 적중·미스 수와 현재 크기를 딕셔너리로 돌려줍니다. 실제 로직은 `_cache_utils.py` 헬퍼가 맡아 위임만 하고 있습니다. `stats()` 가 반환하는 값은 이후 상태 확인 대시보드에서 "캐시 적중률 72%" 같은 숫자로 쓰입니다.

여기까지가 **답변 메모장**입니다. 이제 두 번째 메모장인 **임베딩 메모장**으로 넘어갑니다. 같은 "캐시"라는 이름이지만 **무엇을 얼마나 어디에** 기록하느냐가 완전히 다릅니다.

	ResponseCache 답변 메모장	EmbeddingCache 임베딩 메모장
기록하는 것	LLM이 만든 완성된 답변 문자열	"연차 규정"처럼 텍스트를 숫자 벡터 로 변환한 결과
유효기한	1시간(TTL). 규정이 바뀌면 답도 바뀌어야 하니까	없음 . 텍스트가 같으면 벡터 값은 언제 계산해도 같음
저장 위치	메모리 디렉터리 (서버 재시작 시 소멸)	디스크 파일 <code>cache_dir/{sha256}.pkl</code>
메서드 구조	<code>get()</code> + <code>set()</code> 두 번 호출	<code>get_or_compute(text, compute_fn)</code> 한 번이면 끝

마지막 행이 구현 차이를 만듭니다. 답변 메모장은 `get()` 이 MISS를 반환하면 호출자가 "그럼 LLM을 돌리자"를 직접 결정해야 합니다. 임베딩 메모장은 **MISS일 때 무엇을 계산할지**(`compute_fn`)를 미리 받아 두기 때문에 호출자는 결과만 기다리면 됩니다.

`compute_fn` 이란? 캐시 미스일 때 값을 계산할 함수입니다. "연차 규정" 같은 텍스트를 받으면 768차원 벡터를 돌려주는 임베딩 함수를 여기에 넘깁니다. `EmbeddingCache` 가 미스 시 `compute_fn(text)` 로 호출해 계산하고 결과를 파일에 저장합니다.

파일 I/O의 귀찮은 부분(해시 파일명, `pickle` 직렬화, 히트·미스 카운팅)은 `_cache_utils.py` 의 `embedding_get` · `embedding_set` 이 말합니다. 우리는 이 두 헬퍼를 어떤 순서로 호출할지만 결정합니다.

실습 1 ex07/src/cache.py. EmbeddingCache

```
class EmbeddingCache:
    """로컬 파일 기반 임베딩 캐시."""

    def __init__(self, cache_dir=DEFAULT_EMBEDDING_CACHE_DIR):
        self.cache_dir = Path(cache_dir)
        self.cache_dir.mkdir(parents=True, exist_ok=True)
        self._hits = 0
        self._misses = 0
        logger.info("[EmbeddingCache] 초기화 완료 (캐시 디렉토리: %s)", self.cache_dir)

    def get_or_compute(self, text, compute_fn):
        """캐시 히트면 반환, 미스면 compute_fn으로 계산 후 저장합니다."""
        # TODO: get_or_compute - 캐시 히트면 반환, 미스면 계산 후 저장
        emb, hits_delta, misses_delta = embedding_get(self.cache_dir, text, None)
        self._hits += hits_delta
        self._misses += misses_delta

        if emb is not None:
            return emb

        emb = compute_fn(text)
        embedding_set(self.cache_dir, text, emb)
        return emb

    def stats(self):
        """캐시 통계를 반환합니다."""
        return embedding_cache_stats(self)
```

`make_response_key`, `embedding_get`, `embedding_set` 는 `_cache_utils.py` 의 보조 함수입니다. 해시 계산, 파일 이름 규칙, `pickle` 직렬화 등 지루한 I/O는 전부 헬퍼가 맡고, 우리는 TTL 검사와 용량 관리 로직만 다루는 구조입니다.

7.3 TokenTracker — 업무 일지를 쓴다

화이트보드의 두 번째 네모가 비어 있었습니다. 동료가 던졌던 두 번째 불만이 떠올랐습니다. 얼마나 쓰고 있는 건지 파악이 안 돼요.

오픈이 : "사용량 추적이 안 되고 있어요. 하루에 몇 건 처리하는지, 응답 시간은 평균 얼마인지 전혀 모릅니다."

팀장 : "사서한테 업무 일지를 쓰게 해요."

업무 일지. 매 호출마다 한 줄씩 기록을 남기는 것입니다.

- 오늘 총 몇 건 처리했는가
- 입력, 출력 토큰을 얼마나 썼는가
- 평균 응답 시간은 얼마인가
- API 비용은 얼마가 나갔는가

실무에서는 **Langfuse**, **LangSmith** 같은 전문 모니터링 도구가 이 역할을 합니다. 호출별 토큰, 비용, 지연을 자동 수집해 대시보드로 보여 줍니다. 하지만 도구를 붙이기 전에 **무엇을 측정해야 하는지**를 먼저 이해해야 합니다. 이 책에서는 `TokenTracker` 라는 간단한 클래스를 직접 만들어 감을 잡아 봅니다.

TokenTracker 왜 직접 만드나? Langfuse를 바로 붙이면 대시보드는 예쁘게 나오지만 "무엇이 수집되는지, 왜 그 값이 필요한지"를 이해하지 못한 채 지나가기 쉽습니다. 이 책에서는 **호출 단위로 남길 필드 4개**(입력 토큰·출력 토큰·응답 시간·비용)를 직접 쌓아 보면서 모니터링의 기본기를 체감한 뒤, 실무에서 Langfuse로 교체하는 순서를 권합니다.

실습 2. monitoring.py

`ex07/src/monitoring.py` 의 `TokenTracker` 는 세 부분으로 읽어 나갑니다. **단가 표 → record() 메서드 → 헬퍼 위임 메서드**. 우리가 채울 TODO는 `record()` 한 곳이며, `pass` 를 지우고 아래 코드를 작성합니다.

먼저 **클래스 골격**입니다. 모델별 단가 표는 이미 선언돼 있고, `__init__` 에서는 기록 리스트·누적 합계 카운터·로거만 준비합니다.

실습 2 ex07/src/monitoring.py. TokenTracker — 골격

```
class TokenTracker:
    """LLM API 호출별 토큰 사용량을 추적합니다."""

    # 1. 모델별 단가 표 (달러/1000토큰, 참고용)
    COST_PER_1K_TOKENS = {
        "gpt-4o-mini": {"input": 0.00015, "output": 0.0006},
        "gpt-4o": {"input": 0.005, "output": 0.015},
        "deepseek-r1:8b": {"input": 0.0, "output": 0.0}, # 로컬 모델: 무료
        "llama3.1:8b": {"input": 0.0, "output": 0.0}, # 로컬 모델: 무료
    }

    def __init__(self):
        # 2. 개별 호출 기록 리스트
        self._records = []
        # 3. 누적 토큰 합계 - summary()가 빠르게 읽기 위해 미리 계산
        self._total_input_tokens = 0
        self._total_output_tokens = 0
        self._logger = logging.getLogger(__name__)
```

로컬 모델(deepseek-llama)은 비용이 0이라 단가 표에 0.0 으로 못 박혀 있습니다. OpenAI 모델은 공식 요금제에서 가져온 값입니다. `record()` 에서 이 표를 보고 비용을 계산합니다.

다음은 **TODO가 있는** `record()` 메서드입니다. 한 호출마다 세 가지를 처리합니다. 비용 계산 → 기록 딕셔너리 조립 → 누적 카운터 갱신.

실습 2

ex07/src/monitoring.py. TokenTracker.record

```
def record(self, model, input_tokens, output_tokens, operation="chat", latency_ms=0.0):
    """토큰 사용량을 기록합니다."""
    # TODO: record - 호출별 토큰·비용·지연 누적
    # 1. 모델별 단가로 비용 계산 (헬퍼 위임)
    cost_usd = calculate_cost(model, input_tokens, output_tokens, self.COST_PER_1K_TOKENS)

    # 2. 이번 호출의 메타데이터를 딕셔너리 한 건으로 조립
    record = {
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "model": model,
        "operation": operation,
        "input_tokens": input_tokens,
        "output_tokens": output_tokens,
        "total_tokens": input_tokens + output_tokens,
        "cost_usd": cost_usd,
        "latency_ms": round(latency_ms, 2),
    }

    # 3. 기록 리스트에 추가하고 누적 카운터 갱신 + 로그 한 줄
    self._records.append(record)
    self._total_input_tokens += input_tokens
    self._total_output_tokens += output_tokens

    self._logger.info(
        "[TokenTracker] 사용량 기록: model=%s, input=%d, output=%d, cost=$%.6f, latency=%.0fms",
        model, input_tokens, output_tokens, cost_usd, latency_ms,
    )

    # 4. Rich Table로 호출 요약을 콘솔에 출력 - 라벨 12자 폭 고정으로 값 컬럼 정렬
    table = Table(
        title="TokenTracker 기록",
        title_style="bold cyan",
        title_justify="left",
        show_header=False,
        box=None,
        pad_edge=False,
        padding=(0, 1),
    )
    table.add_column(style="dim", width=12, no_wrap=True)
    table.add_column(style="bold")
    table.add_row("모델", model)
    table.add_row("입력 토큰", f"{input_tokens:,}")
    table.add_row("출력 토큰", f"{output_tokens:,}")
    table.add_row("비용 (USD)", f"${cost_usd:.6f}")
```



```
table.add_row("지연 시간", f"{latency_ms:.0f} ms")
console.print(table)
```

핵심은 3단계의 **누적 갱신**입니다. `_records`에 한 건씩 쌓는 건 "최근 호출 내역"을 보여 주기 위함이고, `_total_*`은 `summary()`가 매번 리스트를 합산하지 않고 바로 꺼내 쓰도록 미리 합쳐 두는 값입니다. 4단계의 **Rich Table 출력**은 운영 로그 가독성을 위해 엮은 한 단계입니다. `Table`과 `console`은 파일 상단에서 `from rich.table import Table`, `from ._rich_logging import console`로 이미 불러와 있어 그대로 쓰면 됩니다. 캡처 이미지에서 본 "TokenTracker 기록" 상자가 이 몇 줄에서 나옵니다.

마지막으로 **조회 메서드 두 개**는 이미 완성돼 있습니다. 우리가 구현할 부분은 없고 헬퍼로 위임만 합니다.

설명 ex07/src/monitoring.py. TokenTracker.summary / recent

```
def summary(self):
    """누적 토큰 사용량 요약을 반환합니다."""
    return token_summary(self)

def recent(self, n=5):
    """최근 n개의 호출 기록을 반환합니다."""
    return token_recent(self, n)
```

`calculate_cost`, `token_summary`, `token_recent`는 `_monitoring_utils.py`의 헬퍼입니다. 비용 공식(`input_tokens * input_price / 1000 + output_tokens * output_price / 1000`)과 요약 계산은 헬퍼가 맡고, 우리는 **기록을 쌓는 부분만** 구현합니다.

7.4 Retry와 ConnectHRAgent — 운영용 래퍼

세 번째 네모에는 슬랙의 "에러가 났어요" 메시지가 떠올랐습니다. 팀장이 마커로 그 위에 작게 적었습니다. **재시도 매뉴얼**.

팀장: "사서한테 매뉴얼 한 장 줘여 줘요. '한 번 실패하면 3번까지 다시 시도해. 시도 사이에 2초씩 쉬고.'"

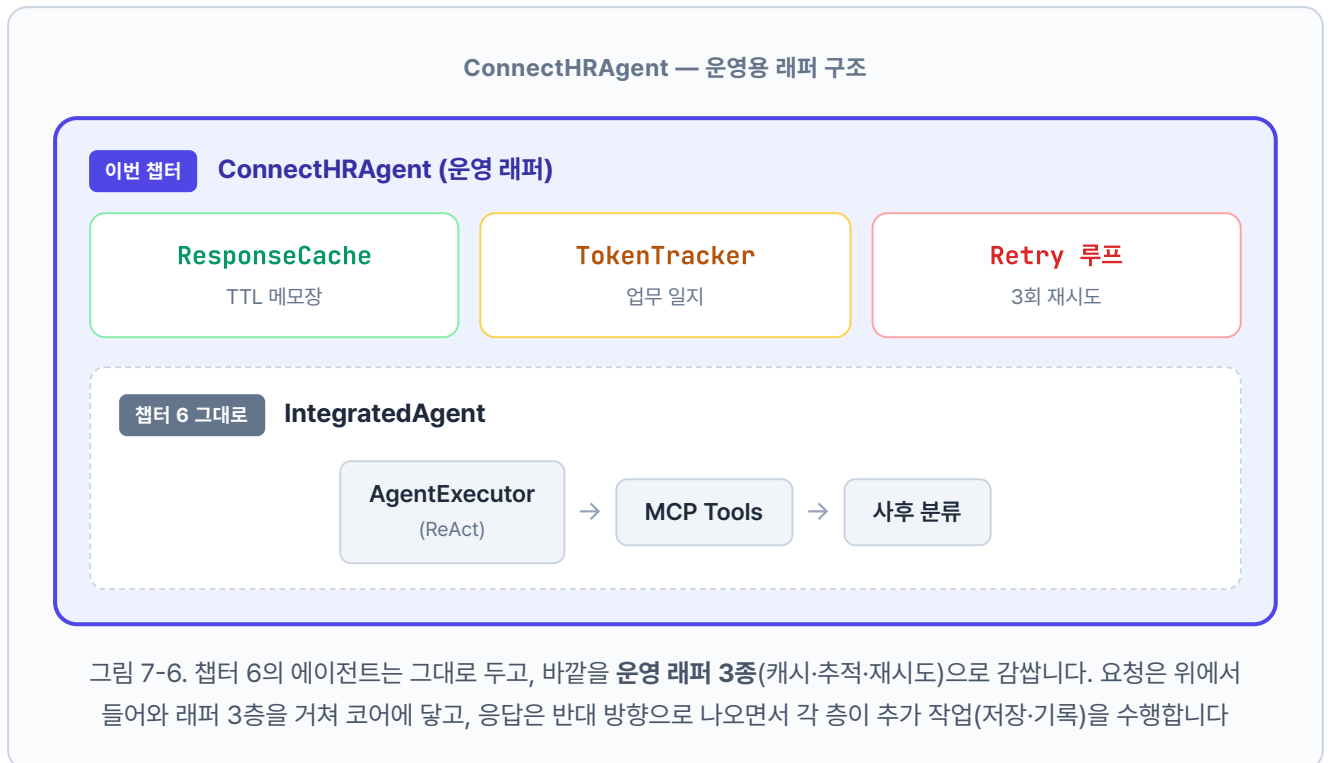
오픈이: "그 정도면 일시적 오류는 대부분 흡수되겠네요."

네트워크가 잠깐 끊기거나 모델이 과부하에 걸리는 일시적 문제는 잠깐 뒤에 다시 시도하면 대부분 풀립니다. 점검 중이던 선반도 잠깐 뒤엔 열려 있을 수 있기 때문입니다.

이 재시도 규칙까지 합쳐 챗터 6의 `IntegratedAgent` 에 캐시·모니터링·재시도를 얹은 버전이 **ConnectHRAgent**입니다. 새 기능을 추가하는 것이 아닙니다. 기존 에이전트를 **안정적으로** 운영할 수 있게 감싸는 것이 목표입니다.

기능	역할	파일
응답 캐시	답변 메모장 (1시간 TTL)	<code>cache.py</code>
임베딩 캐시	단어 → 벡터 변환 결과 저장	<code>cache.py</code>
토큰 추적	사서의 업무 일지	<code>monitoring.py</code>
재시도 로직	"3번까지 다시 시도"	<code>agent_config.py</code>

도구도 구조가 바뀝니다. 챗터 6에서 `mcp_tools.py` 하나에 몰아넣었던 도구 4개를 `tools/` 디렉토리 아래 파일별로 분리했습니다(`leave_balance.py`, `sales_sum.py`, `list_employees.py`, `search_documents.py`). 도구가 늘어도 파일 하나씩 추가하면 되는 구조라 관리하기 쉽습니다.



실습 3. agent_config.py

ex07/src/agent_config.py의 ConnectHRAgent.run() 메서드 TODO의 pass를 지우고 5단계 운영 파이프라인을 구현합니다. 챗터 6의 IntegratedAgent 엔진(AgentExecutor + ReAct 루프)은 그대로 쓰고, 바깥에 캐시·재시도·토큰 기록을 두르는 게 이 챗터의 핵심입니다. Router는 ex06에서 하던 것처럼 질문 유형 라벨을 붙이는 용도로만 호출합니다. 재시도 루프와 AgentExecutor 빌드는 _agent_utils.py의 build_agent_executor() · run_with_retry() 헬퍼가 맡고, 우리는 단계를 조립하는 데만 집중하면 됩니다.

run_with_retry는 상수 두 개로 동작을 조절합니다. RETRY_MAX_ATTEMPTS (기본값 3)가 최대 시도 횟수, RETRY_DELAY_SECONDS (기본값 2.0)가 시도 사이 대기 시간입니다. 두 값 모두 _agent_utils.py 상단에 선언돼 있고 챗터 6의 에이전트 실행 로직을 이 루프 안으로 옮겨 넣습니다.

참고 ex07/src/_agent_utils.py. 재시도 루프 (완성 코드)

```
# 재시도 상수
RETRY_MAX_ATTEMPTS = 3      # 최대 시도 횟수
RETRY_DELAY_SECONDS = 2.0   # 시도 사이 대기 시간 (초)

def run_with_retry(agent_executor, query, chat_history=None):
    last_error = None
    # 1. 최대 3회까지 시도 루프
    for attempt in range(1, RETRY_MAX_ATTEMPTS + 1):
        try:
            # 2. Agent 실행 - 성공하면 결과를 그대로 반환
            result = agent_executor.invoke({
                "input": query,
                "chat_history": chat_history or [],
            })
            return result
        except Exception as exc:
            # 3. 실패 시 에러를 기억하고 경고 로그 남김
            last_error = exc
            logger.warning(
                "[ConnectHRAgent] 시도 %d/%d 실패: %s",
                attempt, RETRY_MAX_ATTEMPTS, exc,
            )
            # 4. 마지막 시도가 아니면 2초 쉬고 다시 도전
            if attempt < RETRY_MAX_ATTEMPTS:
                time.sleep(RETRY_DELAY_SECONDS)

    # 5. 3회 모두 실패 - 사용자에게 보여줄 안전한 에러 메시지를 반환
    return {"output": f"처리 중 오류가 발생했습니다: {last_error}"}
```

`run_with_retry` 는 최대 3회까지 `agent_executor.invoke` 를 재시도하고 시도 사이에 2초씩 쉽니다. 매 실패에 `logger.warning` 으로 몇 번째 시도가 왜 실패했는지 남기고, 전부 실패하면 에러 메시지를 담은 결과를 반환합니다. 이제 `ConnectHRAgent.run()` 차례입니다.

실습 3 ex07/src/agent_config.py. ConnectHRAgent.run

```
class ConnectHRAgent:
    def run(self, query, chat_history=None, use_cache=True):
        """사용자 질문을 처리하고 답변을 반환합니다."""
        # TODO: run - 캐시 → 분류 → 에이전트 실행(재시도) → 토큰 기록 → 캐시 저장 (5단계)
        start_time = time.time()
        logger.info("[ConnectHRAgent] 질문 수신: %s", query[:80])

        # 1. 캐시 조회 - 같은 질문이면 LLM·DB를 건너뛰고 즉시 반환
        if use_cache:
            cached = response_cache.get(query)
            if cached is not None:
                cached["from_cache"] = True
                logger.info("[ConnectHRAgent] 캐시 응답 반환")
                return cached

        # 2. Router로 질문 유형 분류 - 라벨(structured/unstructured/hybrid)만 붙임
        query_type = self._router.classify_query(query)

        # 3. Agent 실행 - ex06의 IntegratedAgent 엔진 그대로, run_with_retry가 최대 3회 재시도
        if self.agent_executor is not None:
            result = run_with_retry(self.agent_executor, query, chat_history)
        else:
            result = {
                "output": "죄송합니다. 에이전트 서비스를 사용할 수 없습니다.",
                "intermediate_steps": [],
            }
        result["query_type"] = query_type
        result["from_cache"] = False

        # 4. 토큰 사용량 기록 - 모델·입출력 토큰·지연시간을 TokenTracker에 누적
        latency_ms = (time.time() - start_time) * 1000
        provider = os.getenv("LLM_PROVIDER", "ollama").lower()
        if provider == "openai":
            model = os.getenv("OPENAI_MODEL", "gpt-4o-mini")
        else:
            model = os.getenv("OLLAMA_MODEL", "deepseek-r1:8b")
        token_tracker.record(
            model=model,
            input_tokens=len(query.split()) * 2,
            output_tokens=len(result["output"].split()) * 2,
            operation="agent_run",
            latency_ms=latency_ms,
        )

        # 5. 캐시 저장 - 다음에 같은 질문이 오면 1단계에서 바로 꺼내 쓰도록
        if use_cache:
            response_cache.set(query, result)
```

```
logger.info(
    "[ConnectHRAgent] 처리 완료 (유형: %s, 소요: %.0fms)",
    query_type,
    latency_ms,
)
return result
```

1단계와 **5단계**가 실습 1에서 만든 `response_cache` 와 맞물리고, **4단계**가 실습 2의 `token_tracker` 에 기록을 쌓습니다. 챗터 6의 에이전트 실행 로직은 **3단계**의 `run_with_retry` 안으로 그대로 들어갔습니다. Router는 ex06에서처럼 질문 유형 라벨만 붙이고 실행 경로는 나누지 않으므로, 검색 도구(`search_documents`)가 반환한 문서는 AgentExecutor의 `intermediate_steps` 에 고스란히 남아 응답 UI의 근거 패널로 흘러갑니다. 결국 `run()` 메서드 하나가 **캐시·분류·재시도·모니터링**을 순서대로 엮는 허브가 됩니다. 모델 이름은 `LLM_PROVIDER` 값에 따라 `OPENAI_MODEL` 또는 `OLLAMA_MODEL` 환경변수에서 가져오고, 토큰 수는 `단어 수 × 2` (한국어 근사)로 추정합니다. Ollama 응답에 토큰 메타가 없어서 쓰는 임시 계산입니다.

7.5 서버 실행 + 캐시 적중 확인

챗터 6의 `ex06` 서버가 켜져 있다면 `Ctrl+C` 로 먼저 종료합니다. `ex07` 도 같은 8000 포트를 사용하기 때문에, 이전 서버를 내리지 않으면 새 서버가 뜨지 않습니다.

터미널 서버 실행

```
python run.py
```

브라우저에서 `http://localhost:8000/chat` 을 열고, 동료가 처음 했던 질문을 그대로 두 번 던져 보세요.

첫 호출. LLM을 실제로 돌리므로 20초 근처.

![../assets/챗터 7/terminal/07_log-first-query.png] 그림 7-7. 첫 질문 로그. 입력, 출력 토큰과 소요 시간이 TokenTracker에 기록됩니다

두 번째 호출. ResponseCache HIT.

 그림 7-8. 같은 질문에 캐시 HIT. 0.1초 이내 응답 반환

첫 호출과 캐시 적중을 나란히 비교하면 비용 차이가 한눈에 보입니다.

	첫 질문	캐시 적중
LLM 호출	2회 (분류 + 답변)	0회
DB 조회	1회	0회
소요 시간	약 16초	즉시 반환
로그	AgentExecutor 체인 전체 출력	<code>[ResponseCache]</code> 적중 한 줄

`잔여 TTL: 3386초`. 메모장에 적힌 지 약 3분이 지났다는 뜻입니다($3600 - 3386 = 214$ 초). TTL이 0이 되면 메모가 만료되고, 다음에 같은 질문이 들어오면 다시 서가에서 찾아옵니다.

`/stats` 엔드포인트를 열면 호출마다 쌓인 누적 통계가 나옵니다. 실제 운영 환경이라면 이 화면이 하루, 일주일 단위로 의미 있어집니다.

 그림 7-9. 누적 호출 수, 토큰, 비용, 캐시 적중률 대시보드

같은 질문을 다시 던지자 로딩 스피너가 한 번도 뜨지 않고 답이 바로 나왔습니다. 20초가 0.1초로 줄었습니다. 오픈이는 의자 등받이에 기대며 모니터를 바라봤습니다.

이제 동료한테 한 소리 덜 들겠네.

슬랙에 스크린샷을 하나 올렸습니다. 같은 질문을 두 번 연달아 던진 타임스탬프가 0.1초 간격으로 찍혀 있었습니다.

동료: "캐시 적중률 72%네요. 10번 중 7번은 LLM을 안 돌린 거잖아요."

오픈이: "그리고 실패한 것도 재시도로 자동 복구됩니다. 이제 '고장났다'는 메시지가 줄어들 거예요."

7.6 더 알아보기

TTL을 얼마로 설정해야 할까요? 사내 문서가 자주 갱신되면 짧게(30분), 변동이 거의 없으면 길게(24시간) 잡습니다. 기본값 1시간은 대부분의 사내 환경에 무난합니다. `.env`의 `CACHE_TTL=3600` 값을 프로젝트마다 조정합니다.

서버를 재시작하면 ResponseCache가 사라지지 않나요? 맞습니다. 인메모리 딕셔너리라 프로세스 종료와 함께 날아갑니다. 운영 환경에서는 **Redis** 같은 외부 캐시로 교체하는 것이 일반적입니다.

`get()` · `set()` 메서드 안의 `self._store` 조작을 Redis 클라이언트 호출로 바꾸면 동일 인터페이스를 유지하면서 영속성을 얻을 수 있습니다.

토큰 수가 추정값인 이유는? Ollama는 응답 메타데이터에 토큰 사용량을 포함하지 않습니다. 그래서 이 책에서는 `단어 수 × 2`를 한국어 기준 대략적 추정치로 씁니다. 정확한 과금이 필요한 OpenAI API는 응답 객체의 `usage.prompt_tokens` / `usage.completion_tokens`를 읽어 실제 값으로 대체하면 됩니다.

Router로 실행 경로를 나누면 더 빠르지 않을까요? 비정형 질문은 문서 검색 체인으로, 정형 질문은 DB 도구로 경로를 나눠서 쓰면 ReAct 한 단계를 절약할 수 있습니다. 다만 그렇게 분기를 두면 답변은 체인에서 나오고 근거 문서는 파이프라인 안에 머물러 응답 UI의 근거 패널과 연결이 끊깁니다. 챗터 7은 ex06의 에이전트 엔진을 그대로 쓰면서 캐시·재시도·토큰 기록만 덧씌우는 게 목표이므로, Router는 질문 유형 라벨 용도로만 남기고 실행은 AgentExecutor 단일 경로로 유지합니다. 검색 도구가 찍은 결과는 `intermediate_steps`에 자동으로 남아 근거 패널이 그대로 작동합니다. 경로 분기로 속도를 짜내는 튜닝은 챗터 8부터 다루는 검색 품질 최적화 영역입니다.

7.7 전체 구성도에서 챗터 7의 자리

전체 구성도. 짙은 박스가 챗터 7 범위

사내 직원 · 관리자

챕터 2

대시보드

FastAPI

REST API · 관리자 웹

챕터 6 + 챕터 7

오케스트레이션 + 운영

+ 캐시 · 재시도

ConnectHRAgent

ReAct · 사후 분류 + TokenTracker

+ 임베딩 캐시

MCP Tools

DB 조회 · 문서 검색

챕터 5

검색 (실시간)

LCEL Chain

검색기 → 프롬프트 → LLM

챕터 4

파싱·벡터화 (오프라인)

챕터 3 문서 규칙

PDF·Word·Excel·HWP

Doc Pipeline

파싱·청킹·임베딩

ChromaDB

벡터 저장소

챕터 2

데이터

PostgreSQL

직원·연차·매출

Ollama LLM (외부)

챕터 7은 새 박스를 엮지 않고 **챕터 6 오케스트레이션 세 박스를 운영 래퍼로 감싼 챕터**입니다. 같은 에이전트지만 이제는 캐시, 토큰 추적, 재시도가 붙어 실제 배포에 견딥니다. 챕터 8부터는 검색 품질 튜닝(Hybrid Search, Reranker) 영역으로 넘어갑니다.

종료합니다

`Ctrl + C` 로 서버를 종료합니다. Docker 컨테이너는 챕터 8에서도 같은 PostgreSQL·ChromaDB를 쓰므로 그대로 두세요.

용어 정리

본문 속 표현	진짜 용어	정식 정의
"답변 메모장"	ResponseCache	TTL 기반 인메모리 응답 캐시. SHA-256 해시 키로 질문을 식별
"임베딩 메모장"	EmbeddingCache	파일 기반 임베딩 벡터 캐시. pickle로 저장·유지
"업무 일지"	TokenTracker	LLM 호출별 입출력 토큰·비용· 응답시간을 누적 집계
"3번까지 다시 시도"	Retry 로직	LLM 호출 실패 시 최대 N회까지 재시도. 간격 포함
"운영 래퍼"	ConnectHRAgent	챕터 6의 IntegratedAgent에 캐시· 모니터링·재시도를 통합한 운영용 에이전트

이것만은 기억하자

- **같은 질문은 같은 답이 빠르게.** ResponseCache(TTL)로 20초 → 0.1초. 하지만 TTL 만료, 규정 개정 시엔 무효화가 필요하다는 점을 잊지 마세요.
- **측정이 개선의 시작입니다.** TokenTracker로 토큰, 비용, 응답시간을 쌓아 두면 "느린 질문", "비싼 질문"이 눈에 보입니다. 실무에서는 Langfuse, LangSmith 같은 도구를 얹어 대시보드화합니다.
- **실패는 기본입니다.** 네트워크 오류, 파싱 실패가 드물지 않습니다. Retry 루프(3회, 2초 간격)만 있어도 체감 장애 빈도가 크게 줄어듭니다. 챕터 8부터는 이 위에서 **검색 품질 자체**를 튜닝합니다.

챕터 8. 엉뚱한 문서를 가져온다. 청킹, 리랭킹, 하이브리드

이번 챕터가 끝나면

- **Fixed, Recursive, Semantic** 세 가지 청킹 전략을 비교 실험합니다
- **Cross-Encoder Reranker**로 Top-K 검색 결과를 재정렬해 정확도를 끌어올립니다
- **BM25 + Vector** 하이브리드 검색과 **alpha 가중합**으로 키워드와 의미 검색을 결합합니다
- "병가를 물었는데 출장이 나오는" 현상이 왜 생기는지, 어떻게 잡는지 손으로 확인합니다

준비하기. 실습 시작 전 한 번만 설정

1. 실습 폴더 이동

터미널 폴더 이동

```
cd rag-start/ex08
```

파일 구조는 다음과 같습니다.

ex08 디렉토리

```

ex08/
├── docker-compose.yml
├── requirements.txt
├── data/
├── tuning/
│   ├── step1_chunk_experiment/
│   │   ├── __main__.py
│   │   ├── strategies.py
│   │   ├── experiments.py
│   │   └── display.py
│   ├── step2_reranker/
│   │   ├── __main__.py
│   │   ├── reranker.py
│   │   └── experiments.py
│   └── step3_hybrid_search/
│       ├── __main__.py
│       ├── retrievers.py
│       └── experiments.py
└── # [참고] 문서·마크다운·ChromaDB
    # [실습] 청킹 전략 비교
    # [참고] CLI 진입점
    # [실습] Fixed·Recursive·Semantic 3전략
    # [참고] 실험 실행기 (step 1-1~1-5)
    # [참고] Rich 출력
    # [실습] Cross-Encoder 리랭커
    # [참고] CLI 진입점
    # [실습] CrossEncoderReranker
    # [참고] 리랭킹 실험 실행기
    # [실습] BM25 + Vector 하이브리드
    # [참고] CLI 진입점
    # [실습] BM25·Vector·Ensemble 검색기
    # [참고] alpha 파라미터 실험

```

2. 실습 환경 구축

터미널 환경 구성. macOS / Linux

```

cd ex08
cp .env.example .env
python3.12 -m venv .venv
source .venv/bin/activate
docker compose up -d
pip install -r requirements.txt

```

터미널 환경 구성. Windows

```

cd ex08
copy .env.example .env
py -3.12 -m venv .venv
.venv\Scripts\activate
docker compose up -d
pip install -r requirements.txt

```

3. 사용할 라이브러리

패키지	역할
<code>langchain-text-splitters</code>	Recursive-Character-Token 스플리터
<code>langchain-experimental</code>	SemanticChunker (의미 기반 청킹)
<code>sentence-transformers</code>	임베딩 + Cross-Encoder (BAAI/bge-reranker-v2-m3)
<code>rank-bm25</code>	BM25 알고리즘 (키워드 기반)
<code>chromadb</code>	벡터 DB

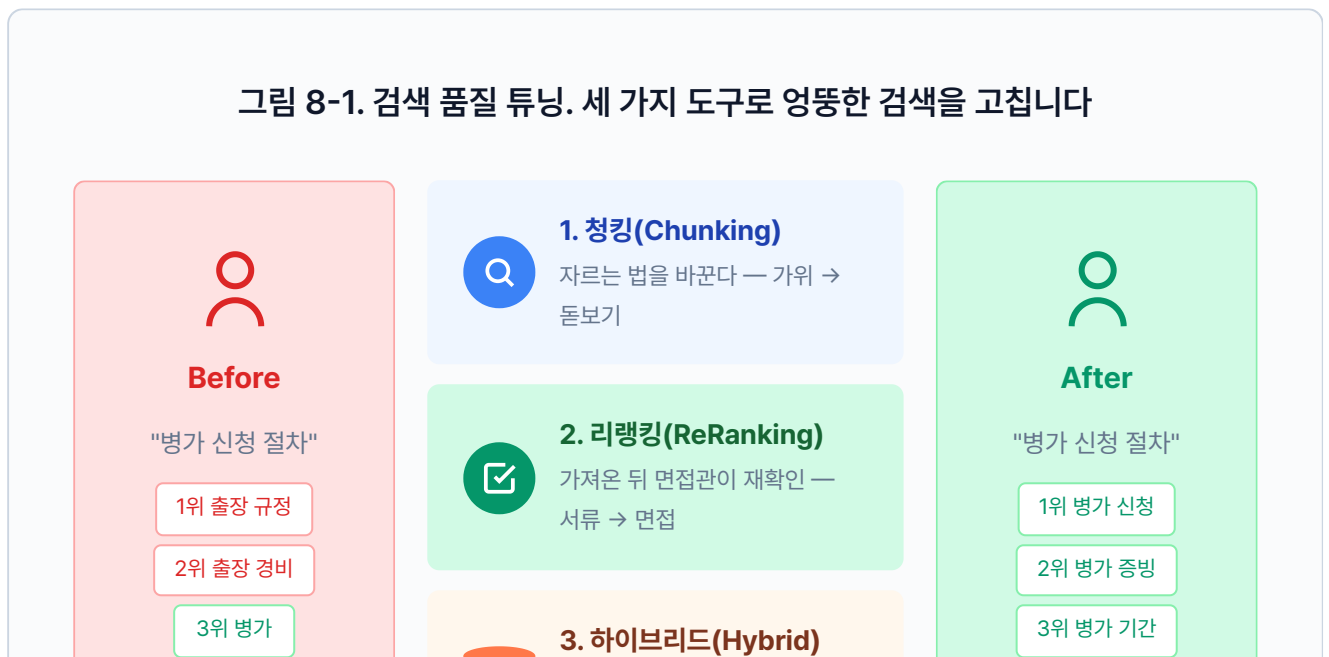
4. 실습 순서

이번 챕터는 **만들기**가 아니라 **실험, 비교**입니다.

1. `python -m tuning.step1_chunk_experiment --step 1-3`. 청킹 전략 비교
2. `python -m tuning.step2_reranker`. Cross-Encoder 리랭커
3. `python -m tuning.step3_hybrid_search`. BM25 + Vector 하이브리드

8.1 "병가를 물었는데 출장 규정이 나왔다"

그림 8-1. 검색 품질 튜닝. 세 가지 도구로 엉뚱한 검색을 고칩니다



상위 3건 중 2건이 출장



두 검색을 합친다 — 의미 + 키워드

상위 3건 모두 병가

챗터 7에서 운영까지 안정화한 뒤 이틀이 지났습니다. 캐시도 돌고 토큰도 추적합니다. 슬랙 채널에 "고장났어요" 메시지가 뜬해진 걸 보며 오픈이는 커피잔에 손을 감쌌습니다. 형광등이 켜지고 한 번 깜빡이더니 다시 안정됐습니다.

동료 : "이거 좀 봐요."

동료가 의자를 끌고 왔습니다. 노트북 화면에 커넥트HR 채팅창이 떠 있었습니다.

동료 : "병가 규정이 뭔지 물었는데, 출장 규정을 가져와서 답하더라고요."

뭐라고?

오픈이가 동료의 노트북을 당겨 봤습니다. 화면에는 "병가 신청 절차를 알려줘"라는 질문 아래 출장 규정이 장황하게 달려 있었습니다. 터미널을 열어 검색 로그를 확인했습니다.

검색 결과 — "병가 신청 절차"

1

HR_취업규칙_v1.0 (p.3)

유사도 85.1%

"제14조(출장) 업무상 출장 시 소요 경비는 실비로 정산하며...제15조(병가) 질병이나 부상으로"

2

HR_취업규칙_v1.0 (p.4)

유사도 82.3%

"출장 기간 중 질병이 발생한 경우 해당 기간은 병가로 전환할 수 있으며..."

3

HR_출장규정_v2.0 (p.1)

유사도 79.8%

"국내 출장은 당일 및 숙박 출장으로 구분하며..."

상위 3건 중 2건이 출장 규정이었습니. 1위 문서를 자세히 보니 이유가 보였습니다. 챗터 4에서 500자마다 기계적으로 잘라 둔 청크에서 **병가 규정 뒷부분과 출장 규정 앞부분이 한 덩어리로** 섞여 있었습니다.

동료 : "에이전트가 멍청한 건가요?"

오픈이 : "아니요, LLM은 받은 문서로 성실하게 답한 거예요. 엉뚱한 문서를 가져다 준 건 검색 단계입니다."

에이전트도 다듬고, 캐시도 붙이고, 모니터링까지 달았는데. 검색의 뿌리가 썩어 있으면 그 위에 뭘 얹어도 소용없다.

건너편 자리에서 팀장이 모니터에서 눈을 떼지 않은 채 한마디를 던졌습니다.

팀장 : "사서가 책을 잘 찾아주냐, 책장을 잘 정리했냐. 뭐가 먼저겠어요?"

8.2 이 챕터의 세 실험

책장 정리. 팀장의 한마디가 머릿속에 걸렸습니다. 검색 결과 로그를 다시 열어 보니 문제가 하나가 아니었습니다. 서랍에서 수첩을 꺼내 펼쳤습니다.

— 문제 정리 —

1. 자르는 방식

- 500자 고정이라 병가 규정 뒷부분과 출장 규정 앞부분이 한 덩어리
- 글자 수가 아니라 의미 단위로 잘라야 함

2. 가져온 뒤 확인

- 유사도 순위 그대로 믿고 있음
- 진짜 답과 먼 문서도 상위에 섞여 올라옴

3. 검색 방법 자체

- 벡터 검색 하나만 쓰고 있음
- 키워드가 정확히 맞는 경우도 놓칠 수 있음

수첩을 덮고 화이트보드 앞으로 걸어갔습니다. 마커 뚜껑을 딸깍 열고 세 칸을 그렸습니다.

이번 챕터는 새 기능을 만들지 않습니다. 챕터 4부터 쌓아 온 검색 파이프라인을 세 방향에서 실험하고 개선합니다.

그림 8-2. 챕터 8의 세 실험. 청킹(Chunking) → 리랭킹(ReRanking) → 하이브리드 검색(Hybrid Search)



8.3 가위 대신 돋보기를 쥐어 주자 — 청킹(Chunking)

오픈이 : "지금 사서는 책을 500자씩 가위로 잘라서 서랍에 넣어 둔 거거든요. 가위가 병가 조항 중간을 지나가면서 출장 규정이란 한 조각이 된 거예요."

팀장 : "그럼 가위 대신 뭘 쥐어 줄 수 있을까요?"

잠시 생각했습니다. 문서에는 구조가 있습니다. 빈 줄, 제목, 주제 전환. 그걸 읽을 줄 아는 사서라면 엉뚱한 곳에서 자르지 않을 겁니다.

가위가 아니라 돋보기를 쥐야 한다.

오픈이 : "자르는 방법부터 바꿔야 할 것 같아요. 세 가지 방법을 보여 드릴게요."

오픈이가 화이트보드에 사내 규정 텍스트를 적었습니다. "제15조 병가... 제16조 출장... 제17조 연차..." 마커 세 자루를 꺼내 놓았습니다.

실습 흐름

1. `tuning/step1_chunk_experiment/strategies.py` 의 TODO를 채웁니다
2. `fixed_size_chunking()` 작성 → `--step 1-1` 로 크기×오버랩 조합 실험
3. `recursive_character_chunking()` 작성
4. `semantic_chunking()` 작성
5. `--step 1-2` 로 세 전략을 같은 문서에 돌려 한 번에 비교

전략 1. 고정 청킹(Fixed Chunking)

오픈이가 빨간 마커를 집었습니다. 자를 꺼내 화이트보드에 대고 500자마다 줄을 그었습니다. 병가 규정 뒤쪽과 출장 규정 앞쪽이 한 칸에 들어갔습니다.

동료 : "이러니까 섞이죠."

제목만 보고 찾는 사서. 챕터 4에서 우리가 만든 사서입니다. 문서를 500자 단위로 기계적으로 잘라 둡니다. "병가"와 "출장"이 같은 문서에 있으면 한 청크에 두 내용이 섞여요.

`tuning/step1_chunk_experiment/strategies.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 1 `tuning/step1_chunk_experiment/strategies.py`. 고정 크기 청킹

```
def fixed_size_chunking(text: str, chunk_size: int = 500, overlap: int = 100) → list[str]:
    # TODO: text를 chunk_size 단위로 잘라 리스트에 추가합니다.
    chunks = []
    start = 0
    # 1. start부터 chunk_size만큼 잘라냄
    while start < len(text):
        end = min(start + chunk_size, len(text))
        chunk = text[start:end]
        # 2. 공백만 있는 청크는 건너뛰
        if chunk.strip():
            chunks.append(chunk.strip())
        if end ≥ len(text):
            break
        # 3. 다음 시작 위치는 end - overlap (겹침 구간)
        start = end - overlap
    return chunks
```

`chunk_size` 는 한 조각의 최대 글자 수, `overlap` 은 조각과 조각이 겹치는 글자 수입니다. 겹침 구간을 두는 이유는 경계에서 잘린 문장이 양쪽 조각에 모두 포함되게 하려는 것입니다. 챕터 4에서 `chunk_size=500` , `overlap=100` 으로 설정했던 함수를 크기와 오버랩을 바꿔 가며 실험해 봅니다.

터미널 Fixed 청킹 크기 × 오버랩 조합 비교 (9개 조합)

```
python -m tuning.step1_chunk_experiment --step 1-1
```

이 실험에서 알아야 할 것

- **크기가 크면(1000자)**: 정보가 많아 LLM이 답하기 좋지만, 관련 없는 내용도 섞이고 토큰도 많이 씁니다.
- **크기가 작으면(300자)**: 주제가 깔끔하지만, 맥락이 잘려서 "이게 무슨 말이지?" 하는 청크가 생깁니다.
- **오버랩이 크면(30%)**: 경계에서 문맥이 끊기지 않지만, 같은 내용이 여러 청크에 중복됩니다.
- **오버랩이 작으면(10%)**: 중복이 적지만, 경계에서 잘린 문장이 검색에 안 걸릴 수 있습니다.

사내 규정처럼 조항이 짧은 문서는 300~500자, 매뉴얼처럼 한 주제가 긴 문서는 800~1000자가 적합합니다.

전략 2. 재귀 청킹(Recursive Chunking)

오픈이가 파란 마커로 바꿨습니다. 이번에는 자를 치우고, 텍스트를 읽으며 빈 줄이 있는 곳에 줄을 그었습니다. 제15조, 제16조, 제17조가 각각 한 칸씩 분리됐습니다.

팀장 : "그런데 빈 줄이 없는 문서도 있잖아요."

오픈이 : "그래서 빈 줄 → 줄바꿈 → 마침표 → 공백 순서로 찾아요. 가장 자연스러운 경계부터 시도합니다."

문단을 보고 찾는 사서. 문서에는 보통 빈 줄이 있습니다. "제1조"와 "제2조" 사이, 단락 전환 지점이요. 이 사서는 빈 줄이 보이면 먼저 거기서 끊습니다. 주제가 섞일 확률이 줄어요.

`tuning/step1_chunk_experiment/strategies.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

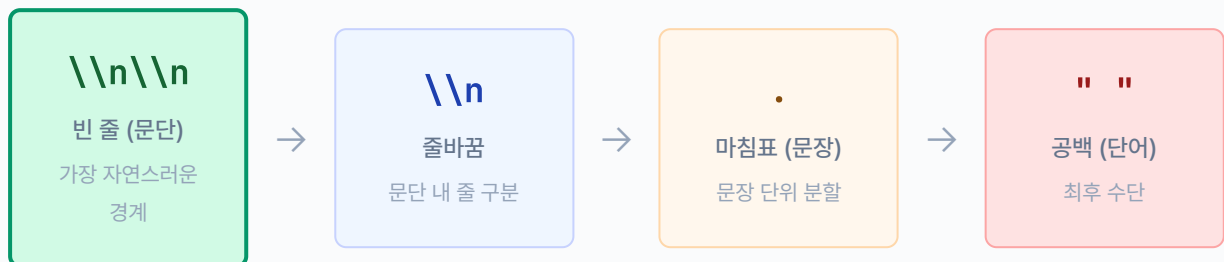
실습 1 tuning/step1_chunk_experiment/strategies.py. 재귀 문자 분할

```
def recursive_character_chunking(text: str, chunk_size: int = 500, overlap: int = 100)
→ list[str]:
    # TODO: RecursiveCharacterTextSplitter로 문단·문장 경계를 우선 분할합니다.
    from langchain_text_splitters import RecursiveCharacterTextSplitter

    # 1. 분할 기준 우선순위: 빈 줄 → 줄바꿈 → 마침표 → 공백
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=overlap,
        separators=["\n\n", "\n", ".", ".", " ", ""],
    )
    # 2. 가장 자연스러운 경계부터 시도하며 분할
    return splitter.split_text(text)
```

`RecursiveCharacterTextSplitter`는 `langchain-text-splitters` 패키지가 제공하는 스플리터입니다. `separators`가 핵심인데, "자를 수 있는 가장 자연스러운 경계"를 우선순위대로 찾습니다. 같은 500자라도 문단이나 문장 경계에서 끊기니 주제 혼입이 줄어듭니다.

그림 8-3. Recursive Separators 탐색 순서



← 자연스러운 경계부터 먼저 시도 · 못 찾으면 다음 단계로 →

Fixed와 같은 `chunk_size`, `overlap` 파라미터를 쓰지만, 자르는 기준이 다릅니다. Fixed는 글자 수만 세지만 Recursive는 문단·문장 경계를 먼저 찾습니다.

전략 3. 의미 청킹(Semantic Chunking)

오픈이가 마지막으로 초록 마커를 꺼냈습니다. 이번에는 글자 수도, 빈 줄도 보지 않았습니다. 문장을 하나씩 읽으며 "여기서 주제가 바뀐다" 싶은 곳에 줄을 그었습니다.

동료 : "사람이 읽는 것처럼 하네요."

오픈이 : "임베딩으로 문장끼리 의미 거리를 재서, 거리가 확 벌어지는 곳에서 자르는 거예요."

목차를 보고 찾는 사서. 더 꼼꼼한 사서입니다. 문서를 자를 때 "여기서 주제가 바뀌네?"를 감지합니다. 연차 이야기에서 출장 이야기로 넘어가는 지점을 알아채고 거기서 끊어요. 빈 줄이 없어도 내용 흐름을 읽습니다.

`tuning/step1_chunk_experiment/strategies.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 1 `tuning/step1_chunk_experiment/strategies.py`. 의미 기반 청킹

```
def semantic_chunking(
    text: str,
    embedding_model: str = "jhgan/ko-sroberta-multitask",
    percentile: int = 70,
) → List[str]:
    # TODO: HuggingFaceEmbeddings + SemanticChunker로 의미 단위 청킹합니다.
    from langchain_community.embeddings import HuggingFaceEmbeddings
    from langchain_experimental.text_splitter import SemanticChunker

    # 1. 임베딩 모델 로드
    embeddings = HuggingFaceEmbeddings(model_name=embedding_model)
    # 2. percentile 기준선으로 의미 전환 감지
    chunker = SemanticChunker(
        embeddings,
        breakpoint_threshold_type="percentile",
        breakpoint_threshold_amount=percentile,
    )
    # 3. 의미가 크게 달라지는 곳에서 분할
    return chunker.split_text(text)
```

`HuggingFaceEmbeddings` 는 문장을 벡터로 바꿔 주는 래퍼이고, `SemanticChunker` 는 문장 벡터 사이의 거리를 계산해 분할 지점을 결정합니다. `embedding_model` 에 한국어 임베딩 모델 (`jhgan/ko-sroberta-multitask`) 을 넘기면 한국어 문맥을 이해합니다.

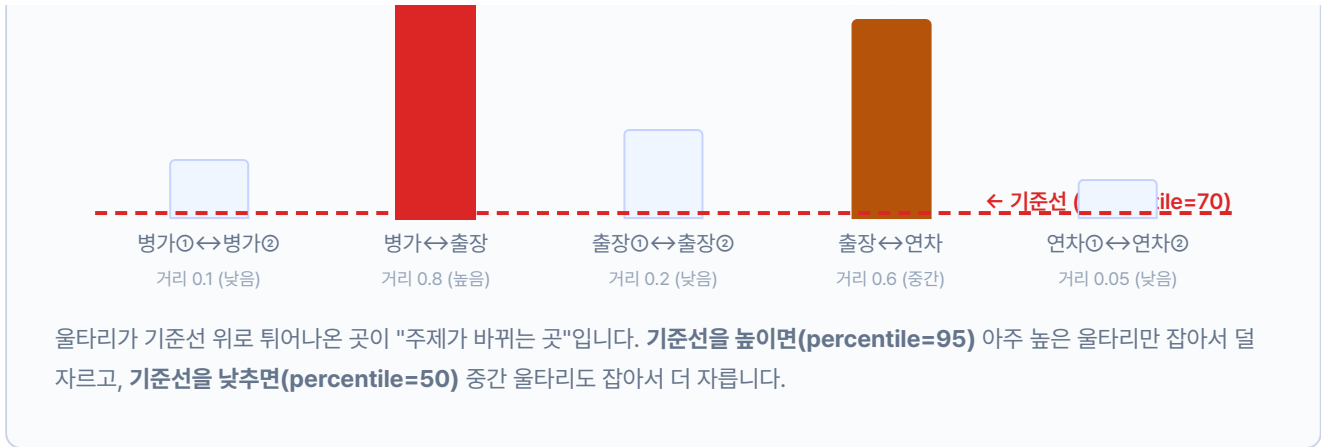
`percentile=70` 은 검문소의 민감도입니다. 문장과 문장 사이에는 "의미 거리"가 있습니다. 같은 주제를 이어가면 울타리가 낮고, 주제가 바뀌면 울타리가 높아집니다. `percentile` 은 "얼마나 높은 울타리에서 자를까"를 정하는 기준선입니다.

그림 8-4. 울타리 비유. 기준선 위로 튀어나온 곳에서 자릅니다

여기서 자름!



여기서 자름!



이 기준선의 높이를 바꾸면 결과가 어떻게 달라지는지 봅시다.



Recursive와 달리 `chunk_size` 파라미터가 없습니다. `percentile` 값 하나로 청크 크기가 자동으로 결정됩니다. 값을 낮추면 더 자주 자르고, 높이면 덜 자릅니다.

세 전략 비교

화이트보드에 빨간, 파란, 초록 줄이 나란히 그어져 있습니다. 같은 텍스트인데 자르는 방법에 따라 결과가 다릅니다.

팀장 : "어떤 게 제일 나은지 숫자로 봐야죠."

세 함수를 모두 채웠으면 한 번에 비교합니다. `--step 1-3` 이 3전략 비교 실험입니다.

터미널 실험 1 실행. Fixed vs Recursive vs Semantic

```
python -m tuning.step1_chunk_experiment --step 1-2
```

그림 8-6. 3전략 비교 결과 (같은 1098자 문서)

전략	청크 수	평균 크기	실행 시간	특징
Fixed (500자)	3	399자	0.000s	균일한 크기, 빠른 처리
Recursive	3	365자	4.4s	문단/문장 경계 존중
Semantic (p=70)	7	155자	7.0s	의미 단위 분할, 최고 품질

같은 문서인데 결과가 다릅니다. Fixed는 글자 수만 세서 3조각(399자), Recursive는 문단 경계를 찾아서 3조각(365자)으로 비슷하지만 잘린 위치가 다릅니다. Semantic은 의미 전환을 감지해서 7조각(155자)으로 가장 잘게 나눴습니다.

Fixed는 단순 문자열 연산이라 0초에 가깝고, Recursive와 Semantic은 임베딩 모델을 로드하느라 각각 4~7초가 걸립니다. 실서비스에서는 문서를 인덱싱할 때 한 번만 돌리므로 속도보다 품질이 중요합니다. 청크가 작아진 만큼 "병가"와 "출장"이 한 조각에 섞일 가능성이 줄어듭니다.

8.4 면접관을 붙여 다시 확인한다 — 리랭킹(ReRanking)

청킹 실험 결과를 보던 오픈이가 고개를 가웃했습니다.

오픈이 : "Semantic으로 잘라도 상위 10개 중에 관련 없는 문서가 섞여 들어올 수 있잖아요."

팀장 : "사람 뽑을 때 서류 심사만으로 뽑나요?"

동료 : "면접을 보죠. 서류 통과한 후보를 다시 앉혀 놓고 하나씩 따져 보는 거잖아요."

오픈이가 화이트보드에 서류 10장을 그렸습니다. 벡터 검색이 가져온 Top-10입니다. 그중 관련 있는 건 2~3장, 나머지는 비슷해 보이지만 엉뚱한 문서입니다. 오픈이가 한 장씩 질문과 대조하며 체크 표시를 했습니다. 10장이 3장으로 줄었습니다.

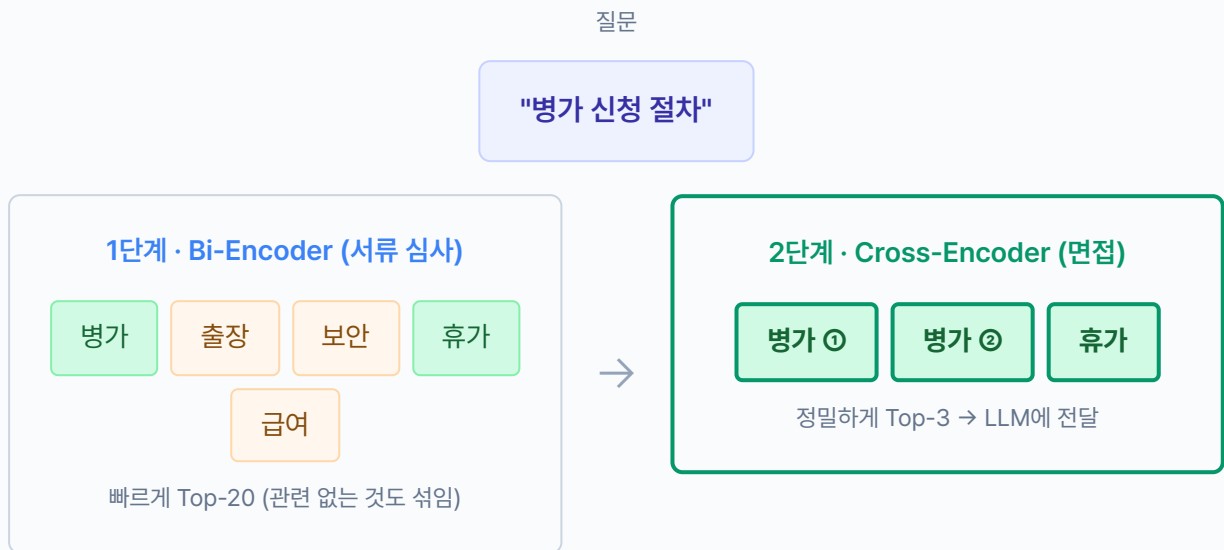
면접관을 붙이는 사서 — 리랭킹(ReRanking). 벡터 검색(Bi-Encoder)은 서류 심사입니다. 질문과 문서를 따로 벡터로 만들어 거리를 잹니다. 빠르지만 대충 푹니다. 리랭커(Cross-Encoder)는 면접관입니다. 질문과 문서를 한 번에 모델에 넣어 직접 "이 문서가 이 질문에 맞나?"를 판단합니다. 느리지만 정확합니다.

오픈이 : "서류로 10명 뽑고, 면접으로 3명 추리는 거네요."

실습 흐름

1. `tuning/step2_reranker/reranker.py` 의 TODO를 채웁니다
2. `python -m tuning.step2_reranker` 로 기본 실험 (`fetch_k=10, top_k=5`)
3. `--fetch-k` , `--top-k` 를 바꿔서 후보 수와 최종 수의 관계를 확인합니다

그림 8-7. 리랭킹 2단계 전략. Bi-Encoder(서류) → Cross-Encoder(면접)



Bi-Encoder vs Cross-Encoder. Bi-Encoder는 서류만 보는 면접관입니다. 질문과 문서를 따로 벡터로 만들어 거리를 잹니다. 빠르지만 대충 푹니다. Cross-Encoder는 직접 대화하는 면접관입니다. 질문과 문서를 한 번에 모델에 넣어 "이 문서가 이 질문에 맞나?" 직접 판단합니다. 느리지만 정확합니다. 그래서 **서류(Bi)**로 많이 뽑고, **면접(Cross)**으로 추리는 2단계 전략을 씁니다.

면접관이 후보를 한 명씩 얹혀 놓고 질문과 대조하는 코드를 작성합니다.

`tuning/step2_reranker/reranker.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 2 tuning/step2_reranker/reranker.py. CrossEncoderReranker

```
from sentence_transformers import CrossEncoder

class CrossEncoderReranker:
    def __init__(self, model_name: str = "BAAI/bge-reranker-v2-m3"):
        self.model = CrossEncoder(model_name)

    def rerank(self, query: str, documents: list[dict], top_k: int = 5) → list[dict]:
        # TODO: 질문과 문서를 쌍으로 만들어 관련성 점수를 매깁니다.
        # 1. (질문, 문서내용) 쌍을 만들
        pairs = [(query, doc["content"]) for doc in documents]
        # 2. Cross-Encoder로 관련성 점수 계산
        scores = self.model.predict(pairs)
        # 3. 각 문서에 점수 추가
        for doc, score in zip(documents, scores):
            doc["cross_encoder_score"] = float(score)
        # 4. 점수 내림차순 정렬 → 상위 top_k개 반환
        ranked = sorted(documents, key=lambda x: x["cross_encoder_score"], reverse=True)
        return ranked[:top_k]
```

`documents` 는 `list[dict]` 형태입니다. 벡터 검색 결과가 `{"content": "...", "score": 0.85}` 딕셔너리로 넘어옵니다. `rerank()` 는 각 문서에 `cross_encoder_score` 키를 추가한 뒤 점수 순으로 정렬하고, 상위 `top_k` 개만 반환합니다. `CrossEncoder` 는 `sentence-transformers` 패키지가 제공하는 클래스이고, `BAAI/bge-reranker-v2-m3` 는 한국어를 지원하는 Cross-Encoder 모델입니다. `predict()` 에 (질문, 문서) 쌍을 넘기면 -10~10 범위의 관련성 점수를 돌려줍니다.

터미널 기본 실험. 10개 가져와서 5개로 추림

```
python -m tuning.step2_reranker
```

터미널 넓게 가져와서 좁게 추리기. 20개 → 3개

```
python -m tuning.step2_reranker --fetch-k 20 --top-k 3
```

그림 8-8. 리랭킹 전후 비교. 면접관(Cross-Encoder)이 순위를 바꿉니다

리랭킹 전 (Vector Search 순위)

리랭킹 후 (Cross-Encoder 순위)

순위	문서	점수
1	연차 신청은 3일 전 서면 제출	0.470
2	연차유급휴가는 15일 부여	0.450
3	재택근무 신청 가능	0.400
4	휴가 시기 조정 가능	0.380
5	성과 평가 기준	0.320

1~5위 점수 차이가 작습니다 (0.470~0.320)

순위	문서	CE 점수
1	연차 신청은 3일 전 서면 제출	0.888
2	연차유급휴가는 15일 부여	0.009
3	재택근무 신청 가능	0.001
4	출장비 정산	0.000
5	육아휴직 신청	0.000

1위가 0.888로 압도적, 나머지는 0점대

서류 심사(벡터 검색)에서는 1~5위 점수 차이가 0.15밖에 안 났습니다. 누가 진짜 적격자인지 구분이 안 돼요. 면접관(Cross-Encoder)이 한 명씩 대화하자 1위(0.888)와 2위(0.009) 사이에 확연한 격차가 생겼습니다. "연차 신청"을 직접 다루는 d02만 0.888로 살아남고, 비슷해 보이지만 다른 주제인 문서들은 0점대로 떨어졌습니다.

`--fetch-k 20 --top-k 3`으로 바꿔 보면 후보를 넓게 잡을수록 면접관이 더 좋은 문서를 찾아낼 수 있지만, 처리 시간도 늘어나는 걸 확인할 수 있습니다.

리랭킹 실험에서 생각해 볼 것들

- **초기 검색 범위(fetch_k):** 10개를 가져와서 5개로 추리면 후보가 적고, 50개를 가져오면 정확도는 올라가지만 Cross-Encoder 추론 시간이 늘어납니다. 문서당 약 0.1초씩 걸립니다.
- **모델 선택:** `bge-reranker-v2-m3`는 한국어 지원이 좋지만 용량이 큼니다. 영어 위주라면 `ms-marco-MiniLM`이 가볍습니다.
- **리랭킹 없이도 괜찮은 경우:** 문서 수가 적거나(100건 이하) 검색 품질이 이미 충분하면 리랭킹 없이 벡터 검색만으로도 됩니다. 비용 대비 효과를 따져 보세요.

8.5 두 검색을 섞는다 — 하이브리드 검색(Hybrid Search)

리랭킹 결과를 확인한 팀장이 한마디를 엿었습니다.

팀장 : "면접관도 만능은 아니에요. 벡터 검색이 애초에 놓친 문서는 면접 대상에도 못 올라오잖아요."

오픈이 : "맞아요. 벡터 검색이 '휴가 사용' 같은 비슷한 의미는 잘 잡는데, '연차 신청'이라는 정확한 단어가 들어간 문서는 놓칠 때가 있거든요."

팀장 : "그럼 단어로 찾는 검색을 하나 더 돌려 봐요."

두 검색을 섞는 사서 — 하이브리드 검색(Hybrid Search). 벡터 검색은 "휴가 사용"처럼 의미가 비슷한 문서를 잘 찾고, BM25는 "연차 신청"이라는 단어가 정확히 들어간 문서를 잘 찾습니다. 각자의 강점이 다르니 둘을 합치면 좋겠는데, 문제가 하나 있습니다.

두 검색이 매기는 점수의 단위가 다릅니다. 시험 점수로 비유하면, 수학은 100점 만점이고 영어는 990점 만점인 셈이에요. 이걸 그냥 더하면 영어 점수가 압도적으로 커져서 수학은 의미가 없어집니다. 그래서 먼저 만점을 맞추고, 그 다음 어느 과목을 더 중요하게 볼지 정합니다.

그림 8-9. 정규화(Normalization). 만점을 맞춰야 합칠 수 있습니다

정규화 전 (만점이 다름)

학생	수학 (100점)	영어 (990점)
A	92	880
B	87	450
C	12	990

그냥 더하면? C가 1위 (12+990=1002)
수학 12점인데 총점 1위라니!

정규화 후 (둘 다 100점 만점)

학생	수학	영어
A	100	80
B	94	0
C	0	100

이제 합산하면? A가 1위 (100+80=180)
두 과목 다 잘하는 학생이 올라옴

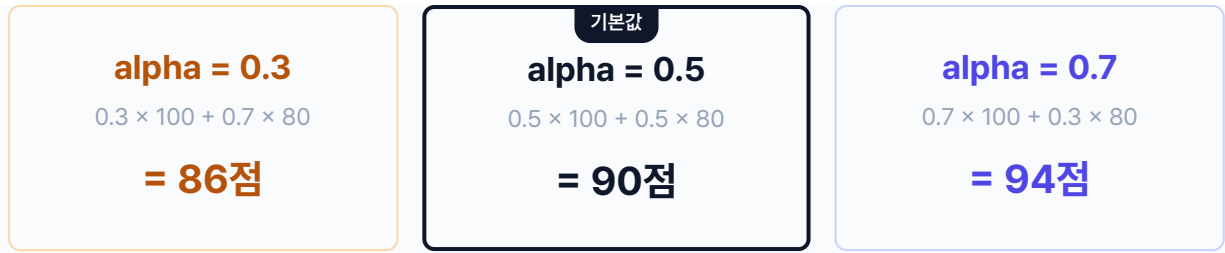
검색도 똑같습니다. 수학 = Vector 검색(만점 1.0), 영어 = BM25 검색(만점 수십). 만점을 맞추지 않으면 BM25 점수가 압도해서 Vector는 의미가 없어집니다.

만점을 맞췄으면 이제 두 점수를 합칩니다. 이때 **가중치(alpha)** 로 비중을 조절합니다. "어느 과목을 더 중요하게 볼까"를 0~1 사이 숫자로 표현한 것입니다.

그림 8-10. 가중치(alpha). 어느 쪽을 더 믿을까?

$$\text{총점} = \alpha \times \text{수학(Vector)} + (1 - \alpha) \times \text{영어(BM25)}$$

학생 A: 수학 100점, 영어 80점



실제 검색에서는 이렇게 비중이 바뀝니다

alpha = 0.3 → 영어(BM25) 비중 높음. "제15조 병가"처럼 정확한 용어를 찾을 때



alpha = 0.5 → 영어(BM25)와 수학(Vector) 반반 (기본값)



alpha = 0.7 → 수학(Vector) 비중 높음. "휴가 규정 알려줘"처럼 의미 검색이 중요할 때

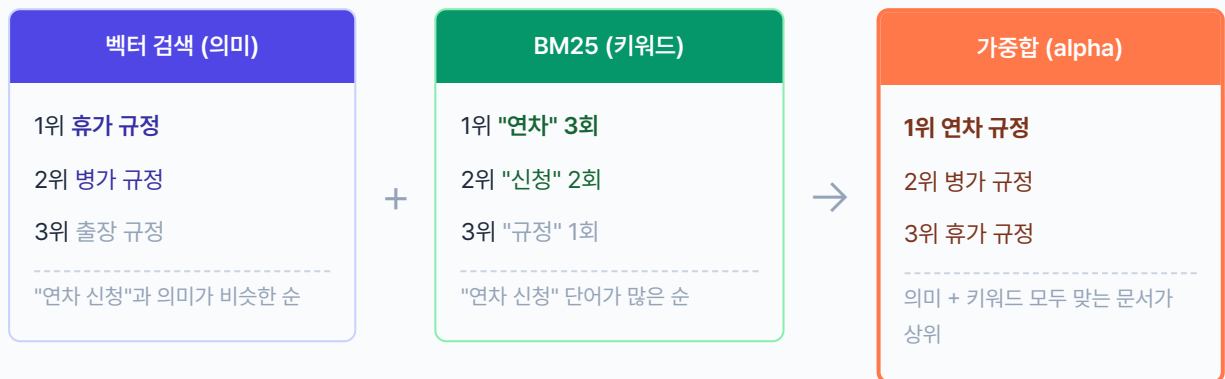


← 영어(BM25·키워드) 비중 · alpha가 클수록 수학(Vector·의미) 비중 →

동료 : "아, 의미를 더 믿을지 단어를 더 믿을지 비율을 정하는 거네요."

이 구조를 코드로 만들어 봅시다.

그림 8-11. 하이브리드 검색. 벡터(의미) + BM25(키워드) → 가중치(alpha) 합산



`retrievers.py` 에 세 클래스가 들어 있습니다. BM25와 Vector 각각의 검색기를 먼저 만들고, EnsembleRetriever가 둘을 합칩니다.

BM25Retriever — 키워드 검색

설명 tuning/step3_hybrid_search/retrievers.py. BM25 키워드 검색기

```
class BM25Retriever:
    def __init__(self, documents: list[str], metadatas: list[dict] | None = None):
        self._build_index(documents)

    def _build_index(self, documents: list[str]):
        # TODO: BM25kapi 인덱스를 생성합니다.
        from rank_bm25 import BM25kapi
        # 1. 각 문서를 소문자 변환 + 공백 기준 토큰화
        tokenized = [doc.lower().split() for doc in documents]
        # 2. BM25 인덱스 생성
        self.bm25 = BM25kapi(tokenized)
```

`BM25kapi` 는 `rank-bm25` 패키지가 제공하는 단어 빈도 기반 검색 알고리즘입니다. 각 문서를 소문자로 변환하고 공백 기준으로 잘라 토큰화한 뒤, 질문 토큰이 문서에 몇 번 등장하는지 세어 점수를 매깁니다. "연차 신청"이라는 단어가 정확히 들어간 문서를 잘 찾지만, "휴가 사용"처럼 다른 표현으로 된 문서는 놓칩니다.

EnsembleRetriever — 두 검색 합치기

`tuning/step3_hybrid_search/retrievers.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 3 tuning/step3_hybrid_search/retrievers.py. EnsembleRetriever

```
class EnsembleRetriever:
    def __init__(self, bm25_retriever, vector_retriever, alpha: float = 0.5):
        self.bm25 = bm25_retriever
        self.vector = vector_retriever
        self.alpha = alpha

    def search(self, query: str, top_k: int = 5, fetch_k: int = 10) → list[dict]:
        # TODO: 두 검색 결과를 정규화한 뒤 alpha 가중합으로 합칩니다.

        # 1. BM25와 Vector 각각 fetch_k개씩 검색 후 0~1로 정규화
        bm25_results = self._normalize_scores(self.bm25.search(query, fetch_k))
        vector_results = self._normalize_scores(self.vector.search(query, fetch_k))

        # 2. 두 결과를 content 기준으로 합침
        scores = {}
        for doc in bm25_results:
            scores[doc["content"]] = {"bm25": doc["normalized_score"], "vector": 0.0}
        for doc in vector_results:
            scores.setdefault(doc["content"], {"bm25": 0.0, "vector": 0.0})
            scores[doc["content"]]["vector"] = doc["normalized_score"]

        # 3. alpha 가중합: hybrid = alpha × vector + (1 - alpha) × bm25
        merged = []
        for content, s in scores.items():
            hybrid = self.alpha * s["vector"] + (1 - self.alpha) * s["bm25"]
            merged.append({"content": content, "hybrid_score": hybrid})

        # 4. hybrid_score 내림차순 정렬 → 상위 top_k개
        return sorted(merged, key=lambda x: x["hybrid_score"], reverse=True)[:top_k]
```

`search()` 는 BM25와 Vector 각각에서 `fetch_k` 개를 가져온 뒤 합쳐서 `top_k` 개를 반환합니다. `_normalize_scores()` 는 점수를 0~1 범위로 맞추는 보조 함수입니다. 만점이 다른 두 검색의 점수를 합치려면 반드시 정규화가 먼저 필요합니다(그림 8-9 참조). `alpha` 는 두 검색의 비중을 조절하는 손잡이입니다. `hybrid_score = alpha × vector_score + (1 - alpha) × bm25_score` 공식으로 합산합니다.

alpha	의미	적합한 경우
0.0	BM25만 (키워드 100%)	"제15조 병가"처럼 정확한 용어를 찾을 때
0.3	BM25 70% + Vector 30%	한국어 특수용어, 약어가 많을 때
0.5	반반 (기본값)	어떤 유형이 많을지 모를 때
0.7	Vector 70% + BM25 30%	"휴가 규정 알려줘"처럼 의미 검색이 중요할 때
1.0	Vector만 (의미 100%)	키워드가 중요하지 않을 때

터미널 실험 3 실행. alpha 파라미터 비교 + 하이브리드 데모

```
python -m tuning.step3_hybrid_search
```

그림 8-12. 하이브리드 검색 결과 (alpha=0.5, 쿼리: "연차 신청 절차와 승인 방법")

순위	BM25	Vector	Hybrid	내용
1	0.896	1.000	0.948	연차 신청은 3일 전까지 서면 제출
2	0.838	0.581	0.709	재택근무 신청 가능
3	1.000	0.000	0.500	개인 USB 사용 금지 (IT보안팀 승인)
4	0.000	0.495	0.248	연차유급휴가 15일 부여
5	0.000	0.490	0.245	고충처리 서면 신청

1위 문서는 BM25(0.896)와 Vector(1.000) 모두 높아서 Hybrid 0.948로 확실한 1위입니다. 3위의 "USB 사용" 문서는 BM25가 1.000(키워드 "신청"이 정확히 매칭)이지만 Vector가 0.000(의미가 완전히 다름)이라 Hybrid 0.500에 머물렀습니다. BM25만 썼다면 이 문서가 1위로 올라왔을 거예요.

alpha를 바꿔서 실험해 보세요

`--alpha` 옵션으로 비중을 조절할 수 있습니다. 위 다이어그램의 막대 비율이 바뀌는 걸 결과에서 확인할 수 있습니다.

```
python -m tuning.step3_hybrid_search --alpha 0.7 # Vector 70%, BM25 30%
python -m tuning.step3_hybrid_search --alpha 0.3 # Vector 30%, BM25 70%
```

8.6 세 실험을 마치며

오픈이가 화이트보드의 빨간, 파란, 초록 줄을 바라봤습니다. 실험 결과가 나란히 적혀 있습니다.

동료 : "그런데 세 개 다 적용해야 하는 거예요?"

오픈이 : "아니요, 문서 성격에 따라 골라 쓰는 거예요. 사내 규정처럼 조항이 짧으면 Recursive 청킹만으로도 충분하고, 검색 결과가 애매하면 리랭커를 붙이고, 키워드와 의미 둘 다 중요하면 하이브리드를 쓰는 거죠."

팀장 : "도구가 세 개 생긴 거죠, 세 개를 다 쓰라는 건 아니에요."

이번 챕터에서 실험한 도구는 세 가지입니다. 청킹 전략, 리랭킹, 하이브리드 검색. 아직 실제 파이프라인에 적용한 건 아닙니다. 어떤 조합이 우리 문서에 맞는지 실험으로 확인한 거예요. 다음 챕터 9에서는 **질문 자체**의 모호함을 잡습니다. "병이 어떻게 써?"처럼 불완전한 질문을 다루는 쿼리 재작성입니다. 그리고 챕터 10에서 이 도구들 중 필요한 것만 골라 하나의 파이프라인으로 조립합니다.

팀장 : "검색이 단단해졌으니까, 이번엔 질문 쪽도 들여다봐요."

오픈이 : "질문이요?"

팀장 : "사람들이 항상 깔끔하게 질문하진 않잖아요."

8.7 튜닝 구성도

이번 챕터에서 실험한 세 가지 튜닝과, 앞으로 다룰 튜닝을 한 장으로 정리합니다. 모든 튜닝을 다 적용할 필요는 없습니다. 문서와 질문 성격에 맞춰 필요한 것만 골라 씁니다.

그림 8-13. RAG 튜닝 메뉴판. 필요한 것만 골라 씁니다

	튜닝	무엇을 바꾸나	언제 쓰나	챕터
✓	청킹 전략	문서를 자르는 방법 (Fixed → Recursive → Semantic)	주제가 섞이는 문제가 있을 때	8
✓	리랭킹	가져온 문서를 Cross-Encoder로 재정렬	검색 결과 상위에 엉뚱한 문서가 섞일 때	8
✓	하이브리드 검색	벡터 + BM25를 alpha 가중합으로 합침	의미 검색과 키워드 검색 둘 다 필요할 때	8
-	쿼리 재작성	불완전한 질문을 LLM이 이해할 수 있는 형태로 변환	"병이 어떻게 써?"처럼 모호한 질문이 많을 때	9
-	파이프라인 조립	위 튜닝 중 필요한 것만 골라 하나의 파이프라인으로 조립	실서비스에 적용할 때	10

✓ 이번 챕터에서 실험한 항목 · - 다음 챕터에서 다룰 항목

용어 정리

본문 속 표현	진짜 용어	정식 정의
"제목만 보고 자르기"	Fixed-size Chunking	일정 글자 수(500자)로 기계적 분할. 빠르지만 주제 혼입 위험
"문단을 보고 자르기"	Recursive Character Chunking	빈 줄·단락 경계를 우선 분할 기준으로 삼는 스플리터
"목차를 보고 자르기"	Semantic Chunking	임베딩 거리 변화로 주제 전환을 감지해 자름
"다시 확인하기"	Cross-Encoder Reranker	질문·문서를 한 번에 모델에 넣어 직접 관련성 점수를 매김
"두 방법 섞기"	Hybrid Search (BM25 + Vector)	키워드 기반 BM25와 의미 기반 벡터 검색을 병행
"두 점수 합치기"	Alpha 가중합 (Weighted Sum)	각 검색 점수를 0~1로 정규화한 뒤 $\alpha * \text{vector} + (1 - \alpha) * \text{bm25}$ 로 합산

이것만은 기억하자

- **검색이 바뀌면 답이 바뀝니다.** RAG의 품질은 LLM이 아니라 Retriever가 결정합니다. 엉뚱한 문서가 들어가면 LLM은 성실히 엉뚱한 답을 합니다.
- **세 도구를 다 쓸 필요는 없습니다.** 청킹 전략, 리랭킹, 하이브리드 검색은 문서와 질문 성격에 맞춰 골라 씁니다.
- **다음 챕터에서는 질문을 다듬습니다.** 챕터 9에서 쿼리 재작성, 챕터 10에서 원하는 튜닝만 골라 하나의 파이프라인으로 조립합니다.

챕터 9. 질문을 제대로 이해 못한다. HyDE와 Multi-Query

이번 챕터가 끝나면

- **Parent Document Retriever**로 조각이 아닌 원본 문서 전체를 참고하게 합니다
- **Contextual Compression**으로 검색 결과에서 핵심 문장만 압축합니다
- **HyDE(Hypothetical Document Embeddings)** 로 "상상 답변을 먼저 만들어 검색"합니다
- **Multi-Query**로 하나의 질문을 여러 관점으로 풀어 재현율을 올립니다
- **약어, 동의어 확장**으로 "WFH" ↔ "재택근무" 같은 단어 차이를 흡수합니다

준비하기. 실습 시작 전 한 번만 설정

1. 실습 폴더 이동

터미널 폴더 이동

```
cd rag-start/ex09
```

파일 구조는 다음과 같습니다.

ex09 디렉토리

```
ex09/
├── tuning/
│   ├── step1_query_rewrite/           # [실습] HyDE / Multi-Query / 약어 확장
│   │   ├── __main__.py
│   │   ├── hyde.py                   # 실험 1-1
│   │   ├── multi_query.py           # 실험 1-2
│   │   ├── abbreviation.py          # 실험 1-3
│   │   ├── experiments.py
│   │   └── data.py
│   └── step2_advanced_retriever/      # [실습] Parent Doc / Compression
│       ├── __main__.py
│       ├── parent_doc.py             # 실험 2-1
│       ├── compression.py           # 실험 2-2
│       ├── self_query.py            # 실험 2-3
│       ├── experiments.py
│       └── data.py
```

2. 실습 환경 구축

터미널 환경 구성. macOS / Linux

```
cd ex09
python3.12 -m venv .venv
source .venv/bin/activate
cp .env.example .env
ollama pull llama3.1:8b
pip install -r requirements.txt
```

터미널 환경 구성. Windows

```
cd ex09
py -3.12 -m venv .venv
.venv\Scripts\activate
copy .env.example .env
ollama pull llama3.1:8b
pip install -r requirements.txt
```

3. 사용할 라이브러리

패키지	역할
<code>langchain</code>	Retriever 인터페이스·LCEL
<code>langchain-community</code>	ParentDocumentRetriever, MultiQueryRetriever
<code>langchain-ollama</code>	LLM 연결 (쿼리 재작성용)
<code>langchain-chroma</code> · <code>chromadb</code>	벡터 DB

4. 실행 순서

1. `python -m tuning.step1_query_rewrite` . HyDE, Multi-Query, 약어 확장
2. `python -m tuning.step2_advanced_retriever` . Parent Doc, Compression

9.1 "휴가"라고 물었는데 "연차유급휴가"를 못 찾는다

그림 9-1. 같은 뜻인데 못 찾는다



"휴가 규정 알려줘"

✗ 복리후생 안내가 나옴

"연차유급휴가 알려줘"

✓ 취업규칙 제15조 정확히 검색

"WFH 정책이 뭐야?"

✗ 관련 문서를 찾지 못했습니다

검색 엔진은 잘 돌아갑니다. 질문을 이해 못 하는 게 문제입니다.

청킹을 바꾸고 리랭커를 붙이고 하이브리드까지 적용했습니다. 일주일도 지났죠. 동료도 "병가 규정"을 물으면 출장 규정이 나오던 시절은 끝났다고 생각했습니다. 모니터에 찍히는 검색 로그는 깔끔했고, 오픈이는 의자에 등을 기대며 커피잔을 들었습니다.

검색 엔진은 손덜 데가 없다.

슬랙 알림이 울렸습니다. 동료가 테스트 결과를 스프레드시트로 정리해 올린 겁니다. "질문 20개 테스트" 제목 아래 빨간 셀이 네 개. 오픈이가 커피잔을 내려놓고 몸을 일으켰습니다.

동료 : "질문 20개 돌려 봤는데, 네 개가 이상해요."

스프레드시트를 열었습니다. 첫 번째 빨간 셀. "휴가 규정 알려줘."

직접 쳐 봤습니다. 커넥트HR 채팅창에 "휴가 규정 알려줘"를 입력하고 엔터를 눌렀더니 "복리후생 안내"가 돌아옵니다. 정작 찾고 싶던 **취업규칙 제15조 연차유급휴가**는 빠져 있었습니다.

잠깐.

"연차유급휴가"를 정확히 쳐 봤습니다. 바로 뜹니다. 15일 부여, 3일 전 서면 신청, 팀장 승인. 원하는 내용이 전부 들어 있었습니다.

같은 뜻인데 단어가 달랐습니다. 문서에는 "연차유급휴가"라고 적혀 있고 동료는 "휴가"라고 물었습니다. 검색 엔진은 "휴가"와 "연차유급휴가"가 같은 뜻이라는 걸 모릅니다.

두 번째 빨간 셀. "WFH 정책이 뭐야?"

"관련 문서를 찾지 못했습니다."

문서에는 "재택근무"라고 적혀 있습니다. WFH가 Work From Home이라는 걸 검색 엔진은 모릅니다. 세 번째 빨간 셀도, 네 번째도 비슷한 패턴이었습니다. 단어가 다르거나, 약어를 쓰거나, 질문이 모호하거나.

그림 9-2. 사용자의 단어와 문서의 단어가 다릅니다

사용자가 말한 단어 "휴가"	≠	문서에 적힌 단어 "연차유급휴가"	×
사용자가 말한 단어 "WFH"	≠	문서에 적힌 단어 "재택근무"	×
사용자가 말한 단어 "쉬는 날"	≠	문서에 적힌 단어 "연차유급휴가"	×
사용자가 말한 단어	=	문서에 적힌 단어	✓

"연차유급휴가"

"연차유급휴가"

정확히 같은 단어를 써야만 찾습니다. **검색 엔진이 아니라 질문이 문제입니다.**

챗터 8에서 검색 엔진 자체를 다 고쳤다고 생각했는데, 엔진이 아무리 좋아도 질문을 이해 못 하면 소용이 없다.

건너편 자리에서 팀장이 모니터를 보며 한마디를 던졌습니다.

팀장 : "사서 귀가 안 열린 거 아닌가요?"

오픈이 : "네?"

팀장 : "서가 정리는 잘 해놨잖아요. 그런데 손님이 '쉬는 날'이라고 말하면, 그게 '연차유급휴가' 서랍이란 걸 알아들어야 할 거 아니겠어요."

키보드 위에 올려놓은 손가락이 멈췄습니다. 챗터 8에서 고친 건 서가 정리였습니다. 책장을 다시 짜고 면접관을 붙이고 두 가지 검색을 섞었습니다. 그런데 사서의 **귀**, 그러니까 질문을 알아듣는 능력은 건드리지 않았습니다.

서랍에서 수첩을 꺼냈습니다. 화이트보드 앞에 서기 전에 무엇이 문제였는지부터 다시 적어야 했습니다.

- 문제 정리 -

1. "휴가" ≠ "연차유급휴가"

- 문서 단어와 사용자 단어가 다르면 검색 실패
- 같은 뜻이라는 걸 기계는 모름

2. "WFH" ≠ "재택근무"

- 영문 약어는 한글 문서에 안 걸림
- 사내 용어 사전이 없으면 그냥 놓침

→ 엔진 문제가 아님. 질문 이해 능력의 문제

9.2 사서의 언어 능력을 키운다

오픈이가 화이트보드 앞으로 걸어갔습니다. 마커 뚜껑을 딸깍 열고 왼쪽 끝에 "사서의 언어 능력"이라고 적었습니다. 팀장이 의자를 돌려 팔짱을 껴줍니다.

팀장 : "사서가 똑똑해지려면 뭘 배워야 할까요?"

오픈이가 "휴가 → 연차유급휴가"를 화이트보드에 적고 잠시 마커를 들고 서 있었습니다. 그러다 하나씩 적어 내려가기 시작했습니다.

첫째, 단어를 번역합니다. "WFH" → "재택근무(Work From Home)". "OT" → "초과근무". 사서가 약어 사전을 가지고 있으면 모르는 단어가 들어와도 원래 뜻으로 바꿔 놓을 수 있습니다.

둘째, 답을 먼저 상상합니다. "쉬는 날 규정이라면... 1년 근속 시 15일 유급휴가 같은 내용이겠지." 그 상상한 답과 비슷한 실제 문서를 찾습니다. 질문으로 검색하는 것보다 답변끼리 비교하는 게 거리가 가까운 경우가 많습니다.

셋째, 질문을 여러 갈래로 바꿉니다. "휴가 규정" 하나로 검색하면 한 방향만 봅니다. "연차 사용 방법", "유급휴가 일수", "휴가 신청 절차"로 펼쳐 놓고 각각 검색하면 놓치는 문서가 줄어듭니다.

넷째, 찾은 조각의 원본 전체를 꺼냅니다. "연차 신청은 3일 전 서면"이라는 짧은 청크가 걸리면, 그 청크가 속한 원 문서(제15~16조) 전체를 반환합니다. 맥락이 살아납니다.

다섯째, 긴 결과를 압축합니다. 부모 문서를 통째로 가져오면 쓸데없는 문장이 섞입니다. 질문과 관련된 문장만 추려서 LLM에 넘깁니다.

다섯 줄을 적고 나서 한 발 물러나 화이트보드를 바라봤습니다.

팀장 : "전부 다 켜면 느려져요. 어디가 아픈지 보고 약을 골라야죠."

도구 상자입니다. 다섯 가지를 전부 쓰는 게 아니라, 어떤 질문이 실패하는지 보고 필요한 도구만 꺼내면 됩니다.

그림 9-3. 사서의 언어 능력 다섯 가지

질문을 알뜰히 귀

- 1 단어를 번역한다 — 약어·동의어 확장
WFH → 재택근무 OT → 초과근무
- 2 답을 먼저 상상한다 — HyDE
"쉬는 날 규정" → "1년 근속 시 15일 유급휴가..." → 상상 답변으로 검색
- 3 질문을 여러 갈래로 — Multi-Query
"휴가 규정" → "연차 사용 방법" + "유급휴가 일수" + "휴가 신청 절차"



결과를 정리하는 손

4 조각의 원본 전체를 꺼낸다 — **Parent Doc**
"연차 신청은 3일 전 서면" 조각 → 제15~16조 원문 전체 반환

5 핵심 문장만 압축한다 — **Compression**
부모 문서에서 질문과 관련된 문장만 남기고 나머지 제거

1~3(파란색) = 검색 전 질문 가공 · 4~5(초록색) = 검색 후 결과 가공

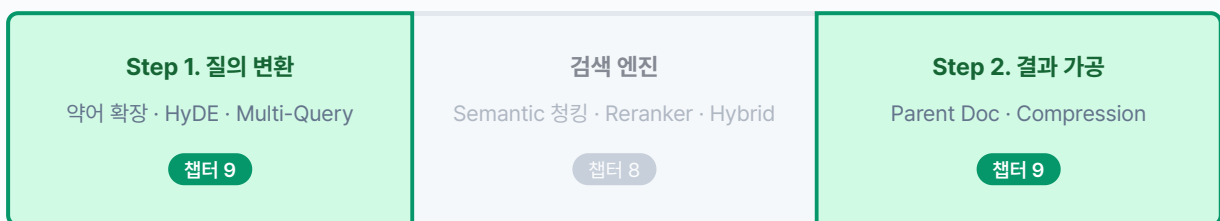
9.3 이번 챕터의 두 묶음

다섯 가지 기법이 두 묶음으로 나뉩니다.

검색 전 질의 변환 (Step 1). HyDE, Multi-Query, 약어 확장. 검색 전에 질문 자체를 가공합니다.

검색 후 결과 가공 (Step 2). Parent Doc, Compression. 검색된 결과를 넓히거나 줄여서 LLM에 넘기기 좋은 형태로 만듭니다.

그림 9-4. RAG 파이프라인에서 챕터 8과 챕터 9의 위치



← 질문 입력 · 검색 엔진 (챕터 8) · 결과 출력 →

9.4 사서의 귀를 열어 주자 — 질의 변환 (step1)

서가는 정리했습니다. 이번에는 **질문 쪽**을 손봅니다. 검색 엔진에 질문을 넘기기 **전에** 질문 자체를 가공하는 도구 세 가지를 하나씩 실험합니다. 실험 결과를 보고 우리 시스템에 필요한 것만 골라 적용할 겁니다.

실습 흐름

1. 가상 답변으로 검색한 결과와 직접 검색 결과를 비교합니다 (`--step 1-1`)
2. 하나의 질문을 여러 표현으로 재작성합니다 (`--step 1-2`)
3. 쿼리의 약어와 동의어를 확장합니다 (`--step 1-3`)

9.4.1 답을 먼저 상상한다 — HyDE

답을 먼저 상상해 보면 어떨까.

사서가 "쉬는 날 규정"이라는 질문을 받으면, 서가로 달려가기 전에 먼저 "1년 근속 시 15일 유급휴가..."라는 답을 상상합니다. 그 상상한 답과 비슷한 문서를 찾으면, 질문과 문서 사이의 어휘 차이를 건너뛸 수 있습니다.

질문 대신 "**이 질문에 어울릴 법한 답변**"을 LLM에 먼저 생성시키고, 그 가상 답변을 임베딩해 검색합니다. 답변과 답변 간 거리가 질문-답변 거리보다 가까운 경우가 많기 때문입니다.

그림 9-5. 직접 검색과 HyDE 검색의 차이

직접 검색

"쉬는 날 규정"

↓ 임베딩

질문 벡터

↓ 문서 벡터와 비교

문서: "연차유급휴가는..."
어휘가 다르면 거리가 멀다

HyDE 검색

"쉬는 날 규정"

↓ LLM 호출

"연차유급휴가는 1년 근속 시 15일..."

↓ 가상 답변을 임베딩

답변 벡터

↓ 문서 벡터와 비교

문서: "연차유급휴가는..."
답변끼리는 거리가 가깝다

핵심: $\text{답변} \leftrightarrow \text{문서 거리} < \text{질문} \leftrightarrow \text{문서 거리}$. 가상 답변이 어휘 차이를 매워 줍니다.

이 흐름을 코드로 구현합니다. 질문 벡터와 가상 답변 벡터를 각각 만들고, 문서 벡터와의 유사도를 나란히 비교하는 함수입니다.

`tuning/step1_query_rewrite/hyde.py`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 1 tuning/step1_query_rewrite/hyde.py. HyDE vs 직접 검색 비교

```
def compare_hyde_vs_direct(query, documents, embeddings=None):
    # TODO: 가상 답변으로 검색한 결과와 직접 검색 결과를 비교합니다.

    # 1. LLM에 가상 답변 생성 요청
    hypo_doc = _generate_hypothetical_doc_llm(query)

    doc_contents = [d["content"] for d in documents]

    # 2. 질문 벡터와 가상 답변 벡터를 각각 생성
    query_vec = embeddings.embed_query(query)
    hyde_vec = embeddings.embed_query(hypo_doc)
    doc_vecs = embeddings.embed_documents(doc_contents)

    # 3. 직접 검색: 질문 ↔ 문서 유사도
    direct_scores = [
        (_cosine_similarity(query_vec, dv), doc)
        for dv, doc in zip(doc_vecs, documents)
    ]

    # 4. HyDE 검색: 가상 답변 ↔ 문서 유사도
    hyde_scores = [
        (_cosine_similarity(hyde_vec, dv), doc)
        for dv, doc in zip(doc_vecs, documents)
    ]

    direct_scores.sort(key=lambda x: x[0], reverse=True)
    hyde_scores.sort(key=lambda x: x[0], reverse=True)

    return {
        "query": query,
        "hypothetical_doc": hypo_doc,
        "direct_results": _to_results(direct_scores),
        "hyde_results": _to_results(hyde_scores),
    }
```

질문 벡터로 검색한 `direct_results`와 가상 답변 벡터로 검색한 `hyde_results`가 나란히 반환됩니다. `embeddings`는 `experiments.py`가 `jhgan/ko-sroberta-multitask` 모델을 로드해서 넘겨주고, `documents`는 `data.py`의 `SEARCH_DOCUMENTS`(사내 규정 10건)입니다. 코드에 등장하는 보조 함수는 같은 파일 위쪽에 미리 정의되어 있습니다.

함수	역할
<code>_generate_hypothetical_doc_llm(query)</code>	LLM에 가상 답변 문서 생성을 요청합니다
<code>_cosine_similarity(a, b)</code>	두 벡터의 코사인 유사도를 계산합니다
<code>_to_results(scored)</code>	(점수, 문서) 리스트를 상위 3개로 잘라 반환합니다

터미널 실험 1-1 실행. HyDE 검색 vs 직접 검색

```
python -m tuning.step1_query_rewrite --step 1-1
```

그림 9-6. 실험 1-1. HyDE (Hypothetical Document Embeddings)

원리: 가상 답변 문서 생성 → 그 문서와 유사한 실제 문서 검색

쿼리: 연차 신청 절차는 어떻게 됩니까?

가상 문서: 연차 신청에 관한 사내 규정에 따르면, 직원은 사용 예정일 3일 전까지 팀장 승인을 받아야 하며, 1년 근속 시 15일의 유급휴가가 부여됩니다...

직접 검색 결과

순위	내용	유사도
1	연차 신청은 3일 전 서면 제출...	0.5917
2	연차유급휴가는 1년 근속 시 15일...	0.5276
3	재택근무 입사 6개월 이상 신청...	0.4455

3위가 재택근무 문서로, 연차와 무관합니다.

HyDE 검색 결과

순위	내용	유사도
1	연차 신청은 3일 전 서면 제출...	0.6691
2	연차유급휴가는 1년 근속 시 15일...	0.6237
3	재택근무 입사 6개월 이상 신청...	0.5888

모든 점수가 올랐습니다. 1위: 0.5917 → 0.6691

직접 검색에서는 1위(0.5917)와 3위(0.4455) 사이 점수 차이가 0.15에 불과해 순위 구분이 어렵습니다. HyDE에서는 가상 답변이 "연차유급휴가"라는 단어를 직접 포함하기 때문에 관련 문서의 유사도가 전반적으로 올랐습니다. 1위 점수가 0.5917 → 0.6691로 상승했고, 상위 문서 간 격차도 뚜렷해졌습니다.

질문을 바꿔서도 실험해 보세요.

터미널 커스텀 쿼리로 HyDE 실험

```
python -m tuning.step1_query_rewrite --step 1-1 --query "쉬는 날 규정"
```

HyDE 실험에서 생각해 볼 것들

- **유사도 점수 차이:** 직접 검색(Direct)과 HyDE 검색의 상위 3개 점수를 비교해 보세요. HyDE 쪽이 전반적으로 높다면, 가상 답변이 질문과 문서 사이의 어휘 차이를 잘 메워 준 겁니다.
- **가상 답변의 품질:** hypothetical_doc 항목을 읽어 보세요. LLM이 만든 가상 답변이 실제 규정과 비슷할수록 검색 정확도가 올라갑니다. 엉뚱한 답변이 나오면 프롬프트를 다듬어야 합니다.
- **순위 변화:** HyDE 검색에서 1위로 올라온 문서가 직접 검색의 1위와 다른지 확인해 보세요. 다르다면 어휘 차이 때문에 직접 검색에서 놓쳤던 문서를 HyDE가 잡아낸 겁니다.

됐다. 오픈이가 터미널을 내려다봤습니다. 유사도가 0.59에서 0.67로 올랐습니다. HyDE는 "쉬는 날"처럼 어휘 차이가 큰 질문에 효과적입니다.

그런데 "복리후생"처럼 의미가 넓은 질문은 어떨까요? 연차도, 재택도, 출장비도 다 걸립니다. 가상 답변 하나로는 한 방향밖에 못 봅니다. 다른 도구를 꺼내 봅니다.

9.4.2 질문을 여러 갈래로 — Multi-Query

동료 : "하나로만 물으면 한 방향밖에 못 보잖아요?"

사서가 서가를 한 방향에서만 훑지 않고 세 갈래로 나눠 찾는 겁니다. LLM에게 "이 질문을 다른 표현으로 3가지 만들어 줘"라고 시키고, 원본 포함 4개 질문으로 각각 검색한 뒤 합집합을 만듭니다.



핵심: 질문 하나를 여러 표현으로 바꾸면 **각 표현이 잡아오는 문서가 다릅니다**. 합집합으로 검색 범위를 넓힙니다.

이 흐름을 코드로 구현합니다. LLM에 프롬프트를 보내 재작성 질문을 받고, 원본 질문과 합쳐 반환하는 함수입니다.

`tuning/step1_query_rewrite/multi_query.py`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 1 `tuning/step1_query_rewrite/multi_query.py`. 다양한 관점의 쿼리 생성

```
def generate_multi_queries(query, num_queries=3):
    prompt = (
        f"다음 질문을 {num_queries}가지 다른 방식으로 재표현하십시오.\n"
        "각 질문은 같은 정보를 구하지만 다른 표현 방식을 사용해야 합니다.\n"
        "번호 없이 각 질문을 한 줄씩 출력하십시오.\n\n"
        f"원본 질문: {query}\n\n"
        "재표현된 질문들:"
    )
    # TODO: LLM을 호출하고 결과를 줄 단위로 파싱합니다.
    llm_result = _call_llm(prompt)

    # 2. 줄 단위로 파싱, 원본 포함 리스트 구성
    lines = [
        line.strip()
        for line in llm_result.split("\n")
        if line.strip() and not line.strip().startswith("#")
    ]
    queries = [query] + lines[:num_queries]
    return queries
```

원본 질문 1개에 재작성 3개를 합쳐 총 4개 질문이 반환됩니다. `num_queries` (기본값 3)가 생성할 재작성 수이고, `_call_llm(prompt)`은 같은 파일에 정의된 보조 함수로 환경변수 `LLM_PROVIDER`에 따라 Ollama 또는 OpenAI를 호출합니다.

터미널 실험 1-2 실행. Multi-Query 재작성

```
python -m tuning.step1_query_rewrite --step 1-2
```

그림 9-8. 실험 1-2. Multi-Query

원리: 1개 질문 → N개 다양한 표현으로 검색 결과 다양화

원본 쿼리: 재택근무 조건과 신청 방법

생성된 쿼리

번호	쿼리	유형
1	재택근무 조건과 신청 방법	원본
2	원격 근무에 관련된 규정 및 절차는?	변형 1
3	원격 근무 신청 방법과 필요한 정보는?	변형 2
4	집으로 출근 조건과 신청 방법은?	변형 3

병합 검색 결과

순위	내용	최고 유사도
1	재택근무는 입사 6개월 이상 정규직에 한해 신청 가능...	0.7355
2	재택근무 중에는 정규 근무시간 준수...	0.6053
3	연차 신청은 3일 전 서면 제출...	0.4531

"재택근무"를 "원격 근무", "집으로 출근" 등으로 바꿔 검색 범위를 넓혔습니다.

"재택근무 조건"이라는 질문 하나를 "원격 근무 규정", "집으로 출근 조건" 같은 다른 표현으로 바꿔 각각 검색합니다. 4개 질문이 잡아온 문서를 합치면 원본만으로 놓쳤을 문서까지 포함됩니다. 1위 문서(0.7355)는 "재택근무"라는 단어가 직접 들어 있어 원본 질문과 강하게 매칭됐고, 변형 질문들이 같은 문서를 다른 방향에서 보강해 점수가 더 올랐습니다.

재작성 개수를 바꿔서 확장 효과를 비교해 보세요.

터미널 쿼리 수를 5개로 늘려 실험

```
python -m tuning.step1_query_rewrite --step 1-2 --num_queries 5
```

Multi-Query 실험에서 생각해 볼 것들

- **재작성 품질:** LLM이 만든 재작성 질문들을 읽어 보세요. 원본과 같은 의도지만 다른 표현을 쓰고 있어야 합니다. 너무 비슷하면 효과가 떨어집니다.
- **검색 범위 확장:** 합집합 결과에 원본 질문만으로는 못 찾았을 문서가 포함됐는지 확인해 보세요. 새로 들어온 문서가 있다면 Multi-Query의 효과입니다.
- **쿼리 수 조절:** `--num_queries 5` 로 늘리면 검색 범위가 넓어지지만, LLM 호출 횟수도 늘어납니다. 3개면 충분한 경우가 많습니다.

HyDE vs Multi-Query 선택 기준. HyDE는 질문과 문서의 **어휘 차이**가 클 때 효과적입니다. "쉬는 날" → "연차유급휴가"처럼 질문에 쓰는 단어와 문서에 적힌 단어가 다를 때 가상 답변이 다리 역할을 합니다. Multi-Query는 질문의 **의미가 넓을 때** 효과적입니다. "복리후생"처럼 여러 규정에 걸치는 질문을 갈래로 쪼개야 할 때 씁니다. 둘 다 LLM을 호출하므로 응답 시간이 늘어나는 점은 같습니다.

Multi-Query는 넓은 질문에 효과적입니다. 네 갈래로 검색하니 한 방향에서 놓쳤던 문서가 합집합에 올라왔습니다. 오픈이가 팀장에게 화면을 보여주려고 고개를 돌렸습니다.

팀장 : "그거 좋은데요. 근데 제가 아까 'WFH 신청'이라고 물어봤거든요? 아무것도 안 나오더라고요."

이건 다른 종류의 문제입니다. 질문의 넓이가 아니라 **단어 자체**를 모르는 겁니다. 사서는 "재택근무"라는 단어만 알고 있었습니다.

9.4.3 약어를 번역한다 — 약어·동의어 확장

사내에서 흔히 쓰는 약어와 줄임말을 사전으로 만들어 두고, 질문에 포함되면 원래 뜻으로 치환합니다.

`data.py` 에 두 종류의 사전이 들어 있습니다.

참고 tuning/step1_query_rewrite/data.py. 약어·동의어 사전

```
ABBREVIATION_MAP = {
    "WFH": "재택근무(Work From Home)",
    "OT": "초과근무(Overtime)",
    "PIP": "성과개선계획(Performance Improvement Plan)",
    "HR": "인사부서(Human Resources)",
    "4대보험": "국민연금, 건강보험, 고용보험, 산재보험",
    "반차": "반일 연차",
    # ... 외 6개
}

SYNONYM_MAP = {
    "연차": "연차유급휴가",
    "휴가": "연차유급휴가",
    "재택": "재택근무",
    "원격근무": "재택근무",
    "퇴직금": "퇴직급여",
    # ... 외 5개
}
```

`ABBREVIATION_MAP` 은 영문 약어를 한글 풀네임으로, `SYNONYM_MAP` 은 구어 표현을 규정집 정식 명칭으로 바꿉니다. 질문에 "WFH"가 들어오면 "재택근무(Work From Home)"로, "연차"가 들어오면 "연차유급휴가"로 치환됩니다.

이 사전을 순회하며 문자열을 치환하는 함수를 작성합니다.

`tuning/step1_query_rewrite/abbreviation.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 1 tuning/step1_query_rewrite/abbreviation.py. 약어·동의어 확장

```
def expand_abbreviations(query):
    # TODO: 쿼리의 약어와 동의어를 확장합니다.
    expanded = query
    applied = []

    # 1. 약어 확장: "WFH" → "재택근무(Work From Home)"
    for abbrev, full_form in ABBREVIATION_MAP.items():
        if abbrev in expanded:
            expanded = expanded.replace(abbrev, full_form)
            applied.append(f"약어 '{abbrev}' -> '{full_form}'")

    # 2. 동의어 확장: "휴가" → "연차유급휴가"
    for term, synonym in SYNONYM_MAP.items():
        if term in expanded and synonym not in expanded:
            expanded = expanded.replace(term, synonym)
            applied.append(f"동의어 '{term}' -> '{synonym}'")

    return {"original": query, "expanded": expanded, "applied": applied}
```

ABBREVIATION_MAP 과 SYNONYM_MAP 을 순회하며 문자열 치환을 수행합니다. applied 리스트에 어떤 변환이 적용됐는지 기록하므로, "WFH 신청 방법"이 "재택근무(Work From Home) 신청 방법"으로 바뀌는 과정을 터미널에서 확인할 수 있습니다.

터미널 실험 1-3 실행. 약어·동의어 확장

```
python -m tuning.step1_query_rewrite --step 1-3
```

그림 9-9. 실험 1-3. 약어/동의어 확장

원리: 사전 기반 매핑으로 약어와 동의어를 정규 표현으로 치환

원본 쿼리	확장된 쿼리	적용 규칙
WFH 정책이 어떻게 됩니까?	재택근무(Work From Home) 정책이...	약어 WFH → 재택근무
OT 신청 절차를 알려주십시오	초과근무(Overtime) 신청 절차를...	약어 OT → 초과근무
연차 신청 방법	연차유급휴가 신청 방법	동의어 연차 → 연차유급휴가
4대보험 가입 시기	국민연금, 건강보험, 고용보험, 산재보험...	약어 4대보험 → 풀네임

사전에 등록된 약어와 동의어가 자동으로 치환됩니다.

"WFH"라고 물었는데 문서에는 "재택근무"라고 적혀 있으면 벡터 검색에서도 놓칠 수 있습니다. 사전이 이 간극을 메워 줍니다. LLM 호출 없이 단순 문자열 치환이라 응답 지연이 없습니다.

약어가 포함된 질문을 직접 넣어 보세요.

터미널 커스텀 쿼리로 약어 확장 실험

```
python -m tuning.step1_query_rewrite --step 1-3 --query "OT 신청 절차"
```

약어 확장 실험에서 생각해 볼 것들

- **적용된 변환:** applied 목록에 어떤 규칙이 적용됐는지 확인해 보세요. 약어(WFH → 재택근무)와 동의어 (휴가 → 연차유급휴가) 두 종류가 있습니다.
- **검색 결과 변화:** 확장 전후로 검색 결과가 달라지는지 비교해 보세요. 문서에 "재택근무"라고 적혀 있는데 질문에 "WFH"를 쓰면 벡터 검색에서도 놓칠 수 있습니다.
- **사전 관리:** data.py 에 없는 약어를 넣으면 그대로 통과합니다. 사내에서 자주 쓰는 약어를 사전에 추가하면 바로 효과가 나옵니다. 슬랙에서 검색 실패 로그를 모니터링하면 "이 단어가 빠져 있구나"를 알 수 있습니다. 사전이 커질수록 검색 실패율이 떨어집니다.

질의 변환 도구 세 가지를 실험했습니다. 질문을 바꾸는 쪽은 여기까지입니다. 그런데 오픈이가 검색 결과 자체를 살펴봤습니다. 가져온 청크 3개를 나란히 놓으니 "연차 신청은 3일 전 서면으로", "팀장 승인이 필요합니다", "긴급한 경우 구두 신청 가능". 각각은 맞는 말인데, 조각조각 끊겨 있습니다.

질문을 잘 바꿔서 찾았는데, 찾아온 결과를 다듬는 도구도 필요하겠는데.

9.5 쪽지가 아니라 규정집을 펼친다 — 결과 가공 (step2)

동료 : "조각 3개 가져왔는데, 각각 한두 줄이면 LLM이 전체 맥락을 알 수 있어요?"

청크 여러 개를 가져와도 각 청크가 짧은 조각이면 LLM은 규정의 전체 흐름을 볼 수 없습니다. 질의 변환이 질문을 다듬는 작업이었다면, 결과 가공은 LLM에 넘기는 문서를 다듬는 작업입니다.

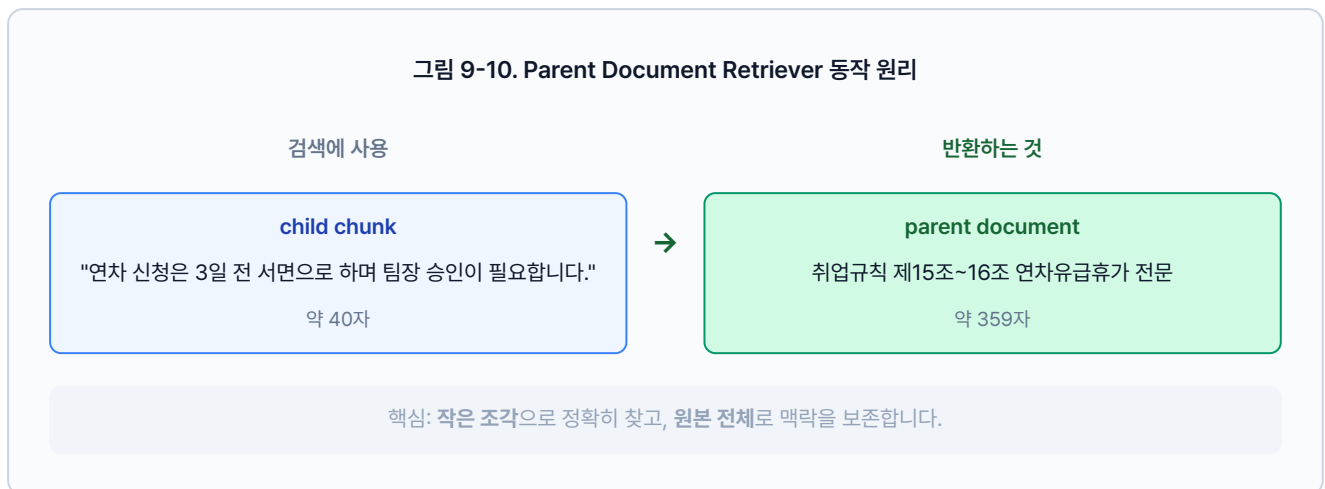
실습 흐름

1. 자식 청크로 검색하고 부모 문서 전체를 반환합니다 (`--step 2-1`)
2. 검색 후 관련 문장만 압축해서 반환합니다 (`--step 2-2`)
3. 질문에서 메타데이터 조건을 추출해 검색 대상을 좁힙니다 (`--step 2-3`)

9.5.1 원본 문서 꺼내기 — Parent Document Retriever

작은 청크로 검색하되, LLM에게는 그 청크가 속한 **부모 문서 전체**를 넘깁니다. 정확도(작은 청크)와 맥락(큰 문서)을 동시에 취하는 방식입니다.

그림 9-10. Parent Document Retriever 동작 원리



이 흐름을 코드로 구현합니다. 자식 청크별 유사도를 계산하고, 매칭된 조각이 속한 부모 문서 전체를 반환하는 클래스입니다.

`tuning/step2_advanced_retriever/parent_doc.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 2

tuning/step2_advanced_retriever/parent_doc.py. Parent Document Retriever

```
class ParentDocumentRetriever:
    def __init__(self, parent_docs, child_chunks, embeddings=None):
        self.parent_docs = {doc["id"]: doc for doc in parent_docs}
        self.child_chunks = child_chunks
        self.embeddings = embeddings

    def search(self, query, top_k=2):
        # TODO: 자식 청크로 검색하고 부모 문서를 반환합니다.
        # 1. 자식 청크별 유사도 계산
        query_vec = self.embeddings.embed_query(query)
        scored_chunks = []
        for vec, chunk in zip(self._child_vectors, self.child_chunks):
            score = _cosine_similarity(query_vec, vec)
            scored_chunks.append((score, chunk))

        scored_chunks.sort(key=lambda x: x[0], reverse=True)

        # 2. 중복 제거: 같은 부모는 한 번만
        seen_parents = set()
        results = []
        for score, chunk in scored_chunks:
            parent_id = chunk["parent_id"]
            if parent_id not in seen_parents and len(results) < top_k:
                seen_parents.add(parent_id)
                parent_doc = self.parent_docs.get(parent_id)
                # 3. 자식 청크 + 부모 문서 전체를 함께 반환
                results.append({
                    "parent_content": parent_doc["content"],
                    "child_chunk": chunk["content"],
                    "score": round(score, 4),
                })
        return results
```

작은 `child_chunk` 로 유사도를 계산하되, 결과에는 `parent_content` 전체가 담깁니다. `parent_docs` 는 `data.py` 의 원본 문서 리스트(ID, 제목, 전체 내용 포함)이고, `child_chunks` 는 원본을 잘게 쪼갠 조각 리스트입니다. 각 조각에는 자신이 속한 부모의 ID(`parent_id`)가 들어 있습니다. `self._child_vectors` 는 `__init__` 에서 모든 자식 청크를 미리 임베딩해 둔 벡터입니다. 같은 부모가 중복 반환되지 않도록 이미 꺼낸 부모는 건너뛸니다.

터미널

실험 2-1 실행. Parent Document Retriever

```
python -m tuning.step2_advanced_retriever --step 2-1
```

그림 9-11. 실험 2-1. ParentDocumentRetriever

원리: 작은 청크로 검색 → 원본 전체 문서 반환 (컨텍스트 풍부)

쿼리: 연차 신청 절차

순위	매칭 청크	부모 문서	부모 길이	유사도
1	연차 신청은 3일 전 서면으로...	취업규칙 제15조~16조 연차유급휴가	359자	0.6272
2	재택근무는 입사 6개월 이상...	재택근무 운영지침	269자	0.4314

매칭 청크는 한 문장이지만, 반환된 부모 문서는 359자 전문입니다.

"연차 신청은 3일 전 서면으로"라는 짧은 조각이 매칭됐지만, 실제로 반환된 건 취업규칙 제15조~16조 전문(359자)입니다. LLM이 "3일 전까지 뭘 해야 하는지" 앞뒤 맥락까지 볼 수 있습니다.

검색 결과 수를 바꿔서 맥락의 양을 조절해 보세요.

터미널 커스텀 쿼리 + top_k 변경

```
python -m tuning.step2_advanced_retriever --step 2-1 --query "병가 규정" --top_k 3
```

부모 문서 검색 실험에서 생각해 볼 것들

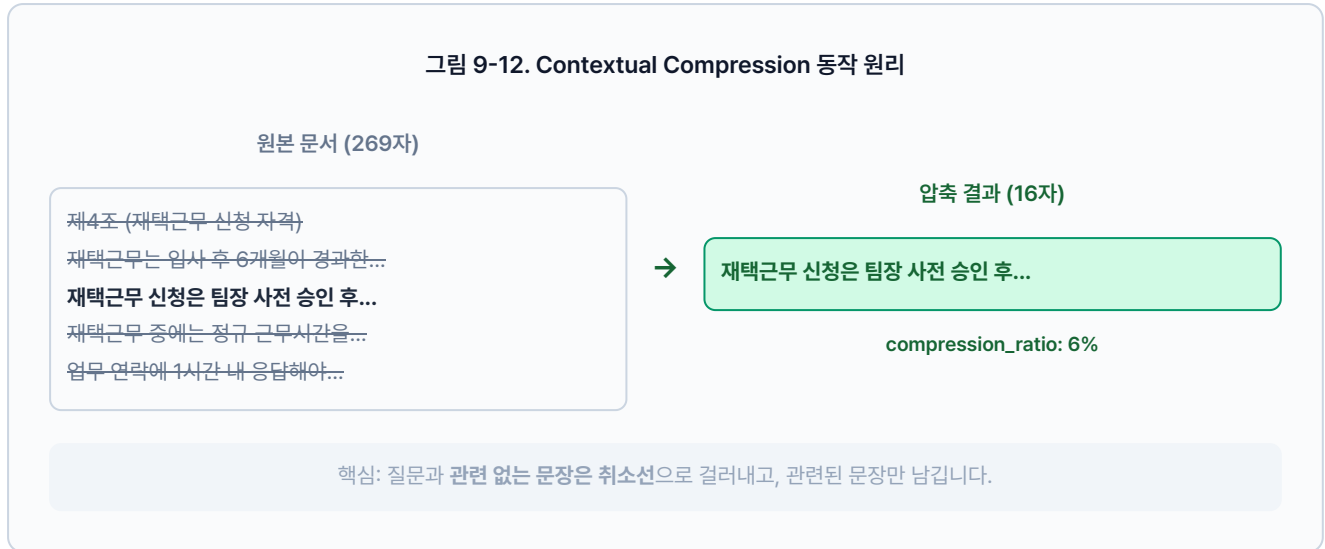
- **크기 차이:** child_chunk(매칭된 작은 조각)와 parent_content(반환된 원본)의 길이를 비교해 보세요. 조각은 100~200자인데 원본은 500자 이상일 겁니다. 이 차이만큼 LLM이 더 넓은 맥락을 볼 수 있습니다.
- **맥락 복원:** 작은 조각만 읽었을 때 의미가 불완전하지 않은지 확인해 보세요. "3일 전까지"라는 조각만으로는 뭘 3일 전까지 해야 하는지 모르지만, 원본에는 앞뒤 문맥이 있습니다.
- **top_k 조절:** `--top_k 5`로 늘리면 더 많은 부모 문서를 가져오지만, LLM에 넘기는 컨텍스트가 길어집니다. 문서 3개면 대부분 충분합니다.

맥락이 살아났습니다. 359자짜리 규정 전문이 LLM에 들어가니 "3일 전까지 뭘 해야 하는지" 앞뒤가 보입니다. 그런데 오픈이가 토큰 사용량을 확인했습니다. 부모 문서를 통째로 넣으니 프롬프트가 눈에 띄게 길어졌습니다.

동료: "규정 전문이 다 필요해요? 질문이랑 관련 없는 조항도 섞여 있잖아요."

9.5.2 핵심 문장만 남기기 — Contextual Compression

부모 문서를 통째로 가져오면 맥락은 살지만 쓸데없는 문장이 섞입니다. 사서가 규정집을 펼쳐 보여주되, 형광펜으로 질문과 관련된 문장만 밑줄 긋고 나머지는 접어 버리는 겁니다.



이 흐름을 코드로 구현합니다. 문서를 문장 단위로 쪼개고, 질문 단어와 겹치는 문장만 골라내는 클래스입니다.

`tuning/step2_advanced_retriever/compression.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 2 tuning/step2_advanced_retriever/compression.py. Contextual Compression

```

class ContextualCompressionRetriever:
    def __init__(self, documents, embeddings=None):
        self.documents = documents
        self.embeddings = embeddings

    def search(self, query, top_k=3):
        # TODO: 검색 결과에서 질문과 관련된 문장만 압축합니다.
        # 1. 유사도 기준으로 문서 정렬
        query_vec = self.embeddings.embed_query(query)
        doc_vecs = self.embeddings.embed_documents(
            [d["content"] for d in self.documents]
        )
        scored = [
            (_cosine_similarity(query_vec, dv), doc)
            for dv, doc in zip(doc_vecs, self.documents)
        ]
        scored.sort(key=lambda x: x[0], reverse=True)

        results = []
        for score, doc in scored[:top_k]:
            original = doc["content"]
            # 2. 핵심 문장만 추출 (최대 3문장)
            compressed = _compress(query, original)
            results.append({
                "original_content": original,
                "compressed_content": compressed,
                "compression_ratio": round(len(compressed) / len(original), 2),
            })
        return results

```

`_compress()` 함수가 문서를 문장 단위로 쪼갬 뒤 질문 단어와 겹치는 문장만 골라냅니다.

`compression_ratio` 가 0.3이면 원본의 70%를 덜어낸 것이니, 부모 문서를 통째로 가져와도 프롬프트 토큰을 크게 절약할 수 있습니다.

터미널 실험 2-2 실행. Contextual Compression

```
python -m tuning.step2_advanced_retriever --step 2-2
```

그림 9-13. 실험 2-2. ContextualCompressionRetriever

원리: 검색된 문서에서 쿼리 관련 부분만 추출 → LLM 토큰 절약

쿼리: 연차 신청 절차

순위	원본 길이	압축 길이	압축률	압축된 내용
1	269자	16자	6%	제4조 (재택근무 신청 자격)...
2	359자	34자	9%	긴급한 경우에는 구두 신청 후 사후 서면 제출...

269자 문서가 16자로 줄었습니다. 질문과 관련된 문장만 남깁니다.

359자짜리 연차 규정 문서가 34자로 줄었습니다. "연차 신청 절차"와 관련된 문장만 남기고 나머지는 잘라낸 겁니다. LLM에 넘기는 컨텍스트가 짧아지니 토큰 비용이 줄고 답변 정확도도 올라갑니다.

압축 검색 실험에서 생각해 볼 것들

- **압축률:** compression_ratio 수치를 확인해 보세요. 0.3이면 원본의 30%만 남긴 겁니다. 질문과 무관한 부분을 70% 덜어낸 셈입니다.
- **정보 보존:** compressed_content를 읽어 보세요. 질문에 답하는 데 필요한 핵심 정보가 남아 있어야 합니다. 너무 짧으면 중요한 내용이 잘려 나갔을 수 있습니다.
- **LLM 부담 감소:** 원본 500자를 150자로 줄이면 LLM이 읽을 양이 줄어듭니다. 문서 10개를 넘기는 상황에서 효과가 두드러집니다.

Parent Doc + Compression 조합. 이 둘은 세트로 쓸 때 효과가 큼니다. Parent Doc이 넓은 맥락을 가져오고 Compression이 핵심만 남기는 구조입니다. Parent Doc 단독 사용 시 부모 문서가 길어 토큰 비용이 늘어나는 문제를 Compression이 해결합니다.

Parent Doc과 Compression은 검색 결과의 양과 질을 조절하는 도구입니다. 마지막으로 하나 더. 오픈이가 동료의 질문 하나를 떠올렸습니다. "인사팀 관련 규정만 보고 싶은데." 이 질문에는 **부서명**이라는 조건이 섞여 있습니다. 벡터 유사도만으로는 "인사팀"이라는 조건을 걸 수 없습니다. 이런 유형의 질문이 들어온다면 쓸 수 있는 도구가 있습니다.

9.5.3 조건이 섞인 질문 — Self-Query Retriever

"2025년 인사팀 문서만" 같은 조건이 섞인 질문에서 **메타데이터 필터**를 자동으로 추출할 수 있습니다. 질문에 포함된 키워드를 감지해 `topic=인사팀`, `year=2025` 같은 조건을 만들고, 전체 문서 중 조건에 맞는 문서만 남긴 뒤 유사도 검색을 수행합니다.

"인사팀 관련 규정"이라는 질문이 들어오면 `{"topic": "인사"}` 필터가 자동으로 만들어지고, 수백 건의 문서 중 인사 관련 문서만 추려서 검색 대상으로 삼습니다. `topic_keywords` 는 `data.py` 에 정의된 토픽-키워드 매핑 딕셔너리로, `{"인사": ["인사", "인사팀", "hr"], "보안": ["보안", "정보보호"]}` 같은 구조입니다.

그림 9-14. Self-Query Retriever 동작 원리



설명 tuning/step2_advanced_retriever/self_query.py. Self-Query 필터 추출 + 검색

```
# self_query.py - 쿼리에서 메타데이터 조건을 자동 추출합니다
class SelfQueryRetriever:
    def __init__(self, documents, topic_keywords=None, embeddings=None):
        self.documents = documents
        self.topic_keywords = topic_keywords or {}

    def extract_filter(self, query):
        # 1. 질문에서 키워드를 감지해 메타데이터 필터 생성
        filters = {}
        query_lower = query.lower()
        for topic, keywords in self.topic_keywords.items():
            if any(kw in query_lower for kw in keywords):
                filters["topic"] = topic
                break
        return filters

    def search(self, query, top_k=3):
        filters = self.extract_filter(query)
        # 2. 필터에 맞는 문서만 남김
        filtered = self.documents
        for key, value in filters.items():
            filtered = [
                doc for doc in filtered
                if doc.get("metadata", {}).get(key) == value
            ]
        # 3. 남은 문서에서 유사도 검색
        scored = []
        for doc in filtered:
            # ... 유사도 계산 (생략)
            scored.append((score, doc))
        return [{"content": doc["content"], "applied_filter": filters}
                for _, doc in scored[:top_k]]
```

`extract_filter()` 가 질문에서 부서명이나 연도 같은 조건을 뽑아내고, `search()` 가 그 조건에 맞는 문서만 추려서 유사도 검색을 수행합니다. 전체 코드는 완성본 레포(ai-qa-lag)를 참고하세요.

터미널 실험 2-3 실행. Self-Query Retriever

```
python -m tuning.step2_advanced_retriever --step 2-3
```

그림 9-15. 실험 2-3. SelfQueryRetriever

원리: 질문에서 메타데이터 필터 자동 추출 → 정확한 필터링

쿼리: 재택근무 조건

자동 추출된 필터: **topic: 재택근무**

순위	내용	토픽	유사도
1	제4조 (재택근무 신청 자격) 입사 후 6개월이 경과한 정규직...	재택근무	0.7136

topic: 재택근무 필터가 자동 추출되어 해당 문서만 검색됩니다.

"재택근무 조건"이라는 질문에서 **topic: 재택근무** 필터가 자동으로 추출됐습니다. 전체 문서를 벡터 검색하는 대신 재택근무 관련 문서만 대상으로 좁혀 검색하니 무관한 문서가 섞이지 않습니다.

부서명이 포함된 질문을 넣어서 필터가 정확히 잡히는지 확인해 보세요.

터미널 커스텀 쿼리로 필터 추출 실험

```
python -m tuning.step2_advanced_retriever --step 2-3 --query "인사팀 관련 규정"
```

Self-Query 실험에서 생각해 볼 것들

- **필터 추출:** applied_filter에 어떤 메타데이터 조건이 추출됐는지 확인해 보세요. "인사팀 관련 규정"이라고 물으면 **topic: 인사** 처럼 부서명이 필터로 잡혀야 합니다.
- **검색 범위 축소:** 필터 적용 전후로 검색 대상 문서 수가 줄어드는지 비교해 보세요. 전체 20건에서 5건으로 줄어들면 그만큼 정확도가 올라갑니다.
- **필터가 안 잡히는 경우:** 질문에 메타데이터 단서가 없으면 필터 없이 일반 검색으로 넘어갑니다. "연차 신청 방법"처럼 부서나 카테고리 힌트가 없는 질문이 이 경우입니다.

다섯 가지 도구를 실험했습니다. 오픈이가 동료들의 질문 패턴을 다시 펼쳐 봤습니다. 약어를 많이 쓰고, 근거로 규정 전문을 보고 싶어합니다. 약어 사전과 Parent Doc — 이 두 개면 충분합니다.

전부 켤 필요 없어. 아픈 데만 약을 바르면 돼.

9.6 사서가 달라졌다

 그림 9-16. 챕터 9 파이프라인. 질문을 변환(확장, 재작성, HyDE)한 뒤 검색하고, 결과를 가공(Parent, Compression)해 LLM에 넘깁니다

동료에게 다시 써 보라고 했습니다. 동료가 키보드를 두드렸습니다.

동료 : "WFH 정책이 뭐야?"

"재택근무" 문서를 정확히 가져와 답변을 내놓습니다. 약어 사전이 "WFH"를 "재택근무(Work From Home)"로 바꿔 놓은 덕이었습니다.

동료 : "쉬는 날 규정 알려줘."

취업규칙 제15조 연차유급휴가 원본 문서 전체가 근거로 올라옵니다. 동의어 확장이 "휴가"를 "연차유급휴가"로 연결하고, Parent Doc이 원본 문서를 통째로 꺼내 왔습니다.

동료 : "...이번엔 진짜 되네요."

오픈이가 의자를 돌려 동료를 봤습니다.

오픈이 : "약어 사전이랑 Parent Doc만 컷거든요. 우리 동료들이 약어를 많이 쓰고, 근거로 규정 전문을 보고 싶어하니까요. 나머지는 필요할 때 켜면 돼요."

다섯 개 중 두 개. 검색 엔진은 안 건드렸는데 이렇게 달라진다.

의자가 삐걱거렸습니다. 팀장이 자리에서 일어나며 화이트보드를 훑듯 봤습니다.

팀장 : "텍스트는 됐는데요. PDF 속 표랑 이미지는?"

오픈이 : "표요?"

팀장 : "규정집에 표가 있잖아요. 그건 텍스트 검색으로 잡히나요?"

9.7 전체 구성도에서 챕터 9의 자리

그림 9-17. 전체 구성도. 짙은 박스가 챕터 9 튜닝 대상

사내 직원 · 관리자

챕터 2

대시보드

FastAPI

REST API · 관리자 웹

챕터 6+7

오케스트레이션 + 운영

ConnectHRAgent

ReAct · 사후 분류 + 캐시

MCP Tools

DB · 문서

챕터 5

검색 (실시간)

챕터 9 질의/결과 튜닝

LCEL Chain

HyDE · Multi-Query · Parent Doc ·
Compression

챕터 4

파싱·벡터화 (오프라인)

챕터 3 문서 규칙

PDF·Word·Excel·HWP

Doc Pipeline

파싱·청킹·임베딩

ChromaDB

벡터 저장소

챕터 2

데이터

PostgreSQL

직원·연차·매출

Ollama LLM (외부)

챕터 9는 LCEL Chain의 **입구(쿼리 변환)** 와 **출구(결과 가공)** 를 다듬었습니다. 같은 ChromaDB, 같은 문서인데 질문 해석력이 올라가 체감 품질이 크게 개선됩니다. 챕터 10에서는 텍스트로 못 잡는 **이미지, 표** 영역을 Vision LLM으로 공략합니다.

9.8 튜닝 구성도

챕터 8에서 검색 엔진 자체를 손봤고, 이번 챕터에서는 검색의 앞뒤를 다듬었습니다. 지금까지 실험한 튜닝과 앞으로 다룰 튜닝을 한 장으로 정리합니다.

그림 9-18. RAG 튜닝 메뉴판. 필요한 것만 골라 씁니다

	튜닝	무엇을 바꾸나	언제 쓰나	챕터
✓	칭킹 전략	문서를 자르는 방법 (Fixed → Recursive → Semantic)	주제가 섞이는 문제가 있을 때	8
✓	리랭킹	가져온 문서를 Cross-Encoder로 재정렬	검색 결과 상위에 엉뚱한 문서가 섞일 때	8
✓	하이브리드 검색	벡터 + BM25를 alpha 가중합으로 합침	의미 검색과 키워드 검색 둘 다 필요할 때	8
✓	HyDE	가상 답변을 먼저 생성해 답변-문서 유사도로 검색	질문과 문서의 어휘 차이가 클 때	9
✓	Multi-Query	한 질문을 여러 표현으로 재작성해 합집합 검색	질문의 의미가 넓어 한 방향만으로 부족할 때	9
✓	약어·동의어 확장	사내 용어 사전으로 약어를 원 표현으로 치환	사내 약어, 줄임말 검색 실패가 잦을 때	9
✓	Parent Document	작은 청크로 검색하되 부모 문서 단위로 반환	조각만 보면 맥락이 부족할 때	9
✓	Contextual Compression	검색된 문서에서 질문과 무관한 부분 제거	부모 문서가 길어 토큰 비용이 부담될 때	9
-	Vision LLM	이미지, 표를 Vision 모델로 읽어 텍스트 변환	PDF 속 표, 차트, 이미지가 검색에 빠질 때	10
-	하이브리드 파서	텍스트 + 시각 요소를 함께 파싱하는 파이프라인	복합 문서(텍스트+표+이미지)를 처리할 때	10
-	RAG 평가 프레임워크	검색 정확도, 답변 충실도를 숫자로 측정	튜닝 전후 품질을 객관적으로 비교할 때	10

✓ 챕터 8~9에서 실험한 항목 · - 챕터 10에서 다룰 항목

용어 정리

본문 속 표현	진짜 용어	정식 정의
"단어를 번역"	약어·동의어 확장	사내 용어 사전을 이용해 약어를 원 표현으로 치환
"답을 먼저 상상"	HyDE	질문에 대한 가상 답변을 LLM이 먼저 생성하고, 그 답변으로 검색
"여러 갈래로 물어보기"	Multi-Query	한 질문을 LLM이 여러 표현으로 재작성해 각각 검색 후 합집합
"원본 문서 전체 꺼내기"	Parent Document Retriever	작은 청크로 검색하되 반환은 큰 부모 문서 단위
"핵심 문장만 추리기"	Contextual Compression	검색된 문서에서 질문과 무관한 부분을 LLM이 제거
"조건을 필터로 변환"	Self-Query Retriever	LLM이 질문에서 메타데이터 조건을 뽑아 <code>where</code> 필터로 심음

이것만은 기억하자

- 검색의 앞뒤를 다듬으면 같은 엔진이 더 똑똑해집니다. 챗터 8이 엔진 자체를 바꿨다면 챗터 9는 입력(쿼리)과 출력(결과)을 가공합니다. 인프라 변경 없이 체감 품질이 가장 크게 오르는 구간입니다.
- 약어, 동의어 사전은 무조건 만드세요. 사내 용어는 공식 문서와 일상 표현이 다릅니다. "WFH ↔ 재택근무" 같은 매핑 한 줄이 실패 질문을 크게 줄여 줍니다.
- Parent Doc + Compression 조합이 안정적입니다. 정확도는 작은 청크, 맥락은 부모 문서, 프롬프트 비용은 압축으로 잡는 전략입니다.
- 챗터 10에서는 텍스트로 못 잡는 이미지와 표를 Vision LLM으로 읽고, RAG 전체 품질을 숫자로 측정합니다.

챕터 10. PDF 이미지까지 잡아라. Vision LLM과 RAG 평가

이번 챕터가 끝나면

- 스캔 PDF / 이미지 문서를 OCR과 Vision LLM으로 읽어 벡터 DB에 넣습니다
- 하이브리드 파서로 이미지가 있는 페이지는 Vision LLM, 텍스트만 있는 페이지는 pypdf로 자동 분기합니다
- RAG 평가 프레임워크로 Precision@k, Recall, Hallucination Rate를 숫자로 측정합니다
- 느낌이 아니라 숫자로 품질을 판단하는 습관을 가집니다

준비하기. 실습 시작 전 한 번만 설정

1. 실습 폴더 이동

터미널 폴더 이동

```
cd rag-start/ex10
```

파일 구조는 다음과 같습니다.

ex10 디렉토리

```

ex10/
├── requirements.txt          # [참고] easyocr · PyMuPDF · chromadb 등
├── data/
│   ├── docs/                # [참고] 원본 PDF (텍스트 + 스캔본)
│   ├── test_questions.json  # [참고] 평가 셋 (질문·정답·근거 문서)
│   └── chroma_db/           # 런타임 생성 (평가 프레임워크 벡터 DB)
├── tuning/
│   ├── step1_document_parser/ # [실습] OCR / Vision LLM 파서 비교
│   │   ├── __main__.py
│   │   ├── ocr.py           # [실습] EasyOCR 파서
│   │   ├── vision.py        # [실습] Vision LLM 파서
│   │   └── display.py
│   ├── step2_hybrid_parser/  # [실습] 텍스트 길이로 Vision 분기
│   │   ├── __main__.py
│   │   ├── hybrid_parser.py # [실습] 하이브리드 파서
│   │   └── display.py
│   └── step3_eval_framework/ # [실습] Precision@k · Recall · 환각률 + A/B/C/
D 조합 평가
    ├── __main__.py
    ├── metrics.py           # [실습] Precision@k · Recall@k · 환각률
    ├── strategies.py        # [참고] A/B/C/D 4 조합 정의
    ├── pipelines.py         # [참고] 파싱·칭킹·쿼리·검색·리랭크 부품
    ├── evaluator.py         # [참고] run_evaluation(strategy)
    └── display.py

```

챕터 7의 FastAPI 서버·채팅 UI·에이전트 레이어는 **챕터 11** · [ex11/ 레포](#)에서 완성 형태로 사용합니다. 챕터 10의 ex10은 튜닝 실험만 다룹니다.

2. 실습 환경 구축

터미널 환경 구성. macOS / Linux

```

cd ex10
python3.12 -m venv .venv
source .venv/bin/activate
cp .env.example .env
ollama pull qwen2.5vl:7b
pip install -r requirements.txt

```

터미널 환경 구성. Windows

```
cd ex10
py -3.12 -m venv .venv
.venv\Scripts\activate
copy .env.example .env
ollama pull qwen2.5vl:7b
pip install -r requirements.txt
```

Vision LLM, 어떤 모델로 실습을 돌릴지 먼저 정하세요

EasyOCR은 `Reader(["ko", "en"])` 처럼 언어 코드만 지정하면 한국어+영어를 함께 인식하니 별다른 선택지가 없습니다. 갈림길은 **Ollama Vision 모델** 쪽입니다. 파라미터 수가 성능과 요구 사양을 동시에 결정하고, **3B 이하**는 노트북 CPU에서도 돌지만 한글·표 인식이 불안정합니다. **7~8B**가 로컬 실무의 현실적 기준선이고, **13B 이상**은 품질이 더 좋지만 GPU가 사실상 필수입니다.

모델	VRAM/RAM	한국어 품질	권장 환경
<code>qwen2.5vl:7b</code>	8GB 이상	중음	RAM 8~16GB 노트북 (기본 권장)
<code>minicpm-v:latest</code>	8GB 이상	괜찮음 (환각 있음)	노트북, 빠른 실습용
<code>llama3.2-vision:11b</code>	16GB 이상	중음	RAM 16GB 이상 데스크톱 (고품질)

- **양자화(Quantization)** : Ollama 태그에 보이는 `q4_0` · `q5_K_M` 는 모델 가중치를 4~5비트로 압축했다는 뜻입니다. 용량과 RAM 사용량을 30~50% 줄여 주는 대신 정확도가 살짝 내려갑니다. 기본 태그(`:latest`)는 대개 품질과 크기의 절충점을 골라 둔 버전이라 실습에서는 그대로 써도 충분합니다.
- **고르는 순서**: (1) 내 머신 RAM·VRAM 확인 → (2) 그 안에 들어가는 후보 1~2개 선정 → (3) 사내 문서 5~10장으로 품질 비교. 벤치마크 점수보다 **우리 문서에서 잘 읽는지가** 최종 기준입니다.
- **상용 API를 쓸 수 있는 경우**: 인사 기록·계약서처럼 **외부 유출이 금지된 사내 문서**는 품질이 좋아도 API를 선택할 수 없습니다. 공개 자료나 외부 공문처럼 유출 제약이 없다면, **GPT-4V가 아니라** `gpt-4o-mini` 부터 검토하세요. Vision 입력을 그대로 지원하면서 **GPT-4o의 약 1/10 요금**으로, 표·스캔본 정도는 품질 차이를 체감하기 어렵습니다.

3. 사용할 라이브러리

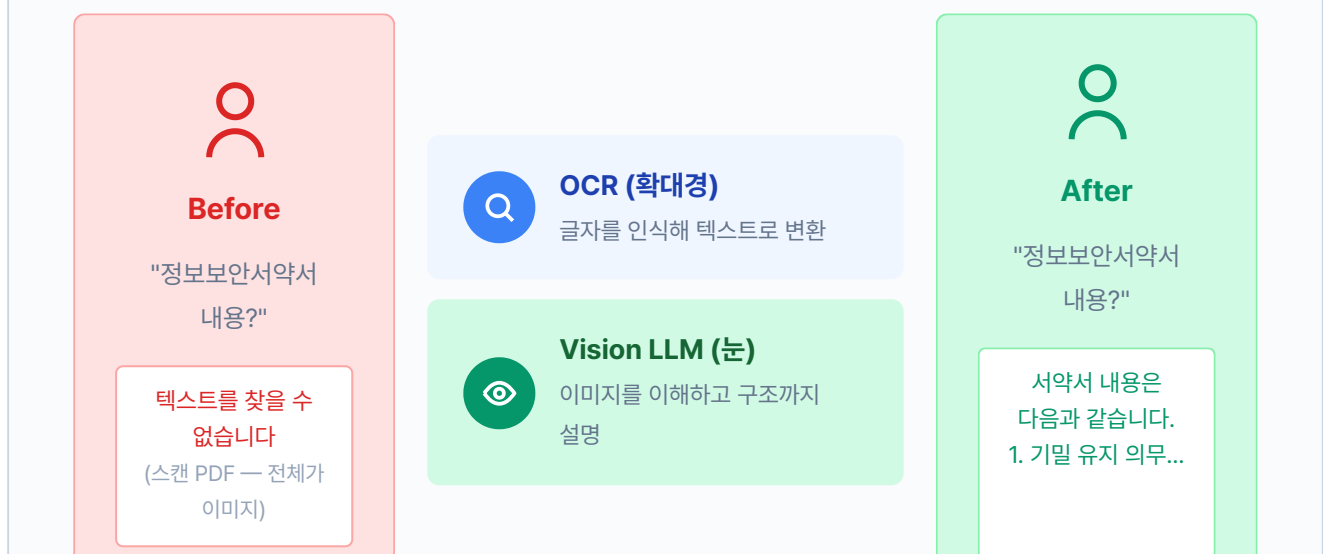
패키지	역할
<code>easyocr</code>	한국어+영어 OCR (EasyOCR)
<code>PyMuPDF</code> (<code>import fitz</code>)	PDF 페이지를 이미지로 변환
<code>Pillow</code>	이미지 전·후처리
<code>langchain-ollama</code>	Vision LLM 호출 (멀티모달 메시지)
<code>pypdf</code>	텍스트 레이어 감지·추출

4. 실습 순서

1. `python -m tuning.step1_document_parser` . OCR vs Vision LLM 비교
2. `python -m tuning.step2_hybrid_parser` . 이미지 감지 후 자동 분기
3. `python -m tuning.step3_eval_framework` . Precision@k, Hallucination Rate 측정

10.1 스캔 PDF: 텍스트가 없다

그림 10-1. 스캔 PDF 문제 해결. OCR + Vision LLM으로 이미지도 읽고, 숫자로 품질을 측정합니다



(Vision LLM이
이미지에서 읽음)

챕터 9까지 검색과 쿼리, 모든 축을 다듬었습니다. 금요일 오후, 키보드 소리만 또룩또룩 울리는 사무실에서 오픈이가 모니터를 정리하고 있을 때 팀장이 다가왔습니다.

팀장 : "이것도 넣어 줘요. 정보보안서약서."

A4 용지를 스캔한 듯한 PDF 한 장. 오픈이가 기존 파이프라인에 넣었습니다. pypdf 결과, 빈 문자열. 한 글자도 안 나왔습니다.

파일을 열어 봤습니다. 스캔본이었습니다. 종이를 스캐너에 올려 찍은 PDF라 페이지 전체가 하나의 큰 이미지. 텍스트 레이어가 없으니 pypdf가 꺼낼 글자 자체가 없었습니다.

사람 눈에는 선명하게 보이는데, 사서한테는 백지나 마찬가지잖아.

오픈이가 docs 폴더를 훑었습니다. 조직도는 박스와 화살표가 얹힌 이미지, 매출 차트는 막대그래프, 서약서는 스캔본. 사내 문서에는 글자만 있는 게 아닙니다. 이 중 어느 것도 챕터 4의 파서로는 제대로 못 읽습니다.

팀장 : "사서한테 눈을 달아야죠."

한마디가 머리에 걸렸습니다.

기존 파이프라인은 그대로, 앞뒤에 두 층만 엮습니다

챕터 4부터 7까지 쌓아 올린 파이프라인은 이번 챕터에서 건드리지 않습니다. 파싱·청킹·임베딩·검색·에이전트·캐시·모니터링까지 모든 층이 원래 자리에 그대로 있습니다. 챕터 8·9에서 실험한 튜닝(단락 청킹·리랭킹·약어 확장·부모 문서 검색 등)은 아직 이 파이프라인에 얹지 않은 부품 상태로 따로 놓여 있습니다. 이번 장의 할 일은 기존 파이프라인의 앞단과 뒷단에 한 층씩 새로 엮고(PDF 이미지 파서·RAG 평가 프레임워크), 뒷단에서 그 평가 도구로 챕터 8·9의 부품들을 조합해 어떤 조합이 정말 수치를 끌어올리는지 확인하는 작업입니다. 이 과정을 마치면 커넥트HR 파이프라인은 챕터 7의 기본 형태에서 스캔본까지 읽고 품질을 수치로 검증하는 **새 버전**으로 올라가 다음 챕터의 조립대 위로 넘어갑니다.

그림 10-2. 챕터 4~07 파이프라인은 그대로 두고 앞뒤에 두 층을 더합니다. 챕터 8·09 튜닝은 뒷단 평가에서 부품으로 조립합니다

챕터 10 추가 · 앞단

PDF 이미지 파서

스캔본·그래프·표가 들어와도 읽어냄. OCR(확대경) · Vision LLM(눈) · 하이브리드(pypdf → Vision 폴백)



챕터 4 ~ 챕터 7 기존 파이프라인 (그대로)

챕터 4

청킹·임베딩·Chroma

챕터 5~07

LCEL · 에이전트 · 운영 래퍼

챕터 8 · 챕터 9 — 파이프라인에 없지 않은 튜닝 부품 (뒷단 평가에서 조합)

챕터 8

검색 튜닝 (실험)

챕터 9

질의 재작성 (실험)



챕터 10 추가 · 뒷단

RAG 평가 프레임워크

답변 품질을 숫자로 측정. Precision@k · Recall · Hallucination Rate · Latency

중간(회색 점선)은 손대지 않습니다. 맨 위(파서)와 맨 아래(평가)만 이번 챕터에서 추가합니다.

10.2 확대경 달기 - OCR

팀장의 말에서 첫 번째 도구가 떠올랐습니다. **확대경**, 즉 **광학 문자 인식(OCR)** 입니다. 이미지 위의 글자를 한 자 한 자 인식해 텍스트로 바꿔 줍니다. 스캔본에서 글자를 꺼낼 때 씁니다.

확대경을 직접 만들어 보겠습니다. `tuning/step1_document_parser/ocr.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 1 tuning/step1_document_parser/ocr.py. EasyOCR로 글자 추출

```
import io
from pathlib import Path
import fitz # PyMuPDF
import numpy as np
from PIL import Image

def parse_pdf_ocr(pdf_path: str | Path, dpi: int = 150) → dict:
    # TODO: EasyOCR로 PDF 페이지를 읽어 텍스트를 추출합니다.
    import easyocr
    # 1. EasyOCR 리더 생성 (한국어+영어)
    reader = easyocr.Reader(["ko", "en"], gpu=False)
    # 2. PyMuPDF로 PDF를 열고 페이지별 이미지 변환
    doc = fitz.open(str(pdf_path))
    page_texts = []
    for page_num in range(len(doc)):
        page = doc[page_num]
        pix = page.get_pixmap(dpi=dpi)
        img = Image.open(io.BytesIO(pix.tobytes("png")))
        # 3. 이미지를 numpy 배열로 변환 후 OCR 실행
        img_array = np.array(img)
        ocr_results = reader.readtext(img_array, detail=0)
        page_texts.append("\n".join(ocr_results))
    doc.close()
    return {"text": "\n\n".join(page_texts)}
```

PyMuPDF가 PDF 페이지를 PNG 이미지로 변환해 주면, EasyOCR이 그 이미지에서 글자를 읽어 텍스트 문자열로 돌려줍니다.

두 도구의 역할은 정반대입니다. 하나는 **그려 내는 쪽**이고 다른 하나는 **읽어 내는 쪽**입니다. PDF에 텍스트 레이어가 있으면 pypdf로 바로 꺼내면 그만이지만, 스캔본은 글자 자체가 이미지로 박혀 있어 그 길이 막혀 있습니다. 그래서 한 발 돌아가는 셈입니다. PyMuPDF로 페이지를 고해상도 이미지로 그려 낸 뒤, EasyOCR에게 그 이미지에서 글자를 짚어 보라고 맡기는 두 단계 파이프라인을 탑니다.

PyMuPDF (`import fitz`). PDF 렌더링 라이브러리. 페이지를 원하는 해상도의 이미지로 그려내는 것이 주 역할입니다. 텍스트 레이어가 없는 스캔본도 겉모습(글자 모양·도장의 위치·표의 선)은 언제나 이미지로 뽑을 수 있습니다.

OCR (Optical Character Recognition, 광학 문자 인식). 이미지 속 글자 모양을 딥러닝 모델로 알아보고 텍스트로 옮기는 기술. **EasyOCR** 은 한글을 포함해 80여 개 언어를 기본 지원하는 오픈소스 구현체입니다.

DPI (Dots Per Inch). 1인치에 점을 몇 개 찍어 이미지를 만드는지 나타내는 해상도 단위. 숫자가 클수록 글자가 또렷해지는 대신 이미지가 커지고 처리 시간이 길어집니다.

첫 단계에서 해상도를 정하는 값이 `dpi=150` 입니다. 72는 화면에 띄우기엔 거칠고, 300은 인쇄용으로 선명하지만 무겁습니다. 그 사이 150은 OCR이 한글·영문 획을 구분할 만큼은 보이면서 페이지당 이미지 크기를 과하게 키우지 않는 실무 기본값으로 굳어졌습니다.

대상 파일은 `data/docs/hr/HR_정보보안서약서.pdf` (스캔본 1p)입니다. 이 장의 파싱 실험은 전부 이 한 장으로 진행합니다.

터미널 실험 1-1 실행. OCR로 스캔 PDF 읽기

```
python -m tuning.step1_document_parser --step 1-1
```

그림 10-3. 실험 1-1. OCR 파싱 (EasyOCR)

대상: HR_정보보안서약서.pdf

전략

OCR (EasyOCR)

추출 글자 수

755자

소요 시간

71.85초

미리보기

가인 승인 Choi Jang 정보보안 서사서 (스스년) 단서면요 2026-HK SEC 002 공기능하 대외비 (Conhidcntil)...

글자는 대부분 읽지만 표 구조는 일렬로 늘어섭니다.

1분 남짓 걸려 글자 대부분을 읽었지만, 미리보기를 보면 '정보보안 서사서'처럼 한글 오인식이 섞여 있습니다. 표는 더 심각했습니다.

그림 10-4. OCR의 한계. 글자는 읽지만 표 구조가 사라집니다

원본 (사람 눈에 보이는 것)

이름	직급	부서
김철수	대리	인사팀
박영희	과장	개발팀

OCR 결과 (확대경이 읽은 것)



이름 직급 부서 김철수 대리 인사팀 박영희 과장 개발팀

셀 구분 없이 일렬로 늘어섭니다

오픈이는 화면을 잠시 내려다봤습니다. 형광등 빛을 받은 커서가 "정보보안 서사서"라는 일곱 글자 옆에서 조용히 깜빡였습니다. 분명 규정집 첫 페이지에 또박또박 박혀 있던 제목. 기계의 눈을 거치자 '약'이 '사'로, '연'이 '년'으로 얼굴이 바뀌어 돌아와 있었습니다. 표도 마찬가지. 행과 열이 만들던 격자는 사라지고, 이름과 직급과 부서가 한 줄로 쏟아져 내렸습니다.

이대로 벡터 DB에 들어가면, 질문을 해도 엉뚱한 답이 돌아오겠지.

키보드 위에 얹혀 있던 손을 거두고 서랍에서 수첩을 꺼냈습니다. 터미널은 그대로 열어 둔 채, 방금 눈으로 본 두 가지를 적어 둘 참이었습니다. 한 시간만 지나도 "OCR이 좀 아쉬웠지"라는 인상만 남고, 정확히 어디서 어떻게 깨졌는지는 뭉툭해집니다. 어디서 어떻게 깨졌는지를 손으로 짚어 두지 않으면, 나중에 다른 도구를 고를 때 같은 함정으로 걸어 들어가기 쉽습니다.

- 실험 메모 -

1. 한글 오인식

- '정보보안 서약서' → '정보보안 서사서'
- '대외비 (Confidential)' → '공기능하 대외비 (Conhiddntil)'
- 도장·고유명사 근처 글자가 뭉개짐

2. 표 구조 소실

- 행·열이 사라지고 한 줄로 늘어섬
- "김철수 대리 인사팀 박영희 과장 개발팀"
- 누가 어느 부서인지 추론 불가

글자는 읽는데, 읽기만 하는구나.

수첩을 덮고 의자 깊숙이 기댔습니다. 화면 속 "스보보안 서의서"와 일렬로 늘어선 이름들이 천천히 위로 올라가다 사라졌습니다. 확대경은 글자를 크게 키워 주긴 했지만, 제 손에 쥐인 것이 서약서인지 출근부인지는 알지 못했습니다. 한 획씩 더듬어 읽는 점자. 딱 그 수준이었습니다.

다른 방법 없을까?

주전자에서 보리차를 따르며 며칠 전 팀장이 지나가듯 던진 말을 곱씹었습니다. "눈을 달아야죠." 그때는 무슨 뜻인지 흘러들었는데, 확대경의 한계를 눈으로 보고 나니 문장이 다르게 들렸습니다. 사람이 문서를 볼 때는 글자 하나하나를 읽기 전에 이미 제목을 알아보고, 표인지 도장인지 구분하고, 네모 칸이 누구를 가리키는지 짐작합니다. 글자를 해독하기 전에 **이해**가 먼저 옵니다.

지금 이 도구에 필요한 건 더 정밀한 확대경이 아니었습니다. 문서를 그림째로 이해해 주는 무언가, 표의 행과 열을 의미로 묶고 "스보보안 서의서"를 "정보보안 서약서"로 복구해 주는 **더 높은 층위의 눈**이 필요했습니다.

10.3 눈 달기 - Vision LLM

두 번째 도구는 **눈**입니다. 글자를 한 획씩 따라 읽는 눈이 아니라, 이미지 전체를 한 장면으로 이해하는 쪽입니다. **Vision LLM**은 사진·도표·스캔본을 보고 그 안에서 일어나고 있는 일을 말로 설명합니다. "이 차트에서 2024년 매출이 얼마야?"라고 물으면 막대그래프의 높이를 눈으로 재고 축의 숫자를 읽어 답을 돌려 줍니다. 조직도 앞에서는 화살표 방향을 따라가며 "마케팅팀장은 대표에게 보고하고, 그 아래 3명이 있습니다" 식으로 관계를 설명합니다. 표는 셀의 위치를 기억해 행과 열을 다시 짚니다.

OCR이 확대경이었다면, Vision LLM은 눈과 두뇌에 가깝습니다. 확대경은 보이는 것을 크게 보여 줄 뿐이지만, 눈은 본 것을 해석합니다. 문장의 의미를 알고, 문맥으로 오타를 복원하고, 표가 표라는 사실을 이해합니다. 확대경이 놓친 자리에 눈을 대면, 앞 실험에서 뭉개졌던 제목과 무너졌던 격자가 다시 제자리를 찾습니다.

이제 에이전트에 눈을 달아 줄 차례입니다. `tuning/step1_document_parser/vision.py`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 1 tuning/step1_document_parser/vision.py. Vision LLM으로 이미지 이해

```
from ._parser_utils import call_vision_llm

def parse_pdf_vllm(pdf_path: str | Path, dpi: int = 150) → dict:
    # TODO: Vision LLM으로 PDF 페이지 이미지를 읽어 텍스트를 추출합니다.
    # 1. PyMuPDF로 PDF를 열고 페이지별 이미지 변환
    doc = fitz.open(str(pdf_path))
    page_texts = []
    for page_num in range(len(doc)):
        page = doc[page_num]
        pix = page.get_pixmap(dpi=dpi)
        # 2. 페이지 이미지를 임시 파일로 저장
        img_path = f"_vllm_page_{page_num + 1}.png"
        pix.save(img_path)
        # 3. Vision LLM에 이미지를 보내 텍스트 추출
        caption = call_vision_llm(img_path)
        page_texts.append(caption)
        Path(img_path).unlink(missing_ok=True)
    doc.close()
    return {"text": "\n\n".join(page_texts)}
```

`call_vision_llm`은 `_parser_utils.py`에 있는 보조 함수로, Ollama의 Vision 모델에 이미지를 보내고 응답을 받습니다. 페이지를 임시 PNG로 저장한 뒤 LLM에 넘기고, 처리가 끝나면 임시 파일을 삭제합니다.

실험 1-1과 같은 `data/docs/hr/HR_정보보안서약서.pdf` (스캔본 1p)를 대상으로 돌려 OCR과 같은 조건에서 결과를 비교합니다.

터미널 실험 1-2 실행. Vision LLM으로 스캔 PDF 읽기

```
python -m tuning.step1_document_parser --step 1-2
```

그림 10-5. 실험 1-2. Vision LLM 파싱

대상: HR_정보보안서약서.pdf(스캔본 1p) · 모델: qwen2.5vl:7b · DPI: 100

전략
Vision LLM

추출 글자 수
788자

소요 시간
21.86초

미리보기

```
```markdown # 정보보안 서약서 (스캔본) **문서번호:** 2026-HR-SEC-002 **보존연한:** 영구 **공개등급:**
대외비 (Confidential) ## 제4조 (생성형 AI 활용 지침)...
```

제목·문서번호·조항 번호까지 마크다운 구조로 살려 냅니다.

글자 수는 OCR과 비슷한데도 제목과 조항 번호까지 정확히 돌려주고, 마크다운 표 구조가 살아 있어 훨씬 다루기 좋은 결과입니다. 대신 페이지 하나에 22초 남짓 걸렸습니다. 로컬 Vision LLM은 정확한 값을 주는 만큼 연산을 쓰고 있는 셈입니다. (qwen2.5vl:7b, CPU, DPI 100 기준. 더 큰 모델이나 GPU를 쓰면 품질은 올라가고 시간은 줄어듭니다)

오픈이는 모니터 앞으로 몸을 당겼습니다. 파이프 기호와 하이픈으로 그려진 마크다운 표. 줄을 맞춰 서 있었습니다. 조금 전까지 "김철수 대리 인사팀 박영희 과장 개발팀"이라고 한 줄로 쏟아지던 이름들이, 이번에는 이름끼리 직급끼리 각자 제자리로 돌아와 앉았습니다. 제목도 마찬가지. "정보보안 서약서" 여섯 글자가, 방금 전 "정보보안 서사서"로 얼굴이 바뀌어 있던 바로 그 자리에 또렷하게 박혀 있었습니다.

이건 확실히 다르네.

확대경은 점을 하나씩 따라가며 읽었는데, 이 친구는 페이지 전체를 한 번에 훑은 모양이었습니다. 표가 표라는 걸 알고, 제목이 제목이라는 걸 알고, 빈칸 다음에 오는 글자가 문맥상 무엇이어서 하는지까지 짐작해서 채워 넣었습니다.

그런데 감탄 끝에 붙은 조건이 마음에 걸렸습니다. 이 눈이 한 페이지를 읽어 낼 때마다 비싼 값을 치릅니다.

#### Vision LLM을 상시로 켜 두기 전에 확인할 비용

- **토큰 비용.** Vision LLM은 한 페이지를 통째로 이미지로 본 뒤 그 안에서 읽어 낸 내용을 토큰으로 쏟아냅니다. 도장·도표가 섞인 스캔본이라면 입력 이미지 토큰만 수천 단위로 불어나고, 답으로 되돌아오는 마크다운 텍스트도 덩달아 쌓입니다. API로 부르면 페이지 수만큼 요금이, 로컬이면 메모리·연산이 그만큼 빠져나갑니다.
- **하드웨어 문턱.** `qwen2.5vl` · `minicpm-v` 같은 7B급 모델은 양자화본 기준 6~8GB 여유 RAM을 요구합니다. CPU만으로도 돌아가지만 페이지당 수십 초 단위로 밀리고, GPU가 있어도 VRAM 8GB 이상이 있어야 안정적입니다. 속도는 단순히 "빠르다/느리다"가 아니라 어떤 머신에서 돌리느냐에 따라 몇 배씩 갈리는 값입니다. (모델 선택 기준은 장 앞쪽 준비하기 tip 참조)
- **중복 비용.** 사내 문서 대부분은 이미 텍스트 레이어가 멀쩡한 PDF입니다. pypdf 한 줄이면 끝날 문서에까지 Vision을 들이밀면 토큰도 메모리도 낭비입니다.

좋은 도구긴 한데, 아무 데나 쓸 건 아니구나.

같은 서약서를 두 도구로 읽은 결과를 나란히 놓으면 차이가 선명합니다.



그림 10-6. OCR은 빠르고 저렴하지만 구조를 모릅니다. Vision LLM은 느리지만 표, 차트, 조직도까지 정확하게 읽습니다

이해는 깊지만 그만큼 토큰과 메모리를 요구합니다. 이미 글자로 잘 꺼낼 수 있는 문서에까지 같은 비용을 치를 이유는 없습니다.

**동료** : "규정집은 텍스트 PDF잖아요. 그것까지 Vision에 돌리면 토큰도 시간도 아깝지 않아요?"

## 10.4 하이브리드 파서 - pypdf가 충분히 읽어 내면 그대로, 아니면 Vision

맞는 말이었습니다. 오픈이가 마우스 휠을 천천히 내리며 docs 폴더를 다시 훑었습니다. 파일 아이콘이 두 갈래로 나뉘어 있었습니다. HR 규정집, 출근 규정, 연차 매뉴얼. 내부에서 워드로 작성해 그대로 PDF로 내보낸 문서들이었습니다. 그 옆에 섞여 있는 정보보안 서약서. 종이에 도장을 찍은 뒤 스캐너를 거친 파일이었습니다.

한 폴더 안에 성질이 다른 두 종류가 있잖아.

두 유형에 같은 도구를 쓰는 것부터 어긋난 접근이었습니다. 규정집까지 Vision에 돌리면 한 줄이면 꺼낼 텍스트를 수십 초에 걸쳐 눈으로 다시 읽는 꼴이 되고, 반대로 pypdf만 고집하면 서약서 앞에서는 손을 놓게 됩니다. 둘 중 하나를 고르는 이분법이 아니라, 페이지마다 상황을 보고 쓰는 도구를 갈라야 했습니다.

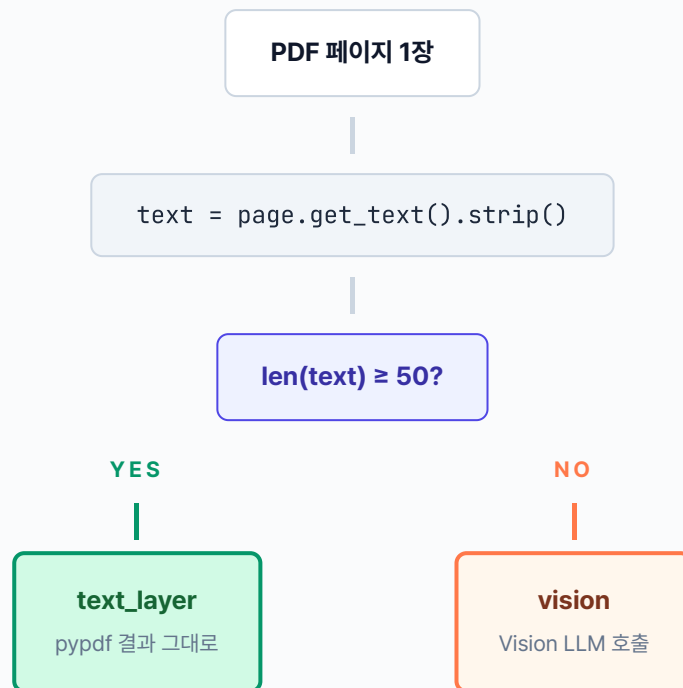
오픈이가 의자를 모니터 가까이 당겼습니다. 머릿속에서 그림이 한 장 그려졌습니다. 페이지가 들어오면 일단 pypdf에 맡긴다. 글자가 제대로 돌아오면 그대로. 빈손이거나 한 줄뿐이면 그제야 눈을 호출한다.

전략은 한 줄이었습니다.

**pypdf가 꺼낸 텍스트가 충분하면 그대로 쓰고, 아니면 눈을 부른다.**

페이지마다 `page.get_text()` 로 텍스트를 꺼낸 뒤, 길이가 50자 이상이면 pypdf 결과를 그대로 쓰고, 그보다 적으면 스캔본이나 차트 페이지로 보고 Vision에 넘깁니다. 판단은 페이지 단위로 독립이라, 10페이지짜리 문서에 스캔 페이지가 1장만 섞여 있어도 그 1장만 Vision으로 가고 나머지는 pypdf가 즉시 끝냅니다.

그림 10-7. 하이브리드 파서의 판단 흐름. 페이지마다 텍스트 길이로 분기합니다



일반 텍스트 페이지는 수백~수천 자, 스캔본·차트 페이지는 0~수십 자. 50자 기준 하나로 자연스럽게 갈라집니다

간단하게 만들어 보겠습니다. `tuning/step2_hybrid_parser/hybrid_parser.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 2 tuning/step2\_hybrid\_parser/hybrid\_parser.py. pypdf 결과로 파서 선택

```
import fitz # PyMuPDF
from ._hybrid_utils import vision_page

MIN_TEXT_LENGTH = 50 # 이 미만이면 스캔본 or 차트 페이지로 간주하고 Vision

def process_page_hybrid(
 page: fitz.Page,
 dpi: int = 150,
 vision_model: str | None = None,
) → dict:
 # TODO: pypdf 텍스트가 충분하면 text_layer, 부족하면 Vision.
 # 1. pypdf 텍스트 레이어 추출
 text = page.get_text().strip()
 # 2. 텍스트 충분 → 그대로 사용 (밀리초 단위)
 if len(text) ≥ MIN_TEXT_LENGTH:
 return {"strategy": "text_layer", "text": text, "char_count": len(text)}
 # 3. 텍스트 부족 → 스캔본 or 차트 페이지. Vision으로 전환
 vision_text = vision_page(page, dpi=dpi, model=vision_model)
 return {"strategy": "vision", "text": vision_text, "char_count": len(vision_text)}
 if vision_text else 0}
```

`page.get_text()` 는 PDF 페이지의 텍스트 레이어를 꺼내는 한 줄이고, `MIN_TEXT_LENGTH` 임계값은 "이 정도도 안 나오면 그림이 자리를 차지하고 있다"는 판단 기준입니다. 스캔본은 0자가 돌아오니 곧장 Vision으로 넘어가고, 텍스트 PDF는 수백 자가 돌아와 그 자리에서 끝납니다. 네이티브 벡터 차트 페이지도 축 레이블 몇 글자 정도라 50자 아래로 떨어지고, 결국 Vision 차례가 됩니다. 10.2절에서 한계를 확인한 OCR은 이 조합에 넣지 않았습니다.

## 이 하이브리드의 경계와 한 걸음 더

이번 파서는 **페이지 단위 텍스트 길이 하나로** 판단합니다. 그래서 커버하는 범위와 남는 한계가 명확합니다.

- **잘 잡는 것:** 스캔본(전체가 이미지), 일반 텍스트 PDF, 네이티브 벡터 차트로만 이뤄진 페이지. 이 챕터의 테스트 문서 전부가 여기 해당합니다.
- **놓치는 것:** 한 페이지 안에 본문 텍스트 + 작은 차트가 섞여 있는 경우. 본문이 50자를 훌쩍 넘기니 `text_layer` 로 분기되고, 차트의 막대 높이·선 기울기 같은 시각 정보는 손실됩니다. 축 레이블·범례 텍스트만 엉뚱하게 남습니다.
- **한 걸음 더:** 이 한계를 넘으려면 페이지를 **영역(bounding box) 단위로 쪼개서** 텍스트 영역은 pypdf, 차트 영역은 Vision으로 따로 보내야 합니다. 실무에서는 `unstructured`, `docling` (IBM), `LayoutParser` 같은 레이아웃 분석 도구가 이 역할을 맡습니다. 대신 설치·학습·디버깅이 한 단계 무거워집니다.
- **언제 확장하나:** 사내 보고서·기획서처럼 본문과 차트가 같은 페이지에 뒤섞인 문서 비중이 의미 있게 커지면 그때 레이아웃 분석 도구를 도입하세요. 스캔본·텍스트 PDF가 대부분이라면 지금 하이브리드로 충분합니다.

**터미널** 실험 2 실행. 하이브리드 파서로 페이지별 자동 분기

```
python -m tuning.step2_hybrid_parser
```

### 그림 10-8. 실험 2. 하이브리드 파싱

대상: HR\_정보보안서약서.pdf(스캔본 1p) · HR\_취업규칙\_v1.0.pdf(텍스트 1p) · Vision: qwen2.5vl:7b · DPI 100

페이지	전략	글자 수
서약서 p.1	vision	788
취업규칙 p.1	text_layer	1,426

전략 분포  
**vision 1/2 · text\_layer 1/2**

총 소요 시간  
**91.94초**

스캔본은 Vision이, 텍스트 PDF는 pypdf가 맡습니다. 한 파이프라인이 두 경로를 자동 선택합니다.

오픈이가 실행 버튼을 누르고 터미널을 지켜봤습니다. 노트북 팬이 제법 오래 숨을 몰아쉬더니 첫 줄이 올라왔습니다. `서약서 p.1 ... vision`. 한 페이지를 읽는 데 1분 30초 남짓. 그러다가 다음 줄에서 속도가 뚝 바뀌었습니다. `취업규칙 p.1 ... text_layer`, 1,426자, 0.17초. 깜빡이 한 번에

끝이었습니다.

파이프라인이 페이지를 한 장씩 들춰 본 다음 스캔본에는 눈을 대고, 텍스트 PDF는 pypdf로 곧장 꺾어 낸 모양새였습니다. 총 91.94초 가운데 91.77초가 Vision 한 번에 쓰였고, 텍스트 한 페이지는 사실상 공짜로 빠졌습니다. 필요할 때만 비싼 도구를 부르는 구조가 숫자로 그대로 드러났습니다. (Vision 시간은 모델을 처음 메모리에 올리는 콜드 스타트를 포함한 값입니다. 한 번 예열되면 호출당 20초 남짓으로 줄어듭니다)

되긴 됐다.

오픈이가 의자 등받이에 몸을 기댔습니다. 어깨에 들어가 있던 힘이 천천히 빠졌습니다. 고개를 돌려 옆자리 팀장을 불렀습니다.

**오픈이** : "팀장님, 서약서 건 돌아갑니다. 스캔본은 Vision으로, 규정집은 pypdf로 페이지마다 알아서 갈라져요."

팀장이 의자를 밀며 다가와 터미널 로그를 훑어봤습니다.

**팀장** : "오, 둘 다 한 파이프라인에서 처리되네요."

**오픈이** : "네. 글자가 충분히 나오면 그대로 쓰고, 부족하면 그때 Vision을 붙이는 식이에요."

팀장이 고개를 몇 번 끄덕이더니 로그를 처음부터 끝까지 한 번 더 눈으로 따라갔습니다. 그러고는 의자에서 일어서며 한 마디 던졌습니다.

**팀장** : "이게 얼마나 좋아진 건지는 어떻게 알아요?"

## 10.5 RAG 평가: 느낌이 아니라 숫자

---

바로 대답이 나오지 않았습니다. 오픈이가 잠시 시선을 내리더니 수첩을 펼쳤습니다. 챕터 8부터 여기까지 손본 튜닝이 한 페이지를 가득 메우고 있었습니다.



— 지금까지 쌓인 튜닝 —

## 1. 청킹

- 의미 기반 분리 (챕터 8)

## 2. 리랭커 (챕터 8)

- CrossEncoder 재정렬

## 3. 하이브리드 검색 (챕터 8)

- BM25 + 벡터 유사도

## 4. HyDE (챕터 9)

- 가상 답변을 먼저 만들어 검색

## 5. 부모 문서 검색 (챕터 9)

- 자식 청크로 찾고 부모 페이지 반환

## 6. 약어 확장 (챕터 9)

- WFH → 재택근무

## 7. 하이브리드 파서 (챕터 10)

- pypdf 부족하면 Vision 전환

생각해 보니 이 파서 하나만이 아니네. 지금까지 엮은 것들도 다 "좋아졌다"고만 말했지, 얼마나 기여했는지는 모른다.

체감만 있고 근거가 없었습니다. 튜닝을 더 엮을지, 이쯤에서 멈출지, 어느 조합으로 갈지. 전부 숫자가 뒤를 받쳐 줘야 답할 수 있는 질문이었습니다. 팀장의 한마디는 이번 파서 하나를 묻고 있는 것이 아니었습니다. 지금까지 쌓인 튜닝 전부에 대한 답을 요구하고 있었습니다.

평가 프레임워크를 꺼낼 차례였습니다.

평가 프레임워크는 RAG 시스템의 품질을 숫자로 재는 채점기입니다. 같은 질문 셋을 여러 튜닝 조합에 던지고, 미리 정해 둔 지표로 결과를 찍어 비교하는 구조입니다. 선생님이 학생들의 시험지를 같은 채점표로 내려 점수를 매기듯, 여러 조합을 같은 잣대 위에 놓아야 어느 튜닝이 정말 값을 하는지 가려낼 수 있습니다.

먼저 어떤 지표로 잴지 정해야 합니다. 이 책에서는 서로 다른 각도를 잡아 줄 네 가지로 추렸습니다.

- **Precision@k** — "검색이 정답을 위에 잘 올렸나?" (상위 k칸의 정확성)
- **Recall@k** — "정답을 빠짐없이 잡았나?" (정답 커버리지)
- **Hallucination Rate** — "답변이 근거 문서 안에서 나왔나?" (지어내기 억제)

- **Latency** — "사용자가 기다릴 만한 속도인가?" (응답 시간)

Precision과 Recall은 검색이 어떻게 돌아가는지를 들여다보고, Hallucination Rate는 만들어진 답변을 뜯어 보고, Latency는 사용자가 실제로 겪는 대기 시간을 잽니다. 한 지표만 붙들면 다른 쪽이 무너져도 모르고 지나칩니다. Recall만 끌어올리다 보면 관련 없는 문서까지 딸려 와 환각이 늘어나고, 속도만 좇으면 품질이 뒷줄로 밀립니다. 네 지표를 한 세트로 놓고 봐야 전체 모양이 보입니다.

지표	의미	계산
<b>Precision@k</b>	상위 k개 중 정답 비율	$(\text{정답 청크 수}) / k$
<b>Recall</b>	전체 정답 중 발견한 비율	$(\text{발견한 정답 수}) / (\text{전체 정답 수})$
<b>Hallucination Rate</b>	답변 중 문서 근거 없는 문장 비율	답변과 근거 문서의 단어 겹침 비율 (더 정확히 잴 땐 별도 LLM에게 채점을 맡깁니다)
<b>Latency</b>	질문당 평균 응답 시간	ms 단위

#### 이 네 지표는 업계 표준과 같은 축을 잽니다

Precision@k-Recall@k는 **문서 검색** 분야에서 오래 쓰인 교과서 지표이고, Hallucination Rate는 RAG가 등장한 뒤 자리 잡은 항목, Latency는 운영 지표의 기본입니다. RAGAS·LangSmith 같은 오픈소스 RAG 평가 도구도 이름만 살짝 바꿔 같은 네 축(혹은 그 변형)을 잽니다. 뒤 10.5.3에서 "이 책의 간단한 구현을 실무에선 어떻게 더 정밀하게 바꾸는지"를 다시 짚습니다.

오픈이가 새 파일을 하나 만들었습니다. `data/test_questions.json`. 비어 있는 배열. 대괄호 사이에서 커서가 조용히 깜빡였습니다.

뭘 기준으로 물어보지?

동료들이 이제까지 던진 질문들을 떠올려봤습니다. "연차 신청은 어떻게 해요?", "병가 며칠까지 돼요?", "USB 써도 돼요?", "이번 분기 매출 얼마예요?" 기억나는 것들을 한 줄씩 적어 내려갔습니다. 매 질문 옆에는 이 답이 어느 문서에서 나와야 하는지, 이상적인 답변은 어떤 내용인지를 함께 적어 뒀습니다.

질문 31개가 쌓였습니다. 각 건은 `query` (테스트 질문), `relevant_sources` (그 답이 담겨 있어야 할 원본 문서명 리스트), `expected_answer` (이상적 답변) 세 축. 앞 두 필드는 Precision@k-Recall 계산에, 마지막 필드는 Hallucination Rate 판정에 쓰입니다.

이게 우리 시스템의 채점지네.

`tuning/step3_eval_framework/metrics.py` 를 열면 Precision@k와 환각률 함수 두 개가 TODO로 비어 있습니다. 하나씩 채워 나갑니다.

### 10.5.1 Precision@k — 상위 k개에 정답 문서가 얼마나 올라왔나

첫 번째 지표는 **Precision@k**. 시스템이 돌려준 검색 결과 상위 k개 중에 정답 문서가 몇 개 들어 있는지를 비율로 재는 지표입니다. 채점지로 치면 "답안지 첫 k칸 중 정답이 몇 칸인가"를 세는 항목입니다. 오픈이가 JSON에 적어 둔 `relevant_sources` 가 정답지가 되고, 시스템이 돌려준 검색 결과 문서명이 답안지가 됩니다.

`calculate_precision_at_k` 함수의 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

**실습 3** tuning/step3\_eval\_framework/metrics.py. Precision@k 계산

```
def calculate_precision_at_k(
 retrieved_sources: list[str],
 relevant_sources: list[str],
 k: int,
) → float:
 # TODO: 상위 k개 결과 중 정답 문서명을 포함한 비율을 반환합니다.
 # 1. 상위 k개만 추린다
 top_k = retrieved_sources[:k]
 # 2. 정답 문서명은 set으로 묶어 조회 속도를 O(1)로 만든다
 relevant_set = set(relevant_sources)
 # 3. 각 top-k 결과 source가 정답 문서명 중 하나라도 포함하는지 센다
 hits = sum(1 for src in top_k if any(rel in src for rel in relevant_set))
 # 4. k로 나눠 비율을 돌려준다 (k=0 방어)
 return hits / k if k > 0 else 0.0
```

매개변수·동작을 풀면 다음과 같습니다.

매개변수	역할
<code>retrieved_sources</code>	시스템이 검색해 가져온 청크의 source 문서명 리스트. 순서가 중요 (위에 올린 것부터 k번째까지만 본다)
<code>relevant_sources</code>	JSON에 적어 둔 정답 문서명 리스트
<code>k</code>	상위 몇 개까지 채점할지 정하는 숫자 (보통 3 또는 5)

문자열 부분 일치(`rel in src`)로 판단한 데는 이유가 있습니다. 검색 결과 source에는 `_chunk_5` 같은 청크 번호 접미사가 붙을 때가 많아서, 정답 문서명이 포함되지만 하면 맞다고 처리해야 합니다. 결과는 0에서 1 사이 값이고, 1에 가까울수록 상위 k 안에 정답 문서가 많이 들어와 있다는 뜻입니다.

예시를 따라가 보겠습니다. `retrieved_sources = ['HR_취업규칙_v1.0_chunk2', 'SEC_보안규정_v1.0_chunk1', 'HR_취업규칙_v1.0_chunk7']`, `relevant_sources = ['HR_취업규칙_v1.0']`, `k=3` 일 때, 상위 3개 중 2개가 `HR_취업규칙_v1.0` 을 포함하니 `Precision@3 = 2/3 ≈ 0.67` 이 됩니다.

그럼 0.67이라는 숫자는 무엇을 의미하는 걸까요. 상위 3칸 중 약 67%가 정답 문서였다는 얘기입니다. 1.0이면 상위 k칸이 전부 정답 문서로 채워진 이상적인 상태, 0.0이면 상위 k칸 어디에서도 정답을 찾지 못한 상태입니다. 실무에서는 0.7 이상이 나오면 검색이 꽤 잘 맞추고 있다고 보고, 0.5 아래로 떨어지면 튜닝이 필요하다는 신호로 읽습니다. 이 숫자 하나로 "검색이 정답을 위에 얼마나 잘 올렸나"를 비교할 바탕이 생긴 셈입니다.

### 10.5.2 Recall@k — 정답 문서를 빠짐없이 찾았나

**Recall@k**는 Precision@k의 반대편에서 같은 검색 결과를 바라봅니다. Precision이 "답안지에 정답이 몇 %냐"를 물었다면, Recall은 "정답지의 문서를 몇 개나 찾았느냐"를 묻습니다. 정답 문서를 놓치는 일이 잦을수록 점수가 떨어지는 지표입니다.

같은 파일의 `calculate_recall_at_k` 함수 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

**실습 3** tuning/step3\_eval\_framework/metrics.py. Recall@k 계산

```
def calculate_recall_at_k(
 retrieved_sources: list[str],
 relevant_sources: list[str],
 k: int,
) → float:
 # TODO: 정답 문서 중 상위 k개 안에 들어온 비율을 반환합니다.
 # 1. 상위 k개만 추출다
 top_k = retrieved_sources[:k]
 # 2. 정답 문서명을 set으로 묶는다
 relevant_set = set(relevant_sources)
 # 3. 각 정답 문서명이 top-k 결과 중 하나라도 포함되는지 센다
 hits = sum(1 for rel in relevant_set if any(rel in src for src in top_k))
 # 4. 정답 문서 개수로 나눠 비율을 돌려준다
 return hits / len(relevant_set) if relevant_set else 0.0
```

Precision@k와 코드가 거의 똑같아 보이지만, **나눗셈의 분모**가 다릅니다.

지표	보는 방향	분모
Precision@k	답안지에 정답이 얼마나?	k (답안 칸 수)
Recall@k	정답 중 얼마나 찾았나?	정답 문서 개수

예시로 감을 잡아 보겠습니다. 정답 문서가 `['HR_취업규칙_v1.0']` 하나인데 상위 3개 중 2개가 이 문서 청크라면, `Precision@3 = 2/3 ≈ 0.67` 이지만 `Recall@3 = 1/1 = 1.0` 입니다. 답안지에 오답이 하나 섞였어도 정답은 빠짐없이 잡아낸 셈입니다. Precision은 답안의 정확성을, Recall은 빠짐없는 정도를 본다고 기억해 두면 됩니다.

Precision과 Recall 두 지표는 검색 쪽 성적표 역할을 합니다. 상위 k개에 정답이 얼마나 들어 있는지 (정확성), 정답이 얼마나 빠짐없이 잡혔는지(커버리지)를 각각 보여 줍니다. 다만 검색을 잘했다고 답까지 맞다는 보장은 없습니다. 올바른 문서를 펴 놓고도 내용을 엉뚱하게 옮기면 답은 틀어집니다. 이번엔 답변 자체를 채점할 차례입니다.

### 10.5.3 환각률 — 답변이 근거 문서에서 벗어났다

두 번째 지표는 **환각률**. 시스템이 문서에 없는 내용을 지어내진 않았는지 재는 지표입니다. 학생이 시험에서 문제지에 없는 답을 써내는 상황을 떠올리면 됩니다.

가장 정확한 방법은 또 다른 LLM을 심사위원으로 세워 문장마다 판정하게 하는 것이지만, 로컬 환경에선 너무 무겁습니다. 그래서 이 책은 **가벼운 방식**을 씁니다. 답변에서 핵심 단어를 뽑아 근거 문서에 실제로 등장하는지 센 다음, 너무 적게 나오면 "문서에서 벗어난 답"으로 판정하는 방식입니다.

같은 파일의 `estimate_hallucination_rate` 함수 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 3 tuning/step3\_eval\_framework/metrics.py. 환각률 추정

```
def estimate_hallucination_rate(
 answers: list[str],
 contexts: list[list[str]],
) → float:
 # TODO: 답변이 컨텍스트에 근거하지 않는 비율을 추정합니다.
 hallucination_count = 0
 # 1. 각 (답변, 컨텍스트) 쌍을 순회한다
 for answer, context_docs in zip(answers, contexts):
 # 2. 컨텍스트 문서들을 하나의 소문자 문자열로 합친다
 context_combined = " ".join(context_docs).lower()
 # 3. 답변에서 4자 이상 단어만 뽑아 핵심 단어 후보로 삼는다
 key_words = [w for w in answer.lower().split() if len(w) > 3]
 if key_words:
 context_words = set(context_combined.split())
 # 4. 핵심 단어 중 컨텍스트에 등장하는 비율이 30% 미만이면 환각으로 카운트
 overlap = len([w for w in key_words if w in context_words]) / len(key_words)
 if overlap < 0.3:
 hallucination_count += 1
 # 5. 환각 답변 수를 전체 답변 수로 나눠 비율 반환
 return hallucination_count / len(answers) if answers else 0.0
```

매개변수	역할
answers	시스템이 낸 답변 문자열 리스트
contexts	각 답변에 쓰인 근거 문서 조각들의 리스트 (답변당 여러 조각이 묶임)

4자 이상 단어만 뽑는 건 한국어 조사나 짧은 단어가 비율을 흐리지 않게 하려는 장치입니다. 30% 기준은 "답변 핵심 단어 10개 중 3개도 문서에 못 찾으면 수상하다"고 보는 실무 감각에서 나온 숫자입니다. 결과는 0에서 1 사이의 비율이고, 0에 가까우면 시스템이 문서에 충실한 쪽, 1에 가까우면 지어낸 답이 많은 쪽입니다.

이 방식의 한계는 **표현만 바꾼 답**을 잡지 못한다는 점입니다. 문서에 "연간 30일"이라고 써 있는데 답변이 "한 해에 30일"로 바꿔 쓰면, 단어가 달라졌다는 이유로 환각으로 오판할 수 있습니다. 더 높은 정밀도가 필요한 실무에선 별도의 LLM에 판정을 맡기는 방식으로 갈아타는 게 좋습니다.

### 이 책의 환각률 측정을 실무에선 이렇게 업그레이드합니다

- **간단한 단어 겹침 대신 LLM 채점:** 이 책의 환각률은 "답변 단어가 근거 문서에 얼마나 겹치나"를 세는 간단한 방식입니다. 실무에선 보통 **별도 LLM에게 문장별로 채점을 맡기는 방식**(업계에선 이 접근을 LLM-as-judge라고 부릅니다)으로 바꿉니다. 단어가 달라져도 뜻이 같으면 옳은 답으로 인정해 줘서 오판이 줄어듭니다.
- **또는 의미 유사도 비교:** 문장을 벡터로 바꾼 뒤 답변 문장과 근거 문장의 벡터가 얼마나 가까운지 잹니다. 챗터 4의 임베딩을 똑같이 씁니다. 단어 겹침보다 "표현만 바꾼 답"에 강합니다.
- **대표 평가 도구:**
  - **RAGAS:** 가장 많이 쓰이는 오픈소스. Faithfulness(답변이 근거에 충실한가) · Answer Relevancy(질문에 잘 답하는가) · Context Precision · Context Recall 네 축을 자동 측정
  - **TruLens:** Groundedness · Context Relevance · Answer Relevance 세 축("RAG Triad"라는 이름으로 통용)
  - **LangSmith:** LangChain의 실행 기록·평가 플랫폼. 서비스에 올라간 뒤 들어오는 요청 로그와 평가를 한자리에서 봅니다
  - **DeepEval:** `pytest` 로 단위 테스트를 쓰듯 RAG 품질을 테스트. 자동 빌드·테스트 환경(CI)에 붙이기 쉽습니다
  - **Arize Phoenix:** 오픈소스 실행 기록 + 평가 도구. 결과를 그래프 대시보드로 보여주는 게 강점
- **출발점 추천:** 지표만 빠르게 보고 싶다면 RAGAS, LangChain으로 시스템을 만들었다면 LangSmith, 자동 테스트 중심이면 DeepEval이 시작점으로 좋습니다.

### 10.5.4 조합별 측정 - 튜닝을 플러그처럼 갈아끼운다

지표는 준비됐지만 아직 절반입니다. 같은 질문 셋을 돌려 보려면 **튜닝 조합을 갈아끼우는 스위치**가 있어야 합니다. 파싱·청킹·쿼리 변환·검색·리랭크 같은 조각을 이름별로 하나씩 꺼내 써서, 조합마다 파이프라인 하나를 조립하는 구조가 필요합니다.

오픈이가 `step3_eval_framework/pipelines.py` 에 부품 함수들을 모았습니다. 한 함수가 한 조각을 담당합니다. 그리고 `strategies.py` 에서 이 부품들을 묶어 A/B/C/D 네 조합으로 정의했습니다. D 조합(챗터 8~10 튜닝이 모두 켜진 최종 형태)을 기준으로, 파이프라인의 네 단계에 어떤 튜닝이 어디에 붙는지 한 장에 담았습니다.

그림 10-9. 튜닝 로드맵. 네 단계를 ㄹ자로 잇고 각 단계에 어떤 튜닝이 붙는지 표시합니다



각 조합(A/B/C/D)은 위 네 단계에서 **얼마나 많은 튜닝 스위치를 켜는지**만 다릅니다. A는 한 개도 켜지 않은 baseline, D는 모두 켜 최종 형태입니다.



참고 tuning/step3\_eval\_framework/strategies.py. 조합 정의

```
from dataclasses import dataclass
from . import pipelines as P

@dataclass(frozen=True)
class Strategy:
 name: str
 description: str
 parse_fn: callable
 chunk_fn: callable
 query_transform_fn: callable
 retrieve_fn: callable
 rerank_fn: callable

STRATEGIES = {
 "A": Strategy("A (baseline)", "페이지 청킹 + 벡터 검색",
 P.parse_pypdf, P.chunk_page, P.query_noop,
 P.retrieve_cosine, P.rerank_noop),
 "B": Strategy("B", "단락 청킹 + 키워드 리랭킹",
 P.parse_pypdf, P.chunk_semantic, P.query_noop,
 P.retrieve_cosine, P.rerank_keyword),
 "C": Strategy("C", "B + 약어 확장 + Parent Doc",
 P.parse_pypdf, P.chunk_semantic, P.query_expand_abbreviations,
 P.retrieve_parent_doc, P.rerank_keyword),
 "D": Strategy("D", "C + 하이브리드 파서",
 P.parse_hybrid, P.chunk_semantic, P.query_expand_abbreviations,
 P.retrieve_parent_doc, P.rerank_keyword),
}
```

`pipelines.py`에는 각 부품의 실제 로직이 들어 있습니다. 예를 들어 하이브리드 파서를 갈아끼우는 부품은 `step2_hybrid_parser`에서 이미 만든 `process_page_hybrid`를 그대로 재사용합니다.

참고 tuning/step3\_eval\_framework/pipelines.py. 하이브리드 파서 부품

```
def parse_hybrid(pdf_path) → list[str]:
 """pypdf 텍스트가 부족하면 Vision으로 전환. step2_hybrid_parser 재사용."""
 from ..step2_hybrid_parser.hybrid_parser import process_page_hybrid
 doc = fitz.open(str(pdf_path))
 pages = []
 for page in doc:
 result = process_page_hybrid(page)
 pages.append(result.get("text", ""))
 doc.close()
 return pages
```

`evaluator.run_evaluation(k, strategy_name)` 이 `STRATEGIES` 에서 조합을 꺼내 부품을 순서대로 호출합니다. A를 돌리면 A 파이프라인이, D를 돌리면 D 파이프라인이 동일 질문 셋에 적용되어 지표가 나옵니다.

**터미널** 조합별 평가 실행

```
조합 하나만
python -m tuning.step3_eval_framework --strategy D --k 3

네 조합을 한 번에 (그림 10-10 비교표 스타일로 출력)
python -m tuning.step3_eval_framework --strategy all --k 3
```

`--strategy all` 을 쓰면 A부터 D까지 차례로 벡터DB를 다시 짓고 같은 질문 셋을 돌려 **네 줄짜리 비교표**를 터미널에 찍어 줍니다.

그림 10-10. RAG 평가 - 조합별 비교

같은 질문 31개로 4가지 튜닝 조합을 비교한 실측 결과표입니다. 환경: data/docs 문서 6건(PDF 3 · DOCX 1 · XLSX 2), k=3, Vision: qwen2.5vl:7b, 로컬 임베딩(ko-sroberta-multitask)

조합	구성	Precision@3	Recall@3	환각률	Latency
A (baseline)	페이지 청킹 + 벡터 검색	0.204	0.452	0.871	78 ms
B	단락 청킹 + 키워드 리랭킹	0.333	0.323	1.000	78 ms
C	B + 약어 확장 + Parent Doc	0.247	0.548	0.903	85 ms
D	C + 하이브리드 파서	0.236	0.532	0.871	80 ms

환각률은 D가 최저(0.871). Recall은 C·D가 베이스라인 A를 0.08~0.10 끌어올렸고, 스캔본까지 본문으로 편입되는 조합은 D뿐입니다

### 실측표를 어떻게 읽을까

- **Precision@3이 낮게 보이는 이유:** 6건짜리 문서 세트는 질문당 정답 후보가 보통 한 건입니다.  $k=3$ 으로 세 칸을 다 채울 때 앞 한 칸만 맞아도 Precision은  $1/3(\approx 0.33)$ 이 위쪽 한계입니다. 표의 B가 그 한계에 거의 맞닿아 있습니다.
- **Recall@3을 최우선 지표로:** 이 시스템은 "정답 문서를 놓치지 않고 올리느냐"가 먼저입니다. C·D가 A(0.452)보다 0.08~0.10 높고, 약어 확장·부모 문서 복원·Vision 편입이 검색 커버리지를 실제로 넓혔다는 뜻입니다.
- **환각률이 왜 0.87~1.00인가:** `estimate_hallucination_rate` 는 `expected_answer` (문서에 없는 가상 정답)와 검색된 청크의 단어 겹침만 봅니다. 예제용 6건 문서 세트에는 이 가상 수치가 그대로 담겨 있지 않아 겹침이 낮게 나옵니다. 수치의 절대값보다 조합 간 상대 비교로 읽으세요.
- **D가 환각률에서 앞서는 이유:** qwen2.5vl이 스캔본에서 "정보보안 서약서" "제4조 (생성형 AI 활용 지침)"처럼 원문 용어를 정확히 복원하기 때문에 검색된 청크에 질문 키워드가 더 자주 등장합니다. 이 차이가 환각률을 C의 0.903에서 D의 0.871로 내렸습니다.

오픈이가 표를 한참 들여다봤습니다. D 행의 환각률 `0.871` 이 눈에 들어왔습니다. 같은 줄의 `0.903` (C), `1.000` (B)보다 한 단계 낮은 숫자. Recall은 C와 0.016 차이로 거의 붙어 있고, Latency도 80 ms 대로 체감 차이 없는 범위였습니다. 대신 D 쪽에는 다른 조합에는 없는 무기가 있었습니다. **스캔본 서약서의 본문이 벡터DB에 들어와 있다는 것.** A·B·C는 서약서 페이지를 아예 인덱싱하지 못했는데, D는 Vision으로 788자를 꺼내 집어넣었습니다.

커서가 D 행 위에서 멈췄습니다.

**오픈이 :** "팀장님, D가 환각률이 제일 낮아요. 서약서 본문까지 올라와 있어서 검색 결과 청크에 질문 단어가 더 자주 등장하는 거예요."

팀장이 잠깐 화면을 내려다보더니 고개를 끄덕였습니다.

**팀장 :** "스캔본이 한 장뿐인데도 숫자에 반영되네요. 종류가 다른 문서를 한 파이프라인으로 다 받는다는 점에서도 D가 맞는 웃이예요."

의자 등받이에 몸이 닿는 순간, 머릿속에서 다른 정리가 됐습니다.

튜닝은 지표 하나를 위로 올리는 게 아니라, 쌓아 둔 부품이 서로 물려 돌아가는지 확인하는 작업이구나.

![../assets/챕터 10/gemini/10\_eval-concept.png] 그림 10-11. 평가 프레임워크. 질문과 정답 쌍을 반복 돌려 숫자로 품질을 측정합니다

## 10.6 튜닝 메뉴판 - 무엇을 골라 쓸까

평가 결과에서 고른 D 조합을 실제로 어떻게 조립할지, 챗터 8~10에 등장한 튜닝을 한 줄씩 놓고 재점검할 차례였습니다. 오픈이가 화이트보드에 튜닝 이름을 하나씩 적어 내려갔습니다.

**오픈이** : "전부 다 켜면 좋겠지만, 느려지고 복잡해집니다. 평가에서 숫자로 증명된 것만 남기겠습니다."

**팀장** : "규정 문서 성격 그대로 가면 돼요. 조항이 짧고, 약어가 많고, 스캔본도 일부 섞여 있고요."

그림 10-12. 챗터 8~10 튜닝 메뉴판. 체크 표시가 커넥트HR 에이전트에 적용한 일곱 조합입니다

튜닝	무엇을 바꾸나	언제 쓰나	LLM 호출	챗터
✓ 청킹 전략	문서 자르는 방법 교체	주제 섞일 때	없음	8
✓ 리랭킹	Cross-Encoder 재정렬	상위 결과 부정확	없음 (reranker 모델)	8
✓ 하이브리드 검색	Vector+BM25 합산	의미+키워드 둘 다	없음	8
HyDE	상상 답변으로 검색	어휘 차이 클 때	있음	9
Multi-Query	질문 여러 갈래	의미가 넓을 때	있음	9
✓ 약어/동义词 확장	WFH→재택근무	약어 많을 때	없음 (사전 치환)	9
✓ Parent Document	자식 청크 검색 → 부모 반환	맥락이 짧게 잘릴 때	없음	9
Compression	핵심 문장만 압축	토큰 절약	있음	9
✓ Vision LLM 파서	스캔PDF/이미지 읽기	스캔본이 섞여 있을 때	있음 (Vision)	10

**오픈이** : "Semantic 청킹으로 단락별로 자르고, 리랭커로 순위를 보정하고, 검색은 BM25와 벡터를 섞은 하이브리드로 넓혀 두고, 약어 사전으로 WFH 같은 말을 풀어 주고, Parent Doc으로 짧은 청크에 맥락을 엮습니다. 스캔본은 Vision 파서로 텍스트를 꺼내 같은 벡터DB에 태우고요. 파이프라인 밖에서는 RAG 평가 프레임워크로 주기적으로 질문셋을 돌려 이 일곱 가지가 실제로 효과가 있는지 숫자로 확인합니다."

**팀장** : "HyDE랑 Multi-Query, Compression은 왜 뺐어요?"

**오픈이** : "셋 다 질문이 들어올 때마다 LLM을 한 번씩 더 부릅니다. 하지만 규정 문서에서는 질문과 문서 어휘가 거의 같아 가상 답변(HyDE)이나 질문 확장(Multi-Query)의 이득이 작고, 청크도 이미 짧아서 Compression이 얻을 게 별로 없었습니다. 비용은 올라가는데 숫자는 제자리였던 거죠."

체크한 일곱 항목이 커넥트HR 에이전트에 실제로 적용한 D 조합입니다. RAG 평가는 조합 밖에서 이 일곱 가지가 제값을 하는지 수치로 검증하는 장치로 따로 돌립니다.

## 용어 정리

본문 속 표현	진짜 용어	정식 정의
"확대경으로 글자 읽기"	<b>OCR (Optical Character Recognition)</b>	이미지에서 문자를 인식해 텍스트로 변환
"눈을 가진 LLM"	<b>Vision LLM</b>	이미지·표·도식을 이해해 자연어로 설명하는 멀티모달 LLM
"이미지 감지 후 분기"	<b>하이브리드 파서</b>	<code>page.get_images()</code> · 텍스트 길이로 Vision LLM과 pypdf를 자동 선택
"정답 비율"	<b>Precision@k</b>	상위 k개 중 정답 청크 비율
"정답 커버리지"	<b>Recall@k</b>	정답 문서 중 상위 k개에 포함된 비율
"근거 없는 답변 비율"	<b>Hallucination Rate</b>	답변 문장 중 문서 근거가 없는 비율. 답변과 근거 문서의 단어 겹침으로 간단히 잡거나, 더 정확히 켈 땀 별도 LLM에 채점을 맡깁니다
"성적표"	<b>RAG Evaluation Framework</b>	평가셋(질문·정답)으로 파이프라인을 돌려 Precision·Recall·환각률을 수치화
"튜닝 조합 스위치"	<b>Strategy Pattern (A/B/C/D)</b>	파서·청킹·쿼리변환·검색·리랭크 부품을 갈아끼워 조합별로 성능을 비교

## 이것만은 기억하자

- **스캔 PDF는 pypdf로 안 읽힙니다.** 텍스트 레이어가 있는 페이지는 pypdf가 살리고, 그렇지 않은 페이지는 Vision LLM이 눈 역할을 합니다. 하이브리드 파서의 분기 기준은 **페이지에서 꺼낸 텍스트 길이**입니다.
- **RAG 평가는 네 지표 세트**입니다. Precision@k(정확성) · Recall@k(커버리지) · Hallucination Rate(근거 충실성) · Latency(속도). 한 면만 보면 다른 쪽이 무너집니다.
- **튜닝은 숫자로 증명된 것만 있습니다.** 우리 문서(규정·짧은 조항·약어 많음·스캔본 일부)에는 D 조합 (Semantic 청킹 + 하이브리드 검색 + 리랭커 + 약어 확장 + Parent Doc + Vision 파서)이 환각률과 커버리지 모두에서 가장 좋은 수치를 냈습니다. 스캔본이 없는 환경이라면 Vision 파서를 빼고 C로도 충분합니다. RAG 평가 프레임워크는 이 조합을 고를 때 쓴 채점기이지 파이프라인에 없는 부품은 아닙니다.

다음 챕터에서는 이 D 조합을 **하나의 파이프라인으로** 조립하고, 답변에 **근거를 붙여** 사용자에게 내보냅니다. 커넥트HR 에이전트가 완성되는 마지막 장입니다.

# 챕터 11. 커넥트HR 에이전트의 완성. 파이프라인과 근거, 그리고 마지막 여정

## 이번 챕터가 끝나면

- 챕터 8~10에서 만든 함수들을 하나의 파이프라인으로 조립합니다
- 답변에 근거를 붙여 줍니다. 정형 질문은 JSON, 비정형·복합 질문은 문서 페이지 이미지
- 챕터 7의 채팅 UI를 재사용해서 실제 사용자 경험으로 완성합니다
- 챕터 1~10의 모든 박스가 채워진 커넥트HR 에이전트의 전체 구성도를 확인합니다

## 준비하기. 실습 시작 전 한 번만 설정

### 1. 실습 폴더 이동

터미널    폴더 이동

```
cd rag-start/ex11
```

파일 구조는 다음과 같습니다.

ex11 디렉토리

```

ex11/
├── run.py # [참고] FastAPI 서버 실행 (챗터 7 UI 재사용)
├── app/ # [참고] 채팅 API + DB
│ ├── main.py
│ ├── chat_api.py # [실습] 근거 분기 로직 추가
│ └── database.py
├── src/
│ ├── pipeline.py # [실습] Application Layer. RagPipeline 조립
│ ├── evidence_capture.py # [실습] PDF 페이지 → PNG 캡처 근거
│ ├── evidence.py # [참고] 출처 추출
│ ├── agent.py # [참고] IntegratedAgent (챗터 6)
│ ├── tuning/
│ │ ├── query_expander.py # QueryExpander - 약어 사전
│ │ ├── bm25_retriever.py # BM25Retriever - rank-bm25 래퍼
│ │ ├── ensemble_retriever.py # EnsembleRetriever - BM25 + ChromaDB 하이브리드
│ │ ├── reranker.py # CrossEncoderReranker (+ SimpleReranker 폴백)
│ │ ├── parent_doc.py # ParentDocumentRetriever - 자식 → 부모 복원 (챗터 9)
│ │ └── hybrid_parser.py # 챗터 10 파서 이식 - 인덱싱 시 pypdf → Vision
│ └── tools/ # [참고] Infrastructure Layer. SQL 3종 + ChromaDB 검색
│ ├── leave_balance.py
│ ├── sales_sum.py
│ ├── list_employees.py
│ └── search_documents.py
├── templates/chat.html # [실습] 근거 이미지·JSON 표시 영역 추가
├── static/
│ └── evidence/ # 런타임 생성 (PDF 캡처 PNG 캐시)
├── data/
│ ├── docs/ # [참고] 원본 PDF
│ ├── chroma_db/ # 런타임 생성
│ └── test_questions.json # [참고]

```

## 2. 실습 환경 구축

**터미널** 환경 구성. macOS / Linux

```

cd ex11
python3.12 -m venv .venv
source .venv/bin/activate
cp .env.example .env
ollama pull llama3.1:8b
ollama pull qwen2.5vl:7b
pip install -r requirements.txt

```

**터미널** 환경 구성. Windows

```

cd ex11
py -3.12 -m venv .venv
.venv\Scripts\activate
copy .env.example .env
ollama pull llama3.1:8b
ollama pull qwen2.5vl:7b
pip install -r requirements.txt

```



### 3. 실습 순서

1. `src/pipeline.py` (D 조합: Semantic 청킹, 하이브리드 검색, 리랭커, 약어 확장, Parent Doc, Vision 파서)을 하나로 조립합니다
2. `src/evidence_capture.py`. PDF 페이지를 PNG로 렌더링해 근거로 제공합니다
3. `app/chat_api.py` + `templates/chat.html`. 정형(JSON)과 비정형(이미지) 근거 분기를 붙입니다

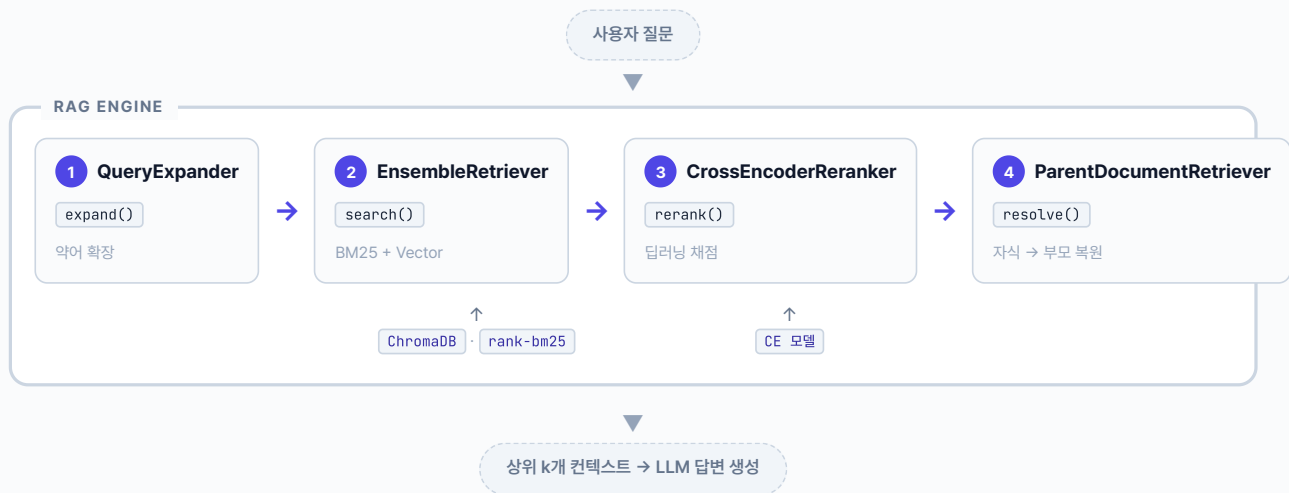
## 11.1 파이프라인 조립 - 튜닝 부품을 한 파일에 엮는다

팀장이 옆자리로 의자를 밀고 왔습니다. 모니터 한 켠엔 챗터 8~10에서 돌린 실험 결과가 여전히 떠 있었습니다.

**팀장** : "이번엔 새 이론을 엮지 말고, 지금까지 본 튜닝을 한 자리에 모아 이어 붙이기만 해요."

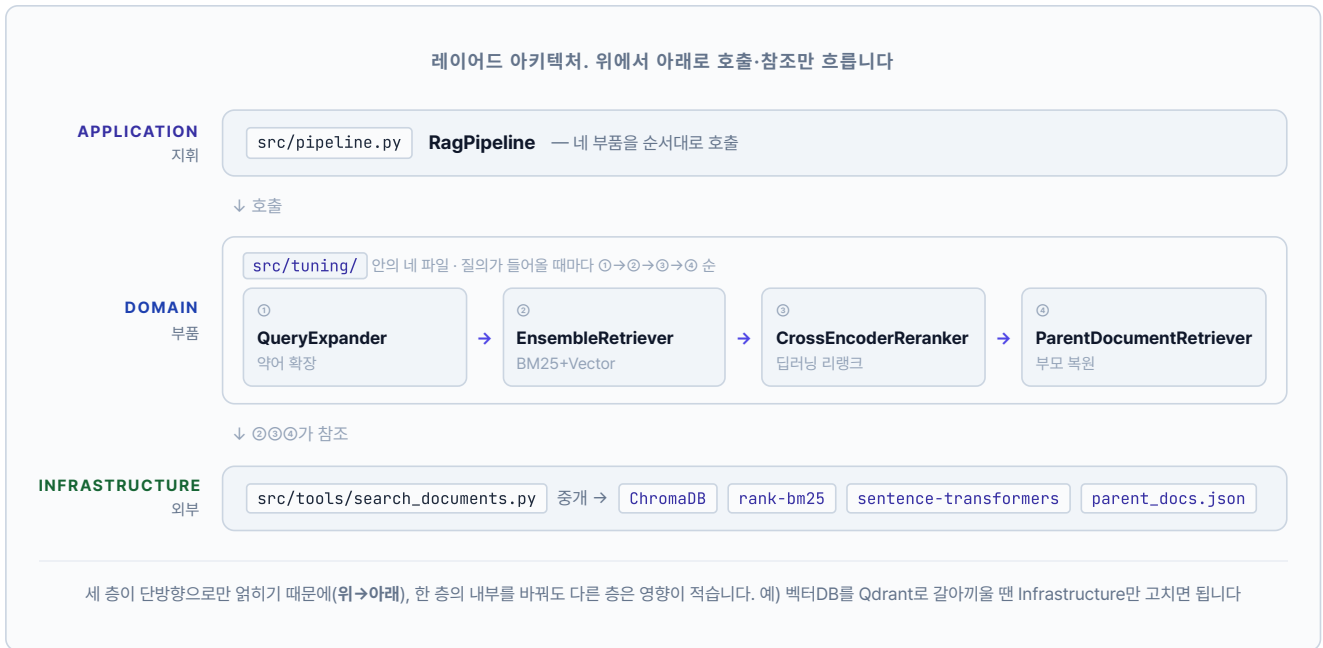
오픈이가 에디터에서 새 파일을 열었습니다. `pipeline.py`. 키보드 위에 손을 올린 채 빈 화면을 잠시 바라봤습니다. 이번 장의 할 일은 지금까지 따로 실험하던 **튜닝 부품**을 한 파일에 모아 질의 흐름 하나로 엮는 것입니다. 챗터 8·9에서는 조각별로 돌려 성능을 비교만 했다면, 여기서는 같은 개념을 함수로 `pipeline.py` 안에 모아 `run_pipeline` 하나로 차례차례 호출합니다.

그림 11-1. RAG 엔진. 질문이 들어와 답변이 나가기까지



최초 실행 시 `ex11/data/docs`를 하이브리드 파서(pypdf → Vision)로 인덱싱해 자식 청크는 ChromaDB에, 부모 페이지는 `parent_docs.json`에 나눠 적재합니다. 네 부품은 질문이 들어올 때마다 자식 검색 → 부모 복원 순으로 실행됩니다

파이프라인을 **레이어드 아키텍처**로 분리합니다. `pipeline.py`의 `RagPipeline`이 질의가 들어올 때마다 네 부품 (`QueryExpander` · `EnsembleRetriever` · `CrossEncoderReranker` · `ParentDocumentRetriever`)을 차례대로 부르고, 그 부품들은 `src/tools/`의 ChromaDB-BM25 같은 외부 저장소를 끌어 씁니다. 벡터DB는 `ex11/data/docs`를 자체 인덱싱해 **자식·부모 두 저장소**로 빌드합니다. 페이지 단위 원문은 부모로 `ex11/data/parent_docs.json`에 두고, 220자 정도로 잘게 쪼갠 자식 청크는 ChromaDB에 `parent_id` 메타데이터와 함께 적재합니다. PDF는 `src/tuning/hybrid_parser.py` (챗터 10의 하이브리드 파서를 이식한 모듈)가 pypdf로 먼저 꺼내 보고 스캔본이면 Vision LLM으로 넘어가 텍스트를 뽑습니다. BM25 인덱스와 Cross-Encoder 모델은 `EnsembleRetriever` · `CrossEncoderReranker` 객체가 최초 사용 시 1회만 초기화합니다.



`src/pipeline.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 1 src/pipeline.py. RagPipeline 조립 (Application Layer)

```
from .tuning import (
 CrossEncoderReranker,
 EnsembleRetriever,
 ParentDocumentRetriever,
 QueryExpander,
)

class RagPipeline:
 # TODO: 4개 도메인 객체를 생성자에서 준비하고 run()에서 순서대로 호출합니다.
 def __init__(self, alpha: float = 0.5) → None:
 from .tools.search_documents import _get_store
 collection, parent_docs = _get_store()
 # 1. 약어 확장 (챕터 9)
 self.expander = QueryExpander()
 # 2. BM25 + ChromaDB 하이브리드 검색 (챕터 8) - 자식 청크 대상
 self.retriever = EnsembleRetriever(collection, alpha=alpha) if collection else None
 # 3. Cross-Encoder 리랭크 (챕터 8)
 self.reranker = CrossEncoderReranker()
 # 4. 자식 → 부모 복원, 중복 parent_id 제거 (챕터 9)
 self.parent_retriever = (
 ParentDocumentRetriever(child_collection=collection, parent_docs=parent_docs)
 if collection and parent_docs else None
)

 def run(self, query: str, k: int = 3) → List[dict]:
 if self.retriever is None or self.parent_retriever is None:
 return []
 expanded = self.expander.expand(query)
 retrieved = self.retriever.search(expanded, k=k) # 자식 청크
 if not retrieved:
 return []
 reranked = self.reranker.rerank(expanded, retrieved, top_k=k * 2)
 parents = self.parent_retriever.resolve(reranked, top_k=k) # 부모 복원
 return parents[:k]
```

`RagPipeline.__init__` 은 생성자 주입 패턴으로 도메인 객체를 한 번 준비해 두고, `run()` 은 다섯 줄 안에서 **확장 → 자식 검색 → 리랭크 → 부모 복원 → 절단**을 호출합니다. 객체별 내부 구현은 `src/tuning/` 각 파일에 나뉘어 있어 개별 교체-테스트가 쉽습니다.

도메인 객체 (tuning/*)	역할
<code>QueryExpander.expand(q)</code>	WFH → WFH(재택근무) 사내 약어 확장
<code>EnsembleRetriever.search(q, k)</code>	BM25 + ChromaDB(Vector)로 자식 청크 검색. 두 점수 정규화 후 가중 합산
<code>BM25Retriever.search(q, k)</code>	rank-bm25 키워드 검색기 (EnsembleRetriever 내부에서 사용)
<code>CrossEncoderReranker.rerank(q, docs, k)</code>	BAAI/bge-reranker-v2-m3 딥러닝 모델로 쿼리-자식 청크 정합성 채점
<code>ParentDocumentRetriever.resolve(docs, k)</code>	자식 청크의 <code>parent_id</code> 로 부모 페이지 원문 복원. 같은 부모 중복 제거

## 11.2 근거 모양을 질문에 맞춘다

파이프라인이 답변을 돌려줄 때 어느 문서에서 가져왔는지 **근거**까지 같이 보여 주어야 독자가 믿습니다. 검색 결과에는 (source, page) 쌍이 이미 붙어 있으니, 그 정보로 PDF의 해당 페이지를 PNG로 떠 주면 답변 옆에 바로 붙일 수 있습니다.

src/evidence\_capture.py를 열고 TODO의 pass를 지우고 아래 코드를 작성합니다.

**실습 1** src/evidence\_capture.py. PDF 페이지 → PNG 렌더 + 캐시

```
import fitz # PyMuPDF
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
DOCS_DIR = BASE_DIR / "data" / "docs"
CACHE_DIR = BASE_DIR / "static" / "evidence"

def render_evidence_image(source_name: str, page_num: int, dpi: int = 150) → str:
 # TODO: PDF 특정 페이지를 PNG로 렌더링하고 /static/evidence/*.png URL 반환.
 # 1. data/docs/ 하위에서 PDF 경로 탐색
 pdf_path = next(DOCS_DIR.rglob(f"{source_name}.pdf"), None)
 if not pdf_path:
 return ""
 # 2. 캐시 디렉토리 준비
 CACHE_DIR.mkdir(parents=True, exist_ok=True)
 out_path = CACHE_DIR / f"{source_name}_p{page_num}.png"
 # 3. 캐시 적중 시 재사용
 if out_path.exists():
 return f"/static/evidence/{out_path.name}"
 # 4. PDF 열어서 해당 페이지 → PNG 저장
 doc = fitz.open(str(pdf_path))
 pix = doc[page_num - 1].get_pixmap(dpi=dpi)
 pix.save(str(out_path))
 doc.close()
 return f"/static/evidence/{out_path.name}"
```

매개변수를 보면 source\_name은 test\_questions.json의 relevant\_sources와 같은 PDF 파일명(확장자 제외), page\_num은 1부터 시작하는 페이지 번호입니다. dpi가 클수록 선명하지만 파일도 커집니다. 150이 화면 표시용으로는 충분합니다.

/static/evidence/ 경로는 FastAPI가 정적 파일로 이미 서빙하고 있어서, 반환한 URL을 그대로 에 쓰면 브라우저에 뜹니다. 캐시 폴더에 한 번 저장된 이미지는 다음 호출에서 재사용되니 성능 부담도 없습니다.

그런데 모든 질문에 PDF 이미지가 근거로 어울리는 건 아닙니다.

**동료** : "서약서나 취업규칙은 이렇게 이미지로 보는 게 좋은데, '1분기 매출 얼마야' 같은 질문은 이미지보다 숫자 그 자체가 더 편해요. 표라면 표, JSON이면 JSON으로 보여 주는 게 낫겠어요."

맞는 말이네. 비정형 답은 문서가 근거지만, 정형 답은 수치가 근거다.

질문의 성격에 따라 **근거의 모양**을 달리해야 합니다. 간단한 규칙으로도 충분합니다. 질문에 숫자·금액·수치·퍼센트 같은 단어가 있으면 **정형**, 그 외는 **비정형**. 정형이면 DB 조회 결과를 JSON 그대로 내보내고, 비정형이면 방금 만든

render\_evidence\_image()로 PDF 캡처 이미지를 붙입니다.

실무에서는 LLM에 "이 질문이 정형이나 비정형이나?"를 먼저 묻는 방식도 쓰지만, 호출이 한 번 더 늘어납니다. 이 책에서는 키워드 규칙으로 시작하고, 필요해지면 LLM 분류로 업그레이드하는 쪽으로 가겠습니다.

`app/chat_api.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

**실습 2** `app/chat_api.py`. 카테고리 분류 + 근거 모양 분기

```
import re

_STRUCTURED_KEYWORDS = [
 "얼마", "몇", "금액", "매출", "수치", "비율", "%", "퍼센트",
 "명", "건", "개수", "총", "평균", "합계", "남은", "남아",
 "언제", "며칠", "시간", "분기", "연도", "년도",
]

_STRUCTURED_RE = re.compile(r"\b\d+[\.%가-힣]?")

def _classify_query_category(query: str) → str:
 # TODO: 질문을 정형(structured) / 비정형(unstructured)로 분류합니다.
 # 1. 숫자 토큰이 있는지
 if _STRUCTURED_RE.search(query):
 return "structured"
 # 2. 정형 키워드가 하나라도 포함되는지
 lower = query.lower()
 for kw in _STRUCTURED_KEYWORDS:
 if kw in lower:
 return "structured"
 # 3. 그 외는 비정형
 return "unstructured"
```

분류 결과는 `ChatResponse.category` 필드로 같이 내려보냅니다. 그리고 기존 엔드포인트 로직에서:

- `category = "structured"` 이면 DB 조회 결과를 `evidence_json` 필드에 담는다
- `category = "unstructured"` 이고 아직 `evidence_images` 가 비어 있으면, 상위 문서의 (source, page)로 `render_evidence_image()` 를 불러 이미지를 첨부한다

두 갈래가 한 엔드포인트 안에서 자연스럽게 갈라집니다.

그림 11-2. 카테고리에 따라 갈라지는 근거 모양

#### 정형 질문 → JSON

Q. 영업부 11월 매출 얼마야?

영업부 11월 매출은 1,240,500,000원입니다.

```
{
 "get_sales_sum": {
 "team": "영업부",
 "month": "2025-11",
 "total": 1240500000
 }
}
```

#### 비정형 질문 → 이미지

Q. 병가 신청 절차 알려줘

병가는 진단서 제출 후 인사팀에 신청합니다...

[PDF 캡처] HR\_취업규칙 p.3

같은 UI에서 질문의 성격에 따라 근거 모양이 자동으로 바뀝니다. 숫자는 숫자로, 문장은 이미지로

**동료** : "이제 매출은 매출답게, 규정은 규정답게 보여지네요."

**팀장** : "좋아요. 이걸로 직원들이 '믿고 쓸 수 있는' 에이전트가 됐어요."

근거의 모양이 곧 답변의 신뢰다.

## 11.3 웹 UI로 확인한다

조립이 끝났습니다. 선택한 D 조합(Semantic 청킹, 하이브리드 검색, 리랭킹, 약어 확장, Parent Doc, Vision 파서)을 엮은 파이프라인으로 챗터 7의 채팅 UI를 다시 띄워 보겠습니다.

**터미널** ex11 서버 실행 (챗터 7 UI + 챗터 8~10 튜닝 적용)

```
python run.py
```

서버 터미널에 기동 순서가 세 단계로 찍힙니다. 벡터DB 준비 → 리랭커 로드 → uvicorn 기동. 여기까지 오는 데 평균 2분이 걸립니다(최초 1회는 모델 다운로드로 더 걸립니다).

```

ex11 - python run.py

———— ex11 커넥트HR 에이전트 ————

VISION_MODEL=qwen2.5vl:7b · PORT=8000

1. 벡터DB 준비
 ✓ 임베딩 모델 준비 완료
 ✓ 기존 인덱싱 재사용 - children=39, parents=7

2. Cross-Encoder 리랭커 로드
 ✓ 리랭커 준비 완료

3. 서버 기동
 ✓ 준비 완료 → http://localhost:8000/chat

INFO: Uvicorn running on http://0.0.0.0:8000

```

그림 11-3. 서버 기동 시 터미널에 찍히는 세 단계

브라우저에서 <http://localhost:8000/chat> 을 엽니다. 챗터 7의 ConnectHRAgent 채팅 UI가 뜹니다. 같은 UI, 다른 엔진.

오픈이가 채팅창으로 손을 올겼습니다. 커서가 입력란에서 깜빡였습니다.

첫 질문. [병가 신청 절차 알려줘](#) .

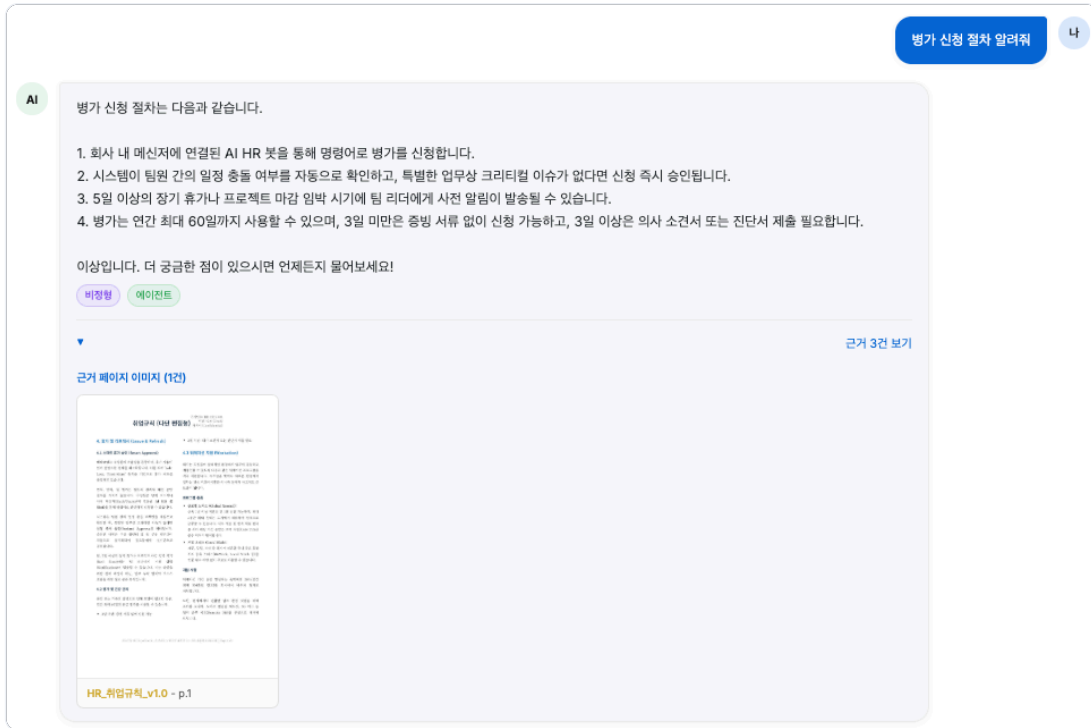


그림 11-4. 병가 규정이 정확히 돌아옵니다. 출장·급여 같은 다른 주제는 섞이지 않습니다

Semantic 청킹이 병가 조항만 한 덩어리로 유지하고, 리랭커가 관련 없는 청크를 밀어낸 결과입니다. 그런데 눈에 보이는 답변만으로는 내부에서 뭐가 일어났는지 알 수 없습니다. 오픈이가 서버를 돌리고 있는 터미널 창으로 시선을 돌렸습니다.



그림 11-5. 병가 질문이 들어온 순간부터 응답이 나갈 때까지의 풀 로그

사전에 등록된 **병가**가 **병가(병가 유급휴가)**로 풀어져 검색으로 넘어갔습니다. 이어서 llama3.1 모델이 검색 결과로 답을 만들어 내는 데 24.9초가 걸렸고, 유형은 비정형(**unstructured**)으로 기록됩니다. 챗터 7에서 붙인 TokenTracker가 챗터 11에서도 그대로 일하고 있습니다.

다음은 약어가 섞인 질문. **WFH 정책 알려줘**.

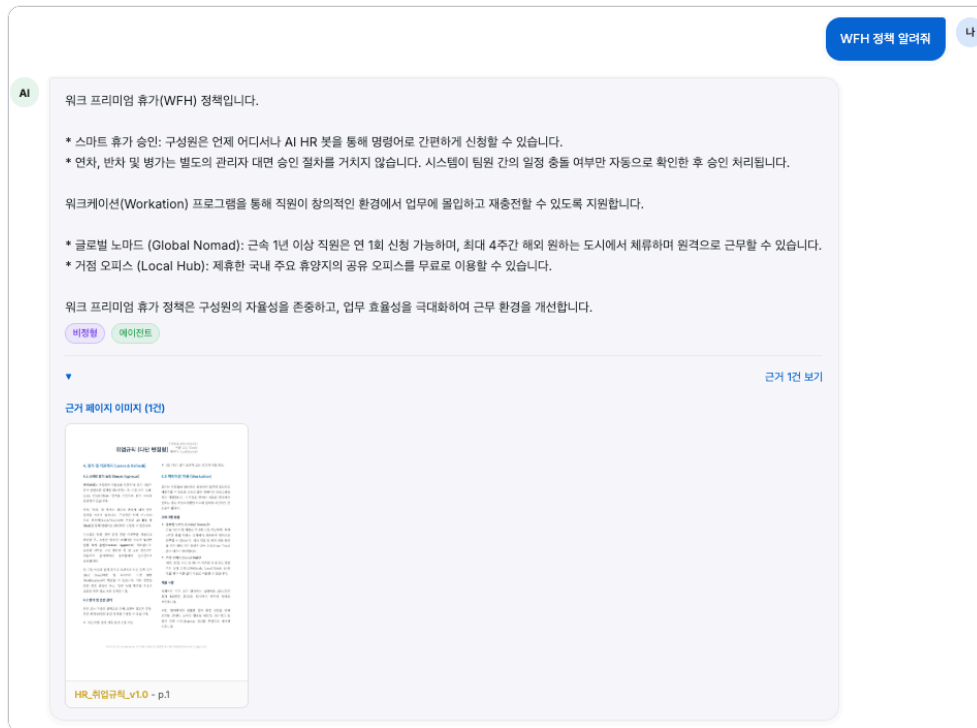


그림 11-6. 취업규칙의 재택·원격 근무 조항이 근거로 돌아옵니다

답은 나왔는데, 이번엔 조금 더 흥미로운 구석이 있습니다. 취업규칙 문서에는 "재택근무"라는 단어가 없고 "원격 근무"만 쓰여 있는데, 답변은 정확히 찾아왔습니다. 오픈이가 다시 터미널을 봤습니다.



그림 11-7. WFH가 재택근무로 확장된 뒤 동일 경로를 밟습니다

사전이 **WFH** 에 **(재택근무)** 를 덧붙였습니다. 그런데 이 확장어는 문서에 없는 표현입니다. 그 간극을 벡터 임베딩이 메워 줍니다. **재택근무** 와 **원격 근무** 는 의미 공간에서 가까운 자리에 놓여 있어 한쪽을 질문으로 들이밀면 다른 쪽 문서가 걸립니다. 챗터 9의 약어 사전이 못 덮는 표현 차이를 챗터 4의 임베딩 모델이 메우는 협업입니다. 출력 토큰이 226→238로 조금 늘어난 것은 LLM이 워크케이션·거점 오피스 두 항목을 합쳐 답했기 때문입니다.

마지막 질문. **정보보안서약서 내용이 뭐야?** .



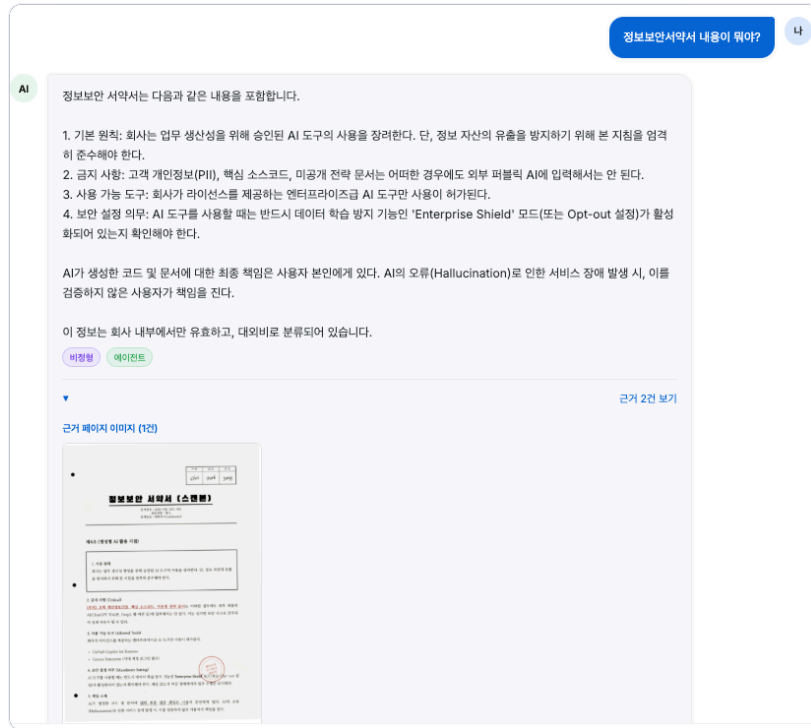


그림 11-8. 제4조 생성형 AI 활용 지침까지 조항 번호가 또렷하게 돌아옵니다

한 박자 기다렸습니다. 노트북 팬이 숨을 한 번 몰아쉬고 서약서 본문이 그대로 돌아왔습니다. 제4조, 제5조, 조항 번호까지 정확합니다. 터미널에는 이런 줄이 찍혔습니다.

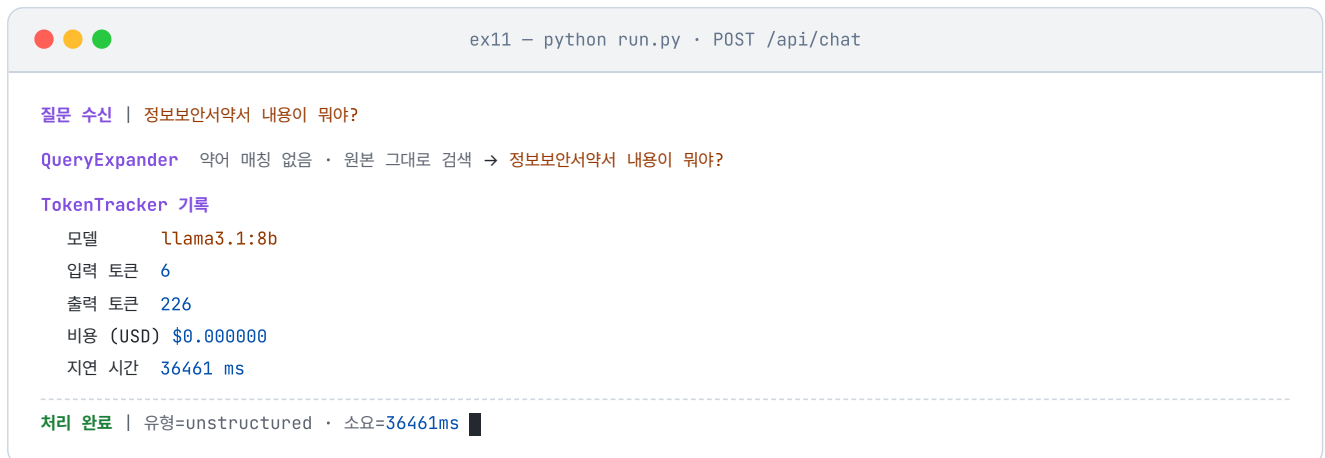


그림 11-9. 약어가 없는 질문은 원본 쿼리가 그대로 흐릅니다. 지연은 늘어납니다

이번엔 사전에 등록된 단어가 없어 원본이 그대로 검색으로 내려갔습니다. 그래도 답이 또렷한 이유는 인덱싱 단계에 있습니다. 챗터 10에서 붙인 Vision 파서가 스캔본 한 장을 텍스트로 꺼내 벡터DB에 올려 뒀기 때문입니다. 지연 시간이 24초 → 36초로 늘어난 것도 눈에 띄니다. 스캔본에서 꺼낸 내용이 구조가 복잡한 표·조항 나열이라 LLM이 답을 엮는 데 시간이 더 걸립니다. 약어 확장이 아니라 인덱싱 때의 투자가 결과를 살린 케이스입니다.

세 개 다 잡았네.

**오픈이** : "병가·WFH·서약서 다 되네요. 서약서는 챗터 10에서 Vision 붙여 둔 덕분이네요."

**동료** : "질문 타입이 세 개 다 다른데 세 번 다 근거까지 붙네요."

## 11.4 엔진의 내부: 파이프라인 한 장

웹 화면에서 본 세 답변이 안에서는 어떤 단계를 거쳐 만들어졌는지, 파이프라인 전체를 한 장에 정리하면 이렇게 보입니다.

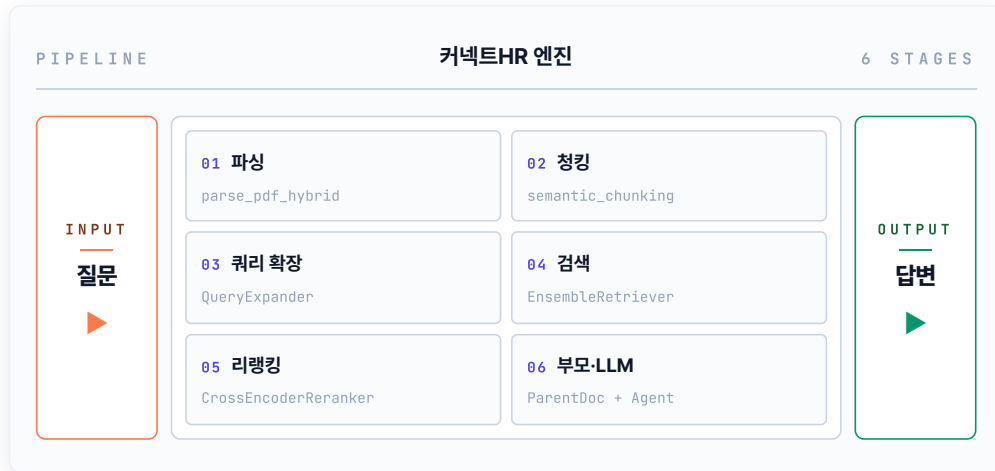


그림 11-10. 질문이 들어와 6단계 파이프라인을 지나 답변이 나가기까지

01(파싱)과 02(청킹)는 서버가 처음 뜰 때 한 번만 도는 **인덱싱** 단계입니다. 문서를 벡터DB에 올리는 일이라 매 질문마다 반복할 필요가 없죠. 나머지 03부터 06(쿼리 확장·검색·리랭킹·부모·LLM)이 질문이 들어올 때마다 도는 **질의** 단계이고, 실습 1의 `run_pipeline`이 이 03~06을 차례로 호출합니다.

챕터 8~10에서 따로 실험하던 부품들이 각 단계에 박혀 있습니다. Semantic 청킹은 02, 약어 사전은 03, 하이브리드 검색은 04, 리랭커는 05, 부모 문서 확장과 LLM 답변 생성은 06. 따로 볼 때는 각자의 평가표만 있었지만, 지금은 한 줄로 꿰어져 한 질문의 응답을 함께 만들어 냅니다.

## 11.5 전체 구성도의 완성

11.4의 파이프라인이 "질문이 답변으로 흐르는 길"이었다면, 아래 구성도는 그 길을 떠받치는 **시스템 전체**를 한 장에 담은 모습입니다. LLM부터 에이전트 루프, 도구, 저장소까지.



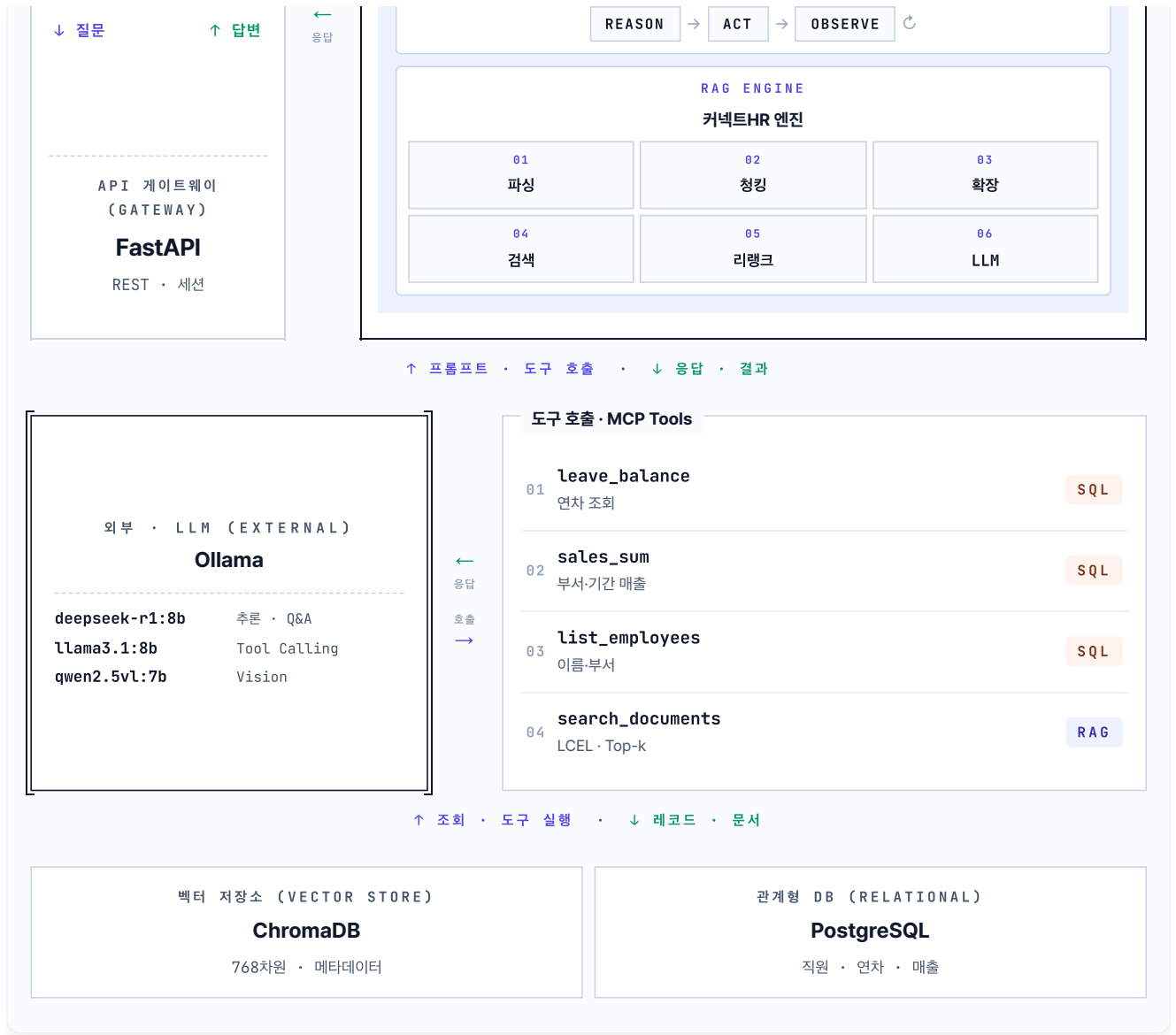


그림 11-11. 커넥트HR 에이전트의 시스템 아키텍처. 사용자부터 LLM, 에이전트 루프, 도구, 저장소까지 한 장에 담았습니다

상단 Ollama가 프롬프트를 받아 답변을 돌려주고, 통합 에이전트 안의 ReAct 루프가 4개 도구를 선택적으로 호출해 PostgreSQL/ChromaDB에서 데이터를 꺼냅니다.

오픈이가 모니터에서 눈을 뗐습니다. 창가 쪽 형광등이 한 번 깜빡였고, 키보드 두드리는 소리가 멀리서 작게 들렸습니다.

자리 뒤로 동료 몇 명이 서 있었습니다. 누군가는 채팅창에, 누군가는 구성도 한 장에 눈이 가 있었습니다.

**동료** : "병가·WFH·서약서 다 잡네요. 고생했어요."

조각들이 한 장에 붙어 버렸네.

챕터 1의 맛보기 RAG에서 출발해 여기까지 왔습니다. 문서 규칙을 세우고(챕터 2~3), 벡터로 옮기고(챕터 4), RAG 엔진을 엮고(챕터 5), 통합 에이전트로 다시 짜고(챕터 6), 운영 안정화(챕터 7), 검색 품질(챕터 8), 쿼리 해석(챕터 9), 멀티모달(챕터 10), 그리고 평가와 조립(챕터 11)까지. 그사이 **커넥트HR 에이전트** 한 명이 완성됐습니다.

팀장이 한참 뒤에 지나가다 모니터를 들여다봤습니다.

**팀장** : "이론 안 엮고 붙이기만 한 거, 이게 이번에 가장 어려운 일이었을 거예요."

책은 여기서 끝나지만, 시스템은 출발선입니다. 오늘 만든 엔진이 내일 어떤 질문에서 무너지는지는 이제 사무실 바깥의 사용자가 알려 줄 겁니다.

## 용어 정리

본문 속 표현	진짜 용어	정식 정의
"한 줄로 엮은 조립"	<b>RAG Pipeline</b>	파싱→청킹→검색→리랭킹→쿼리 재작성 →LLM 답변까지 하나의 데이터 흐름으로 이어붙인 구조
"답변 옆의 증거"	<b>Evidence / Citation</b>	답변 근거가 된 원본 문서 조각. 텍스트 인용· 링크·이미지 캡처 등으로 제공
"정형 / 비정형 분기"	<b>Response Modality Routing</b>	질문 성격에 따라 응답 형태(JSON 수치·문서 이미지 등)를 달리 내보내는 로직

### 이것만은 기억하자

- **조립이 가장 큰 일의 절반입니다.** 챕터 8~10에서 실험한 튜닝 부품을 한 파일에 함수로 모아 `run_pipeline` 하나로 엮으면 파이프라인이 완성됐습니다. 쌓인 부품이 많을수록 조립의 힘이 커집니다.
- **근거는 답변만큼 중요합니다.** 사용자는 답의 내용뿐 아니라 "어디서 나왔는지"로 신뢰를 판단합니다. 정형 수치는 JSON으로, 비정형 내용은 문서 이미지로. 질문의 성격에 맞춰 근거의 모양을 바꿉니다.
- **여기서 끝이 아닙니다.** 평가셋을 늘리고, 사용자 로그로 실패 쿼리를 수집하고, Langfuse 같은 도구로 운영 중 품질을 상시 관측하세요. 커넥트HR 에이전트는 이제 **계속 자라는** 시스템입니다.